从第一步讲起

数据结构

超级纲要 · 教材式初稿

概念 · 推导 · 原图 · 例题 · 自测

例题保留数据的中间状态与公式的推导步骤。
原始课件图标明出处，补充示意按正文模型解释。

2026-10-25 · sTBook
Wiki 与 Typst 共用正文 · 原图均在图注中注明出处

章节目录

如何使用这本笔记 3
1 算法的要素、计算模型与正确性 5
2 增长的快慢：渐近记号、级数与封底估算 13
3 迭代与递归：减治、分治与动态规划 19
4 向量：抽象数据类型、插入删除与动态扩容 31
5 二分查找、插值查找与位图 39
6 起泡排序与归并排序 47
7 列表：循位置访问的链表实现 52
8 栈：调用栈、表达式求值与栈混洗 58
9 队列：先进先出、摊还分析与单调结构 63
10 二叉树：遍历、重建与层次结构 68
11 Huffman 编码与比较下界 71
12 二叉搜索树与 AVL 树 74
13 伸展树、B 树、红黑树与跳转表 77
14 区间树与 kd 树：范围查询的两种武器 80
15 散列：循关键码访问、冲突排解与重散列 82
16 图：遍历、拓扑排序与双连通 85
17 最短路径与最小生成树 88
18 优先级队列：完全二叉堆、Floyd 建堆与左式堆 90
19 字符串匹配：KMP 与 next 表 92
20 快速排序：轴点、划分与希尔排序 94
22 PS01：把“会背复杂度”变成“自己能数出来” 96
23 PS02：向量里的每一次搬动，都要知道搬了谁 110
24 PS03：节点身份、接线顺序与排序成本 124
25 PS04：先把栈里的东西写出来，再谈递归、表达式和单调结构 135
26 PS05：树的箭头、遍历顺序和编码成本，一步一步核对 149
28 把证明里省掉的那一步写出来 164
29 题目没有告诉你用什么结构时，先算一个小输入 171
30 PS04 的 21 个表达式：把每一次压栈、弹栈写全 178
31 换掉数字以后，仍然能重做的六种推理 198
32 习题 1-32：把十六段程序逐层数清楚 203
33 教材、习题解析、PS 与 PA，怎样对着解决同一个问题 213
资料覆盖、原图出处与使用说明 225
读图、读程序和读公式时怎样查符号 227

如何使用这本笔记

这本笔记为三门课而写：数据结构、计算机系统概论、大学物理 B(2)。数据结构部分共 36 章：第 01—20 章沿教师讲义逐章展开主干内容，第 21 章梳理五个 PA 主题，第 22—26 章逐题导航五份 PS，第 27—33 章衔接教材习题与《习题解析》，95、96 两章交代材料来源与全书符号。每一章都遵守同一条写作约定：**概念先落地成具体例子，例子算到每一步看得见，再回到一般结论**。定理和公式都附推导；算法给出完整代码与一次具体输入的执行过程。

各章怎么配合

第一次顺读按 01→20 的顺序。01—03 章打地基：什么是算法、成本怎么数、减治分治与动态规划怎样思考。04—09 章是线性结构（向量、列表、栈、队列），10—14 章是树形结构（二叉树、搜索树、高级搜索树、区间查询），15—20 章是散列、图、堆、字符串与排序。每个新结构都从一次具体操作学起，复杂度分析跟在过程后面，不空谈记号。

11—20 章覆盖编码、搜索树、范围查询、散列、图、堆、串匹配与进阶排序。第 13 章的伸展树、B 树、红黑树各自有自己的规则——伸展树凭“访问即调整”，B 树凭“节点容量上下限”，红黑树凭“颜色约束”；读一种时先按它自己的规则把操作走完，再横向比较另一种，不要把两套中间状态混在一步里。势能分析放在基本操作之后：它解释一串操作的总成本，不是看懂单次旋转的前提。

21—33 章面向做题。PA 与实验解答仅本地保留，公开版不含第 21、27 章。五份 PS 在 22—26 章，保留原题编号；遇到陌生方法，按题内链接回到完整讲解。32 章逐段拆解《习题解析》1—32 题的程序，33 章说明教材、习题、PA、PS 四类材料如何交叉阅读。95 章如实列出各章材料来源与实际覆盖边界——哪些逐题讲过、哪些只讲了代表题，都以那里为准。

怎样读算法的执行过程

书中的每个算法都配一次具体输入的完整执行。数组画成一排带下标的格子，格内是当前值；栈注明哪一端是顶；树注明父子关系与空孩子。一次比较若没有改写数据，图里仍然保留这次比较，因为它决定下一步往哪边走。逐步表格的每一行对应循环的一次迭代，新增的列只放当步变化的东西。判断自己有没有读懂，标准很具体：给你同一结构的另一个输入，你能不能不看书把每一步算出来。做练习题就是用这个标准检验。

复杂度分析一律紧跟在过程之后：先数出这次执行做了多少步，再看规模 n 变大后步数怎么长。记号（O、Ω、Θ）的定义与化简规则集中在 02 章，此后各章只在用到新技巧（均摊分析、势能法、主定理）时现场解释。

网页与 PDF 各自的用法

网页版在正文关键处配交互演示：数组、栈、树的每一步可以翻看、播放，每步下面注明做了什么、为什么可以做。演示只是辅助，PDF 保留了同样的静态步骤图，离线读完不受影响。教师课件里的复杂原图（箭头、配色、表格布局承载信息的）以整页形式收录，正文指出该看图中的哪个部分；若课件与教材的实现策略不同（例如 B 树的先分裂与后分裂），两种过程分别完整画出，不混用。

公式怎么读

每个公式第一次出现时，符号逐个交代。求和号是个典型的例子：

$$\sum_{i=1}^{3} i = 1 + 2 + 3 = 6$$

下标 i 从 1 取到 3，Σ 表示把右端的表达式按 i 的每个取值算一遍再相加。求和号只是“逐项相加”的缩写；如果一时看不出某一项代表什么，回到具体执行表，那里有它对应的每一次实际操作。全书符号的完整清单在 96 章，忘了随时回查。

一条学习建议

这门课的知识点环环相扣：02 章的级数在 18 章建堆时回归，04 章的摊还分析在 05 章位图、09 章队列里反复出现，03 章的最大区间在 14 章线段树里以四项摘要的形态重生。读到后面忘了前面，就顺着书里的反向引用跳回去；每一章的“本章要点”一节，供复习时快速扫读。做题卡住时，先回到对应章节把那一步的理由说清，比多看十道题的题解更有用。

1 算法的要素、计算模型与正确性

计算是人类认识和改造世界的基本方式之一，与理论、实践并列。对计算的原理和过程给出形式化的定义与描述，得到的产物就是**算法**；执行算法的工具与平台，就是**计算机**。计算机的性能年年翻新，但一件计算任务能否真正高效完成，根本因素仍在人对问题本质的理解：设计出合乎问题内在规律的算法，再选用或定制支撑它的数据结构。本章把“算法”这个词的每一层含义拆开落实：它由什么构成、依赖什么工具、怎样证明它正确、又怎样在机器上逐步执行。

1.1 一杆足够大的秤：曹冲称象

大象是称不动的：没有那样大的秤，也没有谁抬得动它。据《三国志》记载，曹操收到孙权送来的一头大象，想知道重量，众人无计可施，年幼的曹冲却轻松设计了一个“分而称之”的算法：

```
weight( elephant )
输入：一头大象
输出：大象的重量
1. 置象大船之上          //将大象牵上一艘船
2. 刻其水痕所至          //在吃水线处刻下标记，然后牵象下船
3. 称物以载之            //把称过重的石块装上船，直到船沉回同一刻痕
4. 校可知矣              //累计石块总重，即为输出
```

曹冲用水和船组成一杆足够大的秤：秤星是吃水线，秤砣是一堆石块。这套装置与现代电子计算机相去甚远，本质却毫无差别——都是支持和实现计算过程的物理机制。不妨就叫它“船石计算机”，上述过程则称为“曹冲算法”。它的正确性由阿基米德原理与质量的可加性保证：吃水深度只取决于总载荷。

1.2 尺规计算机：不许量长度的三等分

再来看一个几何界的算法。欧几里得几何是公理系统的鼻祖，它为大量几何问题提供基于作图的解法，使用的工具只有两件：足够长但不带刻度的直尺，以及半径不限的圆规。不妨称之为“尺规计算机”。它支持的基本操作不外乎五类：过两点作一直线；确定两直线的交点；以任一点为中心作任意半径的圆；确定直线与圆的交点；确定两圆的交点。

给定线段 AB，怎样只用这五类操作把它三等分？量出 $|AB|/3$ 标记两个点就完了，可直尺没有刻度，除法无从做起。尺规计算机给出的解法是算法 1.2：

```
trisect( AB )
输入：线段AB
输出：三等分点C、D
1. 从A发出一条与AB不重合的射线
2. 在射线上依次取C'、D'、B'，使|AC'|=|C'D'|=|D'B'|
3. 联接B'B
4. 过C'作B'B的平行线，交AB于C
5. 过D'作B'B的平行线，交AB于D
```

由平行截线定理（或《几何原本》卷六命题二），$|AC| : |CD| : |DB| = |AC'| : |C'D'| : |D'B'| = 1 : 1 : 1$，三等分完成。其中第 4、5 步都要用到“过直线外一点作平行线”，这本身也是一件尺规可完成的操作：它是相对于三等分问题的**子问题**，其解法称为**子算法**。复杂的原问题分解为简单的子问题，子算法组合起来构成功能更强的算法——这个模式将贯穿整本书。

两件工具、五类操作，量不了任何长度，却完成了除法。所谓**计算模型**，就是这类“工具加规则”的组合：曹冲算法的模型由船、水、刻刀和石块构成，trisect 的模型由直尺和圆规构成。模型决定了哪些操作可以直接兑现、每步代价几何；同一个问题换一个模型，可行性与成本都可能改变。

1.3 算法的要素

算法定义：在特定计算模型下，旨在解决特定问题的指令序列。

由此可以逐项列出算法应当满足的要素。

输入与输出。待计算问题的任何一个具体实例，都要以适当方式描述后才能交给算法，这类描述称为**输入**（input）；经计算得到的信息，即实例对应的答案，需要以适当方式呈现，称为**输出**（output）。曹冲算法的输入是“一头大象”，输出是“大象的重量”；trisect 的输入是“一条线段”，输出是“两个三等分点”。

明确性（definiteness）。每条指令必须语义明确，而非模棱两可。菜谱里“加盐少许”“加糖适量”“煮至半熟”都不是明确的：多少算少许，多熟算半熟，两个人照做会得到两锅不同的菜。算法的每一步无歧义，同一输入重复执行，中间过程与输出都应一致。

可行性（effectiveness）。每条指令对应的操作在计算模型中都能兑现。“把大象装进冰箱”之所以可笑，是因为第二步不可行；但牵象上下船、刻吃水线、堆石块、累计总和都可以兑现。同样，“取长度比为 $1{:}\sqrt{2}$ 的两条线段”在尺规模型下不可行，而作射线、取等距点、作平行线、联接两点都可以兑现。注意各条指令涉及的操作未必都是基本的——“累计一堆石块的总重量”本身就是一个计算问题，需要子算法来完成。

有穷性（finiteness）。对任何输入，算法在执行有限次基本操作后终止并给出输出。这一条最容易出事，下一节专门讨论。

确定性与随机性。算法的执行流程应当由指令唯一决定。随机性会破坏这一点，但它带来的麻烦比想象中微妙，也放到下一节。

1.4 有穷性：看起来正常的循环可能停不下来

考察著名的 Hailstone 序列：从任意正整数 n 出发，偶数折半，奇数变成 $3n+1$，反复进行。比如 $Hailstone(42)=\{42,21,64,32,16,8,4,2,1\}$，8 步就到了 1；又如 $Hailstone(7)$ 经过 17 个数抵达 1。写成程序：

```
int hailstone( int n ) {
   int length = 1;                  //作为递归基，Hailstone(1) = {1}
   for ( ; 1 < n; length++ )        //偶数n：Hailstone(n) = {n} + Hailstone(n/2)
      n = n % 2 ? 3 * n + 1 : n / 2;   //奇数n：Hailstone(n) = {n} + Hailstone(3*n+1)
   return length;                   //返回序列总长度
}
```

这段代码语法正确，可以通过编译、链接并顺利运行，对已尝试过的每个 n 都能终止。可“对任何正整数都终止”是另一回事：$n=27$ 的序列在半空中一路翻涨，冲到过峰值 9232，绕了远路才折回 1，全程 112 个数。是否存在某个 n 使序列永不到达 1？截至本书写作，数学尚无定论，也没人找到反例；Erdős 的评价是：数学还没有准备好对付这类问题。于是在定义层面出现了一件严肃的事：`hailstone()`暂且只能称作程序，不能称作算法——它的正确性与可行性都满足，唯独有穷性悬而未决。

随机性带来另一种微妙情况。假设 `rand()`是理想的随机整数发生器，产生奇数与偶数的概率各为 $1/2$，考查

```
void toss() { while ( rand() & 1 ); }   //rand()为奇数就继续循环
```

循环体为空，作用是反复掷硬币，直到掷出背面。这段程序每一步语义明确，符合明确性；但计算流程取决于随机数，不具备**确定性**（determinism）。对任何 k，循环至少迭代 k 步的概率是 2^{-k}，恒为正，所以“永远不停”始终可能。不过从统计角度看，迭代恰好 k 步（前 $k-1$ 步全为正面、最后一步背面）的概率是 $\left(\frac{1}{2}\right)^k$——这是典型的几何分布（geometric distribution），其数学期望为

$$\sum_{k\geq 1} k\cdot\left(\frac{1}{2}\right)^k = 2,$$

平均两次判定就会停止，迭代次数期望为 1，是常数。一般地，若一次判定遇到背面的概率为 p，同样求和得期望 $1/p$；只要 p 是固定正常数，期望就是常数。这类迭代步数随机、数学期望有限的程序，本书约定仍归入算法之列；但要知道，按最严格的字面定义，它的有穷性只有概率意义上的保证。

Hailstone 与 toss 从两个方向说明了同一件事：一段代码够不够格称作算法，取决于有穷性（和确定性）能否成立。证明了的算数，没证明的存疑。

1.5 正确性：把“排好序”变成可以论证的命题

算法首先必须功能正确（correctness）。曹冲算法的正确性靠阿基米德原理，trisect 的正确性靠平行截线定理；更复杂算法的正确性论证需要专门的方法。这里用一个排序算法做一次完整的示范。任务：把存于数组 A 的 n 个整数调整为非降序。

```
void bubbleSort( int A[], int n ) {
   while ( 1 < n-- )                  //做n-1趟起泡扫描
      for ( int i = 0; i < n; i++ )   //自前向后，逐对检视相邻元素
         if ( A[i] > A[i+1] )         //一旦发现逆序
            swap( A[i], A[i+1] );     //随即交换
}
```

`while ( 1 < n-- )`先判断 `1 < n` 是否成立，再将 `n` 减 1。初始 $n = 7$ 时判断用的是 7，判断后 `n` 变为 6，本趟扫描检视下标 0 至 5 的六对相邻元素。用具体数组执行一遍，取

$$A = \{5, 2, 7, 4, 6, 3, 1\}$$

第 1 趟（扫描范围为下标 0 至 5）。逐对比较，逆序即交换：

比较对	结果	动作	交换后的数组
(0,1)	5 > 2	交换	[2, 5, 7, 4, 6, 3, 1]
(1,2)	5 < 7	不动	[2, 5, 7, 4, 6, 3, 1]
(2,3)	7 > 4	交换	[2, 5, 4, 7, 6, 3, 1]
(3,4)	7 > 6	交换	[2, 5, 4, 6, 7, 3, 1]
(4,5)	7 > 3	交换	[2, 5, 4, 6, 3, 7, 1]
(5,6)	7 > 1	交换	[2, 5, 4, 6, 3, 1, 7]

盯着 7 的位置看：它一路向右换位，每换一次前进一格，最后停在末尾。原因在于，扫描中每处理完一对 $(i, i+1)$，二者中较大者总位于 $i+1$ 处；下一对又以这个较大者为左项继续比较。于是“当前见过的最大者”始终骑在扫描前线上，扫完全部相邻对后必然落到末尾——连贯地看，各元素如同水中的气泡逐个上浮，算法由此得名。第 1 趟结束时数组为

```
下标    0   1   2   3   4   5   6
      [2] [5] [4] [6] [3] [1] [7]
       └──────── 无序前缀 ────────┘ └ 已就位 ┘
```

第 2 趟至第 6 趟做同样的扫描，只是每趟的扫描右端缩短一格：已就位的元素不再参与比较。各趟结束后的完整状态：

趟	扫描范围	趟结束时的数组	新就位的元素
1	0–5	[2, 5, 4, 6, 3, 1, **7**]	7
2	0–4	[2, 4, 5, 3, 1, **6**, 7]	6
3	0–3	[2, 4, 3, 1, **5**, 6, 7]	5
4	0–2	[2, 3, 1, **4**, 5, 6, 7]	4
5	0–1	[2, 1, **3**, 4, 5, 6, 7]	3
6	0	[1, **2**, 3, 4, 5, 6, 7]	2

起泡的一趟扫描，逐步演示如下（同一数组，可与上表逐行对照）：

起泡排序：一趟扫描怎样把7推到末尾 第 1 / 13 步

1. 扫描开始

第1趟：自左向右逐对检视，逆序即交换



从第0对开始。目标：把本趟最大值推到末尾。
为什么：每完成一对比较，较大者都留在靠右的位置。

图 1　逐步演示：起泡排序：一趟扫描怎样把 7 推到末尾

第 6 趟之后判断 `1 < 1` 不再成立，算法停止，排序完成。以上验证了一次运行，下面论证任何输入都正确。

将数组在任何时刻分成两段：前缀 $[0, s)$ 与后缀 $[s, n)$。称一个状态满足**序列模式**，若后缀已经非降序，且后缀中的每个元素都不小于前缀中的任何元素。初始取 $s = n$，后缀为空，模式平凡成立。此后每做一趟扫描：

实质：前缀中的最大元素（若有多个，取最靠后者）必然被推到前缀末尾，即 $s - 1$ 处，称为就位。第 1 趟的逐对比较过程已经演示了这个推进机制，对任意前缀同理。

不变性（loop invariant）：一趟扫描后序列模式依然保持。新进入后缀的元素是原前缀的最大者，它不小于新前缀中的任何元素；后缀其余部分未动，仍非降序。

单调性（monotonicity）：每趟扫描使 s 减 1，后缀长度加 1，问题的实质规模持续削减。

单调性保证至多 $n - 1$ 趟后 s 减到 1；此时后缀 $[1, n)$ 非降序且每个元素都不小于 $A[0]$，而最后剩下的 $A[0]$ 正是最小元素，自然就位——整个数组非降序，正确性得以确立。这个论证的结构值得记住：一条贯穿循环、始终保持的性质（序列模式），加上“每次迭代保持它”（不变性），加上“迭代次数有穷”（单调性），缺一不可。之后各章凡是说明“这样做是对的”，用的都是同一套推理骨架。

这套版本的比较次数与运气无关：每趟比较的相邻对从 $n - 1$ 对递减到 1 对，总计恒为 $n(n-1)/2$，$n = 7$ 时恰是 21 次，与上面的表格一致；交换次数才随输入的混乱程度浮动，最坏（完全逆序）也达到 $n(n-1)/2$。这些数字属于什么量级、算快算慢，下一册笔记有专门的度量工具。

1.6 鲁棒与复用

正确的算法要成为实用的算法，还要经受两类考验。

鲁棒性（robustness）：除常规情况外，还能应对各种极端输入。刚才的 `bubbleSort()`值得逐项检验：数组存在重复元素（甚至全体相等）时，严格不等号`>`保证相等者不交换，算法照常终止；输入已经有序或完全逆序时，扫描照做，结果正确；元素总数只有 2、1 甚至 0 时，`while` 的首次判断就会结束一切，不会越界。碰巧安全的入口条件也不能依赖：若传入的 n 超过数组实际容纳的元素数，循环照样越界读写——接口的前提应在入口处检查，发现违背就明确报告，而不致异常退出。

复用性（reusability）：算法的总体框架能否便捷地用于不同类型的对象。`bubbleSort()`的原理、逻辑和过程与元素类型没有实质关系：无论 `float`、`char` 还是别的什么，只要元素之间可以比较大小，整体框架就能沿用。更进一步，待排序元素只要在逻辑上构成一个序列，即便不以数组形式给出，算法也依然可以施行。第 2、3 章将让起泡排序等算法摆脱这些约束，实现真正的复用。

在满足以上要求之后，评判算法优劣最主要的标准毕竟是运行效率。要实现高效率的计算，就不能只研究算法：高效率的算法需要高效率的数据结构支撑，反过来强大的数据结构也往往是一组强大算法的汇集体。Wirth 把程序概括为“算法 + 数据结构”，本课程在此之上还乘一个效率因子：

$$(\text{Algorithms} + \text{Data Structures}) \times \text{Efficiency} = \text{Computation}$$

效率怎么度量、怎样在不同算法之间公平比较，必须先约定统一的账本。计算模型为此再次出场。

1.7 图灵机：把“一步”拆到不能再拆

要统计算法执行了多少次基本操作，前提是“基本操作”本身有明确定义。为解决希尔伯特判定问题，图灵（Alan Turing）于 1936 年提出了一个形式化模型，即图灵机（Turing Machine, TM）。它把物质世界简化并抽象为一维离散空间，把“万物之灵”抽象为一个读写头：她有“情绪”，任何时候处于某种**状态**（state）；有“视觉”，总“盯着”某个单元，读出其中的符号；有“手”和“笔”，能改写当前单元；有“腿脚”，能沿纸带移动，尽管每次只能移动一格。

构成。一条纸带（tape），水平放置、长度无限，均匀划分为一系列方格单元（cell），每格存有一个符号；符号取自某个有限字母表，未写过的格子统一记作 #。任何时刻，纸带上的内容、读写头的位置及状态合起来构成图灵机的一个**快照**（snapshot）。给定初始快照（相当于给定输入实例）后，机器便从一个快照转换到下一个快照，所有转换依照事先设定的一组**转换函数**（transition function）进行。每条转换函数是一个五元组：

$$(\textbf{当前状态}\ q, \textbf{当前符号}\ c; \textbf{覆写符号}\ d, \textbf{移动方向}\ s, \textbf{新状态}\ p)$$

若读写头处于状态 q 且读取符号 c，则将 c 改写为 d，按 s 是 L 或 R 向左或向右移动一格，转入状态 p。一旦转入状态 h，随即停机（halt），纸带上以约定方式呈现输出。一组互不冲突、整体完备的转换函数，就构成一个图灵机上的算法。从启动到停机经历的快照数量即计算时间 $T(n)$，它同时等于读写头累计移动的步数。图灵机的访问是**顺序访问**：想到达第 1000 格，必须一步步挪过去，没有捷径。

实例：二进制加一。纸带上存有一个非负二进制整数（其余单元均为 #），读写头对准个位，将其加一。加一的实际过程：末尾连续的’1′（可能没有）全部翻成’0′，原先的最低位’0′（或一路翻到头的’#‘）翻成’1′，比如 $10101111 + 1 = 10110000$。五条转换函数构成完整的算法，<表示左行、>表示右行：

```
LMOVE: (<, 1; 0, L, <)   //左行，1->0
UTURN1:(<, 0; 1, R, >)   //掉头，0->1
UTURN2:(<, #; 1, R, >)   //一路翻到头（原数全为1），补最高位
RMOVE: (>, 0; 0, R, >)   //右行
HALT:  (>, #; #, L, h)   //复位并停机
```

UTURN2 不可省略：输入形如 111（全’1′）时，左行扫描越过所有’1′直达’#‘，必须由这条规则写入新的’1′，否则进位丢失，而且机器在状态<读到’#‘时将无规则可用。HALT 里对’#‘的“改写”等于没动、左移也只为进一步——它真正的作用是给出停机信号，并让读写头回到初始位置。

取纸带内容 ##10101111##。整条纸带共 12 格，下标 0 至 11：下标 0、1 与 10、11 是四个 # 边界，数据占据下标 2 至 9，即八位二进制数 10101111（十进制 175）。读写头对准个位，即下标 9，初始状态<。逐拍执行：

拍	状态	读到	动作	此后纸带	头位置（下标）
1	<	1	改 0，左移	##10101110##	8
2	<	1	改 0，左移	##10101100##	7
3	<	1	改 0，左移	##10101000##	6
4	<	1	改 0，左移	##10100000##	5
5	<	0	改 1，右移，转>	##10110000##	6
6	>	0	不动，右移	##10110000##	7
7	>	0	不动，右移	##10110000##	8
8	>	0	不动，右移	##10110000##	9
9	>	0	不动，右移	##10110000##	10
10	>	#	不动，左移，停机	##10110000##	9

先看一遍完整的走带演示，再对照下表读细节：



图 2　逐步演示：图灵机加一：10101111 如何变成 10110000

10 拍之后纸带上是 `10110000`，即十进制 176，恰为 175 加一，而且读写头回到初始位置——这个算法可以反复执行，让纸带上的二进制数不断递增。这里 $n = 8$ 是输入位数，$T(8) = 4 + 1 + 4 + 1 = 10$：左行翻转四个'1'用 4 拍，翻转'0'并掉头 1 拍，右行越过四个'0'用 4 拍，停机 1 拍。一般地，若数长为 n、末尾连续 r 个'1'，左行耗 $r + 1$ 拍、右行耗 $r + 1$ 拍，$T(n) = 2r + 2 \le 2n + 2$，仍是输入规模的线性函数。

1.8 RAM：把“按编号访问一次”当成一步

图灵机外形朴素、操作原始，与我们今天使用的计算机差异不小。为得到更贴近实际的模型，人们提出了随机访问机（Random Access Machine, RAM）。与图灵机无限长的纸带相仿，RAM 拥有无穷多个寄存器，各自存放一个整数；除功能特定的指令寄存器 IR（instruction register）外，其余寄存器顺次编号，构成无限长的序列 $R[0], R[1], R[2], \ldots$。

RAM 算法的描述方式接近高级程序设计语言，支持以下指令，且每条都可直接完成，故称为基本操作：

```
R[j] <- c            //将常数c存入j号寄存器
R[i] <- R[j]         //将j号寄存器的内容转入i号寄存器
R[i] <- R[R[j]]      //将R[j]号寄存器的内容转入i号寄存器（间接寻址）
R[R[i]] <- R[j]      //将j号寄存器的内容转入R[i]号寄存器
R[i] <- R[j] + R[k]  //加法
R[i] <- R[j] - R[k]  //减法
GOTO x               //无条件跳转到第x行指令
IF R[i] = 0 GOTO x   //条件跳转（相等）
IF R[i] > 0 GOTO x   //条件跳转（大于）
STOP                 //算法终止
```

每个 RAM 算法由若干条上述指令构成，从第 0 行开始顺序执行，遇 GOTO 跳转；当前正在执行的指令由 IR 指示。可直接按编号访问任意寄存器，即按秩调用（call-by-rank），正是日后数组与向量访问的原型。在这类模型中，算法的运行时间就是它执行的基本操作次数；$T(n)$ 按惯例取规模为 n 的所有实例中成本最高者——为什么必须取最坏，下一册笔记有专门讨论。

实例：向上整除。这台机器的指令表里没有除法，我们要教一台只会加减与跳转的机器做除法。正整数 c 存于 $R[0]$、正整数 d 存于 $R[1]$，求上整除 $\lceil c/d \rceil$。思路很直接：反复从 $R[0]$ 中减去 $R[1]$，统计在下溢（减成负数）之前减了多少次。程序共七条指令：

```
[0] R[3] <- 1              //常数增量
[1] GOTO 4                 //若R[0]为0，跳过循环
[2] R[2] <- R[2] + R[3]    //x++
[3] R[0] <- R[0] - R[1]    //c -= d
[4] IF R[0] > 0 GOTO 2     //c仍为正则继续减
[5] R[0] <- R[2]           //否则将计数复制到R[0]
[6] STOP                   //返回R[0]
```

取 $c=25$，$d=12$。四个寄存器的初值依次为 25、12、0、0；下表每行记录一步执行，刚被写入的寄存器给出新值并带^，其余寄存器以^表示保持不变：

步	IR	$R[0]$	$R[1]$	$R[2]$	$R[3]$	本步动作
0	[0]	25	12	0	1^	置增量为 1
1	[1]	^	^	^	^	无条件跳至[4]
2	[4]	^	^	^	^	25>0 成立，转[2]
3	[2]	^	^	1^	^	$x=1$
4	[3]	13^	^	^	^	$c=25-12$
5	[4]	^	^	^	^	13>0 成立，转[2]
6	[2]	^	^	2^	^	$x=2$
7	[3]	1^	^	^	^	$c=13-12$
8	[4]	^	^	^	^	1>0 成立，转[2]
9	[2]	^	^	3^	^	$x=3$
10	[3]	−11^	^	^	^	$c=1-12$
11	[4]	^	^	^	^	非正，顺序执行
12	[5]	3^	^	^	^	$R[0]\leftarrow x$
13	[6]	^	^	^	^	停机

14 步快照的逐步演示：



图 3　逐步演示：RAM 上整除：25÷12 的 14 步快照

输出 3 存于 $R[0]$，而 $\lceil 25/12 \rceil = 3$，正确。《荀子·礼论》说“三年之丧，二十五月而毕”——守丧“三年”其实二十五月，按年折算正是向上取整：$\lceil 25/12 \rceil = 3$。循环恰好数出上整除的原因：k 次减法后 $R[0] = c - kd$，循环在第 k 次减法后停止，当且仅当 $c - kd \le 0$ 且 $c - (k-1)d > 0$，即 k 是满足 $k \ge c/d$ 的最小整数，这正是上整除的定义。$c = 0$ 时指令[1]直接跳过循环返回 0，与 $\lceil 0/d \rceil = 0$ 一致。

时间成本等于快照表的高度，即各条指令累计执行次数之和。七条指令分别执行 1、1、4、3、3、1、1 次，合计 14 次：

$$T(25, 12) = 1 + 1 + 4 + 3 + 3 + 1 + 1 = 14$$

同一问题在图灵机上需要来回走带，步数按格子距离计；在 RAM 上只需 14 次指令执行。模型给“一步”定价，也决定了哪种算法在哪台机器上是聪明的。用 C++之类高级语言描述的算法，可以仿照 RAM 算法的分析方法：分别统计每一行代码的执行次数，总和就是时间复杂度。

1.9 两种模型说明了什么

模型改变了“一步”的价格，不改变“算法是基本操作的指令序列”这一本质。两者共同的要点有三条。

其一，模型把平台差异关在门外。同一算法无论用什么语言书写、在什么机器上编译运行，固定模型下的基本操作次数是同一个数，效率评价因此可以客观进行。

其二，时间与空间可以用同一把尺子度量。算法对规模为 n 的所有实例所需的时间量、空间量的最大值，分别称作**时间复杂度**与**空间复杂度**。空间复杂度不计输入数据本身——那属于固有消耗，与算法优劣无关；计入的是转储、缓冲、中继、索引、变换等辅助空间。

其三，模型中的基本操作对应真实机器的真实动作。RAM 的 call-by-rank 就是数组访问，图灵机的纸带是顺序介质的极限情形。后文设计数据结构时，“这项操作要花多少次基本运算”在每个模型下都有确定含义，正是这些模型在背书。

下一册笔记在这个账本上引入大 O 记号：成本数出来了，还要看它随规模增长的趋势。

1.10 本章要点

- 算法定义在特定计算模型之上，是旨在解决特定问题的指令序列；计算模型由工具与规则构成（船石计算机、尺规计算机、图灵机、RAM）。
- 算法要素：输入、输出、明确性、可行性、有穷性，执行流程还须确定（期望有限、约定可归入算法的随机程序除外）。Hailstone 说明有穷性可能悬而未决，这样的代码只是程序。
- 复杂算法可分解为子问题与子算法；正确性靠“一条始终保持的性质 + 每步保持 + 步数有穷”来论证（序列模式、不变性、单调性）。
- 实用的算法还要鲁棒（经得起极端输入）与可复用（框架适用于可比较大小的任何类型）。
- 图灵机以快照和顺序访问定义“一步”；RAM 以十条基本指令和按秩访问定义“一步”；固定模型之后，运行时间才可以客观地数。

2 增长的快慢：渐近记号、级数与封底估算

第 1 章已经造好了两杆精确的标尺：图灵机按拍数计时，RAM 按指令次数计时，`bubbleSort()`在 $n = 7$ 时的 21 次比较可以一格格数出来。但精确标尺单次度量的结果可信，却难以从宏观上给出中肯的评判：有的算法更适应小规模输入，有的正相反；两个算法只要差距不大，完全可以等同视之。更明智的做法，是在精细标尺上只保留几个粗略的刻度，同一等级之内不再区分，不同等级之间才认为有实质差异。这种着眼长远、注重主流的评估策略就是**渐近分析**（asymptotic analysis）。本章的任务：把刻度造出来（渐近记号），把刻度排好（增长阶谱系），把每档刻度换算成人的时间感受（封底估算），并配备分析循环所需的全套数学工具（级数与可视化）。

2.1 大 O 记号

先明确度量的对象。算法的时间复杂度 $T(n)$ 表示：求解规模为 n 的所有问题实例，最多需要花费的基本操作次数。“最多的”是关键字——同等规模的实例成本可以差异极大，比如“判断平面上 n 个点中是否有某三个点构成的三角形面积小于 5.0”，蛮力算法在最好情况下看第一个三角形就能停手，最坏情况下却要检视全部 $\binom{n}{3}$ 个。取最坏作约定，任何输入之下都作数。至于为什么必须选最坏而不是平均值，本册稍后与下一册都会再谈。

有了 $T(n)$，还要再抽象一层：规模足够大之后，$T(n)$ 以多快的速度增长？形式化地说，若存在正常数 $c > 0$ 和 $n_0 \geq 0$，使得对任何 $n \geq n_0$ 都有

$$T(n) < c \cdot f(n),$$

就称 $f(n)$ 是 $T(n)$ 增长速度的渐近上界，记作 $T(n) = O(f(n))$。这就是大 O 记号（big-O notation）。两个允许换来的简化，都可以从定义直接导出：

常系数可等同于 1。对任一常数 $c > 0$，$c \cdot O(f(n)) = O(f(n))$。1000 倍也好，百万分之一倍也好，在渐近意义上都视同 1 倍。

低次项可忽略。对任意常数 $a > 0$、$b > 0$，只要 $k \geq 1$，就有 $n^k + an^{k-1} + \cdots = O(n^k)$：$n$ 足够大后，最高次项之外的一切都被它吸收。

两个来自第 1 章算法的实例。Increase 算法在 n 个比特的输入上读写头最多移动 $2n + 2$ 步，故 $T(n) = O(n)$；`bubbleSort()`实质语句（第 2 至 5 行）的执行次数恒为 $n(n-1)/2$，故 $T(n) = O(n^2)$。它们分别称为这两个算法的**渐近复杂度**（asymptotic complexity）。此后除非特别说明，提到的复杂度都指渐近复杂度。

还有一点值得警醒：$T(n)$ 未必随 n 严格增长，往往有起有伏。比如对整数做素因子分解的蛮力算法（从 2 起逐个尝试），输入若恰好是某种特殊形式的合数，试除很快就会命中；若是素数，则要试到 $\sqrt{n}$。同一个算法在不同输入上的成本时高时低——大 O 记号给出的上界本来就是对这种“包络线”的刻画，不必指望它处处贴身。

2.2 最好、最坏与大 Ω、大 Θ

大 O 记号界定的是保守估计，而算法的实际性能往往因实例而异，甚至差别悬殊。回看 Increase 算法：若输入的所有比特都是 1，读写头确实要走 $2n + 2$ 步；可反过来，只要个位是 0，无论其余各位如何，翻转个位、右移一格、碰到'#'停机，总共只需 2 步。前者称作**最坏情况**（worst case），后者称作**最好情况**（best case）。为了界定最好情况，需要下界记号：若存在正常数 $c > 0$ 和 n_0，使得对任何 $n \geq n_0$ 都有 $c \cdot g(n) < T(n)$，则记 $T(n) = \Omega(g(n))$，读作大 Omega 记号。Increase 算法的时间复杂度为 $\Omega(1)$——最好情况是常数，同时 $O(n)$——最坏情况线性。

`bubbleSort()`则相反：即便输入已经完全有序，没有任何交换发生，$n(n-1)/2$ 次比较照样一次不少。最好与最坏情况的复杂度渐近同阶，此时最贴切的刻画是确界记号：若存在常数 $c_1, c_2 > 0$ 和 n_0，使得对任何 $n \geq n_0$ 都有

$$c_1 \cdot g(n) < T(n) < c_2 \cdot g(n),$$

则记 $T(n) = \Theta(g(n))$，读作大 Theta 记号。可以更精确地说：`bubbleSort()`的时间复杂度为 $\Theta(n^2)$——无论输入好坏。

区分两个层面的概念有助于避免混淆：Ω、Θ、O 描述的是**函数增长趋势**之间的相对关系（下界、确界、上界），而最好、最坏、平均复杂度描述的是**算法在不同实例上的表现**。两者正交：既可以说最坏情况复杂度是 $\Theta(n^2)$（第 1 章的起泡排序），也可以说最坏情况复杂度是 $O(n^2)$ 但最好情况是 $O(n)$（带提前终止的起泡排序，见教材§2D）本课程遵从讲义惯例，多数场合以大 O 记号陈述最坏情况的复杂度。

2.3 增长阶的谱系

沿刻度从低到高，逐档认识常见复杂度。熟悉每一档的典型来源，见到代码就能估出量级。

常数 $O(1)$。与问题规模无关，“一蹴而就”，已是理想的极限。第 1 章的称象与 trisect 只需常数次基本操作；凡是零次（RAM 意义上）循环、分支、调用的算法都属于此类，尽管反过来未必——

```
for ( int i = 0; i < n; i += n/2026 + 1 );   //至多2026次迭代，与n无关
```

含循环而迭代次数恒为常数的算法仍是 $O(1)$。既然时间复杂度是空间复杂度的天然上界，这类算法的空间复杂度必然也是 $O(1)$；只消耗常数辅助空间的算法称作**就地算法**（in-place algorithm），第 1 章的起泡排序就是一例。

对数 $O(\log n)$。定义里没有写底数并非疏漏：对任意常数底 $a, b > 1$，换底公式 $\log_a n = \log_b n / \log_b a$ 表明不同底的对数只差常数倍，渐近意义下等价，通常取 2。更一般地，凡可写成 $O((\log n)^c)$ 形式的都称对数多项式复杂度（poly-logarithmic complexity），比如 $O(\log^2 n)$；再叠加常数幂也无关，$\log(n^c) = c \log n$。对数增长极其缓慢：n 翻一倍，$\log_2 n$ 只加 1。这类算法无限接近于常数，同样是高效率的代名词；有序向量的二分查找、堆的插入与删除都在这一档。

多项式 $O(n^c)$。$c = 1$ 称线性复杂度，相应的算法简称线性算法：树与图的遍历、一次完整的扫描。线性之所以被当作“满意”的底线，是因为多数问题至少要把每个数据项检视一遍，$\Omega(n)$ 常常躲不开。$c = 2$ 的典型来源是“所有对象两两配对”式的比较，起泡排序是代表；$c = 3$ 出现于矩阵乘法。从 $O(n)$ 到 $O(n^2)$ 是本课程主要覆盖的范围：低于 $O(n)$ 者很稀有，有也多已成为经典；高于 $O(n^2)$ 的算法在实际应用中很难令人满意。

在复杂度理论中，多项式还有一个特殊身份：任何问题只要存在多项式时间算法，无论阶次多高，都权且称作**可解的**（tractable）。这个门槛看似宽松笼统，实则出于无奈——相对于下一档，$O(n^2)$ 与 $O(n^3)$ 之间的差异只是小巫见大巫。

指数 $O(2^n)$。只要注意到对任意常数 c，2^n 的增长终会压过 n^c——指数函数把任意高阶的多项式都当作自己的低阶分量——便知多项式与指数之间隔着一道巨大的鸿沟：$\forall c : n^c = O(2^n)$，而反过来不成立。随规模增大，指数复杂度算法的耗时急剧上升，无法真正应用于实际问题；这类“算法”称作程序更加中肯。要留意的是路遥知马力：在 n 还不太大的区间里，指数曲线可能一度低于多项式曲线（比如 $n \le 18$ 时 $2^n/1000 < n^2$），但差距迟早反转并持续扩大，无法保持在多项式倍的范围之内。

一个名副其实的难题：SubsetSum。回到第 1 章开头的称象故事，换一个问法：从那堆称过重的石块里，能否挑出若干块，恰好使吃水线复原？抽掉故事外壳，这就是子集和问题：

> 任给正整数 $S = \{a_1, a_2, ..., a_n\}$，对任何整数 t，问是否存在子集 $S' \subset S$，使 S' 中元素之和恰为 t？

它有一个更现实的化身：选举人团共 51 个选区、538 票，两位候选人是否可能恰好各得 269 票？这就是 $n = 51$（51 个选区各为一枚“石头”，重量是其选举人票数）、$t = 269$ 的子集和，总票数 $538 = 2t$。直觉解法逐个检视 S 的每个子集，递归描述只需两行：

```
bool sSum( S = {a1, a2, ..., an}, t ) {
   if ( t == 0 ) return true;          //递归基：恰好凑出t（零重量的大象必有解）
   if ( n == 0 ) return false;         //递归基：石块耗尽仍没凑出
   S = S \ {an};                       //第n块石头：用，或不用，分两路
   return sSum( S, t - an ) || sSum( S, t );
}
```

正确性毋庸置疑——它确实检查了每个子集。但在最坏情况下，耗时不会少于 S 的幂集规模 2^n。能不能为它找到多项式算法？很遗憾，算法理论早有定论：SubsetSum 属于 NP 完全（NP-complete, NPC）问题，正如绝大多数专家所相信的，基于目前的计算模型，不存在多项式时间算法。若 S 中整数都取自某个确定范围，问题可以转换为背包问题并有效求解——可惜一般场合没有这类约束。还要补充一句更彻底的：比难解（intractable）问题更糟糕的是大量根本不存在有穷计算方法的**不可计算**问题，第 1 章里悬而未决的 Hailstone 就已经站在这类问题的门口。

把谱系汇成一张表，右列是各自的常见来源：

阶	名称	典型来源
$O(1)$	常数	RAM 基本操作、数组按下标访问
$O(\log n)$	对数	二分查找；堆的查询、插入与删除
$O(n)$	线性	树、图的遍历
$O(n \log n)$	线性对数	排序、Huffman 编码
$O(n^2)$	平方	两两比较；起泡排序
$O(n^3)$	立方	矩阵乘法
$O(n^c)$	多项式	可解（P 类）问题的范围
$O(2^n)$	指数	子集枚举；多数难题的平凡解法

2.4 封底估算

渐近刻度造好了，还要能把它换算成人的时间感受。这种在纸片背面寥寥数行就做出可用估算的功夫，称作封底估算（back-of-the-envelope calculation），历史上大师们精于此道：古希腊的 Eratosthenes（前 276—前 194）借助夏至日的日光投影，用极简单的工具相当精准地估出了地球的赤道周长（讲义取约 39350 公里）；1945 年人类首次原子弹试验时，物理学家 Fermi 只看了一眼被冲击波吹落在地的纸屑，爆炸当量便脱口而出，精度惊人。对算法做渐近分析之后，同样应该能够结合实际计算环境，快速给出实际性能的量级——对于还在图纸上的方案，这种估算能帮我们做出正确的取舍。

先建立时间的数量级概念：一天约 10^5 秒，一代人约 30 年即 10^9 秒，一生约一个世纪即 3×10^9 秒，三生三世约 10^{10} 秒。再备好对数与指数的换算：$2^{10} \approx 10^3$，$2^{20} \approx 10^6$，$2^{30} \approx 10^9$，$\log_2 10^3 \approx 10$，$\log_2 10^6 \approx 20$，$\log_2 10^9 \approx 30$。当前常规个人电脑的主频以 GHz 计，按每秒 10^9 次基本运算估算。两张速查表把渐近复杂度与实际性能对应起来——各复杂度在不同规模下的运算次数，以及换算后的运行时间：

复杂度	$n = 10^3$	$n = 10^6$	$n = 10^9$
$\log_2 n$	10 次，约 10 纳秒	20 次，约 20 纳秒	30 次，约 30 纳秒
n	10^3次，1 微秒	10^6次，1 毫秒	10^9次，1 秒
$n \log_2 n$	10^4次，10 微秒	2×10^7次，20 毫秒	3×10^{10}次，30 秒
n^2	10^6次，1 毫秒	10^{12}次，17 分钟	10^{18}次，约 30 年
n^3	10^9次，1 秒	10^{18}次，约 30 年	10^{27}次，约10^{18}秒
2^n	约10^{301}次	约10^{301030}次	无法写下

换算依据：10^{12} 次运算耗时 10^3 秒，约 17 分钟；10^{18} 次耗时 10^9 秒，约 30 年；10^{27} 次耗时 10^{18} 秒，是宇宙现有年龄（约 4×10^{17} 秒）的两倍以上。指数一栏可以感受一下规模：可观测宇宙的原子总数不过约 10^{80}。

作为一次综合演练，考虑真实的计算任务：为全国人口普查数据排序，如何选择算法与硬件平台？问题规模——约 14 亿人口——近似为 10^9。在常规个人电脑上用起泡排序：约 10^{18} 次操作，即 10^9 秒，约 30 年——整整一代人的时间，即便排完，人口数据也早已过时。换中国首台千万亿次超级计算机天河 1A（2010 年曾居世界超算 500 强榜首，逾 10^{15} flops，相当于百万台个人电脑）：$10^{18}/10^{15} = 10^3$ 秒，约 17 分钟（讲义另一页按 2×10^3 秒取整作 20 分钟，取整方式不同）。回到个人电脑改用归并排序（复杂度 $O(n \log n)$，

第 06 章介绍）：约 3×10^{10} 次操作，30 秒。天河 1A 配归并排序：30 微秒。硬件提升六个数量级，赶不上换一个算法；真要追求极致，算法与体系结构两个方面的贡献缺一不可。

2.5 级数：循环分析的备查表

第 1 章说过，分支转向是算法的灵魂；现代语言用规范的循环把 goto 隐藏起来。一个 `for` 循环可拆成四个部分——初始化（`int i = 0`）、继续条件（`i < n`）、更新操作（`i++`）、循环体（`if ... swap ...`）——`while`、`repeat` 本质上同样如此。形式上规范的循环，其执行过程必然呈现简明的规律：纵观各步迭代消耗的时间，往往构成某种级数（series），级数的总和就是循环的复杂度。下面把常用的级数一次备齐。

算术级数（等差数列求和）：

$$\sum_{k=0}^{n-1} k = \frac{n(n-1)}{2} = O(n^2)$$

配对法给出最短证明：把和式正写一遍、倒写一遍，逐列相加每列都是 $n-1$，共 n 列，总和 $n(n-1)$；一排是它的一半。无论首项与公差是多少，这类级数总与其中最大项的平方同阶——起泡排序的 $n(n-1)/2$ 次比较正是一例。

幂方级数：

$$\sum_{k=1}^{n} k^c \approx \frac{n^{c+1}}{c+1} = O(n^{c+1})$$

不必逐个硬记，把级数近似为积分 $\int_0^n x^c\mathrm{d}x = n^{c+1}/(c+1)$ 即可快捷得到所有幂次的渐近级别。规律：比幂次高出一阶。

几何级数（等比数列求和）：

$$1 + a + a^2 + \cdots + a^n = \frac{a^{n+1}-1}{a-1} = O(a^n) \qquad (a>1)$$

求和公式的推导只需一移一减：设 S 为和式，则 $aS = a + a^2 + \cdots + a^{n+1}$，两式相减几乎所有项都对消，$S(a-1) = a^{n+1}-1$。规律：与最大（最后）一项同阶。后面分析递归调用树时会反复用到这一条。

收敛级数。比率 $a<1$ 时级数收敛于常数：$\sum_{k\ge1} 2^{-k} = 1 = O(1)$。第 1 章掷币程序的期望 $\sum_{k\ge1} k\cdot 2^{-k} = 2$ 正是这类级数的应用——迭代步数可能很多，期望却是常数。看似抽象，实则随机算法的分析离不开它。

调和级数：

$$\sum_{k=1}^{n} \frac{1}{k} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \approx \ln n = O(\log n)$$

按 2 的幂分组可以看清下界：$(1) + (\frac12+\frac13) + (\frac14+\cdots+\frac17) + \cdots$，第 k 组含 2^{k-1} 项、每项都不小于 2^{-k}，故每组之和至少 $\frac12$；全式共约 $\log_2 n$ 组，总和因此不低于 $\frac12\log_2 n$。上界同理：第 k 组每项不超过 $2^{-(k-1)}$，每组之和至多 1，总和不超过 $\log_2 n + 1$。

对数级数：

$$\sum_{k=1}^{n} \log k = \log(1\cdot 2\cdots n) = \log(n!) \approx n\log n = O(n\log n)$$

末步用 Stirling 公式 $\ln(n!) \approx n\ln n - n$。第 18 章分析建堆复杂度时，会正面遇到它。基本级数还可以组合出现，比如 $\sum_{k=1}^{n} k\log k = O(n^2\log n)$，处理原则相同：抓住增长最快的那部分。

2.6 可视化：把复杂度看成面积

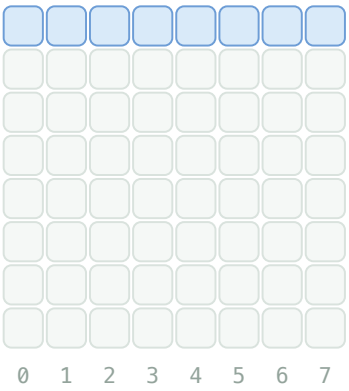
先在网页里把三个循环的点阵看一遍，再对照下文的面积论证：

循环的点阵：正方形、三角形与倍增

第 1 / 19 步

1. 实例A：正方形填到第1行

for i in 0..n−1: for j in 0..n−1 已执行8次



外层i=0这一行，内层j跑满8格，新增8个点。

为什么：两层独立，每行都是8点：面积=n^2。

图 4　逐步演示：循环的点阵：正方形、三角形与倍增

级数公式备好了，但代码风格各异、逻辑千变万化，一味精细计算既费时又易错。可视化的方法往往化抽象为直观：把每次基本操作对应为平面上一个单位面积的方块，总执行次数就是被填充区域的面积。几个典型实例足够说明这种方法的力量。

实例 A：**正方形**。内外两层彼此独立、各迭代 n 步：

```
for ( int i = 0; i < n; i++ )
   for ( int j = 0; j < n; j++ )
      Oop( i, j );        //O(1) operation: 只需常数时间
```

执行过程是对 $n \times n$ 矩形自上而下、从左到右的填充，每次 `Oop()`对应一个单位小块，总次数即矩形面积 n^2。

实例 B：**三角形**。内层长度取决于外层：

```
for ( int i = 0; i < n; i++ )
   for ( int j = 0; j < i; j++ )
      Oop( i, j );
```

第 i 行填 i 个小块，整个区域是等腰直角三角形，面积约为正方形的一半——常系数的差别不影响阶，$O(n^2)$。取 $n = 8$ 逐行数：$0 + 1 + \cdots + 7 = 28$ 次，与算术级数公式一致。

实例 C：**稀疏三角形**。内层步长扩大到 2026：

```
for ( int i = 0; i < n; i++ )
   for ( int j = 0; j < i; j += 2026 )
      Oop( i, j );
```

还是同一个三角形，只是每 2026 列保留一列，面积缩小 2026 倍。实际验证：$n = 20260$ 时执行 111420 次，与理论值 $\frac{n^2}{2\times2026} + \frac{n}{2} = 111430$ 吻合（差 10 来自边界取整）。除以常数不改变阶，总体仍是 $O(n^2)$。

实例 D：**倍增行宽**。外层变量翻倍：

```
for ( int i = 1; i < n; i <<= 1 )
   for ( int j = 0; j < i; j++ )
      Olop( i, j );
```

各行宽度 $1, 2, 4, 8, \ldots$ 构成几何级数，由“与最大项同阶”一眼看出总量 $O(n)$——末行宽度 2^t 落在 $[n/2, n)$，而总和 $2^{t+1} - 1$ 落在 $[n, 2n)$。逐项求和验证（$n = 8$ 时三行合计 $1 + 2 + 4 = 7$ 次）：同一个三角形，行宽逐行加 1 是平方级，行宽逐行翻倍反而线性；增长方式由“怎么长”决定，不由“有几行”决定。

实例 E：对数曲线下的点阵。外层算术递增、内层几何递增：

```
for ( int i = 0; i <= n; i++ )
   for ( int j = 1; j < i; j += j )
      Olop( i, j );
```

第 i 行的 j 取 $1, 2, 4, \ldots$ 直到达到 i，行宽约 $\log_2 i$。点阵贴着对数曲线排布，总次数近似为对数曲线下的积分面积：它小于大矩形 R（宽 n、高 $\log_2 n$）的面积，又至少超过其一半，故与 R 渐近同阶，即 $O(n \log n)$。精确核对 $n = 8$：各行执行 $0, 0, 1, 2, 2, 3, 3, 3, 3$ 次，合计 17 次，与 $\log_2(8!) \approx 15.3$ 同量级。

五个实例反复演示的是同一件事：循环的形状决定级数，级数的末项决定阶。往后的章节里，无论是嵌套扫描还是调用树，都可以先用面积直觉估出量级，再用级数公式核实。

2.7 空间账目

空间复杂度 $S(n)$ 的记法与时间相同，账目规则在第 1 章已经约定：输入数据本身不计（固有消耗），转储、缓冲、中继、索引、变换等辅助空间全部计入。之所以通常更关注时间，是因为二者有一个本质区别——故地或可重游，昨日实难再现：同一空间单元可以被多次基本操作反复使用，而每操作一次时间就永久地流逝一个单位。由此，$T(n)$ 是 $S(n)$ 天然的上界；专门的空间分析自有其意义，尤其在对空间苛刻的场合，后续章节会结合具体问题介绍方法。

递归算法的空间峰值还有专门的分析技巧，递归的复杂度也有自己的递推规律——这些连同分治策略与主定理，是下一册笔记的主题。

2.8 本章要点

- 渐近分析只保留粗刻度。$T(n)$ 取最坏实例上的最大成本；大 O 是渐近上界，两条化简规则：常系数等同于 1，低次项可忽略。
- 最好/最坏复杂度刻画算法在不同实例上的表现，$\Omega/\Theta/O$ 刻画函数趋势的相对关系；Increase 最好 2 步最坏 $2n + 2$ 步，起泡排序最好最坏同为 $\Theta(n^2)$。
- 谱系：$O(1)$、$O(\log n)$（底数无关）、$O(n^c)$（可解的门槛）、$O(2^n)$（与多项式之间的鸿沟）；SubsetSum 是 NP 完全的实例，其平凡解 2^n 已接近最优。
- 封底估算用“每秒 10^9 次”换算：人口普查排序，起泡排序在个人电脑上要 30 年，归并排序只要 30 秒；算法的改进可以胜过百万倍的硬件。
- 级数对应循环形状：算术级数与最大项平方同阶，几何级数与最大项同阶，调和级数 $O(\log n)$，对数级数 $O(n \log n)$；可视化把总执行次数看成填充面积。
- 空间只计辅助部分，时间是它的天然上界；空间可以重复使用，时间不能。

3 迭代与递归：减治、分治与动态规划

高级语言里的函数调用，包括递归调用，本质上同样是通过分支转向实现的。算法的这些部分既是复杂度的主要来源，也是复杂度分析的核心与难点。从设计的角度看，迭代与递归是实现算法逻辑的两种基本手段，它们的背后各有一套简化问题的策略。人脑的任务是找出简化问题的办法；一旦找出来了，剩下的重复劳动尽可以交给计算机：持续地“大事化小”，直至“小事化了”。本册笔记沿这条路走完四站：减而治之、分而治之（附送两件分析利器），一个贯穿四种策略的完整问题（总和最大区间），以及动态规划（Fibonacci 数与最长公共子序列）。

3.1 减而治之：每次削掉一点

面对规模庞大的问题，不必追求毕其功于一役。换个思路：只要能不断地削减问题规模，哪怕每次只有区区一个单位，问题也迟早得以解决。愚公移山正是这个策略的注脚：每天不过挖走一箕半畚，何愁“方七百里，高万仞”的太行、王屋不平？这就是**减而治之**（decrease-and-conquer，简称减治）：把问题分解为两个子问题，其一平凡，另一规模缩减而形式不变；递归求解非平凡的那个，再把解综合回来。

先看迭代版的参照。称象算法的最后一步需要累计所有石块的重量，设重量均为整数、已存入数组，直截了当的写法是：

```
int sumI( int A[], int n ) {
   int sum = 0;                //累计器
   for ( int i = 0; i < n; i++ )
      sum += A[i];             //逐项累计
   return sum;
}  //数组求和（迭代版）
```

运行时间主要消耗于 for 循环，总体 $O(n)$，而且最好、最坏情况一样；除输入数组外只用常数个辅助变量，空间 $O(1)$，属于就地算法。

按减治思路改写为递归：规模为 n 的问题，化为规模 $n-1$ 的同一问题，再加常数时间的一次加法：

```
int sum( int A[], int n ) {
   if ( 1 > n )                //递归基：规模削减到0，直接可得
      return 0;
   else                        //一般情况：前n-1项之和，再累计末项
      return sum( A, n-1 ) + A[n-1];
}  //数组求和（线性递归版）
```

这里出现了递归算法的生命线：**递归基**（base case of recursion）。一旦问题规模削减到足够小（此处为 0），便直接处理而无需继续递归。为确保有穷性，任何递归算法都必须设有这类逻辑分支，而且每个递归实例都要优先测试它。

递归跟踪。递归算法的复杂度怎么算？第一种方法是直接跟踪执行过程。每一次递归调用称作一个**递归实例**（recursion instance）；若实例 X 创建了实例 Y，就从 X 向 Y 引一条边，Y 完成后必会沿原边逆行返回。所有实例按此原理相互连接，构成一棵**调用树**（call tree）：把每个实例各自消耗的时间累加起来，就是总体运行时间。`sum(A, n)`的调用树最简单——退化为一条长度为 n 的链：

```
sum(A,5) -> sum(A,4) -> sum(A,3) -> sum(A,2) -> sum(A,1) -> sum(A,0)
   ↓返回      ↓返回       ↓返回       ↓返回       ↓返回      返回0
   14          9           8           4           3
```

以 $A=\{3,1,4,1,5\}$ 为例，共 $n+1=6$ 个实例，每个只需 $O(1)$ 时间（调用语句本身也需 $O(1)$，但这类开销的总数不超过实例总数，可忽略），总体 $O(n)$——与迭代版相同。这类最多创建一个新实例的递归称为**线性递归**。

空间呢？递归实例各自所需的空间固定（此处 $O(1)$），空间总量取决于同时活着的实例数量的峰值。抵达递归基时实例最多，共 $n+1$ 个，故空间复杂度 $O(n)$——递归版比迭代版更耗空间。

递推方程。递归跟踪直观易行，却不适用于逻辑复杂、体量庞大的算法。第二种方法切换到代数：为求解规模 n 的问题，先求解规模 $n-1$ 的问题，再花常数时间执行一次加法，于是

$$T(n) = T(n-1) + O(1), \qquad T(0) = O(1)$$

两式联立展开即得 $T(n) = O(n)$。递归跟踪与递推方程两种方法，此后将交替使用。

数组倒置。再练一例：倒置数组，令元素前后翻转。减治的切法：花费常数时间令首末元素互换，再对剩下的中缀递归倒置。

```
void reverse( int* A, int n ) {
   if ( n < 2 ) return;       //递归基：0个或1个元素，无需倒置
   swap( A[0], A[n-1] );      //交换首元素与末元素
   reverse( A + 1, n - 2 );   //递归倒置中缀
}  //O(n)时间 + O(n)空间
```

以 $\{3,1,5,2,4\}$ 为例：首末交换得 $\{4,1,5,2,3\}$；对中缀 $\{1,5,2\}$ 首末交换得 $\{4,2,5,1,3\}$；剩余单个元素 5 自然就位。调用树仍是一条单链，时间、空间均为 $O(n)$；用递推方程验证：$T(n) = T(n-2) + O(1)$，同样解出线性。

3.2 分而治之：一分为二

点滴削减是一种耐心的策略；另一种更积极直接的办法，是将原问题分割成若干个规模更小、形式相同的子问题，分别处理，再把子问题的解综合为原问题的解——这就是**分而治之**（divide-and-conquer，简称分治），《孙子兵法》所谓“凡治众如治寡，分数是也”。借助递归机制，划分可持续进行，直到子问题平凡到答案一目了然，终止于递归基。一个递归实例创建多个新实例的递归称作**多路递归**；最常见的是一分为二，即**二分递归**，调用树从此有了真正的分叉。

还是数组求和。以中点为界将数组均衡地分为前后两段，分别递归求和，再相加：

```
int sum( int A[], int lo, int hi ) {   //入口为sum(A, 0, n)
   if ( hi - lo < 2 ) return A[lo];    //递归基：区间宽度不足2
   int mi = (lo + hi) / 2;             //均分为前缀[lo,mi)与后缀[mi,hi)
   return sum( A, lo, mi ) + sum( A, mi, hi );
}  //数组求和（二分递归版）
```

区间采用左闭右开约定：$A[lo, hi)$ 指从第 lo 个元素起、直到第 hi-1 个。整数加法满足结合律，求和结果与执行次序无关，正确性不言而喻。值得追问的是：这样一分为二地折腾一遍，收益在哪里？

递归跟踪：逐层清点。$n = 8$ 时的调用树是一棵二叉树：

```
第0层                          sum(0,8)
第1层               sum(0,4)              sum(4,8)
第2层         sum(0,2)   sum(2,4)   sum(4,6)   sum(6,8)
第3层    (0,1)(1,2)(2,3)(3,4)(4,5)(5,6)(6,7)(7,8)
```

各层实例数量为 $1, 2, 4, \ldots, n$，构成以 2 为倍数的几何级数——上一册笔记的结论直接可用：级数的阶由最大（最后）一项决定。底层对应递归基的实例恰好 n 个：数组中每个元素各自对应一个递归基，这个结论对非 2 的幂也成立。实例总数 $2n-1$（$n = 8$ 时为 15），每个 $O(1)$，总体 $O(n)$——与线性递归版相同，多轮划分并未换来时间上的收益。

空间：忒修斯的绳线。出人意料的是空间。线性递归只有一个递归基，二分递归的底层节点占了调用树的一半，貌似后者更耗空间，实际恰恰相反。把调用树看作一座迷宫，递归算法的执行过程就是希腊英雄忒修斯（Theseus）对迷宫的遍历：将绳线一端固定在起始实例处，手握另一端踏入迷宫；向前一步，绳线延长一截；退回一步，缩短一截。任一时刻正在执行的实例为 Y，需要且只需记录从起点到 Y 的路径上所有实例——绳线的长度就是此刻的空间占用量。每次抵达递归基都意味着随即折返，故绳长在递归基处达到瞬时极大；所有极大值中的最大者称作**递归深度**（recursion depth），它就是算法的空间复杂度。二分递归的递归深度为 $\log_2 n + 1$（$n = 8$ 时为 4 层），空间 $O(\log n)$——反而比线性递归的 $O(n)$ 节省得

多。同一问题，时间相同的两种写法，空间可差一个量级；这个差别在第 08 章（栈与调用栈）之后会更具体。

递推方程与代入验证。从原理概括：求解规模 n 的问题，需先递归求解两个规模 $n/2$ 的问题，再加一次：

$$T(n) = 2T(n/2) + O(1), \qquad T(1) = O(1)$$

解这类方程可以用猜测加验证（guess-and-check）：猜 $T(n) = an + b$，代入右侧得 $2(a\,n/2 + b) + 1 = an + (2b + 1)$，与左侧比较得 $b = 2b + 1$，即 $b = -1$；结合 $T(1) = 1$ 定出 $a = 2$。于是 $T(n) = 2n - 1$，与逐层清点的 $2n - 1$ 个实例完全吻合。猜测经代入验证成立，该算法确为线性复杂度。

3.3 主定理：分治递推的统一解法

猜测加验证面对更复杂的递推会很吃力。好在分治策略的递推不外乎一种形状：

$$T(n) = a \cdot T(n/b) + g(n)$$

含义：原问题分成 a 个规模均为 n/b 的子任务，每个递归实例自身（划分、合并等）耗时 $g(n)$。**主定理**（Master Theorem，亦译大师定理）按 $g(n)$ 与门槛函数 $n^{\log_b a}$ 的相对大小分三种情况给出闭合解：

1. **门槛函数占优**：若存在 $\varepsilon > 0$ 使 $g(n) = O(n^{\log_b a - \varepsilon})$，则 $T(n) = \Theta(n^{\log_b a})$——总成本由叶子层支配。
2. **同阶**：若 $g(n) = \Theta(n^{\log_b a} \cdot \log^k n)$（$k \geq 0$），则 $T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$——各层成本均匀，逐层求和相当于乘上 $\log_b n$ 层。
3. **合并占优**：若存在 $\varepsilon > 0$ 使 $g(n) = \Omega(n^{\log_b a + \varepsilon})$，且满足正则条件（存在 $0 < c < 1$ 使 $a\,g(n/b) \leq c\,g(n)$ 对足够大的 n 成立），则 $T(n) = \Theta(g(n))$——总成本由顶层合并支配。

把本课程将遇到的递推对号入座：

递推式	来源	a,b	门槛 $n^{\log_b a}$	$g(n)$	解
$T(n) = T(n/2) + O(1)$	有序向量二分查找（第 05 章）	1, 2	1	$O(1)$	$\Theta(\log n)$，同阶 $k = 0$
$T(n) = 2T(n/2) + O(1)$	二分递归求和	2, 2	n	$O(1)$	$\Theta(n)$，叶子占优
$T(n) = 2T(n/2) + O(n)$	归并排序（第 06 章）	2, 2	n	$O(n)$	$\Theta(n \log n)$，同阶
$T(n) = T(n/2) + O(n)$	把二分查找强用于列表	1, 2	1	$O(n)$	$\Theta(n)$，合并占优
$T(n) = T(n/2) + O(\log n)$	带对数合并的查找	1, 2	1	$O(\log n)$	$\Theta(\log^2 n)$，同阶 $k = 1$
$T(n) = 2T(n/2) + O(n^2)$	蛮力统计跨界逆序对（第 06 章）	2, 2	n	$O(n^2)$	$\Theta(n^2)$，合并占优

主定理也能指明优化的方向。看第三种情况的例子：$4T(n/2) + O(n)$ 解出 $\Theta(n^2)$，与不分治的蛮力持平。定理提示，降低复杂度只有两个方向——缩小子任务规模 b，或者减少子任务个数 a。下一节的大整数乘法正是沿着第二个方向完成的改进。

3.4 大整数乘法：三次乘法的恒等式

当整数的位数超过系统字宽之后，就不能再把它当作常数，也不能假定算术运算只需常数时间——这样的整数称为大整数。两个 n 位大整数的加法逐位进行并处理进位，需要 $\Theta(n)$；乘法用朴素的竖式逐位相乘，需要 $\Theta(n^2)$。分治能否更快？

把两个乘数各按半切开：$X = a \cdot 2^{n/2} + b$，$Y = c \cdot 2^{n/2} + d$，其中 a, c 是高半段、b, d 是低半段。展开：

$$XY = ac \cdot 2^n + (ad + bc) \cdot 2^{n/2} + bd$$

移位对齐与加法（含进位）都是线性成本。朴素做法递归计算 ac、ad、bc、bd 四次半长乘法：

$$T(n) = 4T(n/2) + O(n) \Longrightarrow \Theta(n^{\log_2 4}) = \Theta(n^2)$$

复杂度与不划分时持平，划分没有产生收益。主定理指出的两个优化方向里，子任务规模 b 已经是 2（均衡切分的极限），只能从个数下手。关键技巧来自一条恒等式：

$$ad + bc = (a - b)(d - c) + ac + bd$$

用小数字验证一遍：计算 12×34，取 $a = 1, b = 2, c = 3, d = 4$：$ac = 3$，$bd = 8$，$(a-b)(d-c) = (-1) \times 1 = -1$；于是 $ad + bc = -1 + 3 + 8 = 10$，与直接计算 $1 \times 4 + 2 \times 3 = 10$ 一致。重组：$3 \times 100 + 10 \times 10 + 8 = 408 = 12 \times 34$。恒等式在带符号运算下依然成立，负的中间项无害。

于是只需三次半长乘法：ac、bd、$(a-b)(d-c)$（多做两次减法，归入线性项）：

$$T(n) = 3T(n/2) + O(n) \Longrightarrow \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$$

n 越大改进越可观：$n = 2^{20}$ 时，$n^2 \approx 10^{12}$ 而 $n^{1.585} \approx 3 \times 10^9$，相差三百多倍。这个算法属于 Karatsuba；此后还有更快的改进，但“减少子任务个数”这一优化方向，是主定理最先指明的。

3.5 Akra–Bazzi 定理：不均衡的分治

主定理覆盖的递推要求子任务规模均等。更一般地，递推可能形如

$$T(x) = \sum_i a_i \cdot T(b_i x + h_i(x)) + g(x)$$

其中子任务分成若干组，各组个数 a_i、规模比例 b_i 不尽相同（$0 < b_i < 1$）。**Akra-Bazzi 定理**说：解出满足 $\sum_i a_i b_i^p = 1$ 的实数 p，则 $T(x) = \Theta\left(x^p(1 + \int_1^x g(u)/u^{p+1}\,\mathrm{d}u)\right)$。第 20 章的线性选择算法将用 $a_1 = a_2 = 1$、$b_1 = 1$、$b_2 = 1 - \varepsilon$ 这类不均衡划分，届时按本式定出 p 即可。现阶段记住两点即可：主定理并非唯一工具；遇到不均衡切分，Akra-Bazzi 定理是更一般的解法。

3.6 总和最大区间：四种策略同台

前面备齐了工具，现在用一个贯穿性的问题检验它们。在整数数组 A 中，把区间 $[lo, hi)$ 内元素的总和记作 $S(lo, hi)$。**总和最大区间**（greatest slice）问题：找出总和最大的那个区间。以教师课件的实例为准：

$$A[0, 19) = \{\, 1, -2, 7, 2, 6, -9, 5, 6, -12, -8, 13, 0, -3, 1, -2, 8, 0, -5, 3 \,\}$$

在这 19 个元素定义的 190 个非空区间中，总和 17 由三个区间取得：$[2, 8)$（$7, 2, 6, -9, 5, 6$，长 6）、$[10, 16)$（$13, 0, -3, 1, -2, 8$，长 6）与 $[10, 17)$（$13, 0, -3, 1, -2, 8, 0$，长 7）。按讲义“短者、靠后者优先”的约定，长 7 的 $[10, 17)$ 落选，等长的两个取靠后者，应报告 $[10, 16)$。有两个接口约定需要先分清，后文所有算法都会标明自己属于哪一个：

- **接口 I（允许空区间）**：零宽度区间的总和为 0，可作候选；全负输入的答案为 0。讲义算法以 `gs = 0` 起始即属此类。
- **接口 II（要求非空）**：答案至少包含一个元素；全负输入的答案为最大的单个元素。讲义另一版本以 `gs = A[0]`起始。

此外，当最大总和有多个区间同时取得时，讲义约定“短者、靠后者优先”；本节以最大和的数值为主线，端点的选取规则在给出完整 trace 后说明。

3.6.1 蛮力：枚举每一个区间

最直接的想法：两重循环枚举所有区间 $[lo, hi)$，每个区间调用第 02 节的求和例程花 $O(n)$ 时间算出总和，择优更新：

```
int gs_BF( int A[], int n ) {          //蛮力版：O(n^3)
   int gs = 0;                         //已知的最大和：零宽度区间，总和为零
   for ( int lo = 0; lo < n; lo++ )    //枚举起点
      for ( int hi = lo; hi < n; ) {   //共O(n^2)个区间
         int s = sum( A, lo, ++hi );   //每次重新求和：O(n)
         gs = max( gs, s );            //择优更新
      }
   return gs;
}
```

n 个起点的区间总数是 $\frac{n(n+1)}{2}$（接口 I 还含空区间），每区间求和 $O(n)$，总体 $O(n^3)$。用小数组 $B = \{2, -3, 4, -1, 3\}$ 看清它浪费在哪里。固定起点 $lo = 0$，蛮力把每个前缀都从头加起：

终点	区间	蛮力的算式	总和
hi=1	[2]	2	2
hi=2	[2,-3]	2−3	−1
hi=3	[2,-3,4]	2−3+4	3
hi=4	[2,-3,4,-1]	2−3+4−1	2
hi=5	[2,-3,4,-1,3]	2−3+4−1+3	5

每一行的加法都在重复上一行的劳动。注意到 $S(lo, hi) = S(lo, hi-1) + A[hi-1]$：同一起点下，相邻区间的总和只差一个元素。

3.6.2 递增：别把加过的再加一遍

利用上述递推关系，固定起点后让终点向右滑动，每步只加新进入的元素：

```
int gs_IC( int A[], int n ) {          //递增版：O(n^2)
   int gs = 0;                         //接口 I：空区间和为0
   for ( int lo = 0; lo < n; lo++ )    //枚举所有起点
      for ( int s = 0, hi = lo; hi < n; ) {
         s += A[ hi++ ];               //递增地得到总和：O(1)
         gs = max( gs, s );            //择优更新
      }
   return gs;
}
```

对数组 B，五个起点各自的递增求和构成一张三角形数表：

起点 lo	递增过程（每步加入的元素与累计和）	该起点下最大
0	2 ⟶ 2−3=−1 ⟶ −1+4=3 ⟶ 3−1=2 ⟶ 2+3=5	5
1	−3 ⟶ −3+4=1 ⟶ 1−1=0 ⟶ 0+3=3	3
2	4 ⟶ 4−1=3 ⟶ 3+3=6	6
3	−1 ⟶ −1+3=2	2
4	3	3

全局最大 6 出现在起点 2、终点 5，即区间 $[4, -1, 3]$——一枚负元素反而被最好的区间收编，因为删掉它总和反而下降。三角形数表的面积是 $\frac{n(n+1)}{2}$，总体 $O(n^2)$：蛮力的立方降下来了，但“换一个起点就全部重来”依然浪费。

3.6.3 分而治之：跨界区间怎么算

以中点 mi 为界把 $[lo, hi)$ 切成前缀 $[lo, mi)$ 与后缀 $[mi, hi)$。$[lo, hi)$ 内的最大区间无非三种情况：完全落在前缀中、完全落在后缀中、跨越切分线。前两类递归求解，只需集中处理第三类：跨界的区间 $[i, j)$ 满

足 $i < mi \le j$，它必由两段拼成——前缀中的非空后缀 $[i, mi)$，加上后缀中的非空前缀 $[mi, j)$。两类候选可以独立取最大再相加：分别用一趟递增扫描枚举所有后缀和、前缀和，各花 $O(n)$：

```
int gs_DC( int A[], int lo, int hi ) {          //分治版：O(n*logn)
   if ( hi - lo < 2 ) return A[lo];             //递归基：单个元素（接口 ‖ 语义）
   int mi = (lo + hi) / 2;
   int gsL = 0, gsR = 0;                        //前缀的最大后缀和 | 后缀的最大前缀和
   for ( int s = 0, i = mi - 1; lo <= i; i-- )
      gsL = max( gsL, s += A[i] );              //所有后缀[i,mi)
   for ( int s = 0, j = mi; j < hi; j++ )
      gsR = max( gsR, s += A[j] );              //所有前缀[mi,j)
   return max( gsL + gsR,
               max( gs_DC( A, lo, mi ), gs_DC( A, mi, hi ) ) );
}
```

正确性的关键在跨界情形的分解：任何跨界区间唯一地分解为一个左侧非空后缀加一个右侧非空前缀，而两者取最大后拼起来仍是一个合法的跨界区间，所以分别最大化再相加不会错。对数组 $B = \{2, -3, 4, -1, 3\}$，$mi = 2$，逐项算跨界候选：左半 $\{2, -3\}$ 的后缀和为 -3（$[1, 2)$）与 -1（$[0, 2)$），最大 $gsL = -1$；右半 $\{4, -1, 3\}$ 的前缀和为 4、3、6，最大 $gsR = 6$；跨界候选 $-1 + 6 = 5$。左半递归得 2（$[2]$），右半递归得 6（$[4, -1, 3]$），三者取最大，全局答案 6。要当心一类错误：左半内部最大的区间是 $[2]$（和 2），右半内部最大的区间是 $[4, -1, 3]$（和 6），两者相加得 8，立即超过真值 6——拼接出的两段根本不连续。跨界候选必须是左半的后缀接右半的前缀，二者都紧贴切分线。

每个递归实例耗时集中于并列的两个 for 循环，共 $O(n)$，递推 $T(n) = 2T(n/2) + O(n)$，由主定理同阶情况解得 $O(n \log n)$。

3.6.4 减治：一次扫描的答案

出人意料的是，这个问题存在线性算法。诀窍是换一个角度：盯住最大区间的起点，从右向左扫描。设 sum 保存“以当前下标 k 为起点的、向右延伸的最大区间和”（所有候选都非正时记 0，即空区间）。读入 $A[k]$ 时无非两种情况：若旧 sum 为正，接上它继续延伸；若旧 sum 已非正，再接任何东西都不如从 $A[k]$ 重新开始——这正是代码中 `sum = max(sum, 0)` 的剪除动作。每步再用 gs 记录历史最大：

```
int gs_LS( int A[], int n ) {           //减治版：O(n)，接口 I
   int gs = 0;                          //已知的最大和 = S[n,n) = 0
   for ( int sum = 0, k = n - 1; 0 <= k; k-- ) {  //当前区间[k,n)
      sum += A[k];                      //递增地更新"以k为起点的最大和"
      gs = max( gs, sum );              //择优更新
      sum = max( sum, 0 );              //剪除总和非正的起点
   }
   return gs;
}
```

以教师课件 19 元素实例的完整 trace 核对（自右向左，i 为当前下标，S 为“以 i 为起点的最大和”，加粗行为更新 gs 的时刻）：

i	$A[i]$	S（先加$A[i]$）	更新后 gs	剪除后 sum
18	3	3	**3**	3
17	−5	−2	3	0
16	0	0	3	0
15	8	8	**8**	8
14	−2	6	8	6
13	1	7	8	7
12	−3	4	8	4

i	$A[i]$	S（先加$A[i]$）	更新后 gs	剪除后 sum
11	0	4	8	4
10	13	17	**17**	17
9	−8	9	17	9
8	−12	−3	17	0
7	6	6	17	6
6	5	11	17	11
5	−9	2	17	2
4	6	8	17	8
3	2	10	17	10
2	7	17	17	17
1	−2	15	17	15
0	1	16	17	16

最终 `gs = 17`，答案正确。看两个关键行：$i = 17$ 时，-5 与右侧已积累的 3 拼成 -2，非正，剪除归零——若不剪，它会把后面所有的起点候选都往下拖；$i = 10$ 时 S 冲到 17，历史最大被刷新，对应区间正是 $[10, 16)$。注意 $i = 2$ 时 S 同样达到 17 但未再刷新：`max(gs, sum)` 在平局时保留先到者，而自右向左的扫描先遇到的是更靠右的 $[10, 16)$——恰与“靠后者优先”的全局约定一致（两个同和区间等长 6，取靠右的 $[10, 16)$）。教师课件同页 trace 表的最后两行 S 值印作 14、15，按递推 $17 - 2 = 15$、$15 + 1 = 16$ 应为 15、16，属笔误；本表已按递推改正。

自右向左扫描的完整过程，先看逐步演示，再对照下表：



图 5　逐步演示：最大区间：从右向左一次扫描（19 元素实例）

接口 II（非空）的版本剪除动作不变，只改两处：`gs` 初始化为 `A[0]`（空区间不再作为候选），更新时用严格大于 `if ( gs < s ) gs = s;`。19 元素实例在接口 II 下的完整 trace（剪除动作一致，故 S 列与接口 I 完全相同，差别只在 `gs` 的初值与更新时机）：

i	$A[i]$	S	更新后 gs	剪除后 sum
18	3	3	**3**	3
17	−5	−2	3	0
16	0	0	3	0
15	8	8	**8**	8
14	−2	6	8	6

i	$A[i]$	S	更新后 gs	剪除后 sum
13	1	7	8	7
12	−3	4	8	4
11	0	4	8	4
10	13	17	**17**	17
9	−8	9	17	9
8	−12	−3	17	0
7	6	6	17	6
6	5	11	17	11
5	−9	2	17	2
4	6	8	17	8
3	2	10	17	10
2	7	17	17	17
1	−2	15	17	15
0	1	16	17	16

接口 II 下答案同为 17。接口 I 的运行时间主要消耗于 for 循环，每个位置仅常数时间，总体 $O(n)$：四种策略里代码最短、耗时最少。回头看，它正是递增求和与聪明剪枝的结合：递增避免了蛮力的重复加法，剪除非正起点避免了递增版“换个终点重新来”的浪费。四种策略一条比一条快，靠的都是不重复劳动。

3.6.5 附：一次返回四项的变体

分治版每次为算跨界候选要重新扫描边界。若让每个区间的递归返回时顺便带回父亲需要的全部信息，扫描就免了。对非空区间约定四项摘要：总和 S、最大前缀和 P（必须从首元素开始）、最大后缀和 Q（必须在末元素结束）、最大区间和 M（任取非空连续区间）。以左半 $\{-2,3,-1\}$ 为例：$S=0$；前缀候选 $-2,1,0$，最大 $P=1$；后缀候选 $-1,2,0$，最大 $Q=2$；内部最佳是单项 $[3]$，$M=3$。左右孩子的摘要合并规则：

$$S=S_L+S_R,\qquad P=\max(P_L,S_L+P_R),\qquad Q=\max(Q_R,S_R+Q_L),\qquad M=\max(M_L,M_R,Q_L+P_R)$$

每条规则的道理相同：父段的前缀要么完全停在左段内（取 P_L），要么贯穿左段（取整个左段加右段前缀 S_L+P_R）；最大区间照左、右、跨界三分。合并只需常数时间，整个算法 $T(n)=2T(n/2)+O(1)$，由主定理得 $O(n)$，递归深度 $O(\log n)$。第 14 章的线段树将保存同样的四项摘要，使任意相邻查询段都能直接拼接——这套“带足上下文返回”的想法，那里会大放异彩。若改用接口 I，基础情况的 P、Q、M 应取 $\max(0,x)$，总和仍为 x；两种定义不可在同一棵递归树里混用。

3.7 Fibonacci 数：重复实例是浪费的根源

减治与分治是两种基本设计模式，但直接以递归实现它们，有时会导致复杂度爆炸。动态规划（dynamic programming）往往是思路对称、功能等效而效率极高的替代路线。第一个例子：计算 Fibonacci 数列的第 n 项，该数列按 $fib(n)=fib(n-1)+fib(n-2)$ 递归定义，$fib(0)=0$、$fib(1)=1$。

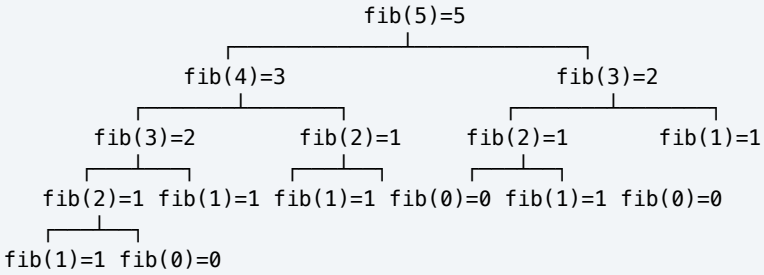
```
uint64_t fib( int n ) {              //二分递归版
   return ( 2 > n ) ? n : fib( n-1 ) + fib( n-2 );
}  //fib: O(φ^n), φ ≈ 1.618
```

实测会发现它慢得出奇。复杂度是指数级的，证明只需一步代入：猜 $T(n)\le c\cdot\varphi^n$，其中黄金分割比 $\varphi=\frac{1+\sqrt{5}}{2}\approx 1.618$。代入递推式右侧：

$$c\varphi^{n-1}+c\varphi^{n-2}+O(1)=c\varphi^{n}\cdot\underbrace{\left(\frac{1}{\varphi}+\frac{1}{\varphi^{2}}\right)}_{=1}+O(1)=c\varphi^{n}+O(1)$$

最后一步用到 $\varphi^2=\varphi+1$（黄金分割的定义式，两边同除以 φ^2 即得）。右侧仍被 $O(\varphi^n)$ 吸收，猜测成立，$T(n)=O(\varphi^n)$。封底估算一下：$\varphi^{43}\approx 10^9$，所以在这台“每秒十亿次”的参考机器上，大约从 $fib(43)$ 开始就能感觉到秒级延迟，随后愈发明显。

病根：同一小题反复做。画出 $fib(5)$ 的调用树，问题一目了然：



先后出现的递归实例共 15 个，但按规模归并后不过 6 种：$fib(5)$ 出现 1 次、$fib(4)$ 1 次、$fib(3)$ 2 次、$fib(2)$ 3 次、$fib(1)$ 5 次、$fib(0)$ 3 次。实例总数有个漂亮的闭式：$C(n)=C(n-1)+C(n-2)+1$，$C(0)=C(1)=1$，归纳可证

$$C(n)=2\,fib(n+1)-1$$

（验证：$C(5)=2\times 8-1=15$，与树上数出的完全一致。）反复出现、结果不变的计算，何必一次次重做？

记忆化。对付这类重复，最朴素的武器是速查表——人尽皆知的九九乘法表正是这样一张表：表中每项当初仍需花工夫习得，此后无论是 9×2 还是 3×7，都能不假思索地直接报出答案。改造方法完全从外在形式入手，不必了解算法背后的原理：算过就记账，再问直接查。

```
extern uint64_t M[];                //快查表：前两项预置0、1，其余均为UINT64_MAX
uint64_t fibM( int n ) {            //记忆化版：O(n)
   if ( UINT64_MAX == M[n] )        //仅当该子问题尚未解出
      M[n] = fibM( n-1 ) + fibM( n-2 );
   return M[n];                     //否则直接引用
}  //递归基的答案已预存，判断亦可省去
```

$M[n]=fib(n)$ 对 $n<2$ 预置，其余初始化为哨兵值 UINT64_MAX。每个子问题只会在第一次求值时照常递归，此后一律直接查表：按规模分层看，每种规模至多计算一次，总计算量为 $O(n)$（递归调用的实例仍有约 $2n-1$ 个，但绝大多数是一次查表即返的廉价调用）；代价是速查表占用的 $O(n)$ 空间。从 `fib()` 到 `fibM()`的改造手法称**记忆化**（memoization），对任何“重复实例众多”的递归算法都适用。

动态规划。速查表毕竟占空间，尤其当子问题涉及多个参数时表会膨胀。细审 `fibM()`的执行过程会发现：它其实是从递归基出发、按规模递增的次序填表。既然如此，何不干脆掉转方向，自底向上直接迭代？

```
uint64_t fibI( int n ) {            //迭代版：O(n)时间 + O(1)空间
   uint64_t f = 1, g = 0;           //初始化：fib(-1) = 1, fib(0) = 0
   while ( 0 < n-- )
      { g += f;  f = g - f; }       //经n次加减，g逐步推进
   return g;
}
```

这里 f,g 始终只记录相邻两项，f 的初值 $fib(-1)=1$ 来自斐波那契向负方向的延伸（$fib(-1)+fib(0)=fib(1)$）。不变式：迭代 k 轮之后 $g=fib(k)$、$f=fib(k-1)$。逐轮核对（$n=5$）：

轮次开始时的 n	本轮执行 g += f; f = g−f 之后	(g, f)	含义
5	(1, 0)	$g=fib(1), f=fib(0)$	迭代 1 轮，$g=fib(1)$
4	(1, 1)	$g=fib(2), f=fib(1)$	迭代 2 轮，$g=fib(2)$

轮次开始时的 n	本轮执行 g += f; f = g−f 之后	(g, f)	含义
3	(2, 1)	$g=fib(3), f=fib(2)$	迭代 3 轮，$g=fib(3)$
2	(3, 2)	$g=fib(4), f=fib(3)$	迭代 4 轮，$g=fib(4)$
1	(5, 3)	$g=fib(5), f=fib(4)$	迭代 5 轮，$g=fib(5)$
0	循环退出，返回 g = 5		

时间 $O(n)$，更重要的是任一时刻只需记住最后两个子问题的解，空间降至理想的 $O(1)$。从递归到记忆化到迭代，三个步骤合起来就是**动态规划**的核心思想：分析问题自身的逻辑，找出足以推导下一个解的少量关键信息，只保存它们。很多递归算法都有这种性质，敏锐地发现并利用它，性能即有数量级的改善。

3.8 最长公共子序列：二维的动态规划

再看一个更典型、也更需要动态规划的例子。若干字符顺次排成一行构成**序列**（sequence）；从序列中挑出若干字符、保持原次序重新串接，得到**子序列**（subsequence）——比如“sina”之于“tsinghua”，“cute”之于“computer”。两个序列（把空序列也视作序列）共同拥有的子序列中，包含字符最多者称作**最长公共子序列**（longest common subsequence, LCS）。LCS 可能同时有多个：对序列“didactical”与“advantage”，长度为 4 的公共子序列只有“data”一个字符串：d 取下标 0、a 取 3、t 取 5、a 取 8（didactical 的字母表中没有 e，“date”并不成立）；不过同一个“data”挑字符的方式可以不止一种。教材图 1.26 另以“data”与“dana”一对序列展示 LCS 的多解与歧义。长度是唯一确定的，以下只讨论长度。

递归分解。任给序列 $A[0,n)$ 与 $B[0,m)$，考查两者最末端的字符 $A[n-1]$ 与 $B[m-1]$，无非两种情况。若二者相同，则至少存在一个 LCS 同时采用它们（若某 LCS 不用，总可换上这对末字符而不变短），于是减治：剪除这对字符，递归求 $A[0,n-1)$ 与 $B[0,m-1)$ 的 LCS，再加 1。若二者不同，则任何 LCS 都不会同时采用它们，于是分治：分别剪除 $B[m-1]$ 与 $A[n-1]$，递归求出两个子问题的 LCS，取更长者为解。

```
uint32_t lcs( const char* A, int n, const char* B, int m ) {
   if ( n < 1 || m < 1 ) return 0;                  //递归基：空序列
   if ( A[n-1] == B[m-1] )                          //尾字符相同：减治
      return 1 + lcs( A, n-1, B, m-1 );
   else                                             //否则：分治
      return max( lcs( A, n-1, B, m ), lcs( A, n, B, m-1 ) );
}  //每经一次比对，至少一个序列缩短一位，必能终止
```

最好情况并不坏：当一个序列恰好是另一个的后缀时，算法持续沿减治分支线性递归，$O(n)$ 收工。最坏情况则完全相反：若两序列毫无公共字符，减治分支一次也不会触发，每层分裂为二，递推 $T(n,m)\geq T(n-1,m)+T(n,m-1)$，递归实例总数不超过从 (n,m) 走到边界的单调路径数的两倍减一，即 $2\binom{n+m}{n}-1$（$n=m=10$ 且无公共字符时实测 184755 个叶子、369509 个实例）——由 Stirling 公式约为 $2^{n+m}/\sqrt{n+m}$，标准的指数爆炸。

子问题空间。与 Fibonacci 同理，病根在重复实例。可能出现哪些子问题？把子问题铺成 $(n+1)\times(m+1)$ 的矩阵：格子 (a,b) 对应“序列 A 的前缀 $A[0,a)$ 与序列 B 的前缀 $B[0,b)$ 的 LCS 长度”。减治分支把格子移向左上对角，分治分支移向正左或正上。无论走法如何，格子总数不过 $(n+1)(m+1)$——对“didactical”与“advantage”是 110 种，而递归实例成千上万。重复劳动的理由充分，该动用速查表了。

记忆化。给每个格子备一格快查表（0 行 0 列预置 0，其余置哨兵），子问题只在首次求值时照常递归。时间降至 $O(n\times m)$——每个格子只算一次、每次 $O(1)$。

动态规划。同样可以掉转方向自底向上：以第 0 行为基础（任一序列与前缀 $A[0,0)$ 的 LCS 恒为 0），逐行递推。转移规则与前述递归完全一致：行尾字符与列尾字符相同则左上格加 1，否则取正上格与左格的较大者。完整算一下行序列 A = “didactical”（d,i,d,a,c,t,i,c,a,l）对列序列 B = “advantage”（a,d,v,a,n,t,a,g,e）：

行 \ 列	0	a	d	v	a	n	t	a	g	e
0	0	0	0	0	0	0	0	0	0	0
d	0	0	1	1	1	1	1	1	1	1
i	0	0	1	1	1	1	1	1	1	1
d	0	0	1	1	1	1	1	1	1	1
a	0	1	1	1	2	2	2	2	2	2
c	0	1	1	1	2	2	2	2	2	2
t	0	1	1	1	2	2	3	3	3	3
i	0	1	1	1	2	2	3	3	3	3
c	0	1	1	1	2	2	3	3	3	3
a	0	1	1	1	2	2	3	4	4	4
l	0	1	1	1	2	2	3	4	4	4

逐格挑几处看转移规则如何工作。第 1 行（字符 d）与第 2 列（字符 d）相交：字符相同，取左上格 0 加 1，得 1；第 1 数据列（字符 a，即 B 的首字母）与第 2、3 行（字符 d、i）均不匹配，三格都是 0。第 4 行（字符 a）与第 1 列相交：相同，取左上（第 3 行第 0 列）的 0 加 1，得 1；与第 4 列（B 中第二个 a）相交：相同，取左上（第 3 行第 3 列）的 1 加 1，得 2——这一跳正是“data”中第二个字符 a 被收入公共子序列的时刻。第 6 行（字符 t）与第 6 列（字符 t）：左上格 2 加 1，得 3，“dat”成形。第 9 行（字符 a）与第 7 列（字符 a）：左上 3 加 1，得 4，“data”完工。其余格子不发生字符匹配，只在上方与左方之间挑更大者继承。右下角给出 $LCS = 4$，整表与教师课件 p90 的 DP 表逐格一致（课件以 advantage 为行、didactical 为列，与本表互为转置）。

整表耗时 $O(nm)$。依赖方向是单向的：第 i 行只依赖第 $i-1$ 行与本行左侧，于是始终只需滚动保存两行：

```
uint32_t* lcs1 = new uint32_t[m+1];      //上一行
uint32_t* lcs2 = new uint32_t[m+1];      //当前行
memset( lcs1, 0, sizeof(uint32_t)*(m+1) );  lcs2[0] = 0;   //哨兵列
for ( int i = 0; i < n; swap( lcs1, lcs2 ), i++ )
   for ( int j = 0; j < m; j++ )
      lcs2[j+1] = ( A[i] == B[j] ) ? 1 + lcs1[j]
                                   : max( lcs2[j], lcs1[j+1] );
uint32_t solu = lcs1[m];  delete[] lcs1;  delete[] lcs2;  return solu;
```

入口前若 $n < m$ 可交换两序列，保证按短者开表，空间 $O(\min(n, m))$。若还需要还原出 LCS 本身（而不只是长度），可另备一张同样大小的方向表，回溯时沿“左上（匹配）”或“上/左（取大者）”走到边界即可，空间仍是 $O(nm)$——这就是“省时间”与“省空间”之间的典型权衡。

3.9 同名不同义：LCS 与编辑距离

字符串相似度有两套常见题目，别混为一谈。LCS 问“最多能保留多少字符”；编辑距离问“最少多少步编辑能把一个串变成另一个”。若只允许插入与删除（各计 1 步），答案有公式 $D = n + m - 2L$：A = “ABC”、B = “ADC”时 $L = 2$（“AC”），$D = 3 + 3 - 4 = 2$（删 B、插 D）。若允许成本 1 的替换，一步把 B 改成 D 就够，$D = 1$。两者的 DP 都是二维前缀状态，但转移集合不同：只插删的版本不允许“对角替换”转移。遇到题面先列出允许的操作集合，再把每个操作翻译成一条转移，就不会串题。第 21 章的 Filename 题将利用预算小这一特点，只在带状区域内做这份 DP。

3.10 动态规划适用的条件

把本册三个 DP 实例放在一起，可以提炼出适用条件。其一，**重复子问题**：同一小问题反复出现（$fib(3)$ 算三遍；110 种格子被走成千上万遍），速查表才有收益。其二，**最优子结构**：大问题的答案能由小问题的答案按明确规则组合（fib 的加法、LCS 的取大、最大区间的三分）。其三，**状态够用而不冗余**：状态

要包含决定未来的全部信息——LCS 用“两个前缀长度”就够，因为剩余问题只取决于这两者，与此前如何跳字符无关；最大区间问题里“全局最优”推不动下一步（最优区间可能早已结束），必须另设“以当前下标为起点”的状态。三者齐备，就可以递归 + 记忆化起步，能找到自底向上的迭代次序就转成 DP，能看出滚动依赖就压空间。这套流程后面章节会反复走。

3.11 练习

1. 按本册的方式画出 $fib(6)$ 的完整调用树，标出每个实例的返回值；验证实例总数 $C(6) = 2\,fib(7) - 1 = 25$，并统计 $fib(2)$ 与 $fib(3)$ 各出现多少次（对照：$C(7) = 41$）。
2. 对数组 $A = \{4, -3, 5, -2, -1, 2, 6, -2\}$ 手工执行接口 I 的减治版最大区间扫描（自右向左），列表记录每步的 i、$A[i]$、S、剪除后的 `sum` 与 `gs`，并指出取得最大和的区间。
3. 用滚动两行的方法手工复算“advantage”对“didactical”的 LCS 表（行、列对调后再算一遍），核对两次的右下角都是 4；再找出两个长度为 4 的 LCS，并写出各自挑字符的下标。
4. 蛮力版、递增版、分治版、减治版最大区间算法在 $n = 100$ 的输入上，基本操作次数大约分别是多少？用上一册的封底估算表换算成时间，体会四个版本的差距。

3.12 本章要点

- 减治每次削掉一个单位（线性递归），分治一分为二（二分递归）；递归基是确保有穷性的生命线。
- 复杂度分析两板斧：递归跟踪数实例，递推方程解代数式；空间看“同时活着的实例”峰值，即忒修斯绳线的最大长度（递归深度）。
- 主定理按 $g(n)$ 与 $n^{\log_b a}$ 的相对大小三分；不均衡切分找 Akra-Bazzi。大整数乘法靠 $ad + bc = (a - b)(d - c) + ac + bd$ 把子任务从 4 个减到 3 个。
- 总和最大区间的四种策略逐级加速：蛮力 $O(n^3)$、递增 $O(n^2)$、分治 $O(n \log n)$、减治 $O(n)$；接口 I 允许空区间（`gs` 从 0 起）、接口 II 要求非空（`gs` 从 $A[0]$ 起），不可混用。
- Fibonacci 的三个版本：二分递归 $O(\varphi^n)$，记忆化 $O(n)$ 时间 $O(n)$ 空间，迭代 $O(n)$ 时间 $O(1)$ 空间；病根都是重复实例，药方都是“算过就记，再问直接查”。
- LCS 把分治减治揉进二维格子：递归 $O(2^{n+m})$，记忆化与 DP $O(nm)$，滚动两行后空间 $O(\min(n, m))$。

4 向量：抽象数据类型、插入删除与动态扩容

向量是本书第一个正式的数据结构，也贯穿全书：后续各章的栈、队列、散列表、图邻接矩阵都以它为底层构件。本章做三件事：约定向量的对外接口（抽象数据类型），作为应用者完整地使用一遍这些接口，再作为实现者完成接口背后的全部工作——插入、删除、动态扩容，并回答一个关键问题：为什么扩容必须采用容量加倍策略。

4.1 抽象数据类型与数据结构：先定接口，再做实现

抽象数据类型（Abstract Data Type, ADT）= 数据模型 + 定义在该模型上的一组操作。它给出抽象定义：只描述外部的逻辑特性，即各操作的语义；不考虑时间复杂度，也不涉及数据的存储方式。**数据结构**（data structure）= 基于某种特定语言、实现 ADT 的一整套算法。它给出具体实现：负责内部的表示与实现，包含完整的算法；同一个 ADT 可以有多种实现，而且实现与复杂度密切相关。一句话概括两者的关系：

$$\text{Application} = \text{Interface} \times \text{Implementation}$$

在数据结构的具体实现与实际应用之间，ADT 就分工与接口制定了统一的规范。实现一方的任务是高效兑现接口操作，好比做冰箱、造汽车；应用一方的任务是通过接口便捷地使用数据结构，好比用冰箱、开汽车。有了这套规范，高层算法设计者可以与底层数据结构实现者分工协作，不同的算法与数据结构可以便捷地组合借用，每种操作接口只需统一实现一次：代码篇幅缩短，安全性加强，软件复用度提高。

数组是向量的出发点。C/C++语言中，数组元素与编号一一对应：`A[0], A[1], ..., A[n-1]`；若每个元素占用 s 个字节的空间，则 `A[i]` 的物理地址为 $A + i \times s$（可能有对齐填充）。也就是说，只要拿到起始地址和下标，任何元素的地址一次算出、一次访达——这是第 1 章 RAM 模型“按秩调用”（call-by-rank）的物质基础。

向量（vector）是数组的抽象与泛化：由一组元素按线性次序封装而成，各元素与 $[0, n)$ 内的**秩**（rank）一一对应，向量 V 中秩为 r 的元素记作 $V[r]$。秩是下标概念的泛化，实现为无符号整数：

```
using Rank = uint32_t;
```

与直接使用数组相比，经由向量来操作、管理数据更加简化、统一与安全；元素类型可以灵活选取，也便于定制复杂数据结构——比如 B 树节点用 `Vector<BTNodePosi<T>>`记录所有孩子，图结构用二维向量 `Vector<Vector<Edge<Te>*>>`实现邻接矩阵存放边集。

制造汽车之前要先写好使用说明书，数据结构也是这样。向量 ADT 需要支持的接口如表 2.1：

表 2.1　向量 ADT 的操作接口及其功能

操作接口	功能
`size()` `empty()`	报告元素总数；判断向量是否为空
`insert(r, e)`	将 e 作为秩为 r 的元素插入（后继元素顺次后移）
`remove(r)`	删除秩为 r 的元素（后继元素顺次前移）
`sort()` `unsort()`	排序；置乱
`find(e)`	在无序向量中，查找与 e 相等者（如有多个，取最靠前者）
`search(e)`	在有序向量中，查找不大于 e 的最大者
`dedup()`	从无序向量中，剔除相等的元素（保留最靠后者）
`uniquify()`	从有序向量中，剔除相等的元素（保留最靠前者）
`traverse(visit())`	对每个元素统一执行 visit 操作

这些接口可以有多种实现方式，复杂度也不尽相同。把对外接口与内部实现分离之后，无论具体采用哪种实现，应用者都可以遵照统一的接口规范来调用。也就是说，向量结构哪怕还停留在“图纸”上，应用者就已经可以“假想”地使用它了——下面第一件事就是这样的假想使用。

4.2 作为应用者：一次完整的操作实例

表 2.2 摘自教材 2026 版，记录一个字符向量从创建开始，经由 ADT 接口依次执行一系列操作的完整过程。约定'^'表示“向量内容同上”。

表 2.2　向量操作实例

操作	输出	向量内容
初始化构造		空
size()	0	^
empty()	true	^
insert(0,'N')		N
insert('A')		N A
insert(0,'D')		D N A
size()	3	^
insert(1,'A')		D A N A
remove(2)	'N'	D A A
insert(2,'T')		D A T A
size()	4	^
find('N')	−1	^
find('T')	2	^
find('A')	1	^
insert(2,'I')		D A I T A
dedup()		D I T A
insert(1,'I')		D I I T A
remove(4)	'A'	D I I T
insert(2,'G')		D I G I T
empty()	false	^
sort()		D G I I T
search('I')	3	^
search('X')	4	^
search('@')	−1	^
uniquify()		D G I T
remove(0)	'D'	G I T

逐段看这张表。第 4 至 10 行是插入与删除。insert('A')省略了秩参数，含义是插到末尾，即 insert(size(), 'A')；insert(0,'D')是头部插入，原有的 N A 整体后移一位，得到 D N A；insert(1,'A')后向量已是 D A N A；remove(2)删除秩为 2 的元素'N'，把被删元素作为返回值，其后的元素顺次前移，得到 D A A；再经 insert(2,'T')补回 T，向量定格为 D A T A，规模 4。

第 12 至 14 行是 find。它面向无序向量，查找与目标相等者，如有多个取最靠前者；失败返回-1。find('N')返回-1：向量 D A T A 中没有 N。find('T')返回 2：唯一的 T 在秩 2。find('A')返回 1：A 有两个，分别位于秩 1 和秩 3，按约定取靠前的 1。

第 15 至 16 行演示无序去重。insert(2,'I')后向量为 D A I T A，其中 A 重复（秩 1 与秩 4）。dedup()剔除相等的元素并保留最靠后者，删去秩 1 的 A，得到 D I T A。对照第 25 行的 uniquify()：它面向有序向

量，保留每个相等区段的最靠前者——从 D G I I I T 剔去一个 I，得到 D G I T。两个接口都返回被删除元素的个数（教材表中省略了这一输出）。

第 21 至 24 行是 search。它面向有序向量，查找不大于 e 的最大者（返回其秩），为后续插入 insert(1 + search(e), e)提供位置。sort()后向量为 D G I I T（非降序）：search('I')返回 3——不大于 I 的最大元素是秩 3 的 I，两个相等者取秩大者；search('X')返回 4——不大于 X 的最大者是秩 4 的 T；search('@')返回-1——所有元素都大于'@'，无候选。注意 search 与 find 的语义差异：前者假设有序、允许失败返回-1，但通常返回的是插入位置的参照；后者只要求可判等。

以上是应用者的全部视角：不管谁来实现向量，只要遵照表 2.1 的接口规范，这些操作必可如期兑现。

有一处版本差异需要说明。本章配套讲义课件的同类示例沿用旧版教材，其中 find('A')的输出为 3，与表 2.1"取最靠前者"的定义矛盾（D A T A 中最靠前的 A 在秩 1）。2026 版教材的表 2.2 已更正为 1，本书从教材。

4.3 作为实现者：Vector 模板类

切换到实现者视角。向量模板类私有地持有三个成员：数据区指针_elem、实际容量_capa、当前规模_size；_elem[0, _size)是真实存放的元素，[_size, _capa)是尚未使用的预留空位：

```
template <typename T>
class Vector {
private:
   T* _elem;         //数据区
   Rank _capa;       //容量
   Rank _size;       //规模
protected:
   /* 内部接口：copy、expand、shrink、各排序算法等 */
public:
   /* 开放接口：表2.1的各操作 */
};
```

循环秩访问通过重载下标操作符实现，两个版本分别支持读和写：

```
template <typename T>   //可作为左值：V[r] = (T) (2*x + 3)
T& Vector<T>::operator[]( Rank r ) { return _elem[ r ]; }

template <typename T>   //仅限右值：T x = V[r] + U[s] * W[t]
const T& Vector<T>::operator[]( Rank r ) const { return _elem[ r ]; }
```

本课程采用简易的方式处理意外和错误：入口参数约定$0 \le r < _size$，不做越界检查——正式工程实现应当加上断言或异常，这里为突出算法主线而省略。构造函数按预估规模开出足够大的空间，并在末尾始终预留一个哨兵单元；析构函数释放整个数据区：

```
Vector( Rank s = DEFAULT_SIZE )          //容量为s+1，末尾预留哨兵
{ _elem = new T[ _capa = s + 1 ]; _size = 0; }
~Vector() { delete [] _elem; }           //释放数据区
```

区间复制是多数操作的公共底层，将 A[lo, hi)复制至_elem[r, r-lo+hi)，原内容直接覆盖，耗时$O(hi - lo)$：

```
template <typename T>
void Vector<T>::copy( Rank r, const T* A, Rank lo, Rank hi ) {
   while ( lo < hi ) _elem[ r++ ] = A[ lo++ ];  //逐单元复制
}
```

赋值操作实现深复制：先清空自身，再确保容量足够，最后整体复制——若 T 非基本类型，需要重载=以完成元素的深复制：

```
template <typename T>
void Vector<T>::operator=( const Vector<T>& V ) {
   _size = 0;                    //清除原有内容
   expand( V._size );            //确保容量足够
   copy( 0, V._elem, 0, _size = V._size );   //整体复制
}
```

4.4 插入与删除：以区间为基本形式

插入与删除都有区间版本和单元素版本，实现时以区间版本为基本形式，单元素版本是它的特例。

区间插入：将 `A[lo, hi)`共 `d = hi - lo` 个元素插入至秩 `r`。先把后缀`[r, _size)`整体后移 d 位，腾出空位，再复制入新元素。后移必须自后向前进行：若自前向后，前面的元素尚未挪走就会覆盖尚未搬动的元素。

```
template <typename T> void Vector<T>::insert( Rank r, const T* A, Rank lo, Rank hi ) {
   Rank d = hi - lo;  expand( d );      //若必要，先扩容，确保足以新增d个元素
   for ( Rank i = _size - 1; i != r - 1; i-- )
      _elem[ i + d ] = _elem[ i ];      //自后向前，后缀逐单元后移
   copy( r, A, lo, hi );  _size += d;   //插入d个元素
}
```

区间删除：删除`[lo, hi)`。把后缀`[hi, _size)`整体前移，覆盖被删区间，然后更新规模。前移必须自前向后进行，道理与插入对称。

```
template <typename T> void Vector<T>::remove( Rank lo, Rank hi ) {   //0 <= lo < hi <= n
   while ( hi < _size ) _elem[ lo++ ] = _elem[ hi++ ];  //O(n - hi)：正比于后缀长度
   _size = lo;   shrink();                              //更新规模，必要时缩容
}
template <typename T> T Vector<T>::remove( Rank r )   //单元素版本：区间版本的特例
{ T e = _elem[ r ];  remove( r, r + 1 );  return e; } //O(n - r)，仍正比于后缀长度
```

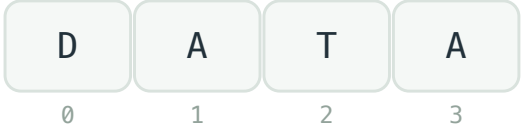
插入搬移的逐步演示（以本章表 2.2 第 15 行的 `insert(2,'I')`为例）：

向量插入：后缀怎样整体后移

第 1 / 5 步

1. 初态

insert(2, 'I')：把后缀[2,4)整体后移一位，再写入I



搬移方向：自后向前。

_size = 4 → 5；容量足够，无需扩容

向量D A T A，在秩2插入字符I。

为什么：后缀[2,4)（T、A）必须整体右移腾位。

图 6　逐步演示：向量插入：后缀怎样整体后移

两个方向的关键事实都写在注释里：**删除的成本正比于后缀长度 $n-hi$，与被删区间的宽度 $hi-lo$ 无关**。

为什么单元素实现为区间版本的特例，反过来却不行？以删除为例算一笔账。取 $n=1000$，删除区间 $[100,900)$ 共 800 个元素。区间删除把秩 900 之后的 100 个元素一次性前移，移动 100 次。若改作逐个调用 `remove(100)`：第一次删除时后缀长度为 $1000-101=899$；此后规模逐次减 1，后缀长度依次为 898、897、……、100，总移动次数为

$$899+898+\cdots+100=\sum_{k=100}^{899}k=\frac{(100+899)\times 800}{2}=399600$$

区间版本移动 100 次，逐个删除移动 399600 次，相差约 4000 倍。逐个删除的每一次都在反复搬运同一段后缀。这条经验值得记住：**重复性的任务应尽可能集中起来，批量地完成**。第 05 章的 uniquify 从$O(n^2)$改进到$O(n)$，靠的也是同一思路。

4.5 动态扩容：装填因子与加倍策略

上一节的 `insert` 在必要时会调用 `expand()`扩容。为什么需要扩容？因为内部数组在创建时容量固定，而向量应对的输入规模无法预知。申请空间时留有足够余地，可以在相当长时间内免于追加；但对任何固定的容量`_capa`，总会遇到装不下的时候。为量化空间的紧张程度，定义**装填因子**（load factor）

$$\lambda=_size/_capa$$

两个极端：$\lambda=100\%$ 时已无空位，再插入就会**上溢**（overflow）；λ 远小于 50% 时元素寥寥无几，属于**下溢**（underflow），空间利用率过低。为保证向量持续高效运转，需要令其根据实际情况动态调整容量，始终维持

$$25\%\le\lambda<100\%$$

扩容的策略如同蝉蜕：身体经过一段时间的生长，蜕去原先的外壳，代之以更大的新外壳。向量则在即将上溢时，适当扩大内部数据区的容量：

```
template <typename T> void Vector<T>::expand( Rank d = 1 ) {
   if ( _size + d < _capa ) return;      //足以新增d个元素，无需扩容
   T* oldElem = _elem;                   //记下原数据区
   _capa = ( _size + d ) * 2;            //容量加倍
   _elem = new T[ _capa ];               //分配新空间
   copy( 0, oldElem, 0, _size );         //转移数据
   delete [] oldElem;                    //释放原空间
}
```

扩容的最好情况（无需扩容）只需$O(1)$，最坏情况（扩容搬迁）需要$O(n)$，两者相差悬殊且非此即彼。怎样从整体上界定这类操作的性能？平均分析在这里失效，必须换用均摊分析。

4.6 均摊分析：加倍摊还 O(1)，递增摊还 O(n)

先澄清两种“平均”的区别。**平均复杂度**（average complexity）即期望复杂度：根据各种操作出现的概率，对运行时间做加权平均。它不适用于 `expand()`——对一个真实的向量，最好情况与最坏情况的出现并非随机，而是完全确定的。换个角度：对任何一个真实的向量，考查其完整的、足够长的操作历史（简化起见，设只有插入、无删除，且都是单元素插入），`expand()`的各次调用成本虽然忽高忽低，但整个历史累加起来是完全确定的量。**均摊复杂度**（amortized complexity）考查的正是这个量：对一个真实算法连续、足够多次的调用，将总共花费的时间摊还至单次调用。

容量加倍策略。设从容量 0 开始，逐个插入 n 个元素。每次插入本身$O(1)$；上溢触发扩容的时机是第 2、4、8、16……次插入（容量翻倍至 2、4、8、16……），相应需要搬移的数据量为 1、3、7、15……即

$$1+3+7+\cdots=(2-1)+(4-1)+(8-1)+\cdots=(2+4+8+\cdots)-(\text{项数})<2n-\log_2 n$$

上一册笔记的结论直接可用：几何级数与末项同阶，末项不超过 n，故搬移总量 $O(n)$。连同 n 次 $O(1)$ 的插入本身，n 个元素的总成本仍是 $O(n)$，摊还至每次插入不过 $O(1)$。用一个具体规模核对：$n=13$ 时

扩容发生在第 2、4、8 次插入，搬移量为 $1+3+7=11$，加上 13 次插入本身，总成本约 24 个基本操作，摊还每次不足 2 次。

容量递增策略。作为对照，设每次上溢时容量只增加固定增量 I：则每隔 I 次插入就要扩容一次，各次搬移量为 $I, 2I, 3I, \ldots$，构成算术级数，总搬移量为

$$I + 2I + \cdots + \frac{n}{I} \cdot I \approx \frac{n^2}{2I}$$

摊还至每次插入高达 $O(n/I) = O(n)$。用数字感受差距：$n = 1000$、$I = 5$ 时，递增策略的总搬移约 100500 次，而加倍策略不足 2000 次。

两种策略的对比汇成一张表：

策略	扩容触发	总搬移量	摊还复杂度
容量加倍	上溢时_capa 翻倍	$O(n)$（几何级数）	$O(1)$
容量递增 I	上溢时_capa 增加 I	$O(n^2/I)$（算术级数）	$O(n)$

同样是为上溢做准备，加倍与递增一个天一个地，差别全在“每次扩多少”：加倍让下一次上溢所需的插入次数也翻倍，便宜的调用与昂贵的调用交替出现且比例恒定；递增则让昂贵的调用越来越贵。这正是第 02 章几何级数与算术级数的区别在工程中的直接体现。

最后补齐容量管理的另一半：**缩容**。删除操作使 λ 下降，当 $_size/_capa < 25\%$ 时，`shrink()`按类似的方式将容量减半。缩容阈值取 25% 而不是 50%，是有意留出的迟滞带：若容量减半恰好发生在 $\lambda = 50\%$ 处，那么在规模 $n = _capa/2$ 附近反复地插入、删除，将使每一次操作都触发 $O(n)$ 的扩容或缩容搬迁，均摊分析随之失效。留出从 25% 到 100% 的宽裕区间后，这类抖动被吸收，均摊复杂度的结论依然成立。

4.7 遍历与函数对象

遍历（traverse）指对向量中的每一个元素统一实施同一操作。操作的“号令”如何传达？借助**函数对象**（function object）：本是一个对象，重载`()`之后等效于一个函数，可以作为参数传递：

```
template <typename T> template <typename VST>
void Vector<T>::traverse( VST& visit ) {
   for ( Rank i = 0; i < _size; i++ ) visit( _elem[i] );
}
```

统一地将所有元素各自加一，只需定义一个加一函数对象并传入：

```
template <typename T>   //假设T可直接递增或已重载++
struct Increase                         //函数对象：通过重载operator()实现
{ void operator()( T& e ) { e++; } };
template <typename T> void increase( Vector<T>& V )
{ V.traverse( Increase<T>() ); }
```

更复杂的任务同样可以封装。统计总和，可以让函数对象持有外部累加器的引用，遍历时逐项累加；统计相邻逆序对个数，可以让它记住前驱元素，每访问一个元素就与前驱比较、更新计数——计数为零当且仅当向量整体有序。统一的遍历机制加上可定制的函数对象，使这些彼此不同的任务共用同一个遍历算法。

4.8 无序查找与去重：判等器、哨兵与 O(n²)

表 2.1 中`find(e)`与`search(e)`的一个隐蔽差别值得先说清楚：前者面向无序向量，只要求元素可判等（e == x）；后者面向有序向量，依赖元素可比较大小（e < x）。两者对应两种不同的接口约定。以词条（entry）为例——它由键（key）与值（value）组成：

```
template <typename K, typename V> struct Entry {
   K key;  V value;                    //键、值
   bool operator== ( const Entry<K, V>& e ) { return key == e.key; }  //判等：只比键
   bool operator<  ( const Entry<K, V>& e ) { return key <  e.key; }  //比较：只比键
   bool operator>  ( const Entry<K, V>& e ) { return key >  e.key; }  //比较：只比键
};
```

得益于判等器与比较器，此后的算法不必区分词条及其对应的键。

顺序查找。在无序区间[lo, hi)内查找 e，最直接的做法是逐个比对。实现里有一个省事的小技巧：先把 e 存入_elem[hi]充当哨兵，这样即便 e 不存在，扫描也必然在 hi 处命中终止，每次比对因此免去一次越界判断：

```
template <typename T>   //0 <= lo < hi <= _size
Rank Vector<T>::find( const T& e, Rank lo, Rank hi ) const {   //O(hi - lo)
   T bak = _elem[hi];  _elem[hi] = e;      //将目标存入哨兵
   while ( e != _elem[lo] ) lo++;          //自前向后，逐个比对
   _elem[hi] = bak;  return lo;            //哨兵还原；返回最靠前的命中者，或hi示意失败
}
```

顺序查找是**输入敏感的**（input-sensitive）：最好情况 e 恰在 lo 处，一次比对即返回，$O(1)$；最坏情况 e 不存在或位于末尾，需要$O(hi - lo)$。同一算法在不同输入上的成本有实质差别，这与二分查找“无论成败都约$\log n$次”的表现不同。

去重（deduplicate）。无序向量的重复元素位置杂乱，利用减治思想：从秩 k = 0 开始，在前缀[0, k)中查找与_elem[k]相等者；找到就删除前者（保留靠后者），否则 k 前移一位：

```
template <typename T> Rank Vector<T>::dedup() {   //剔除相等的元素，保留最靠后者
   for ( Rank k = 0; k < _size; ) {
      Rank i = find( _elem[k], 0, k );   //在前缀[0,k)中查找与_elem[k]相等者：O(k+1)
      if ( i < k )
         remove( i );                    //若命中（必为最靠前者），删除之：O(n-k)
      else
         k++;                            //前缀中无重复，k前移
   }
   return _size - oldSize;               //返回被删除元素的总数
}  //O(n^2)
```

以本章表 2.2 第 16 行的操作为实例：对 D A I T A 执行 dedup()，逐步为

k	find(_elem[k], 0, k)的结果	动作	此后向量
0	前缀空，返回 0	k++	D A I T A
1	前缀无 A，返回 1	k++	D A I T A
2	前缀无 I，返回 2	k++	D A I T A
3	前缀无 T，返回 3	k++	D A I T A
4	前缀秩 1 命中 A	删除秩 1	D I T A
4	前缀无 A，返回 4	k++，循环结束	D I T A

复杂度：前缀查找的比对次数最坏为$1 + 2 + \cdots + n = racn(n + 1)2$（输入全无重复时恰好如此），加上每次删除$O(n)$的移动，总体$O(n^2)$。第 05 章将看到：同样的去重，在有序向量上只需$O(n)$。

4.9 本章要点

- 抽象数据类型 = 数据模型 + 定义其上的操作，只管外部逻辑语义；数据结构 = ADT 的一种具体实现，与复杂度密切相关。同一 ADT 可以有多种实现，接口规范让实现与应用可以分工。

- 向量是数组的抽象与泛化：元素与秩$[0, n)$一一对应，支持循秩访问；表 2.1 定义了全部接口，表 2.2 给出了一次完整的使用实例（讲义课件旧示例中 find('A')输出 3 与其接口定义矛盾，教材 2026 版已更正为 1）。
- 插入与删除以区间版本为基本形式：插入自后向前后移，删除自前向后前移；删除成本$O(n - hi)$只取决于后缀长度。逐个删除 800 个元素要移动约 399600 次，区间删除只需 100 次：重复性任务应批量完成。
- 动态扩容以装填因子λ度量空间利用率，目标$25\% \le \lambda < 100\%$；expand()容量加倍。
- 均摊复杂度考查连续多次调用的总成本摊还：容量加倍的摊还复杂度为$O(1)$（几何级数，总搬移$O(n)$），固定增量递增为$O(n)$（算术级数，总搬移$O(n^2/I)$）；缩容阈值取 25% 以留出迟滞带，防止插删抖动触发反复搬迁。
- 遍历以函数对象传达统一操作：重载 `operator()`的对象可以像函数一样传入 traverse()。

5 二分查找、插值查找与位图

有序向量比无序向量多一层结构：元素之间已按大小排好队。这一层结构能换来什么？本章给出三级答案。其一，同样的去重，无序要$O(n^2)$，有序只需$O(n)$。其二，查找从顺序扫描的$O(n)$降到二分的$O(\log n)$，且经 Fibonacci 切分、单次比较、迭代不变性三步打磨，常系数逐步压低。其三，当向量极长且分布有规律时，插值查找把效率再推到$O(\log\log n)$。最后介绍位图：当问题只关心“有没有”，一个比特就够，海量数据的去重与筛法因此变得异常轻快。

5.1 有序去重：uniquify 的两个版本

有序向量的重复元素必然彼此紧邻，构成一段连续区段——这一条结构性质就是全部优化的来源。

低效版。与无序版 dedup()同为减治思路：逐个检查相邻元素，相等就删除后者：

```
template <typename T> Rank Vector<T>::uniquify_slow() {
   Rank oldSize = _size;  Rank i = 1;
   while ( i < _size )
      if ( _elem[i-1] == _elem[i] )   //若相邻者相等
         remove( i );                 //则删除后者：O(n-i)
      else i++;                       //否则继续扫描
   return oldSize - _size;
}  //O(n^2)：最坏情况下每次remove都要搬运整个后缀
```

输入全为同一元素时，$n-1$ 次 remove()平均各搬 $\frac{n}{2}$ 个后继，总计 $O(n^2)$。与第 04 章区间删除的教训相同：反复搬运同一段后缀。

高效版。既然重复者相邻，同一区段只需保留一个代表。用两个指针：i 指向已确认无重复前缀的末尾，j 为扫描指针；_elem[j]与_elem[i]相等则 j 右移跳过，不等则 i 前移一格并把新元素收编进保留区：

```
template <typename T> Rank Vector<T>::uniquify() {
   Rank oldSize = _size;  Rank i = 0, j = 0;   //i：保留区末尾；j：扫描指针
   while ( ++j < _size )                       //逐一扫描，直至j越界
      if ( _elem[i] != _elem[j] )              //发现新元素
         _elem[++i] = _elem[j];                //收编至保留区末尾之后
   _size = ++i;  shrink();                     //直接截除尾部多余元素
   return oldSize - _size;
}  //O(n)：每个元素恰好被j访问一次
```

以教材讲义的实例 $[3,3,5,5,5,5,8,8,13,13,13,13]$ 执行，逐步演示与状态对照：



图 7　逐步演示：uniquify：双指针一次扫描去重

扫描结束后向量收缩为 $[3,5,8,13]$。这里删除元素的方式不再是 `remove()`，而是整体前移覆盖：`_size` 直接截到 `i+1`，尾部旧元素无须理会。对比悬殊：同样输入，低效版要搬运几十次后缀，高效版每个元素只被访问一次。第 04 章“重复性任务批量完成”的经验，在有序结构上以这种双指针形式反复出现。

5.2 二分查找（版本 A）：轴点与减治

有序向量的每个元素都可以充当**轴点**（pivot）：以任意元素 $S[mi]$ 为界，前缀 $[lo, mi)$ 的元素都不大于它，后缀 (mi, hi) 的元素都不小于它。于是查找 e 只需三个分支：e 更小则深入前缀，e 更大则深入后缀，相等即命中。每经一次比较，候选区间宽度减半：

```
template <typename T>   //在有序区间[lo, hi)内查找e
Rank binSearch( T* S, const T& e, Rank lo, Rank hi ) {
   while ( lo < hi ) {                    //三个分支；成功可提前终止
      Rank mi = (lo + hi) / 2;            //取中位数作轴点
      if      ( e < S[mi] ) hi = mi;      //深入前缀[lo, mi)
      else if ( S[mi] < e ) lo = mi + 1;  //深入后缀(mi, hi)
      else return mi;                     //命中于mi
   }
   return -1;                             //查找失败
}  //O(logn): T(n) = T(n/2) + O(1)
```

用讲义实例执行。向量由 7 个字母按字典序排成（下标 0 至 6）：

$$S = \{\,\mathrm{B}, \mathrm{I}, \mathrm{L}, \mathrm{L}, \mathrm{O}, \mathrm{W}, \mathrm{Y}\,\}$$

命中情形 `binSearch('O')`：$mi = (0+7)/2 = 3$，$S[3]$ =L；‘O’更大（两次比较），$lo = 4$；$mi = (4+7)/2 = 5$，$S[5]$ =W；‘O’更小（一次比较），$hi = 5$；$mi = (4+5)/2 = 4$，$S[4]$ =O，两次比较后命中。经 $2+1+2 = 5$ 次元素比较，命中于[4]。**失败情形** `binSearch('F')`：$mi = 3$：F 更小，一次比较，$hi = 3$；$mi = 1$：F 既不小于 I、I 也不小于 F……准确地说$S[1]$ =I 更小，两次比较，$lo = 2$；$mi = 2$：F 更小，一次比较，$hi = 2$；$lo = hi$，循环退出。经 $1+2+1 = 4$ 次比较失败——F 若存在应位于秩 1（I）与秩 2（L）之间，但返回值-1 完全不能指示这一位置。这是版本 A 的接口缺陷，稍后由版本 B、C 弥补。

复杂度。线性递归 $T(n) = T(n/2) + O(1)$，解得 $O(\log n)$，大大优于顺序查找；递归深度 $O(\log n)$，各递归实例仅耗时 $O(1)$。**平均查找长度**（average search length, ASL）把比较次数按等概率平均。以 $n = 7$ 的向量 $S = \{\mathrm{B}, \mathrm{D}, \mathrm{F}, \mathrm{H}, \mathrm{J}, \mathrm{L}, \mathrm{N}\}$ 为例，逐个元素数成功情形的比较次数：H 命中只需 2 次（一次“小于？”加一次“大于？”皆否），B 需 $1+1+2 = 4$ 次，D 需 $1+2 = 3$ 次……七者合计 29，平均

$$\mathrm{ASL}_{succ} = 29/7 \approx 4.14$$

失败情形八种位置的平均为 4.50（教材图 2.12）。两项都符合 $O(1.50 \log_2 n)$ 的通式——每层的深入在“左侧 1 次比较”与“右侧 2 次比较”之间不均衡，这个系数 1.50 正是可以改进的地方。

5.3 Fibonacci 查找：把切分点移向黄金分割

版本 A 不均衡的根源写在递推式里：深入前缀只需 1 次比较，深入后缀却要 2 次，而两种子向量的宽度相同。改进思路之一：**调整前后宽度**，让“贵的方向”短一些、“便宜的方向”长一些。具体地，把切分点从正中移到黄金分割位置。为简化叙述，设向量长度 $n = fib(k) - 1$，取

$$mi = fib(k-1) - 1$$

则前、后子向量长度分别为 $fib(k-1) - 1$ 与 $fib(k-2) - 1$：深入后缀时新宽度 $fib(k-2) - 1$ 更短（那里每次要 2 次比较），深入前缀时 $fib(k-1) - 1$ 更长（那里只需 1 次）。这一处理可反复套用，直至命中或区间宽度收缩为零：

```
#include "fibonacci/Fib.h"       //Fib类：get()返回当前Fib项，prev()前移一项
template <typename T> static Rank fibSearch( T* A, T const& e, Rank lo, Rank hi ) {
   Fib fib( hi - lo );                          //创建不小于n的最小Fib项：O(logn)
   while ( lo < hi ) {                          //每步至多两次比较、三个分支
      while ( hi - lo < fib.get() ) fib.prev();  //向前滚动（分摊O(1)）
      Rank mi = lo + fib.get() - 1;             //按黄金比例切分
      if      ( e < A[mi] ) hi = mi;            //深入前半段[lo, mi)
      else if ( A[mi] < e ) lo = mi + 1;        //深入后半段(mi, hi)
      else return mi;                           //命中
   }
   return -1;
}  //第3版教材代码2.22
```

用 7 元素向量 $S = \{2,4,6,8,10,11,12\}$ 查找 11 走一遍：Fib 从 8 开始，$hi - lo = 7 < 8$ 滚动到 5，$mi = 0 + 5 - 1 = 4$：$S[4] = 10 < 11$，$lo = 5$；$hi - lo = 2 < 5$ 滚动到 3 再到 2，$mi = 5 + 2 - 1 = 6$：$S[6] = 12 > 11$，$hi = 6$；$hi - lo = 1 < 2$ 滚动到 1，$mi = 5 + 1 - 1 = 5$：$S[5] = 11$ 命中。探测序列 $4 \to 6 \to 5$，逐步演示：

Fibonacci查找：切分点4→6→5 第 1 / 5 步

1. 准备

fib.get() = 8；**切分点** mi = lo + fib.get() − 1 = None

2	4	6	8	10	11	12
0	1	2	3	4	5	6

初始。

lo=0, hi=7

Fib对象从不小于n=7的最小Fibonacci项8开始。

为什么：n取fib(k)−1形式时可反复套用黄金分割切分。

图 8　逐步演示：Fibonacci 查找：切分点 4→6→5

平均查找长度的对比仍以 $n = 7$ 为例：成功情形 $28/7 = 4.00$，失败情形 $35/8 = 4.38$，相对版本 A 的 4.14 与 4.50 确有改进。精确分析给出 $O(1.44 \log_2 n)$，较版本 A 的 $O(1.50 \log_2 n)$ 常系数更低。需要说明版本出处：2026 版教材的 `search()`统一接口（代码 2.6）仍随机调用 `binSearch` 与 `fibSearch`，但正文已不再展开 fibSearch 的细节，本节按第 3 版教材代码 2.22 收录。

5.4 二分查找（版本 B）：每次迭代只比一次

改进思路之二：**统一两个方向的比较次数**，都压到 1 次。代价是放弃三分支：在每个轴点只做一次比较，小则深入前缀，否则深入后缀——算法从“能提前终止”变成“必须一路收缩到区间宽度为 1”：

```
template <typename T>
Rank binSearch( T* S, const T& e, Rank lo, Rank hi ) {
   if ( e < S[lo] ) return lo - 1;        //相当于e = -∞：特判并返回-1
   while ( 1 < hi - lo ) {                //不变性：S[lo] <= e < S[hi]
      Rank mi = (lo + hi) / 2;
      ( e < S[mi] ) ? hi = mi : lo = mi; //每次迭代仅1次元素比较
   }                                      //出口时hi = lo + 1
   return lo;                             //返回不大于e的最大者
}  //2026版教材算法2.13
```

新接口的语义：无论命中与否，返回**不大于 e 的最大元素的秩**（e 小于全体元素时特判返回-1）。这个约定专为后续插入服务：`insert(1 + search(e), e)`总能把 e 插到正确的位置并保持有序。设 $S = \{A, B, ..., K\}$（11 个字母），`search('D')`：$mi = 5$，D<F，$hi = 5$（1 次比较）；$mi = 2$，D<C 不成立，$lo = 2$（1 次比较）；$hi - lo = 3$，$mi = 3$，D<D 不成立，$lo = 3$；$hi - lo = 1$ 退出，返回 3。三次比较、三次迭代，恰好都是 $\log_2 11$ 量级。而且相等元素有多个时，版本 B 总是返回其中**最靠后者**：先插入的居前、晚来者居后——这种稳定性对某些算法至关重要。同理 `search('X')`返回 10（最大者 K），于是 `insert(11, 'X')`正确追加到末尾。

5.5 二分查找（版本 C）：不变性与返回值

版本 B 仍有可调校之处：特判、出口条件、返回值各占一行。对它略作修改得到版本 C：

```
template <typename T>
Rank binSearch( T* S, const T& e, Rank lo, Rank hi ) {
   while ( lo < hi ) {                 //每步一次比较、两个分支；不能提前终止
      Rank mi = (lo + hi) / 2;
      if ( e < S[mi] ) hi = mi;        //深入前缀[lo, mi)
      else lo = mi + 1;                //深入后缀(mi, hi)
   }                                   //出口时lo == hi
   return lo - 1;                      //返回不大于e的最大者
}  //2026版教材算法2.14
```

与版本 B 的差异共四点：省掉特判；出口条件是宽度缩短至 0 而非 1；返回 `lo-1` 而非 `lo`；最关键的是分支处后缀左边界为 `mi+1`。细看会生疑：无论向哪个方向深入，轴点[mi]都被排除在外，若唯一的 e 恰是 S[mi]，岂不遗漏？有趣的恰恰在这里——这一担忧虽有情理，却无必要。版本 C 具有迭代不变性：

$$\textbf{前缀}[0, lo) \textbf{ 中的元素皆不大于 } e\text{；}\quad \textbf{后缀}[hi, n) \textbf{ 中的元素皆大于 } e$$

启动时 $lo = 0$、$hi = n$，前后缀皆空，不变性自然成立；此后每步迭代，要么后缀向前扩展（其中元素由不小于 $S[mi]$ 而由传递律大于 e），要么前缀向后拓展（其中元素不大于 $S[mi]$ 故亦不大于 e），不变性保持。终止时 $lo = hi$：前缀末元素 $S[lo - 1]$ 不大于 e，后缀首元素 $S[lo]$ 大于 e，故返回的 $lo - 1$ 确系不大于 e 的最靠后者——即便 e 本身存在于向量中。区间收缩与返回值的完整过程：

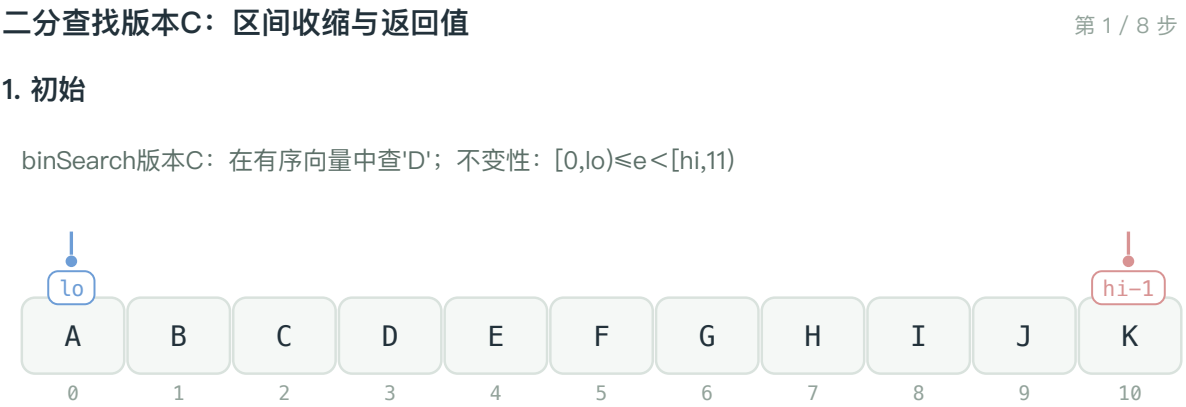


图 9　逐步演示：二分查找版本 C：区间收缩与返回值

版本 C 的调用树完美平衡，成功与失败的平均查找长度同为 $O(\log n)$，常系数已是渐近意义下的极致。

5.6 插值查找：按分布猜位置

版本 C 已经把“每次比一次”做到极致，但它仍刻板地取中位数，对数据分布不闻不问。超长的有序向量由大数定律保证往往蕴含强规律：若元素近似服从均匀分布，则各元素的秩与数值近似呈线性关系。英文词典里查“data”会直接翻到大约 2/26 处，查“structure”会翻到 19/26 附近——这就是**插值查找**（interpolation search）：每步按线性关系猜测轴点：

$$mi = lo + (hi - lo) \cdot \frac{e - S[lo]}{S[hi] - S[lo]}$$

以讲义实例执行，19 个元素 $5, 10, 12, \ldots, 92$，查找 $e = 50$：

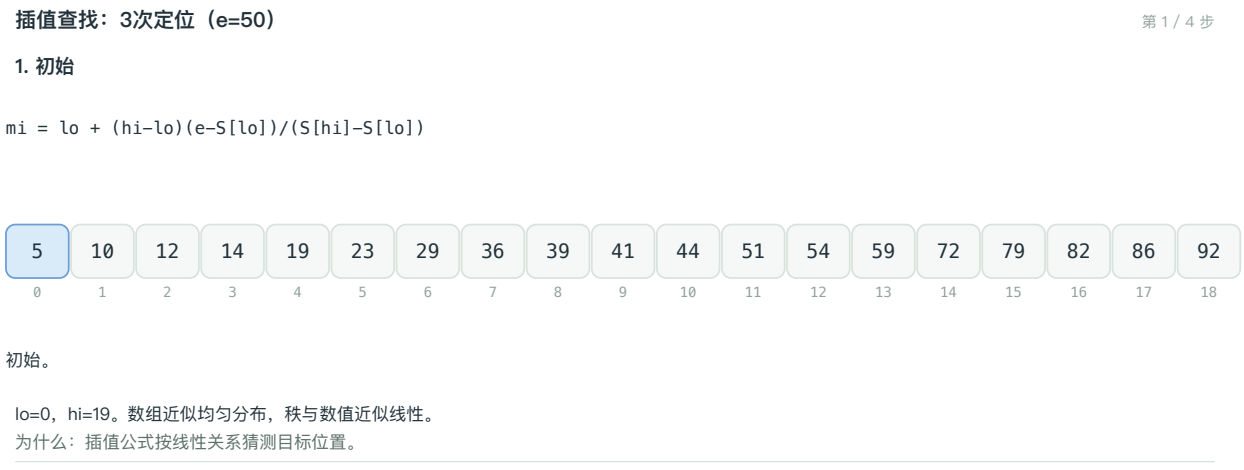


图 10　逐步演示：插值查找：3 次定位（e=50）

三次插值依次得 $mi = 9$（$S[9] = 41 < 50$，$lo = 10$）、$mi = 11$（$S[11] = 51 > 50$，$hi = 11$）、$mi = 10$（$S[10] = 44 < 50$，区间宽度归零），返回 10——不大于 50 的最大者正是 44。对照二分需要约 4 次比较，插值只用 3 次。

效率的理论结论（Perl 与 Yao 等独立证明）：对满足独立同分布均匀假设的有序向量，每经一次插值与比较，待查找区间的宽度按**平方根**的速度缩减，即从 n 缩到 $\sqrt{n}$；从位宽的角度看，就是 $\log_2 n$ 位折半为 $\frac{1}{2}\log_2 n$ 位，故总共只需

$$O(\log \log n)$$

次迭代。但别急着欢呼：一是 n 不够大时大数定律效应不明显，$O(\log\log n)$ 相对 $O(\log n)$ 的优势微乎其微（比如 $n=2^{32}$ 时 $\log_2 n=32$、$\log_2\log_2 n=5$，即便忽略插值引入的乘除法也只省了 27 次比较）；二是插值易受畸形分布的干扰，最坏 $O(n)$——取 $S=\{1,2,3,...,999,10^{12}\}$，查找 1000：每次插值算出的 mi 都只比上一次前进一格，退化成顺序扫描。实际可行的方法是**算法接力**：先插值迅速缩小范围，待区间宽度缩减到一定程度转二分，最后当数据项只剩两三百个时改用顺序查找。

5.7 位图：用一位回答“有没有”

数据结构并不总是要记住“是什么”，有时只需要回答“有没有”。判断一个整数是否存在，用向量存它要占整个元素的空间，用位图则只需**一个比特**：

```
class Bitmap {                          //位图Bitmap：空间效率极高的存在性判断结构
   uint8_t* M;  Rank n;                 //M[]为位图所存放的字节序列
   void init( Rank n ) { M = new uint8_t[(n + 7) / 8]; memset( M, 0, (n + 7) / 8 ); }
   bool test( Rank k ) { return M[ k >> 3 ] & ( 0x80 >> ( k & 0x07 ) ); }
   void set( Rank k )   { if ( !test(k) ) M[ k >> 3 ] |= ( 0x80 >> ( k & 0x07 ) ); }
   void clear( Rank k ) { M[ k >> 3 ] &= ~( 0x80 >> ( k & 0x07 ) ); }
};
```

定位规则看一个具体数就明白：$k=13$ 落在字节 $13>>3=1$ 的第 $137=5$ 位，对应掩码 $0x80>>5=0x04$：

位图：一个比特回答"有没有" 第 1 / 4 步

1. 初始：全0

字节1（位8 ~ 15）：k=13 → 字节 13>>3=1，位 13&7=5，掩码 0x80>>5 = 0x04

0	0	0	0	0	0	0	0
8	9	10	11	12	13	14	15

位图清零。

24个比特分3个字节存放，初始全0。
为什么：每一位只回答一个"有没有"。

图 11　逐步演示：位图：一个比特回答“有没有”

5.8 位图应用：海量去重与筛法

海量去重。设需对 10^{10} 个 24 位整数去重：每个整数的取值范围为 $[0,2^{24})$，直接开位图需要 2^{24} 比特即 2 MB——而把全部整数存进向量需约 40GB。逐个读入整数，test 判重、set 登记，一遍流式扫描即可完成去重，时间 $O(10^{10})$、辅助空间仅 2MB。

筛法。Eratosthenes 筛法求 $[2,n)$ 内的全部素数，是位图最经典的应用：

```
void Eratosthenes( const Rank n ) {
   Bitmap B( n );  B.set( 0 );  B.set( 1 );        //0、1不是素数
   for ( Rank i = 2; i < n; i++ )                  //从2起逐个
      if ( !B.test( i ) )                          //确认i为素数后
         for ( Rank j = i * i; j < n; j += i )     //从i^2起，以i为步长
            B.set( j );                            //筛除其倍数
   //此时test(i)为假者即素数
}
```

内循环从 i^2 起步是关键的小优化：更小的倍数 $i \times k$（$k < i$）必含更小的质因子，已在更早的轮次被筛除。以 $n = 30$ 为例完整走一遍：

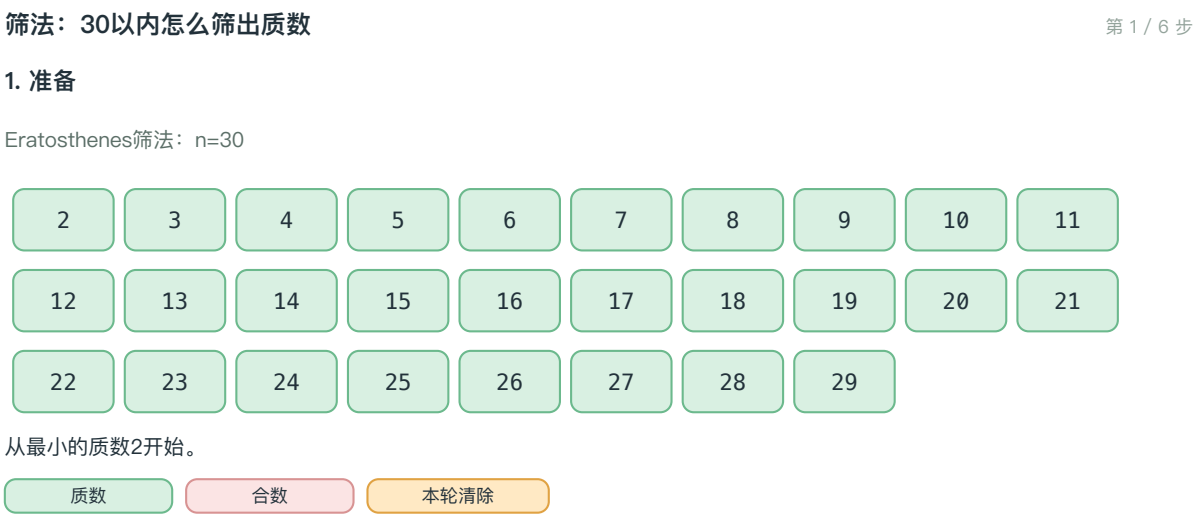


图 12　逐步演示：筛法：30 以内怎么筛出质数

筛完剩下 10 个素数：2, 3, 5, 7, 11, 13, 17, 19, 23, 29。复杂度上，内循环的启动次数为各素数各一次，每次迭代 $\lfloor n/p \rfloor$ 步，总次数 $\sum_{p \text{ 为素数}} n/p$ 由素数定理约为 $n \ln \ln n$——几乎线性的时间。

5.9 位图的快速初始化：校验环

位图还有一个隐蔽的成本：`init()`里的 `memset()`要把全部字节清零，耗时 $O(n)$。当 n 达到 2^{32} 时，仅清零就要耗去半 GB 内存的搬运时间，而后续真正的计算可能远比这轻。能否让“初始化”也降到 $O(1)$？技巧是校验环（Hopcroft, 1974）：不再依赖位图本身的可信性，而是用两张登记表交叉验证：

```
class Bitmap {                          //借助校验环快速初始化的位图
   Rank* F;  Rank* T;  Rank size;       //F[k]登记k的入场顺序，T[]为按序登记的名单
   bool test( Rank k ) { return ( F[k] < size ) && ( T[ F[k] ] == k ); }
   void set( Rank k ) { if ( !test(k) ) { F[k] = size;  T[size++] = k; } }
   void clear( Rank k ) {
      if ( test(k) && --size ) {        //一般情况：令校验环k，与拖尾的校验环交换
         F[ T[size] ] = F[k];  T[ F[k] ] = T[size];
      }
   }
   void reset() { size = 0; }           //O(1)整体重置
};
```

test 的两道关卡缺一不可：`F[k] < size` 只说明 k 曾登记过——若 F 是新分配的内存，其中的垃圾值可能碰巧小于 size，所以还要第二关 `T[F[k]] == k` 确认登记名单里 k 的座位号确实写着 k。clear 的交换删除保持名单紧凑：把被删元素与名单末尾交换，size 减一。走一遍小例子：`set(1)`、`set(4)`、`set(8)`后 $T = [1, 4, 8]$、

$F[1]=0$、$F[4]=1$、$F[8]=2$、$size=3$；`clear(4)`：$size$ 减到 2，交换使 $T=[1,8]$、$F[8]=1$、$F[4]=1$（陈旧值无害）——此后 `test(4)`两关分别为真、假，报告 false；`test(8)`两关皆真，报告 true。整套操作除登记表本身外无需清零任何空间，`reset()`更是一行 `size = 0`：O(1)。

5.10 本章要点

- 有序向量的重复元素相邻：uniquify 双指针一次扫描$O(n)$，无序 dedup 需$O(n^2)$。
- 二分查找版本 A 三个分支、可提前终止，但深入前后代价 1:2 不均衡，ASL $O(1.50\log_2 n)$，且失败时-1 不指示位置；BILLOWY 实例：‘O’命中 5 次比较、‘F’失败 4 次。
- Fibonacci 查找按黄金分割切分（$mi=fib(k-1)-1$），ASL 降至 $O(1.44\log_2 n)$；n=7 实例探测序列 4⟶6⟶5。
- 版本 B 每次迭代一次比较，返回不大于 e 的最大者（特判 e<S[lo]）；版本 C 去掉特判、出口宽度为 0、返回 lo−1，靠不变性“[0,lo)≤e<[hi,n)”保证正确，且杜绝了“轴点被排除”的担忧。
- 插值查找按秩与数值的线性关系猜轴点，均匀分布下 $O(\log\log n)$；畸形分布退化 $O(n)$；实际用插值⟶二分⟶顺序的三级接力。
- 位图用一比特回答存在性：test/set/clear 皆 O(1)；海量去重只需 2MB 筛 10^10 个 24 位整数；Eratosthenes 筛法 $O(n\log\log n)$，内循环从 i^2 起步。
- 校验环位图把初始化降为 O(1)：F/T 双表交叉验证，test 两道关卡缺一不可，clear 用交换删除保持名单紧凑。

6 起泡排序与归并排序

第 01 章已经完整论证过起泡排序的正确性，第 04 章把它集成进向量 ADT。本章把这个最朴素的排序算法打磨到极致——提前终止、区间跳跃，并借它引出排序算法的一个重要品质：稳定性。随后请出本章的主角归并排序：由冯·诺依曼于 1945 年首次在数字式电子计算机 EDVAC 上编程实现，第一个在最坏情况下仍保持 $O(n \log n)$ 的排序算法。它的分治框架、“前缀复制、后缀直用”的归并实现，以及顺带统计逆序对的额外收获，都是后文反复引用的基础。

6.1 排序器：一个接口，多种实现

向量的 `sort()`接口只有一个，背后的实现却有多种。2026 版教材代码 2.7 把它们收拢在一个统一入口：

```
template <typename T> void Vector<T>::sort( Rank lo, Rank hi ) {
   if ( lo + 2 > hi ) return;       //退化情况，统一处理
   switch ( rand() % 6 ) {          //随机选择算法以充分测试
      case 1 :  bubbleSort( lo, hi ); break;     //起泡排序
      case 2 :  selectionSort( lo, hi ); break;  //选择排序（习题）
      case 3 :  mergeSort( lo, hi ); break;      //归并排序
      case 4 :  heapSort( lo, hi ); break;       //堆排序（第12章）
      case 5 :  quickSort( lo, hi ); break;      //快速排序（第14章）
      default : shellSort( lo, hi ); break;      //希尔排序（第14章）
   }
}
```

测试阶段随机挑选算法，以便用同一批输入检验各种实现的正确性与效率；实际应用中则视问题特点灵活取舍。本章完成其中的起泡排序与归并排序，选择排序留作习题，其余分别在堆、快排等章节介绍。

6.2 起泡排序：基本版

第 01 章的版本只需套上向量模板的区间约定，即成基本版：

```
template <typename T>
void Vector<T>::bubbleSort( Rank lo, Rank hi ) {      //0 <= lo < hi <= _size
   while ( lo < --hi )                 //逐趟起泡扫描：最大元逐个就位
      for ( Rank i = lo; i < hi; i++ )
         if ( _elem[i] > _elem[i+1] ) //若逆序
            swap( _elem[i], _elem[i+1] );   //则交换
}  //算法2.15，基本版
```

它的比较次数对任何输入都恒为 $n(n-1)/2$（第 01 章已用 $n = 7$ 时的 21 次验证过），连输入已经有序的情况也不省一分力气——$\Theta(n^2)$，不因输入的好坏而改变。可实际上，不同输入的混乱程度毕竟有别：如果扫描到一半发现前缀已然有序，本该提前收工。

6.3 提前终止版：一个标志位

怎样及时发现“已然有序”，同时总体复杂度不增？用一个逻辑标志位 `sorted`：每趟扫描开始时置为 true，一旦发现并交换逆序对就置回 false；某趟结束时若它仍是 true，说明整趟未发现任何逆序，前缀已经有序：

```
template <typename T>
void Vector<T>::bubbleSort( Rank lo, Rank hi ) {
   for ( bool sorted = false; sorted = !sorted; hi-- )  //逐趟扫描，直至不再交换
      for ( Rank i = lo + 1; i < hi; i++ )
         if ( _elem[i-1] > _elem[i] )
            { swap( _elem[i-1], _elem[i] );  sorted = false; }
}  //算法2.16，提前终止版
```

循环条件 `sorted = !sorted` 一举两得：进入循环前将标志取真（本趟先假定有序），趟末再据实况翻转。对完全有序的输入，一趟扫描 $n-1$ 次比较后即停，最好情况 $O(n)$；最坏情况仍是 $O(n^2)$。

6.4 跳跃版：last 记住最后一次交换

提前结束的机会毕竟难得，更常见的情形是“扫到中途，尾部已经就位”。观察一条规律：**前一趟扫描中，若最后一次交换发生在`[last-1]`与`[last]`之间，则后缀`[last, hi)`必然全部就位**——那次交换之后，再没有任何元素被挪动过，而每趟扫描恰恰把当时的最大元推到了扫描末端。于是下一趟可以跳过整个后缀，直接对`[lo, last)`继续扫描：

```
template <typename T>
void Vector<T>::bubbleSort( Rank lo, Rank hi ) {
   for ( Rank last; lo < hi; hi = last )          //确定待扫描区间后，跳跃收缩
      for ( Rank i = ( last = lo ) + 1; i < hi; i++ )
         if ( _elem[i-1] > _elem[i] )             //若逆序
            { swap( _elem[i-1], _elem[i] );  last = i; }  //交换，并记住位置
}  //算法2.17，跳跃版
```

用具体数组执行一遍。取

$$A = \{2, 1, 3, 5, 4, 6, 7\}$$

输入只有两处逆序（2>1 与 5>4），正是跳跃版大显身手的场面：

跳跃版起泡：last怎样让区间跳跃收缩 第 1 / 12 步

1. 初态

跳跃版：本趟扫描区间[lo, hi)=[0, 7)，last=0



last每趟初始化为lo。

lo=0，hi=7。输入仅两处逆序：2>1与5>4。

为什么：跳跃版的看点：一趟扫描能一次跳过多少尾部。

图 13　逐步演示：跳跃版起泡：last 怎样让区间跳跃收缩

第 1 趟扫描：交换(2,1)时 `last=1`，交换(5,4)时 `last=4`；此后直到扫描结束再无交换。于是 `hi = last = 4`，第 2 趟只扫`[0,4)`——后缀`[4,7)`中的 5、6、7 被整段跳过。第 2 趟全无交换，`last` 保持 0，`hi=0`，循环退出。全程共 $6 + 3 = 9$ 次比较；而基本版需要恒定的 21 次。每趟的时间消耗正比于本趟的实际扫描宽度，乱得越局部，跳得越多。

6.5 稳定性：相等元素的次序

时间效率之外，排序算法还有一项更细致的品质要求——**稳定性**（stability）：相等的元素在排序之后，能否保持其在输入序列中的相对次序。看讲义的例子，输入中三个 7 带上标以示区分：

```
输入：6, 7a, 3, 2, 7b, 1, 5, 8, 7c, 4
稳定输出：1, 2, 3, 4, 5, 6, 7a, 7b, 7c, 8
不稳定输出：1, 2, 3, 4, 5, 6, 7a, 7c, 7b, 8
```

三个 7 在数值上无法区分，但它们身上附带的 satellite 信息（比如三条同名记录的原始先后）会因次序是否保持而有截然不同的含义。多关键字排序正是以此为基础：先按次关键字排一遍，再用稳定的算法按主关键字排序，次关键字的次序便自动保留。

本章的三个起泡排序版本都是稳定的，原因有两条，缺一不可：交换只发生在相邻元素之间，绝无跨距离的对调；比较一律使用严格不等号>而非>=，相等的元素根本不会触发交换。稍后将看到，归并排序同样可以做到稳定，但需要实现上的刻意安排。

6.6 归并排序：分而治之

起泡排序无论如何优化，最坏情况仍是 $\Theta(n^2)$；归并排序则无论如何都不超过 $O(n \log n)$，属于最坏情况最优的排序算法。它的框架正是第 03 章的分而治之，三步：均分为前缀、后缀（$O(1)$）；递归排序子序列（$2T(n/2)$）；合并有序子序列（$O(n)$）。前两步毫无技巧可言，实质的计算全在最后的归并（merge）：

```
template <typename T>
void Vector<T>::mergeSort( Rank lo, Rank hi ) {   //0 <= lo < hi <= _size
   if ( hi - lo < 2 ) return;                     //单元素区间自然有序，递归基
   Rank mi = ( lo + hi ) / 2;                     //均分为前缀、后缀
   mergeSort( lo, mi );  mergeSort( mi, hi );     //各自递归排序
   merge( lo, mi, hi );                           //归并：实质的计算，O(n)
}  //算法2.18，O(nlogn)
```

具体实例的宏观过程：长度 8 的输入 $[8,7,6,5,4,3,2,1]$ 被逐层均分，直到 8 个单元素子向量（递归基，天然有序）；随后相邻的有序子向量捉对合并，有序段长度自 1、2、4 而 8，数量自 8、4、2 而 1，最终合成完整的有序向量。

6.7 二路归并：前缀复制，后缀直用

归并怎么做？要把两个有序子向量就地合并，最大的障碍是合并会覆盖尚未读取的元素。实现采取“半复制”策略：把前缀复制出来备用，后缀则留在原地——因为输出 S 从 lo 开始写入，最早只会写到后缀的起始位置，后缀中尚未归入的元素天然不会被冲掉：

```
template <typename T>
void Vector<T>::merge( Rank lo, Rank mi, Rank hi ) {  //归并[lo,mi)与[mi,hi)
   Rank s = 0;  T* S = _elem + lo;                    //就地归并至S[0, hi-lo)
   Rank a = 0, la = mi - lo;  T* A = new T[la];       //复制前缀A
   for ( Rank i = 0; i < la; i++ ) A[i] = S[i];
   Rank b = 0, lb = hi - mi;  T* B = S + la;          //后缀B直接使用
   while ( ( a < la ) && ( b < lb ) )                 //反复比较A、B的首元素
      S[s++] = ( A[a] <= B[b] ) ? A[a++] : B[b++];    //小者优先归入S
   while ( a < la ) S[s++] = A[a++];                  //B先耗尽：A残余整体归入
   delete [] A;
}  //若A先耗尽，B残余本就在S中，无需任何搬动
```

教材的注释里埋了一道思考题：“若 A 先耗尽呢？”答案是：无需任何处理。B 的残余本就排在 S 的尾部，位置不变、次序不变。逐步演示看一次完整的归并——$A = [1,3,6,7]$ 与 $B = [2,4,5,8]$ 合并：

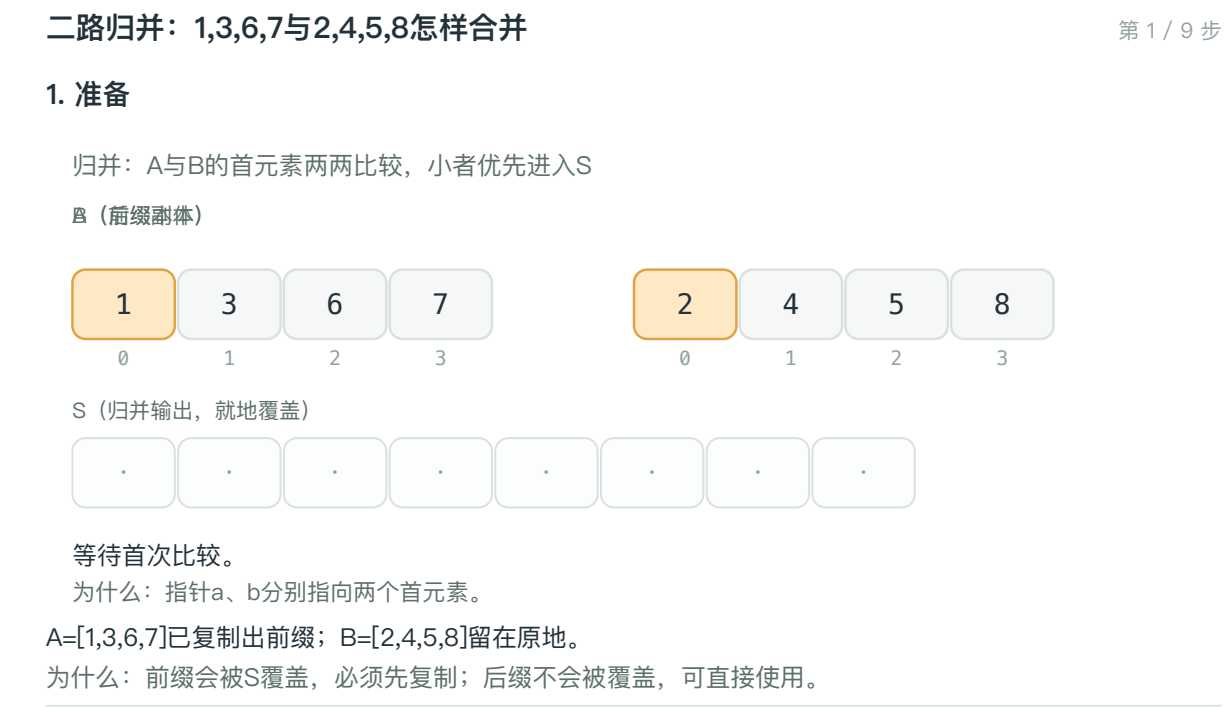


图 14　逐步演示：二路归并：1,3,6,7 与 2,4,5,8 怎样合并

九步之后 $S=[1,2,3,4,5,6,7,8]$。正确性一目了然：每次归入 S 的都是两个有序序列当前首元素中的较小者，故 S 始终有序。注意取等号时优先取 A——这一点正是归并排序稳定性的保证。复杂度方面，无论过程如何，$a+b$ 终将到达 $la+lb=n$，每步输出恰好一个元素，归并必为线性；代回主定理，

$$T(n)=2T(n/2)+O(n)\Longrightarrow O(n\log n)$$

6.8 综合评价：优点与不足

归并排序的各项品质值得逐条盘点，它们直接决定使用场景。

优点：不需随机读写，完全顺序访问——尤其适用于列表之类的序列、磁带之类的设备；只要实现恰当，可保证稳定；可扩展性极佳，十分适宜于外部排序；同层子任务彼此独立，易于并行化。作为最坏情况最优的首个排序算法，它的 $O(n\log n)$ 上界不依赖输入的任何运气。

不足：反复 `new/delete` 复制前缀耗时——可优化为整个排序只做一次分配；需要 $O(n)$ 辅助空间——适当多花时间，可改造为就地版本；即便输入已接近甚至完全有序，仍需 $\Omega(n\log n)$ 时间——对比起泡提前终止版的 $O(n)$，这是为最坏情况保障付出的代价；序列足够短之后，递归的效率骤降——适时变招，改用起泡排序一类的简单算法。这些“不足”没有一个是死穴，每一项都有对应的工程解法，代价是代码复杂度。

6.9 逆序对：归并的另一份收获

排序的交换次数与序列的**逆序对**（inversion）数目直接相关：逆序对指满足 $i<j$ 且 $V[i]>V[j]$ 的下标对。相邻逆序对（只统计相邻位置）用一个遍历函数对象即可数出——第 04 章的 `Compare` 就是现成的实现，计数为零当且仅当向量整体有序。全局逆序对的统计也有一个漂亮的做法：在归并的过程中，每当 B 的首元素小于 A 的首元素而被归入 S 时，A 中剩余的 $la-a$ 个元素都与这个 B 元素构成逆序对，累加即可。这样统计全部逆序对的时间与归并排序同阶，为 $O(n\log n)$——它正是教材的一道习题，也是 PS 题目的常用工具；逐对枚举的蛮力做法需要 $O(n^2)$。换句话说，归并的过程本身就是一趟逆序对的清点：排序与计数，一次完成。

6.10 本章要点

- `sort()`统一入口下有多种实现（起泡、选择、归并、堆、快排、希尔），退化区间统一处理。

- 起泡排序基本版恒为 $\Theta(n^2)$；提前终止版以 sorted 标志实现最好 $O(n)$；跳跃版以 last 记录最后一次交换位置，令后缀整体跳过——断言`[lo,last) < [last] <= (last,hi)`。实例 $\{2,1,3,5,4,6,7\}$ 只需 $6+3=9$ 次比较。
- 稳定性要求相等元素保持输入次序；起泡排序稳定的两个前提：只交换相邻元素、比较用严格不等号。多关键字排序依赖这一性质。
- 归并排序 $T(n)=2T(n/2)+O(n)=O(n\log n)$，最坏情况最优；二路归并“前缀复制、后缀直用”，A 先耗尽时 B 残余无需搬动；取等号优先 A，保证稳定。
- 归并的优点：顺序访问、稳定、宜外部排序、易并行；不足：辅助空间 $O(n)$、对有序输入无优惠、短序列递归低效——各有工程解法。
- 相邻逆序对可由遍历函数对象 $O(n)$ 统计；全局逆序对在归并中顺带统计为 $O(n\log n)$，蛮力法则需 $O(n^2)$。

7 列表：循位置访问的链表实现

向量与列表同属序列（sequence）：元素按线性次序排列，各有秩，相邻者互称前驱、后继。差别在于一条设计决策——**循秩访问**把元素的物理地址与其逻辑次序绑定在一起，因此向量擅长静态操作（按下标读取、二分查找），动态操作却显吃力：插入、删除动辄搬迁整个后缀。**列表**（list）把这条绑定剪断：各元素的物理次序不再与逻辑次序直接相关，逻辑上紧邻的一对前驱与后继，物理上既不必保持先后，更未必相互紧邻。作为补偿，每一对逻辑紧邻的元素彼此记录对方的物理位置——**链接**（link）。链接抽象为元素的**位置**（position）之后，访问方式从循秩换成**循位置访问**（call-by-position）：正如借助名片由某位朋友找到他的亲戚，进而同事、同事的战友、战友的同学——循着位置，常数时间内从一个元素转入下一个，即可抵达列表中的任何一个元素。动态操作只涉及局部的常数个元素，与全局规模无关，这就是列表多数动态操作可在 $O(1)$ 完成的机制保证。

7.1 节点与链表：两级封装

列表的典型实现是**链表**（linked list），分两级封装。对应于列表的元素，基本单位是**节点**（node）：既封装数据项 `data`，也以指针记录前驱、后继节点的位置：

```
template <typename T> using LNP = ListNode<T>*;   //列表节点位置
template <typename T> struct ListNode {           //简洁起见，完全开放而不再严格封装
   T data;  LNP<T> pred, succ;                    //数值、前驱、后继
   ListNode() : data(), pred( nullptr ), succ( nullptr ) {}
   ListNode( const T& e, LNP<T> p = nullptr, LNP<T> s = nullptr )
      : data( e ), pred( p ), succ( s ) {}        //类T须已定义复制方法
   LNP<T> insertPred( const T& e );               //前插入
   LNP<T> insertSucc( const T& e );               //后插入
   void movePred( LNP<T> s );                     //转移至s之前
   void moveSucc( LNP<T> p );                     //转移至p之后
};
```

其上再封装列表模板类。对外接口与向量 ADT 几乎一致（`size/empty/first/last/insert/remove/sort/find/search/dedup/uniquify/traverse`），内部则私有持有规模`_size`与一对**哨兵节点** `head`、`tail`——二者外部不可见、不存数据，却让首、末、中间节点处于完全同等的地位：首节点的秩可理解为 0，末节点为 $n-1$，头哨兵为 -1，尾哨兵为 n。初始化只做一件事：让两个哨兵互指：

```
template <typename T> void List<T>::init() {
   head = new ListNode<T>;  tail = new ListNode<T>;
   head->succ = tail;  tail->pred = head;  _size = 0;
}
```

7.2 模仿循秩访问：能用，但勿多用

为了兼容旧代码，列表也能模仿循秩访问——但代价立现：

```
template <typename T> LNP<T> List<T>::operator[]( Rank r ) const
{ return *first() + r; }                  //O(r)：虽方便，勿多用
template <typename T> LNP<T> ListNode<T>::operator+( Rank r ) {
   for ( LNP<T> p = this; ; p = p->succ, r-- )   //从当前节点出发
      if ( 0 == r ) return p;                    //顺数第r个节点即目标
}  //秩 == 前驱的总数
template <typename T> Rank ListNode<T>::operator-( LNP<T> h ) {   //h <= this
   for ( Rank d = 0; ; h = h->succ, d++ )
      if ( h == this ) return d;                 //区间内节点挨个报数
}  //O(n)
```

`V[r]`在向量上是 $O(1)$，在列表上要数 r 步；两个位置相减（求秩差）同样要 $O(n)$。列表的用法应当是：拿到一个位置之后，就沿着位置前进，不回头换算秩。

7.3 无序表的插入、删除与移动

插入接口有四个对外形态，内部全部落在节点的 insertPred（或对称的 insertSucc）上：

```
LNP<T> List<T>::insert( const T& e, LNP<T> p ) { _size++;  return p->insertPred( e ); }  //e作p的前驱
LNP<T> List<T>::insert( LNP<T> p, const T& e ) { _size++;  return p->insertSucc( e ); }  //e作p的后继
LNP<T> List<T>::insertFirst( const T& e ) { _size++;  return head->insertSucc( e ); }    //e作首节点
LNP<T> List<T>::insertLast( const T& e ) { _size++;  return tail->insertPred( e ); }     //e作末节点

template <typename T> LNP<T> ListNode<T>::insertPred( const T& e ) {
   LNP<T> x = new ListNode( e, pred, this );   //创建新节点（pred或为head）
   pred->succ = x;  pred = x;  return x;       //列表接纳新节点（操作次序不可颠倒）
}
```

前插入共三根指针的写入，次序是刻意安排的：第一步构造新节点时就把 X 的两根外向指针接好；第二步 pred->succ = x 接通前驱的正向链；第三步 pred = x 补齐后继的反向链。逐步演示能看清为什么次序不能颠倒——中途任何一步停下，链表都保持可达：

前插入：三根指针的次序 第 1 / 4 步

1. 初态

初态



尚无动作。

为什么：新节点此刻还不存在。

链表片段A → B。任务：把新节点X插入B之前。

为什么：insertPred需要改动三处指针，次序不可颠倒。

图 15　逐步演示：前插入：三根指针的次序

删除是对称的操作，两根指针让前后节点互相绕过被删者：

```
template <typename T> T List<T>::remove( LNP<T> p ) {
   T e = p->data;                                  //备份数据项
   p->pred->succ = p->succ;  p->succ->pred = p->pred;   //pred|succ或为head|tail
   delete p;  _size--;  return e;                  //释放节点，更新规模，返回数据项
}
```

删除节点：让两侧指针绕过它 第 1 / 3 步

1. 初态

remove(p)：让两侧指针绕过被删节点



尚未动作。

为什么：被删节点的pred、succ分别指向A与C。

链表片段A → p → C。要删除p。

为什么：p的两侧指针必须先改，否则后续节点失联。

图 16　逐步演示：删除节点：让两侧指针绕过它

移动（move）是插入与删除的组合拳，却只需 $O(1)$：

```
template <typename T> void ListNode<T>::movePred( LNP<T> s ) {   //s != this
   pred->succ = succ;  succ->pred = pred;                //取出当前节点
   pred = s->pred;  pred->succ = this;  succ = s;  s->pred = this;   //转移至紧靠于s之前
}  //moveSucc(p)完全对称，O(1)
```

这类操作是向量完全不具备的：把一个元素搬到任何地方，列表只动常数根指针。

7.4 复制与析构

深复制（deep copy）需递归复制蓝本的每一层嵌套结构。列表版本的思路朴素：从蓝本区间出发，逐节点 `insertLast` 到新列表：

```
void List<T>::copy( LNP<T> p, Rank n ) {          //复制蓝本区间[p, p+n)
   for ( init(); n--; p = p->succ )               //先建空表
      insertLast( p->data );                      //依次复制，O(1)每个
}
```

耗时线性正比于节点总数，以及每个 T 对象深复制所需的时间。**析构**反过来逐个释放：

```
template <typename T> List<T>::~List() {
   for ( LNP<T> x = head; x != tail; delete ( x = x->succ )->pred );
   delete tail;
}
```

一道思考题随课件附上：若换成只有 succ 指针的单向链表，又当如何析构？——提示：每删一个节点之前，先把它的 succ 存下来。

7.5 无序查找与去重

无序查找沿用第 04 章的哨兵技巧，只是从下标换成位置：

```
template <typename T>   //在无序列表区间(h, t)内，找到等于e的最靠前者
LNP<T> List<T>::find( LNP<T> h, const T& e, LNP<T> t ) const {   //h < t
   T bak = t->data;  t->data = e;                //预置哨兵
   while ( e != ( h = h->succ )->data );         //自前向后逐个比对
   t->data = bak;  return h == t ? nullptr : h;  //命中或出界
}  //O(n)
```

去重与向量版同为减治：对每个节点 p，在前缀中查找相等者，命中即删除。复杂度的表达却多了一层细腻——

```
template <typename T> void List<T>::dedup() {     //O(n^2)
   for ( LNP<T> p = first(); p != tail; p = p->succ )   //O(n)
      if ( LNP<T> q = find( head, p->data, p ) )         //O(n)
         remove( q );                                    //O(1)!
}
```

最坏情况（比如元素互异）为 $O(n^2)$，与 `Vector::dedup()`一样；最好情况（比如全等）只需 $O(n)$——得益于摆脱了循秩访问的窠臼：向量的 `remove(i)`每次要搬移 $O(n-i)$ 个后继，列表的 `remove(q)`只动常数根指针。

7.6 遍历：函数指针与函数对象

遍历接口与向量一致，沿 `succ` 链走一遍即可；统一操作同样以函数指针或函数对象传达。第 04 章的三个例子原样可用：`Increase` 统一加一、`Cumulate` 借助外部累加器统计总和、`Compare` 每访问一个元素便与前驱比较，累加相邻逆序对——计数为零当且仅当列表整体有序。

7.7 有序列表：去重与查找

摒弃循秩访问换来的动态操作红利，在有序列表上继续兑现；但凡事一利一弊，静态操作的性能可能反降。

去重：重复元素必然紧邻，一趟扫描即可：

```
template <typename T> void List<T>::uniquify() {   //只需遍历一趟，O(n)
   if ( _size < 2 ) return;                         //过短的列表自然无重复
   for ( LNP<T> p = first(), q = p->succ; q != tail; q = p->succ )
      if ( p->data != q->data )                     //p与其后继q不等
         p = q;                                     //则前进一步
      else                                          //相等：直接删除后者
         remove( q );                               //不必如向量那样间接删除
}
```

对比第 05 章向量版 uniquify 的双指针搬运——列表的物理删除本就是 $O(1)$，无需保留区挤压的技巧。

查找：有序列表的 `search` 找不到比 $O(n)$ 更好的办法——没有随机访问，二分无从谈起。课件版本自后向前扫描并预置哨兵：

```
template <typename T>   //有序列表区间(h, t)：等效于h->data == -∞
LNP<T> List<T>::search( LNP<T> h, const T& e, LNP<T> t ) const {
   T bak = h->data;  h->data = e;                //预置哨兵
   while ( e < ( t = t->pred )->data );          //自后向前逐个比较
   h->data = bak;  return t;                     //不大于e的最靠后者
}  //O(n)
```

7.8 列表排序三例：选择、插入、归并

列表的排序器同样随机挑选算法测试，本节择要讲三个：选择排序、插入排序与归并排序——它们恰好构成“从 $O(n^2)$ 到 $O(n\log n)$、从搬数据到搬指针”的一条演进线。

选择排序。温故而知新：起泡排序每趟要做多次交换，但那趟扫描的实质无非是找出当前最大元素 m 并令其就位——经 $n-1$ 次比较确定 m 之后，其实一次交换就足够了。列表版把这一实质挑明：

```
template <typename T> void List<T>::selectionSort( LNP<T> h, LNP<T> t ) {   //h < t
   for ( LNP<T> x = t->pred; h->succ != x; t = x, x = t->pred )   //无序前缀(h, x]
      swap( getMax( h, t )->data, x->data );   //找出最大者，交换令其就位
}   //亦可 getMax(h,t)->movePred(t)：同为O(1)，且能保证稳定性

template <typename T> LNP<T> List<T>::getMax( LNP<T> h, LNP<T> t ) {   //2 <= t-h
   LNP<T> m = t->pred;                          //当前最大者
   for ( t = m->pred; h != t; t = t->pred )     //自后向前逐个比较
      if ( m->data < t->data ) m = t;           //见贤思齐：严格更大才更新
   return m;                                    //即便有多个，也总取最靠后者
}  //O(n)：自后向前 + 严格比较 = 稳定性
```

性能上，n 步迭代：`swap()`累计 $O(n)$，`getMax()`累计 $O(n^2)$——对任何输入都没有最好情况，主要消耗来自比较。好的一面：移动操作只需 $O(n)$ 次，远少于起泡排序；而就常系数而言，移动远贵于比较，实际收益可观。两条改进线索课件已经点出：`getMax()`若借助高级数据结构可降到 $O(\log n)$，立即得到 $O(n\log n)$ 的排序算法（第 12 章的堆排序正是这条路）；原地交换的浪费可借助**轮换**（cycle）视角精确刻画——任何排列都可分解为若干轮换（最多 n 个、最少 1 个），交换前 m 与 x 同属一个轮换 C 且前后紧邻，交换后 m 从 C 中逸出自成轮换、C 长度减一：C 含 k 个元素就会出现 k 次原地交换。对互异元素的随机排列，期望而言原地交换的比例并不高，却也无法根除。

插入排序。减而治之的又一形态：序列永远分成有序前缀 $S[0,r)$ 与待排序后缀 $U[r,n)$；反复取出后缀首元素 e，在 S 中查得适当位置并插入。列表版把这个过程写成一行：

```
template <typename T>   //逐一引入节点r，持续拓展有序前缀(h, r)
void List<T>::insertionSort( LNP<T> h, LNP<T> t ) {
   for ( LNP<T> r = h->succ, s = r->succ; r != t; r = s, s = r->succ )
      r->moveSucc( search( h, r->data, r ) );   //将[r]转移至(h,r)内的适当位置
}  //仅使用O(1)辅助空间：就地算法
```

得益于 `search()`“返回不大于 e 的最靠后者”的接口语义，前缀总是保持有序，而且稳定——前缀中与 r 相等的元素都在 r 之前，r 被安插到它们之后。可自行验证几种边界：前缀中有元素与 r 相等；r 在前缀中最小或最大；前缀为空。复杂度是**输入敏感**的（input-sensitive）：为插入 $e=A[r]$，比较次数恰为逆序对记账数 $I(r)$ 加 1，总体比较 $\Theta(n+I)$、移动 $O(n)$——序列含多少逆序对，就要花多少时间，正如衣物有多脏，洗涤就用多久。顺带回答一个自然的问题：前缀总是有序的，为何不二分？**有实质的差异**：`binSearch()`抵达中点本身在列表上就是 $O(r)$，省下的比较全贴进了寻址。最后一个特质是**在线**（online）：打牌的人都是边发边理牌，发完即理好——插入排序在数据完全就绪之前即可开始计算，这个能力多数排序算法并不具备。

归并排序。列表版的分治主算法与向量版同构，只是切分要靠位置运算：

```
template <typename T> void List<T>::mergeSort( LNP<T> h, LNP<T> t ) {   //h < t
   Rank n = *t - h - 1;  if ( n < 2 ) return;      //区间长度不足2，递归基
   LNP<T> x = *h + n / 2;                          //切分：(h,t) = (h,x] + (x,t)
   mergeSort( x, t );  mergeSort( h, x = x->succ );//后缀、前缀各自排序
   merge( h, x, t );                               //归并
}  //归并线性 → 总体O(nlogn)
```

列表的归并不再复制前缀——搬指针即可。实质上它是**增量式的插入排序**：逐个把后缀节点转移进前缀的适当位置，小者优先（相等者优先取前缀侧）以保证稳定：

```
template <typename T>   //将[y,t)中的节点，分别转移至(h,y)中的适当位置
LNP<T> List<T>::merge( LNP<T> h, LNP<T> y, LNP<T> t ) {
   for ( LNP<T> x = h->succ; ( x != y ) && ( y != t ); )
      if ( x->data > y->data )                    //y更小：转移到x之前
         ( y = y->succ )->pred->movePred( x );
      else                                        //否则x自然前进
         x = x->succ;
}  //O(n)，线性正比于节点总数
```

若归并可在线性时间内完成，整体运行时间即 $O(n \log n)$。

7.9 逆序对：从计数到归并统计

逆序对 (inversion)：若 $i < j$ 且 $S[i] > S[j]$，则 (i, j) 构成一个逆序对。为便于统计，统一把逆序对记到后者 j 的账上：$I(j)$ = j 之前大于 $S[j]$ 的元素个数。讲义的三组实例：$\{5,3,1,4,2\}$ 各账目为 $0+1+2+1+3$，共 7 个；完全有序为 0；完全逆序为 $0+1+2+3+4=10=n(n-1)/2$，是上界。逆序对总数与序列的"乱度"精确对应：交换一对逆序元素，逆序对至少减 1（相距越远的逆序对，一次交换期望修复越多）。由此重审本章算法：起泡排序每次交换的都是紧邻逆序对，逆序对只能逐一修复，最坏必然 $O(n^2)$；插入排序针对 $e = A[r]$ 的迭代恰做 $I(r)+1$ 次比较，若全序列含 I 个逆序对，总比较 $\Theta(n+I)$——它的时间就是序列脏污程度的直接读数。全局逆序对的统计也不必蛮力 $O(n^2)$：套用归并排序的框架，把逆序对按位置分成偏于前缀、偏于后缀、跨界三类，前两类递归统计；实质的跨界统计利用一个不变事实——假想归并排序进行至后缀元素 j 即将出列的时刻，前缀中剩下的恰是所有与 j 构成跨界逆序的元素，将此时的剩余个数累加到 j 的账上（$I(j) \mathrel{+}= |\mathrm{LR}|$）即可，次序虽经调整，总数不变。全部分摊下来，$O(n \log n)$。

7.10 本章要点

- 列表与向量同属序列；列表以链接解耦物理与逻辑次序，循位置访问使动态操作只动局部常数根指针。
- 链表两级封装：ListNode（data + pred/succ）与 List（_size + head/tail 哨兵，秩−1、0、n−1、n）；`operator[]` 为 O(r)的模仿品，勿多用。
- 前插入三根指针次序不可颠倒：构造时接好 X 的外向两根，再改 pred->succ，最后改 pred；删除让两侧指针绕过被删者；移动=取出+转移，O(1)。
- 复制逐节点 insertLast；析构逐个 delete 并留意单向链表的变体。
- 无序 find 带哨兵 O(n)；dedup 最坏 $O(n^2)$最好 O(n)；遍历配函数对象（Increase/Cumulate/Compare）。
- 有序 uniquify 一趟 O(n)直接删除后继；search 自后向前 O(n)——动态之利与静态之弊的对照。
- 列表排序：选择排序 getMax 自后向前+严格比较保稳定、getMax 累计 $O(n^2)$无最好情况（堆排序是其上限改进）；插入排序 $\Theta(n+I)$ 输入敏感、在线、就地稳定；归并排序搬指针不搬数据，O(n log n)。
- 逆序对记到后者账上；交换一对逆序至少修复 1 个；插入排序时间=脏污读数；归并框架下跨界逆序对在"j 即将出列"时刻统计，总计 O(n log n)。

8 栈：调用栈、表达式求值与栈混洗

栈（stack）是受限的序列：只允许在栈顶（top）操作——`push()`入栈、`pop()`出栈、`top()`查询栈顶，外加`empty()/size()`。先进后出（LIFO）的限制换来一条铁律：**任何时刻，可访问的元素只有栈顶一个**。本章沿讲义 04 章的主线走五站：调用栈（递归的隐式栈与尾递归消除）、进制转换（显式手工栈的第一个应用）、括号匹配（单种括号的计数器为何不够）、表达式求值（双栈协作与优先级表），最后以栈混洗收束——把 n 个元素按一个栈的规矩重排，恰好对应 Catalan 数。

8.1 调用栈：递归的隐式栈

程序运行时，每次函数调用都会在**调用栈**（call stack）上压入一帧：局部变量、参数、加上返回地址。以课件的三层调用为例——`main()`调用`funcA()`（返回地址 1000），`funcA()`调用`funcB()`（返回地址 1500），`funcB()`又递归调用自己（返回地址 1820）——栈帧自底向顶依次为 main、funcA、funcB、funcB，每帧记录各自的局部变量与返回地址；函数返回时弹出一帧，跳回该帧记录的地址。递归算法的空间复杂度主要取决于**递归深度**（同时活着的帧数），而非递归实例总数——第 03 章忒修斯绳线的比喻，在这里落实为具体的栈帧。

尾递归（tail recursion）：线性递归的特例，唯一的那句递归恰好位于函数末尾（return 之前）。第 03 章的数组倒置就是一例。它的性质耐人寻味：一旦抵达递归基，会触发一连串返回地址相同的 return，调用栈连续 pop 瞬间清空；而且每有新帧入栈，老帧随即废弃——何不直接复写老帧？越来越多的编译器能自动识别并做**尾调用优化**（TCO）：时间复杂度有常系数改进，空间复杂度或有渐近改进（$O(n) \to O(1)$）。手工消除尾递归也只三步：把递归形式改写为转向标志`next:`，把递归调用改为`goto next`，把参数更新移到 goto 之前——`reverse()`由此从$O(n)$空间降到$O(1)$。头递归（如进制转换的递归版）没有这份幸运：它必须在下潜到最深处之后才开始做事，调用栈无法省略——要消除，只能显式地请出一章真正的栈。

8.2 进制转换：手工栈的第一课

给定十进制非负整数，转换为 b 进制的表示。回忆短除法：反复整除取商、模余取数位，即可自低而高得出各位——比如$2026_{(10)}$：$2026 = 3 \times 8^3 + 7 \times 8^2 + 5 \times 8 + 2$，即$3752_{(8)}$（讲义另举$12345_{(10)} = 30071_{(8)}$；作为文化背景，巴比伦楔形文字使用 60 进制，正方形对角线记作$1°24'51''10'''$，即$1 + 24/60 + 51/60^2 + 10/60^3 \approx 1.41421296$，误差小于$0.6 \times 10^{-6}$，即便边长 1 公里误差不足 1 毫米）。

问题在输出次序：数位自低向高产生，却要自高向低打印。递归版用头递归的调用栈完成这一反转：

```
void convert( uint64_t n, int base ) {              //递归版：隐式调用栈
   static const char digit[] = "0123456789ABCDEF";
   if ( n < 1 ) return;                             //递归基
   convert( n / base, base );                       //头递归：先输出所有高位
   output( digit[ n % base ] );                     //再输出最低位
}  //时间、空间均为O(logn)
```

迭代版把隐式的调用栈换成显式的手工栈——模余入栈、除商更新，最后整体弹出：

```
void convert( uint64_t n, int base ) {              //迭代版：显式手工栈
   const char digit[] = "0123456789ABCDEF";
   Stack<char> S;
   while ( 0 < n ) { S.push( digit[ n % base ] );  n /= base; }
   while ( !S.empty() ) output( S.pop() );          //按常规次序输出各数位
}  //同为O(logn)，常系数更小
```

逐步演示（$2026 \to 3752_{(8)}$）：

进制转换：2026怎样变成3752(8) 第 1 / 10 步

1. 准备

convert(2026, 8)：反复模余入栈、除商更新；再整体弹出

2026 ÷ 8 = 253 ⋯ 2 ; 253 ÷ 8 = 31 ⋯ 5 ; 31 ÷ 8 = 3 ⋯ 7 ; 3 ÷ 8 = 0 ⋯ 3

输出（自高到低）：（尚无）

等待模余。

目标：把十进制2026转成八进制。
为什么：短除法自低向高产生数位：先得到低位，后得到高位——输出顺序恰好相反，正是栈的用武之

图 17　逐步演示：进制转换：2026 怎样变成 3752(8)

8.3 括号匹配：计数器为何不够，栈又为何够

表达式{[()]}里，读到)时应配对最近一个尚未闭合的(。原则：**消去一对紧邻的成对括号，原表达式匹配与否不受影响**——减而治之。实现只扫一趟、绝不回头：左括号一律进栈暂存；右括号弹栈核对。

先看一种括号的简化情形：其实只需一个计数器——左括号加一、右括号减一，中途转负即失配，终了归零即匹配。可惜多种括号并存时，多个计数器仍可能误报：(]会让两个计数器同时归零，配对关系却是错的。还须回头求助于栈：左括号一律进栈；右括号必须与栈顶左括号同类：

```
bool paren( const char A[], Rank n ) {                  //多类括号匹配：一趟扫描
   Stack<char> S;                                       //无论多少种括号，只需一个栈
   for ( Rank i = 0; i < n; i++ )
      switch ( A[i] ) {                                 //左括号一律进栈
         case '(': case '[': case '{': S.push( A[i] ); break;
         case ')': if ( S.empty() || '(' != S.pop() ) return false; break;
         case ']': if ( S.empty() || '[' != S.pop() ) return false; break;
         case '}': if ( S.empty() || '{' != S.pop() ) return false; break;
      }
   return S.empty();                                    //最终栈空，当且仅当匹配
}
```

原理是栈的一条天然性质：**任何时刻，栈内运算符（此处是左括号）的滞留次序与配对次序严格对应**——栈顶永远是“最急需配对”的那个。这一性质还能推广：XML 支持自定义元素，等于自定义无数种括号，一个栈依然够用。

括号匹配：{[()]}怎样通过一个栈 第 1 / 7 步

1. push '{'

paren()：左括号一律进栈；右括号必须与栈顶同类

栈（自底向顶）： {



字符0：'{'是左括号，进栈。

字符0：'{'是左括号，进栈。
为什么：栈里记录的是已扫到、尚未匹配的左括号。

图 18　逐步演示：括号匹配：{[()]}怎样通过一个栈

8.4 表达式求值：双栈协作

求算术表达式的值是编译器、shell、电子表格的共同需求。求值策略仍是减而治之：优先级最高的局部先计算，并以其数值替代——洋葱芯一趟顺序扫描即可找到，但要保证“洋葱芯”随时可见、随手可剥，还须借助栈：**已扫到、但尚待处置的部分用栈缓存**。用两个栈分工——运算数栈 `opnd`、运算符栈 `optr`——关键的一条不变式是：任何时刻，optr 内运算符的优先级必然自底向顶渐次升高。于是栈顶部分能否立即计算，只取决于栈顶运算符与当前运算符的**相对优先级**。
各种优先级情况被归纳为一张9 × 9的表（栈顶运算符×当前运算符，取+ − ∗ / ! ()与哨兵 $$ $ 九种），并用**完美散列**压缩为长度 347 的一维数组：

```
const char Prio[] =                 //栈顶 \ 当前: + - * / ^ ! ( ) $
   "~<  >   >  >       < .   >     <"
" <  .>   >      >        <>     "
"    .   >< <           >    x< <"
"         >    <        >  >  < <"
" . >   >      >  <   <   <      "
"  .<  >< < <         >   <       "
"x   >  >>  >          >  >  <    "
"           >< <   >   >      <<"
". <   >           .  >      ~ < "
" .   x   <<<           >  <";
char prio( char t, char c ) { return Prio[ ( 37*t + 47*c ) % 347 ]; }   //完美散列
```

主算法随之水落石出。逐个处理各字符，遇操作数（可能多位、含小数）读入数栈；遇运算符则查 `prio(栈顶, 当前)`，按四种结果处置：

- `'<'`：**推迟计算，静待时机**——当前运算符进符栈（如 `PRIO('-', '^') = '<'`：乘方优先级更高，减号只能等）；
- `'>'`：**时机已到，执行计算**——弹出栈顶运算符与所需操作数，计算结果压回数栈（如 `PRIO('!', '/') = '>'`：阶乘必须先行）；
- `'~'`：**终须了断**——左右括号相遇则脱去括号（`PRIO('(', ')') = '~'`）；双哨兵`\$`相遇则求值结束（`PRIO('\0', '\0') = '~'`，“从创世纪到世界末日”）；
- `'='`：当前运算符与栈顶同级（如连续加减），按结合性处理。

以 `1 + 2 * 3` 走一遍：数 1 进栈；+与哨兵比得<进符栈；数 2 进栈；*与+比得<进符栈（乘号更高，加号等得起）；数 3 进栈；读到\$，*与\$ 比得>，弹出*与 2、3 算得 6 压回；+与\$ 比得>，算得 7 压回；\$ 与 \$ 相会，弹出唯一的 7。逐步演示：

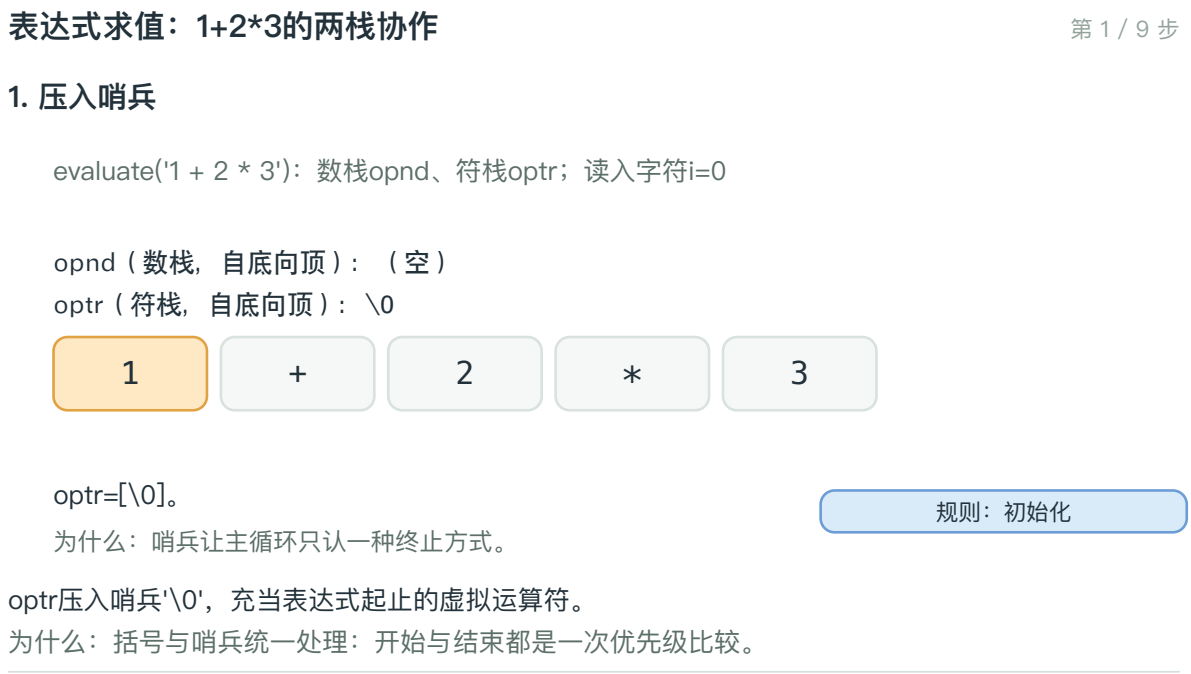


图 19　逐步演示：表达式求值：1+2*3 的两栈协作

8.5 逆波兰表达式：擦去左括号

逆波兰表达式（RPN, Reverse Polish Notation）由 Lukasiewicz 提出：运算符置于操作数之后（故称后缀式），不使用括号即可表示带优先级的运算关系；作为补偿，须额外引入分隔元字符（空格）。比如 `0 1 2 + + 3 ^ 4 5 + / 6 ! * 7 8 + 9 * -`。

它与本章的求值算法血缘极深：**中缀转后缀，就是把转换过程中输出的所有左括号擦掉**。`evaluate()` 已经同时生成 RPN——把读入的操作数、以及每次'>'执行的运算符顺序接入 RPN 串即可（代码中 `append(RPN, ...)`两处）。反过来，RPN 自身的求值更简单：**只需一个运算数栈**——遇操作数入栈；遇运算符弹出所需个数的操作数、计算、结果压回；扫完后栈中唯一剩值即结果。练习：手工把`( 0 + ( 1 + 23 ) / 4 * 5 * 67 - 8 + 9 )`转为 RPN 并求值（讲义实例：结果 2025）。

8.6 栈混洗：Catalan 数

给定输入序列$A = [1, 2, ..., n]$（只许从 A 端弹出），借助一个中转栈 S，通过两类操作重排为输出序列 B：**A 操作**=A 顶弹入 S，**B 操作**=S 顶弹入 B。这样得到的 B 称作 A 的**一个栈混洗**（stack permutation）。同一输入可有多种混洗：$[1, 2, 3, 4\rangle$、$[4, 3, 2, 1\rangle$、$[3, 2, 4, 1\rangle$.....但$[3, 1, 2, ...\rangle$必然不行。混洗总数$SP(n)$ =?——既然$[3, 1, 2\rangle$不可生成，$SP(n) < n!$，更精准的呢?

两次原理计数。考查排头的元素 1：混洗之后它将居于第k位，共n类可能（**加法原理**）；第k类如何生成?无非四步：1 之前的$k-1$个元素先于 1 入栈、1 入栈、1 即刻弹出居第 k 位、其余$n-k$个元素完成各自的混洗——前后两个子问题相互独立（**乘法原理**）。于是

$$SP(n) = \sum_{k=1}^{n} SP(k-1) \cdot SP(n-k), \qquad SP(0) = 1$$

这正是 **Catalan 数**的递推公式，通项$cat(n) = \frac{(2n)!}{n!\,(n+1)!}$，且$cat(n) = \Theta(4^n / n^{1.5})$：$SP(3) = 5 < 3! = 6$，$SP(4) = 14 < 4! = 24$。

哪些排列不是栈混洗? 禁形：对任何$1 \le i < j < k \le n$，模式$\{..., k, ..., i, ..., j, ...\}$（k 在 i 之前而 j 在最后）必非栈混洗——$[3, 1, 2\rangle$就是$n = 3$的唯一禁形。反过来，不含任何禁形的序列一定是栈混洗，于是有$O(n^3)$

乃至$O(n^2)$的甄别算法。但返璞归真，借助栈直接模拟只需$O(n)$：逐个检视目标元素，若它在（或能腾挪至）S 顶部则执行 B 操作，否则无法完成：

```
Rank spCheck( Vector<Rank>& B ) {        //B = unsort[1,n]
   Stack<Rank> S;  Rank A = 1;           //中转栈、输入栈（后者只记栈顶）
   for ( Rank n = B.size(), r = 0; r < n; S.pop(), r++ )
      if ( S.empty() || S.top() != B[r] )
         if ( n < A || B[r] < A )        //A已空，或不含B[r]
            return r;                    //卡住：不是栈混洗
         else
            do S.push( A++ );            //连续A操作
            while ( S.top() != B[r] );
   return B.size();                      //成功
}  //O(n)
```

用禁形$[3,1,2\rangle$走一遍即知它为何卡住：

栈混洗甄别：[3,1,2]为什么卡住 第 1 / 4 步

1. 初始

spCheck([3,1,2])：A输入栈只记栈顶；S为中转栈；逐项尝试B操作

A输入栈顶：1　　目标序列B：3 1 2　　已核对到第0项

等待核对B[0]=3。

A输入栈=[1,2,3]（只需记录栈顶1）；S空；目标B=[3,1,2]。

为什么：逐项尝试B操作：目标元素若能腾挪至S顶部就弹出，否则卡住。

图 20　逐步演示：栈混洗甄别：[3,1,2]为什么卡住

栈混洗还有一副对称的面孔：每个混洗对应于 n 次 A 操作与 n 次 B 操作构成的序列，且任一前缀中 A 操作不少于 B 操作——把 A、B 对应于左、右括号，n 个元素的栈混洗与 n 对括号的匹配表达式一一对应（第 08 章开头刚讲过它的判定）。Catalan 数在这里第二次现身。

8.7 本章要点

- 栈 = 只许栈顶操作的受限序列；调用栈是递归的隐式栈，空间取决于递归深度；尾递归可被 TCO 消除，或手工三步转迭代。
- 进制转换：短除法自低向高产生数位，手工栈反转输出次序，$O(\log n)$。
- 括号匹配：单种括号计数器足矣，多类括号必须用栈——栈顶永远是“最急需配对”的左括号；可推广至 XML。
- 表达式求值：opnd/optr 双栈，优先级表完美散列压缩，四种情形< > ~ =；不变式：符栈优先级自底向顶递增。
- RPN：中缀转后缀 = 擦去左括号；RPN 求值只需一个数栈。
- 栈混洗计数$SP(n)$为 Catalan 数；禁形$\{k,i,\ldots,j\}$刻画非混洗；直接模拟$O(n)$甄别；混洗与括号匹配一一对应。

9 队列：先进先出、摊还分析与单调结构

队列（queue）与栈相反地受限：只允许在队尾插入（enqueue()/rear()）、在队头删除（dequeue()/front()）——**先进先出**（FIFO），也形象地叫“后进后出”（LILO）。FIFO 恰是“公平”的数学形状：谁等得最久，谁先被服务。本章从接口与实现出发，经轮转调度、滑动窗口、银行模拟三个应用，落到两个更深的课题：双栈当队的**摊还分析**（聚合法、记账法、势能法一次集齐），以及单调栈与单调队列（直方图最大矩形与 MaxQueue）。本章也是摊还分析三种方法的正式课堂——第 04 章的加倍扩容只用过其中最直观的一种。

9.1 队列：接口与实现

队列同样是受限的序列，直接基于第 03 章的列表派生即可：

```
template <typename T> struct Queue: public List<T> {
   void enqueue( const T& e ) { insertLast( e ); }   //入队：O(1)
   T dequeue() { return remove( first() ); }         //出队：O(1)
   T& front() { return first()->data; }              //队首：O(1)
};  //列表：相当于把水平列表顺时针旋转90度
```

借助列表的内部结构，三个接口都只需 $O(1)$。课后有一道对照练习：若基于向量派生（enqueue = insertLast、dequeue = remove(0)），各接口效率如何？——dequeue 要搬迁全部后缀，$O(n)$；这正是本章末尾双栈技巧要解决的课题。若坚持纯数组实现，答案另有其人：循环数组以_front = (_front + 1) % _capa 的方式让队头指针绕圈，代价是扩容逻辑要另行打理。

9.2 应用一：轮转调度

一组客户共享同一资源（多个进程共享 CPU、实验室成员共享打印机），如何兼顾公平与效率？**轮转调度**（RoundRobin）：所有客户组成一个队列，服务关闭前反复地——令队首客户出队，接受一个时间片 quantum 的服务，然后重新入队：

```
Queue Q( clients );                    //共享资源的所有客户组成队列
while ( !ServiceClosed() ) {
   e = Q.dequeue();                    //队首出队
   serve( e, quantum );                //接受一段时长为quantum的服务
   Q.enqueue( e );                     //重新入队
}
```

逐步演示（A、B、C、D 四个客户）：

轮转调度：出队、服务、再入队 第 1 / 6 步

1. 初始队列

轮转调度第1轮：队首出队→接受一个时间片→重新入队



等待调度。

A、B、C、D共享一个CPU，按FIFO公平轮转。
为什么：队列保证谁等得最久谁先被服务。

图 21　逐步演示：轮转调度：出队、服务、再入队

每轮循环后队列循环左移一格，没有客户会被饿死。CPU 的分时系统、打印机的排队打印，用的都是这个模型。

9.3 应用二：前缀和与滑动窗口

前缀和。队列$A=[3,1,4,1,5,9]$，要就地把它变成前缀和序列$[3,4,8,9,14,23]$。利用队列的循环性质，一趟完成：

```
void qPreSum( Queue<int>& A, Rank n ) {
   int sum = 0;
   while ( n-- ) A.enqueue( sum += A.dequeue() );   //出队累加、结果再入队
}  //O(n)
```

滑动窗口和。给定序列与窗口宽度w，求每个连续w项之和。技巧：先算前缀和，窗口和就是两个前缀和之差；窗口在队列上滑动，旧队首出队、新元素入队：

```
void qWinSum( Queue<int>& A, Rank n, Rank w ) {      //w <= n
   qPreSum( A, n );                                  //先求前缀和
   Queue<int> W;  W.enqueue( 0 );                    //建立窗口（左哨兵0）
   for ( Rank i = 1; i < w; i++ ) W.enqueue( A.dequeue() );   //充满窗口
   for ( Rank i = w - 1; i < n; i++ ) {
      A.enqueue( A.front() - W.dequeue() );          //求差：窗口和
      W.enqueue( A.dequeue() );                      //滑动窗口
   }
}  //O(n)
```

以$[3,1,4,1,5,9]$、$w=3$为例：前缀和为$[3,4,8,9,14,23]$，窗口和依次为$8,6,10,15$。两段代码展示了队列的一个独特用法：**让数据在队列里循环流动**，新结果作为新元素入队，无需辅助数组。

9.4 应用三：银行服务模拟

多窗口银行一小时的客流怎么模拟？每个窗口各维护一个队列，记录顾客的服务时长：

```
void simu( int winds, int duration ) {                  //窗口数、营业时长
   Queue<int>* wind = new Queue<int>[winds];
   while ( duration-- ) {                               //每个单位时间
      for ( int i = 0; i < winds; i++ )                 //各非空队列的队首顾客
         if ( !wind[i].empty() && ( 1 > wind[i].front()-- ) )
            wind[i].dequeue();                          //服务完毕出列，后继接替
      if ( rand() % ( 1 + winds ) )                     //新顾客以wins/(wins+1)概率到达
         wind[ rand() % winds ].enqueue( 1 + rand() % 99 );    //随机选窗口、定时长
   }
   delete [] wind;
}
```

模型的三个要素——到达过程（随机入队）、服务过程（队首倒计时）、排队规则（入随机窗口）——每个环节都是一个队列操作。这类离散事件模拟是队列最典型的应用场景之一。

9.5 双栈当队：摊还分析的三种方法

队列也能用两个栈实现：**R 栈只管 enqueue**（新元素压入 R），**F 栈只管 dequeue**（从 F 顶弹出）；仅当 F 为空时，才把 R 整体逐一弹出压入 F——两次 LIFO 复合出一次 FIFO：

```
void enqueue( const T& e ) { R.push( e ); }             //O(1)
T dequeue() {                                           //F空则先倒腾
   if ( F.empty() ) while ( !R.empty() ) F.push( R.pop() );
   return F.pop();
}  //最好O(1)、最坏O(n)；平均呢？—均摊！
```

逐步演示：



图 22　逐步演示：双栈当队：R 进 F 出，倒腾一次

单看 dequeue()，最好$O(1)$、最坏$O(n)$，平均看似难以界定——这正是摊还分析的用武之地。三种方法在此齐齐登场：

聚合法（aggregate）：考虑d次 dequeue()与e次 enqueue()（$d \le e$）之后的任意时刻。全部操作的耗时：每次 enqueue 一次 push 共e次；每个元素至多经历一次倒腾（R 弹出、F 压入）共$2d$次；每次 dequeue 一次 pop 共d次。总计不超过

$$e + 2d + d \le 4n = O(n)$$

折算到每个操作，均摊成本$O(1)$。

记账法（accounting）：给每个元素派发 4 枚金币——1 枚支付它自己的 `enqueue()`，2 枚预付将来的倒腾（R 弹出、F 压入），最后 1 枚支付 `dequeue()`。每个操作都预先付清了账单，结构永远不会破产；任意n个操作的总代价至多$4n = O(n)$。记账法还能精确分析更复杂的算法：以第 04 章的 `Vector::dedup()`为例，给秩为k的元素派发$n - k$枚金币，不变式是“考查到$[k]$时，前缀$[0, k]$中每个元素恰剩$n - k$枚，后缀(k, n)的金币原封未动”；无论接下来在$[i]$处删除（账单$n - i - 1 = (k - i) + (n - k - 1)$恰好由$[i]$与$(i, k]$各自埋单），还是继续前移，不变性都保持——比对与移动的总成本于是有了逐元素对账的证明。

势能法（potential）：定义势能函数Φ为双栈问题里 F、R 两栈长度的乘积，验证任何一步操作的**摊还成本 = 实际成本 + $\Delta\Phi$非负**，则总摊还成本即初始势能加总面积的保守估计。

三种方法同一结论：双栈队列的每个操作均摊$O(1)$。它们各有所长——聚合法最直接，记账法最利于逐元素对账，势能法最善于处理状态连续变化的算法；此后遇到均摊问题，可从中择顺手者。

9.6 直方图内最大矩形：单调栈

给定非负整数直方图$H[0, n)$，求其中最大的正交矩形（歧义时取最靠左者）。蛮力枚举每根柱子$H[r]$支撑的最大矩形——向两侧扩展到第一根更矮的柱子为止——需要$O(n^2)$。单调栈把它压到$O(n)$：

```
Rank* s = new Rank[n];   Stack<Rank> S;
for ( Rank r = 0; r < n; r++ )                        //自左向右扫描
   while ( !S.empty() && ( H[ S.top() ] >= H[r] ) ) S.pop();   //直到栈顶更矮
s[r] = S.empty() ? 0 : 1 + S.top();   S.push( r );   //S始终严格递增
while ( !S.empty() ) S.pop();
```

单调栈（monotonic stack）：栈内元素（的高度）自底向顶严格递增。新柱入栈前，把所有不低于它的柱子弹出——**弹出的时刻恰是它们“此后再无用处”的时刻**：右边界出现了；而弹出后的新栈顶恰好提供左边界。每个柱子至多入栈、出栈各一次，均摊$O(n)$。$t(r)$（右边界）由反向的第二次扫描同理求得，于是所有极大矩形$O(n)$时间、$O(n)$空间。课件还提出更高的要求：若直方图以就地（in-place）、在线（on-line）方式给出，$t(r)$必须等全部输入就绪才能确定——能否一趟扫描同时算出$s(r)$与$t(r)$？答案仍是单调栈，配合末尾哨兵，每个出栈的柱子当场结算：

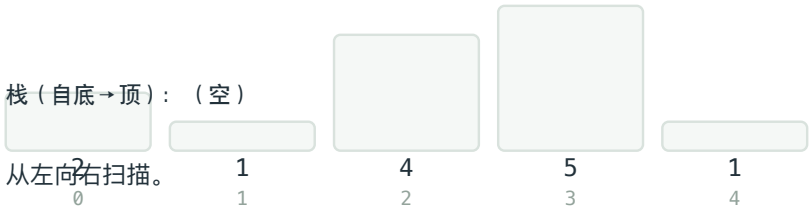
```
Stack<Rank> SR;   uint64_t maxRect = 0;   //SR次顶即s(r)-1，顶即r
for ( Rank t = 0; t <= n; t++ )                       //均摊O(n)
   while ( !SR.empty() && ( t == n || H[ SR.top() ] >= H[t] ) ) {
      Rank r = SR.pop(), s = SR.empty() ? 0 : SR.top() + 1;
      maxRect = max( maxRect, H[r] * ( t - s ) );   //当场结算
   }
```

以$[2, 1, 4, 5, 1]$为例完整走一遍（各块结算面积 2、8、5、4，最大 8）：

直方图最大矩形：单调栈结算 第 1 / 12 步

1. 准备

单调栈求最大矩形：栈内高度自底向顶严格递增



直方图[2,1,4,5,1]，加哨兵高度0于两端。

为什么：最大矩形必以某根柱子为高度、向两侧扩展到极限。

图 23　逐步演示：直方图最大矩形：单调栈结算

9.7 MaxQueue：单调队列

滑动窗口的进一步要求：窗口滑动的同时，随时查询**当前窗口内的最大元素**。朴素做法每次$O(w)$；借助辅助队列X（记录可能成为最大者的候选）可以做到 `getMax()` $O(1)$，但 `enqueue()`最坏$O(n)$且可能持续发生。真正的解法是**单调队列**：辅助结构Y只保留 Q 中的关键元素，且其值自队首向队尾严格递减——

```
enqueue( e ) {                          //均摊O(1)
   Q.enqueue( e );  c = 1;              //c：e在Y中的计数
   while ( !Y.empty() && ( e >= Y.rear()->data ) ) c += Y.dequeueRear()->count;
   Y.enqueue( <&e, c> );                //从队尾弹掉所有不大于e的候选
}
dequeue() {                             //O(1)
   if ( --Y.front().count < 1 ) Y.dequeue();   //队首候选计数减到0才出队
   return Q.dequeue();
}
getMax() { return Y.front()->data; } //队首即当前最大者：O(1)
```

原理与单调栈同源：新元素 e 入队尾时，所有不大于 e 的旧候选**永远不可能再成为最大者**（e 比它们新且大，会活到最后），逐个从队尾弹掉；每个元素至多入队、出队各一次，均摊$O(1)$。计数器$< e, c >$记录该候选“替身”的个数——出队时计数减到零才真正出队。滑动窗口最大值、带时限的调度优化，都是它的直接应用场景。

9.8 本章要点

- 队列 = FIFO 受限序列；基于列表派生三个接口均$O(1)$；向量版 `dequeue` 为$O(n)$，纯数组方案是循环数组。
- 轮转调度：出队→服务→入队的循环，FIFO 即公平；前缀和与滑动窗口展示“数据在队列里循环流动”的就地技巧；银行模拟是离散事件模拟的典型模型。
- 双栈当队：F 空才倒腾的纪律下，聚合法（$e + 2d + d \le 4n$）、记账法（每元素 4 枚金币）、势能法（两栈长度乘积）三种摊还分析殊途同归$O(1)$；记账法可逐元素对账 `Vector::dedup()`。
- 单调栈：栈内高度严格递增，出栈时刻即“此后无用”时刻，直方图最大矩形均摊$O(n)$，末尾哨兵就地结算。
- 单调队列：辅助队列自队首向队尾严格递减，新元素从队尾弹掉无用候选，配合计数器实现 enqueue/dequeue/getMax 均摊$O(1)$。

10 二叉树：遍历、重建与层次结构

树是有根的层次结构：除根外每个节点有唯一的父节点，沿父链接上溯必达根、无环。二叉树（binary tree）进一步规定每个节点至多两个孩子，且左右位置分明——“只有一个孩子”也必须说明是左是右。这条看似苛刻的限制换来极高的结构规整性：本章的遍历、重构与计数，几乎每条结论都带一个漂亮的常数。

10.1 概念与记号

树的要素：节点与联边；指定根之后即为**有根树**。任给若干棵有根子树，引入新节点 r 并向各子树根发出一条边，即得更大的树——r 与各子树根互称**父**（parent）、**子**（child），一个节点的子节点数为其**度**（degree）；零度节点称**叶**（leaf），其余为**内部节点**。从任一祖先沿边下溯到后代，沿途边数为**路径长度**；若起点是根，该长度即节点的**深度**（depth）。节点所有后代的最大深度减去自己的深度，是该节点的**高度**（height）；全树的高度即根的高度，空树高度约定为 -1。

真二叉树（proper binary tree）：每个节点度数为 0 或 2。任何二叉树都可以“补齐”成真二叉树——为所有度数为 1 的节点补上一个空的叶。这个规范化在重构问题里马上会用到。

10.2 BinNode 与 BinTree

```
template <typename T> using BNP = BinNode<T>*;   //二叉树节点位置
template <typename T> struct BinNode {
   T data;                          //数值
   BNP<T> parent, lc, rc;           //父、左孩子、右孩子
   int height;                      //高度
   Rank size();                     //子树规模
   BNP<T> insertLc( const T& e ); //作为左孩子插入
   BNP<T> insertRc( const T& e ); //作为右孩子插入
   BNP<T> succ();                   // （中序意义下）直接后继
};
template <typename T> int BinNode<T>::updateHeight()          //O(1)
{ return height = 1 + max( lc ? lc->height : -1, rc ? rc->height : -1 ); }
template <typename T> void BinNode<T>::updateHeightAbove()    //O(n = depth(x))
{ for ( BNP<T> x = this; x; x = x->parent ) x->updateHeight(); }
```

BinTree 持有_root 与_size，insert 在孩子位置挂新节点并 updateHeightAbove()，attach 把整棵子树接入某节点的空位——这些操作本身 $O(1)$，高度更新沿父链 $O(\text{depth})$。

10.3 遍历：三种深度优先 + 一种广度优先

访问节点的次序不同，便得到不同的遍历。深度优先的三种只在“访问根的时机”上不同：**先序**（根-左-右）、**中序**（左-根-右）、**后序**（左-右-根）。递归实现各三行；不依赖递归的迭代版才是本节的重头。

先序遍历的藤路径。沿着最左侧藤（起始于根的长子通路）观察：整个遍历可分解为——自上而下访问藤上节点，再自下而上遍历各右子树。各右子树的遍历彼此独立、自成子任务，于是手边需要一个容器把右子树“存起来待办”——栈。迭代版：沿藤访问并入栈；弹栈取右子树重复：

```
template <typename T, typename V> void visitAlongVine( BNP<T> x, V& visit, Stack<BNP<T>>& S )
{
   while ( x ) { visit( x->data );  S.push( x->rc );  x = x->lc; }   //当前藤自顶向下
}
template <typename T, typename V> void preorder( BNP<T> x, V& visit ) {
   Stack<BNP<T>> S;
   while ( true ) {
      visitAlongVine( x, visit, S );   //访问藤上节点，右子树入栈暂存
      if ( S.empty() ) break;
      x = S.pop();                     //弹出下一棵右子树的根
   }
}  //O(n)
```

中序遍历的藤路径。中序的访问时机在左子树之后——于是沿最左侧藤下潜时只入栈不访问，抵达藤根（最深的节点）才开始访问并转向右子树：

```
template <typename T, typename V> void inorder( BNP<T> x, V& visit ) {
   Stack<BNP<T>> S;
   while ( true ) {
      while ( x ) { S.push( x );  x = x->lc; }   //沿藤下潜，逐步入栈
      if ( S.empty() ) break;
      x = S.pop();  visit( x->data );            //弹出并访问藤根
      x = x->rc;                                 //转向右子树
   }
}  //O(n)
```

直接后继与统一范式。中序意义下节点 x 的**直接后继** `succ()`：若 x 有右孩子，则是右子树中**最左**的节点（右子树藤的终点）；否则是 x 的**最低左祖先**——沿父链上溯，第一个作为左孩子进入的祖先。有了 `first()`（最左下潜）与 `succ()`，中序遍历可以写成极简的

```
for ( x = first(x); x; x = x->succ() ) visit( x->data );
```

任何遍历都可套用这个范式，只要写好各自的 `first()`与 `succ()`。最坏情况下单次 `succ()`需 $O(h)$，但整个遍历摊还 $O(n)$——每条边至多被走过两次。

后序遍历：访问时机最靠后，迭代版需要记住“从哪条边返回”（左孩子还是右孩子），栈中需要存标志位，思路与中序藤一致。

层次遍历。深度优先三兄弟用栈；按层访问则用**队列**：出队→访问→孩子依次入队。FIFO 恰保证“先访问者的孩子先访问”，同层次序由入队次序自然保证：

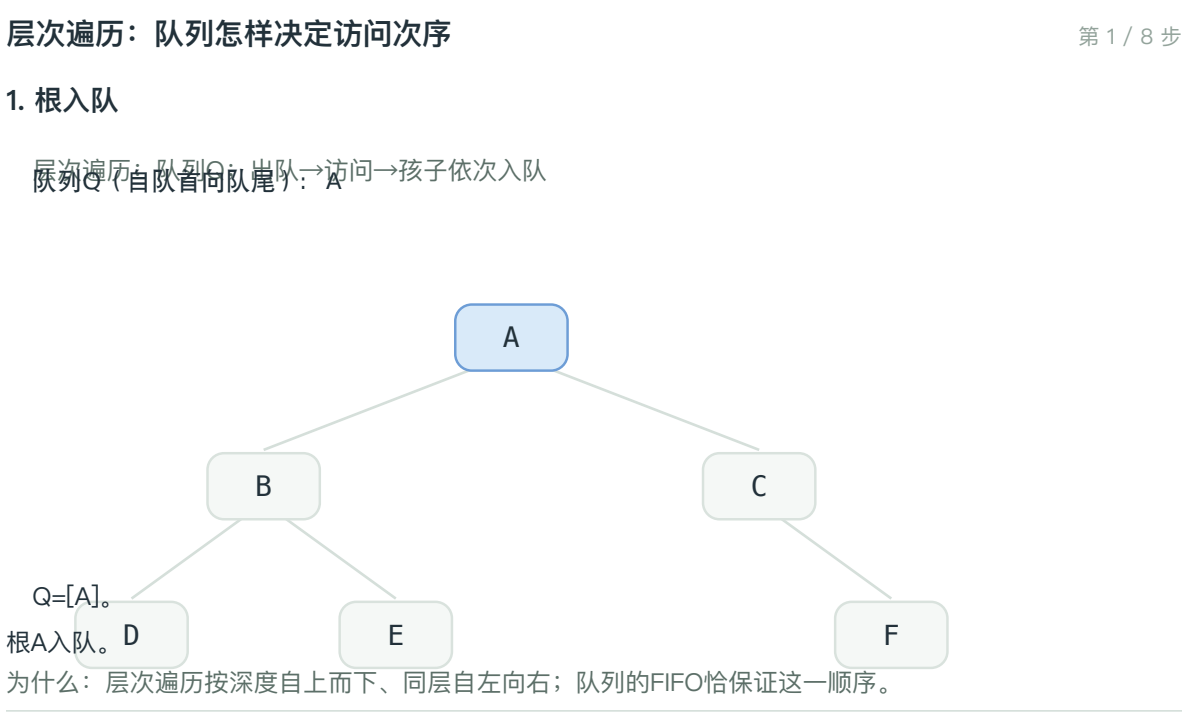


图 24　逐步演示：层次遍历：队列怎样决定访问次序

以 $A(\,B(D,E),C(-,F)\,)$ 为例：$Q=[A]$ → 访问 A、BC 入队 → 访问 B、DE 入队 → 访问 C、F 入队 → 访问 D、E、F。序列 AB CDEF，同层次序与子层次序都严格保持。

10.4 重构：中序是关键

遍历序列能否唯一还原一棵二叉树？**先序或后序单独都不够**——它们丢失了“空子树在哪”的信息。**中序 + 任一先序/后序**则可以唯一重构（元素互异时）。分治论证：先序的首元素必为根；在中序中定位它，左

侧即左子树的中序、右侧即右子树的中序；数出左子树规模 k，先序中随后的 k 个元素即左子树的先序、其余为右子树的先序——递归至空区间。后序同理，只是根在末尾。

算法实现时为快速定位，需先对中序建**散列表**（第 15 章）或按元素查表 $O(n)$；分治递归总体 $O(n)$（互异元素、散列表定位时）。

真二叉树的中序 + 层次也可唯一重构：层次序列首元素为根；在中序中定位后，左右子树的规模即层次序列中相邻两段的长度。非真二叉树时该法可能歧义——这正是“补齐成真二叉树”这一规范化的用武之地。

10.5 计数：n 个节点的二叉树有多少棵

借道栈混洗的结论：n 个节点的二叉树形态数 = Catalan 数 $cat(n) = \frac{(2n)!}{n!(n+1)!}$——每棵二叉树对应一个中序-先序对，而 n 个节点的中序遍历次序与栈混洗一一对应。这也解释了为何“先序单独”无法重构：一棵先序序列平均对应约 $cat(n)$ 棵不同的树。

10.6 本章要点

- 树的记号：度、叶/内部节点、深度、高度（空树-1）；二叉树左右分明，可补齐为真二叉树。
- BinNode：data/parent/lc/rc/height；updateHeightAbove 沿父链 O(depth)。
- 四种遍历：先序/中序/后序/层次；先序迭代=藤上访问+右子树入栈；中序迭代=藤上下潜只入栈、弹出即访问；层次遍历用队列。
- succ()直接后继：有右孩子取右子树最左，否则取最低左祖先；中序遍历可写成 for(first; x; x=succ())，摊还 O(n)。
- 重构：中序+先序（或后序）唯一；真二叉树中序+层次唯一；先序或后序单独不够。
- n 节点二叉树形态数 = Catalan 数 cat(n)，与栈混洗一一对应。

11 Huffman 编码与比较下界

数字通讯中，字符要变成二进制位串才能上线传输。若每个字符固定用等长的编码，总长度可以预算；但不同字符的出现频度天差地别——高频字符配短码、低频字符配长码，总长可以大幅压缩。代价是必须前缀无关（prefix-free）：任何字符的编码都不能是另一字符编码的前缀，否则解码者面对 001 无法判断它是字符 I 还是 NM 的开头。本章解决两个问题：怎样构造总编码长度最小的前缀无关编码（Huffman 树），以及——一个更大的主题——**怎样证明某个算法已经不可能再改进**（复杂度下界、比较树、线性归约）。

11.1 前缀无关编码与编码树

把字母表Σ中的每个字符各作为一个叶节点，随意组成一棵**真二叉树**：从根到叶 x 的通路即 x 的编码，左、右分支分别记 0、1。通路上除终点外都是内部节点——叶不可能出现在另一叶的路径上，前缀无关性由树结构自然保证。这样的树称 **PFC 编码树**；字符的编码长度即对应叶的深度，整体成本为所有字符编码长度之和 CL。PFC 并不唯一，何者 CL 最短？CL 最短者称**最优编码树**（OCT），必然存在（真二叉树只有有限多棵）——虽然未必唯一。

不加权的情形有一个干净的刻画：**OCT 的叶子只能出现在倒数两层**——否则深叶与浅叶交换总能降低 CL；而“完全”的真二叉树即是 OCT，$CL = n \times \lceil \log_2 n \rceil$ 量级。真正的难点在带权情形：字符的使用频度各不相同，衡量尺度换成**带权平均编码长度**（叶深 × 字符频度的总和，即 WPL）。均衡树在频度悬殊时（比如“小米加步枪”对“飞机加坦克”的悬殊对比）反而远远不是最优——需要的算法必须把频度考虑进来。

11.2 Huffman 树：贪心合并

Huffman 算法（1952）一句话说完：反复从森林中取出权重最小的两棵树，合并为一棵（新树权重为二者之和），插回森林；$n-1$ 次迭代后森林中唯一所剩的那棵，即 Huffman 编码树。字符节点封装为：

```
struct HuffChar {                       //Huffman字符：字符+频数
   char ch;  uint32_t weight;
   bool operator<( const HuffChar& hc ) //比较器
      { return weight > hc.weight; }    //故意大小颠倒
};                                      //便于delMax()取出最小者
```

森林就是一个优先级队列。贪心合并的实现：

```
HuffTree huffTree( HuffForest& F ) {
   while ( 1 < F.size() ) {                         //每迭代一步，森林减一
      HuffTree T1 = F.delMax(), T2 = F.delMax();  //取出最小两棵（T2权重不小于T1）
      HuffTree S;  S.insert( HuffChar( Weight(T2) + Weight(T1) ) );
      S.attach( T2, S.root() );  S.attach( S.root(), T1 );   //合并
      F.insert( S );                                //插回森林
   }
   return F.delMax();   //层次序列必然单调非增
}
```

以频数 $A=2$、$B=3$、$C=5$、$D=8$ 为例的完整合并过程：

Huffman贪心合并：2,3,5,8 第 1 / 5 步

1. 初始森林

Huffman贪心：森林F中取出权重最小的两棵合并，插回森林



初始：四棵单节点树。

四个字符按频数作为单节点树：A2、B3、C5、D8。
为什么：编码目标：总编码长度（带权路径长WPL）最小。

图 25　逐步演示：Huffman 贪心合并：2,3,5,8

结果：A=110、B=111、C=10、D=0，$WPL = 3\times2 + 3\times3 + 2\times5 + 1\times8 = 33$——频数最高的 D 只占 1 位，最低的 A 占 3 位，高频短码、低频长码。

正确性（归纳法）。关键引理：**频数最低的两个字符 x 与 y，必在某棵 OCT 中互为兄弟**——若在某最优树中不是兄弟，可通过两次交换使它们成对且 WCL 不增。设$|\Sigma| < n$时 Huffman 算法生成 OCT；对$|\Sigma| = n$，令Σ'为把 x、y 合并成新字符 z（权重相加）后的字符集，两个字符集的 PFC 树集合之间存在一一对应（z 展开为 x、y 兄弟 ⟺ x、y 合并为 z），且两种树形的 WCL 恰差一个固定值 $w(x)+w(y)$——“差距固定，故最优必对应于最优”。归纳完成。

一个需要约定的细节：OWCT 通常有多棵（同深子树可互换，甚至不同深也可能），Huffman 算法对某些细节也未明确——**约定以实现的版本为准**：左兄弟（深层者）不小于右兄弟。

11.3 Huffman 森林的实现：从 O(n²)到 O(n)

运行时间取决于森林的 delMax()与 insert()。**列表实现**（ListPQ：insert 插队尾、delMax 摘最大）两操作合计每步$O(n)$，总体$O(n^2)$。改用完全二叉堆或左式堆（第 12 章）每步$O(\log n)$，总体$O(n\log n)$。有意思的是，**直接用最简单的数据结构也能做到$O(n)$**——森林=栈+队列：

```
struct HuffForest {                          //一个栈 + 一个队列
   Stack<HuffTree> S;  Queue<HuffTree> Q; //两者全程各自有序
   HuffForest( uint32_t* freq, char s, char t ) {   //初始化：字符按频数排序后入栈
      for ( char c = s; c < t; c++ ) S.push( HuffTree().insert({ freq[c-s], c }) );
      S.sort();                                      //频数逆序
   }
   void insert( HuffTree t ) { Q.enqueue( t ); }      //合并树只进队列
   HuffTree delMax() {                                //最小树只可能在栈顶或队首
      return S.empty() || ( !Q.empty() && ( Q.front() > S.top() ) )
             ? Q.dequeue() : S.pop();
   }
};
```

原理：按频数排序后所有字符入栈（频数逆序，栈顶最小）；此后只出不进；合并生成的树单调不降，恰好构成一个有序队列。两个有序序列的 delMax 只需比较栈顶与队首，$O(1)$；insert 也是$O(1)$。第 09 章“有序序列合并”的思想在此再度发力。

11.4 下界与最优：比较树

同一问题的不同算法，复杂度可能相差悬殊；但针对任何一个具体计算问题，算法的改进不可能持续获得——**改进的尽头称作该问题的复杂度下界**（lower bound），达到下界的算法即**最坏情况最优算法**（worst-case optimal, WCOA）。怎样证明下界？看一个热身：从 4 个外观相同的苹果中找出唯一的重量不同者，借助天平只需 2 次称量——为什么 2 次就够、又不会再少？

比较树（comparison tree）。基于比较的算法（comparison-based algorithm, CBA）的每次比较至多有分支数种走向，整个运行过程对应一棵树：运行的输出对应叶节点，运行的过程对应从根到叶的路径，消耗的时间即路径长度，最坏情况耗时即**树的高度**。若问题有 n 种输出（比如 n 个互异元素的一种排序），树至少需要 n 片叶；而高为 h 的二叉树至多2^h片叶，故

$$n \le 2^h \Longrightarrow h \ge \lceil \log_2 n \rceil$$

这就是比较树的下界通式。**苹果甄别**：4 种输出（谁是异重者），下界$\lceil \log_2 4 \rceil = 2$次称量，且 2 次称量的算法存在——下界紧。**n 个元素的排序**：共有$n!$种输入/输出，下界

$$\lceil \log_2 n! \rceil = \Omega(n \log n)$$

由 Stirling 公式$\log_2 n! \approx n \log_2 n - 1.44 \log_2 n$。归并排序的$O(n \log n)$与之匹配，故属于 WCOA——第 06 章“最坏情况最优”的断言在此兑现。

11.5 线性归约：下界的传染性

每个问题的下界都要这样一对一地证明吗？不必。**线性归约**（linear-time reduction）：若问题 W 可在$O(n)$时间内转化为问题 U 的输入，且 U 的输出可$O(n)$转化回 W 的输出，则 W 的任何算法都对应于 U 的一个算法（曲径通幽：输入转化→U 算法→输出转化），于是

$$\Omega(n \log n) \le |W| \le |U|$$

U 的下界“传染”给 W。两个实例：

SORTING ≤N HUFFMAN：排序的每个输入（n 个实数）可以直接理解为 Huffman 的一个输入（n 个字符的频数）；Huffman 输出的编码树经层次遍历，必然得到有序序列（包含所有内部与叶节点）——$O(n)$双向转换。因此 Huffman 本身也具有$\Omega(n \log n)$的下界：上一节栈+队列的$O(n)$实现并不与此矛盾——下界针对的是基于比较的模型，$O(n)$实现绕开的正是比较（它利用了“频数可排序预处理”这一额外信息）。

SORTING ≤N RBM（红蓝匹配）：给平面上互不重合的红、蓝点各 n 个，用 n 条无交线段联结异色点。排序的每个输入映射为 x 正半轴上的 n 个红点（预处理$O(n)$），y 正半轴上有序的 n 个蓝点即成 RBM 实例；RBM 的输出一趟遍历即可转回排序的输出。故 RBM 也是$\Omega(n \log n)$。

下界一旦确立，便沿归约关系成片“传染”——这正是第 02 章“SubsetSum 是 NP 完全”那一页的另一种回声：难，是成批地难；同样，最优，也是成批地最优。

11.6 本章要点

- 前缀无关编码=编码树（叶=字符，通路=编码）；OCT 叶子只在倒数两层，均衡真二叉树即 OCT；带权情形衡量尺度为 WPL。
- Huffman 算法：贪心合并最小两棵，$n-1$次迭代；正确性归纳基于“最小字符成对”引理与“差距固定”对应。
- 森林实现：列表$O(n^2)$、堆$O(n \log n)$、**栈+队列$O(n)$**（有序序列合并思想，且绕开比较模型、不受$\Omega(n \log n)$下界约束）。
- 比较树：CBA 对应二叉树，n 种输出⇒高≥⌈log₂n⌉；排序下界 Ω(nlogn)，归并排序 WCOA；苹果甄别⌈log₂4⌉=2。
- 线性归约传递下界：SORTING ≤N HUFFMAN、SORTING ≤N RBM，成片确立 Ω(nlogn)。

12 二叉搜索树与 AVL 树

有序向量的查找是$O(\log n)$，插入却是$O(n)$；列表的插入$O(1)$，查找却是$O(n)$。**二叉搜索树**（BST）把“有序”搬进分叉关系里：每个节点的左子树所有键不大于它、右子树所有键不小于它（互异时严格）。于是查找只需每步比较一次、走向一侧——形态随机时$O(\log n)$；而插入与删除只在局部改几根指针。本章还处理它唯一的隐患——树形可能失衡退化成单链——由此引出第一个平衡搜索树：AVL。

12.1 BST：中序次序即全局次序

BST 的**顺序性**有一条等价刻画：**中序遍历序列单调不降**。这使“树上查找”与“有序向量二分”在逻辑上完全同构。接口实现：

```
template <typename T, bool H> BNP<T> BST<T, H>::search( const T& e ) {   //查找
   return searchIn( _root, e, _hot = nullptr );
}
template <typename T, bool H>   //典型实现：_hot指向命中节点的父节点（或将来接入处）
static BNP<T> searchIn( BNP<T> v, const T& e, BNP<T>& hot ) {
   if ( !v || ( e == v->data ) ) return v;    //递归基：命中或空树
   hot = v;
   return searchIn( ( e < v->data ) ? v->lc : v->rc, e, hot );
}  //O(h)：h为树高
```

插入：先 search()，失配之处即新节点的接入位置——新节点必为叶：

```
template <typename T, bool H> BNP<T> BST<T, H>::insert( const T& e, int h ) {
   BNP<T> x = search( e );
   if ( x ) return x;                          //已存在，不再插入
   x = new BinNode<T>( e, parent = _hot );     //新节点作为叶子
   ( _hot ? ( e < _hot->data ? _hot->lc : _hot->rc ) : _root ) = x;
   _size++;  _p->updateHeightAbove();          //派生的AVL、红黑树会重写并自行更新
   return x;
}
```

删除分两种情形。**单枝或叶**：直接用唯一的孩子顶替；**双枝**：实际删除的是它的**直接后继** succ()（右子树最左节点）——交换两者数据，问题转化为删除 succ（succ 必无左孩子，回到单枝情形）：

```
template <typename T, bool H> bool BST<T, H>::remove( const T& e ) {
   BNP<T> x = search( e );
   if ( !x ) return false;
   removeAt( x, _hot );  _size--;
   return true;
}
template <typename T> static BNP<T> removeAt( BNP<T> x, BNP<T>& hot ) {
   BNP<T> w = x;                               //实际被摘除者
   BNP<T> succ = x->rc ? removeAt后的succ : nullptr;
   if ( !HasLChild( *x ) ) x = x->rc;          //左空：右子树（或nullptr）顶替
   else if ( !HasRChild( *x ) ) x = x->lc;     //右空：左子树顶替
   else { w = succ( x );  swap( x->data, w->data );   //双枝：与直接后继换数据
          BNP<T> u = w->parent;                //问题转化为摘除w（无左孩子）
          ( u == x ? u->rc : u->lc ) = x = w->rc; }
   hot = w->parent;
   if ( x ) x->parent = hot;
   delete w;  return x;
}
```

三种接口都正比于树高$O(h)$。好形态（随机插入序）$h = O(\log n)$；坏形态（按序插入）退化成单链$h = O(n)$——remove()频繁 succ()还可能加剧倾侧（可通过随机使用 succ()/pred()缓解）。治本的方案是平衡。

12.2 等价二叉搜索树与旋转

两棵 BST 若**中序遍历序列相同**，则等价——“上下可变，左右不乱”。局部等价变换的基本构件是**旋转**：zag（逆时针，左旋）与 zig（顺时针，右旋）。以 zag 为例：g的左孩子p上浮为子树根，g带自己的右子树下沉为p的右孩子，p原来的右子树转而挂在g的左肩。旋转改变“上下”（深度），不改变“左右”（中序次序）——这正是 BST 重平衡的全部自由度：失衡靠旋转修复，正确性靠中序不变保障。

12.3 AVL 树：渐近平衡

平衡因子（balance factor）：$balFac(x) = h(x.lc) - h(x.rc)$。AVL 树（G. Adelson-Velsky & E. Landis, 1962）约定所有节点 $|balFac| \le 1$：

```
auto BalFac = []( auto x ) { return Height( x->lc ) - Height( x->rc ); };
auto AvlBalanced = []( auto x ) { return ( -2 < BalFac( x ) ) && ( BalFac( x ) < 2 ); };
template <typename T> class AVL : public BST<T, false> {
public:
   BNP<T> insert( const T& );   //重写
   bool remove( const T& );     //重写
};                              //BST::search()等接口直接沿用
```

AVL 是渐近平衡的：高度h的 AVL 树规模至少为$S(h) = S(h-1) + S(h-2) + 1$（最瘦情形：根的两侧分别是高$h-1$与$h-2$的最瘦 AVL）——即 Fibonacci 数！$S(h) = fib(h+2) - 1$，数列依次为 1, 2, 4, 7, 12, 20, 33, 54, 88, 143, 232, ...。于是n节点的 AVL 高度$h = O(\log n)$——最瘦也不失对数高度。Fibonaccian 树内部节点的 balFac 均为+1 或−1，删除任何节点高度都会下降。

12.4 AVL 插入：失衡与双旋

插入或删除后，沿父链上溯的某些祖先可能失衡。**最低的失衡节点 g**（不低于新节点的祖父）是复衡的入口；定义 $p = \text{TallerChild}(g)$、$v = \text{TallerChild}(p)$——名分唯一确定。按 p 与 v 的同侧/异侧分两种情形：

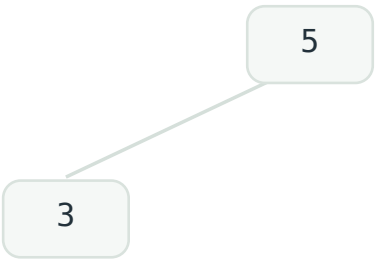
单旋（LL 或 RR）：p 与 v 同侧，一次 zag（或 zig）即可。子树不仅恢复平衡，**高度也复原**——更高祖先即便此前失衡也必一揽子复衡，整个插入至多做一次旋转。

双旋（LR 或 RL）：p 与 v 异侧，先以 v 为轴旋 p（zag 或 zig），再旋 g。以插入序列 5、3、4 为例——4 落在 3 的右肩，g=5 失衡，p=3 与 v=4 异侧（LR）：

AVL双旋：LR失衡与复衡 第 1 / 4 步

1. insert(3)。

insert(3)。



插入3：成为5的左孩子，树仍平衡（各点|balFac|≤1）。

为什么：平衡因子balFac = h左 − h右。

插入3：成为5的左孩子，树仍平衡（各点|balFac|≤1）。
为什么：平衡因子balFac = h左 − h右。

图 26 逐步演示：AVL 双旋：LR 失衡与复衡

双旋后 v=4 升为子树根，3、5 各归两侧；中序序列 3<4<5 保持不变。同样，高度复原、更高祖先一揽子复衡。故 AVL 插入的总代价为$O(\log n)$——search $O(\log n)$ + 旋转$O(1)$ + 高度更新$O(\log n)$。

12.5 AVL 删除：可能连锁失衡

删除导致的失衡更顽固：g 复衡后子树高度**可能下降**，更高祖先可能接连失衡——需要逐层上溯检查、逐一旋转，最坏$O(\log n)$次旋转。因此 AVL 的 remove()虽仍为$O(\log n)$，常数与实现复杂度都高于 insert()。

12.6 本章要点

- BST 顺序性=中序单调；search/insert/remove 均$O(h)$；插入必为叶，双枝删除转化为删除直接后继。
- 坏形态退化为$O(n)$单链：需要重平衡。
- 等价 BST=中序相同；旋转 zig/zag 上下可变、左右不乱，是重平衡的全部自由度。
- AVL：$|balFac| \le 1$；$S(h) = fib(h+2) - 1$渐近平衡，$h = O(\log n)$；Fibonaccian 树是最瘦 AVL。
- 插入失衡：最低失衡 g、p=TallerChild(g)、v=TallerChild(p)；同侧单旋、异侧双旋；高度复原，至多一次旋转。
- 删除失衡可能连锁：逐层上溯，最坏$O(\log n)$次旋转。

13 伸展树、B 树、红黑树与跳转表

平衡是手段，不是目的。AVL 用严格的平衡因子换来了$O(\log n)$的稳定高度，但四个场景里它各有短板：访问分布极度偏斜时（少数热点键占据绝大多数查询），维持全局平衡是在伺候冷数据；数据在外存时（磁盘一次读一页），二叉树每下一层都要高概率做一次 I/O，两层之间只运一个键太浪费；并发与持久化场景要求每次修改的结构变化越少越好（加锁与版本复制只关心真正变动之处）。本章按讲义 08 章依次介绍四种回应：**伸展树**（自适应，把热点拉到根）、**B 树**（多路多键，一层一个页）、**红黑树**（结构变化至多$O(1)$的重平衡）与**跳转表**（用随机化替代严格平衡）。

13.1 伸展树：访问后即伸展

逐层伸展：访问节点 v 后，沿父链逐层旋转把 v 推到根——zag/zig 处理父侧失衡（与 AVL 同型），zig-zag/zag-zig 处理异侧。深度为 d 的节点经 d 次旋转均可到根。但“一步一步往上爬”有致命弱点：若按序访问$\{1, 2, 3, \ldots, n\}$，每次访问后树高仍保持$O(n)$，总耗时$O(n^2)$——逐层伸展对这类访问序列不起作用。

双层伸展（Sleator & Tarjan, 1985）的构思精髓：**向上追溯两层，而非一层。**反复考察祖孙三代$g = parent(p)$、$p = parent(v)$、v，按相对位置经两次旋转使 v 上升两层、成为子树根：

- **zig-zag / zag-zig**（异侧）：与逐层伸展效果相同——zag(g)后 zig(p)（或对称），v 按中序居中上位；
- **zig-zig / zag-zag**（同侧）：关键差异在此——先对 g、p 旋转，再对新的 p、v 旋转（即先 zig(g)后 zig(p)）。v 直升两层，且原高处的子树被“拦腰折断”甩到另一侧，树高在此后大概率骤降。

```
template <typename T> BNP<T> Splay<T>::splay( BNP<T> v ) {   //将v伸展至根
   BNP<T> p, g;
   while ( ( p = v->parent ) && ( g = p->parent ) ) {        //自下而上，反复双层
      BNP<T> gg = g->parent;                                 //每轮之后v以gg为父
      if ( IsLChild( *v ) == IsLChild( *p ) ) {              //zig-zig / zag-zag
         p->parent->rotateAbove( g );  v->parent->rotateAbove( p );
      } else {                                               //zig-zag / zag-zig
         v->parent->rotateAbove( p );  v->parent->rotateAbove( g );
      }
      if ( !gg ) _root = v;                                  //v已到根
      else ( g == gg->lc ) ? gg->lc = v : gg->rc = v;        //接入更高层
      updateHeightAbove( gg );                               //gg以下的高度
   }
   if ( p = v->parent ) { p->rotateAbove( v ); updateHeight( p ); }   //奇数层：最后单旋
   v->parent = nullptr;  _root = v;  return v;
}  //每次伸展 amortized O(logn)
```

接口上，伸展树的 `search()`在查找（无论成败）之后都把最后访问的节点 `splay` 到根——`return _x ? splay(_x) : _p ? splay(_p) : nullptr;`：命中者、或失配时的_hot，随即成为根。`insert/remove` 亦复如此（remove 后让其前驱/后继伸展至根）。整套操作的摊还复杂度$O(\log n)$（证明见教材§8.1.4 的势能分析），而且它**不需要记录与维护任何平衡标志**。自适应特性意味着：若访问序列有强烈的时间局部性（刚访问的键很快再被访问），实际运行时间可以远小于$O(m \log n)$。代价是“结构变化剧烈”（最差每次旋转$O(\log n)$次）——这对并发加锁不友好，却催生了下面的红黑树。

13.2 B 树：为分级存储而生

动机。在分级存储体系中（内存–磁盘），BBST 查找虽然渐进$O(\log n)$，但每深入一层都高概率要做一次 I/O；一次 I/O 的代价约为一次内存访问的10^5倍。改善方向有二：充分利用每个缓存页（一页装多个键），尽可能以短步长访问页内数据。等价变换：BST 每$\lceil m/2 \rceil$代合并为一个**多路多键节点**——这样的节点恰好存入一个缓存页，I/O 次数随树高降至常规 BST 的$\log_2 m$倍。

结构。m阶 B-树（(a,b)-树，Bayer & McCreight 1970；实际常取$m = 200 \sim 300$）：

- 外部节点（nullptr）：理想平衡——深度全部相等，约定以h为树高（比常规 BST 的高度约定大 1）；
- 内部节点：分支数上限m（关键码至多$m - 1$）；除根以外下限$\lceil m/2 \rceil$（关键码至少$\lceil m/2 \rceil - 1$）；根至少 2 个分支。

叶节点=外部节点的父亲，同在深度$h-1$。高度$h \le \log_{\lceil m/2 \rceil}(\frac{n+1}{2})$——即便最瘦，仍是对数级。

插入与上溢。查找自根下行至叶，新关键码插入叶内。若节点关键码超出$m-1$个，发生**上溢**：取**中间关键码上提**一层插入父节点，该节点分裂为左右两半；父节点也可能因此上溢，分裂逐层向上传播。唯一使树高增加的情形：根上溢分裂，m路中分，产生新的根。

B树插入：[5,8,11]怎样分裂 第 1 / 5 步

1. 插入5

3阶B树（(2,3)–树）：节点至多2个关键码、3个分支

根=[5]。

为什么：规模1，无上溢。

空树：5成为根，单节点[5]。

为什么：B树插入总是先落在叶节点。

图 27 逐步演示：B 树插入：[5,8,11]怎样分裂

删除与下溢。删除自根下行定位；即便目标不在叶，也可经后继交换使之“成为”叶（与 BST 的双枝删除同理），最终总在叶节点中删除。删除后若节点关键码少于$\lceil m/2 \rceil - 1$，发生**下溢**：先尝试**旋转**——从左（右）兄弟借一个关键码经父节点周转；兄弟也不够时执行**合并**——自身、父节点的一个关键码与左（右）兄弟三合为一，父节点因此可能下溢，合并逐层向上传播。树高降低的唯一情形：根下溢且无关键码，两个孩子合并为新的根。

13.3 红黑树：至多 O(1)的结构变化

动机（并发性与持久性）。数据库的每次修改需要在结构变动处加锁，访问延迟取决于需要加锁的局部数量：伸展树最差$O(\log n)$处、AVL 的 insert 可保证$O(1)$但 remove 可能$O(\log n)$。另一场景是**持久化**（persistence，支持对历史版本的访问）：蛮力为每个版本独立保存需$O(nh)$空间；利用相邻版本的结构共享可压到$O(n)$——前提同样是单次操作的结构变化要少。红黑树保证**无论 insert/remove，结构变化（重构+染色）至多$O(1)$**，重染色虽可$O(\log n)$，但不涉及结构的锁与复制。

性质。红黑树是二叉搜索树，每个节点附加红/黑颜色，满足：

1. 树根必为黑色；
2. 外部节点（nullptr）视作黑色叶；
3. 红节点之子必黑——树中不存在双红；
4. 任一节点到其后代外部节点的路径上，黑节点数目相同（**黑深度**一致）。

由此，红黑树的高度不超过$2\log_2(n+1)$：黑高度至多$\log_2(n+1)$，红色至多把路径拉长一倍。红黑树与 4 阶 B 树等价——黑节点与其红孩子合并即成 4 阶 B 树节点，红黑树正是 4 阶 B 树的二叉树化身（红=内部通道，黑=关键码），B 树的理想平衡由此转化为红黑的高度上界。

插入：双红修正。新节点染红插入（满足性质 4，不破坏黑高度），唯有与黑父相邻的双红违例需修复。设 g（祖父）为黑、p（父）红、v 红：

- **叔父 u 为黑**：以 v 为轴做 **3+4 重构**（三次节点、四个子树重排），p 转黑、g 转红——一次终止；
- **叔父 u 为红**：g 转红、p 与 u 转黑（子树黑高度不变），双红可能上移到 g 与其父——逐层向上重染色，至多$O(\log n)$次（但全部只是染色，无结构变化）。

删除：双黑修正。删除后可能引发双黑（路径黑数少一）下溢，情形更多（兄弟为黑有红孩子：借位重构一次终止；兄弟为黑无红孩子：兄弟转红、双黑上移；兄弟为红：旋转变形后归入前两类），至多$O(\log n)$次重染色、$O(1)$次重构。同样满足"结构变化至多$O(1)$"的承诺。

13.4 跳转表：概率平衡

Skip List（Pugh, 1990, "A Probabilistic Alternative to Balanced Trees"）：放弃维持严格平衡，改用随机化。每个元素是一个塔（Tower）——由底向上以概率 1/2 逐层"生长"；整表即一个塔列表（`Skiplist : List<Tower<T>>`），同一层的塔横向以 next 链串接，构成一条有序链。

查找自顶层向右下行：当前层能前进就前进，否则下一层。每层链长期望减半，期望比较次数$O(\log n)$；塔的期望高度$O(\log n)$。插入：先按底层有序链定位、随机决定塔高，沿途层间用水平链接入——只动局部指针；删除对称。无需旋转、无需染色，实现简单，缺点是常数因子大于红黑树、且性能只有概率保证（最坏情形可以构造出很坏的单链，但概率可忽略）。

13.5 本章要点

- 逐层伸展对顺序访问序列无能为力（$O(n^2)$）；双层伸展 zig-zig 先旋 g 后旋 p，摊还$O(\log n)$；伸展树免平衡标志、自适应局部性，但结构变化可达$O(\log n)$处。
- B 树$=(a,b)$-树：分支$[\lceil m/2\rceil, m]$，外部节点理想平衡；高度$O(\log_m n)$，每层一次 I/O、一页一节点。
- B 树插入上溢：中间码上提分裂，可逐层传播；树高+1 仅当根分裂。删除下溢：先旋转借位、后三合一合并，可传播；树高−1 仅当根下溢。
- 红黑树四性质（根黑、外黑、无双红、黑深度一致）$\Rightarrow$高度$\le 2\log_2(n+1)$；红黑树=4 阶 B 树的二叉化身。插入双红：叔父黑 3+4 重构一次终止，叔父红重染色上溯；删除双黑至多$O(\log n)$染色+$O(1)$重构——并发的锁粒度、持久化的版本共享由此受益。
- 跳转表以随机塔高替代平衡维护：期望$O(\log n)$查找/插入/删除，实现最简、只有概率保证。

14 区间树与 kd 树：范围查询的两种武器

一维与二维的范围查询，是向量、栈、树之后的新形态问题：给定一组区间，问哪些刺穿了某个点（**穿刺查询**，stabbing query）；给定平面上一组点，问哪些落在矩形里（**正交范围查询**）。蛮力逐个检视当然可行，但"区间集相对固定、查询高频"的场景里，预处理成专门的结构才能换来输出敏感的在线查询。本章按讲义 07 章介绍三类结构：区间树（一维穿刺）、kd 树（多维划分）与四叉树（空间递归压缩），并顺带认识离散化与作用域这两个通用技巧。

14.1 穿刺查询与离散化

在 x 轴上给定 n 段区间，对任意查询点 qx，筛选出所有包含它的区间。先看一类简化情形：所有区间的交非空，取公共点 m。预处理按"由远及近"排序所有左、右端点；此后对任意 qx（不妨设 qx 在 m 左侧），所有区间左端点都在 qx 左侧，**是否被刺中仅取决于右端点是否足够靠右**——从远到近逐个检视右端点即可。一般情形用**离散化**（discretization）：把 n 段区间的$2n$个端点排序，相邻端点之间形成$2n-1$段**初级区间**（elementary interval, EI）。关键观察：**同一段 EI 内的任何位置，穿刺查询的输出必然完全相同**。于是把所有 EI 排成有序向量，各段预先记录对应的查询输出，查询时二分定位、直接输出——$O(\log n + r)$（r 为命中数）。隐患在最坏情况的空间：每段区间可能横跨$\Theta(n)$段 EI，总共$O(n^2)$——**空间爆炸**。

14.2 区间树：作用域与分治

区间树以分治化解空间爆炸。取所有端点的中位点m为界，把区间分作三类：横跨m的（左端点在左、右端点在右）、整体在左、整体在右。**横跨者存入当前根节点**，其余交给左右子树递归处理，直至无需划分。每个节点维护**作用域**$R(v)$：叶节点的作用域是对应的 EI，内部节点的作用域是孩子作用域之并；叶节点预先记录$Int(v)$=横跨$R(v)$的所有区间。查询时沿树下行到 qx 所在的叶，沿途累加节点中的区间：

$$\text{刺中 qx 的区间} = \bigcup_{v\ \text{在路径上}} Int(v) \cup \text{沿途各节点的横跨区间}$$

每层只需$O(1)$或$O(\log n)$处理横跨者（按右端点有序），总查询$O(\log n + r)$，预处理空间$O(n)$——输出敏感、在线，空间爆炸不复存在。

14.3 kd 树：平面交替二分

kd 树处理二维（乃至 d 维）的**正交范围查询**。构造（buildKdTree）：根节点对应整个平面的包围矩形；**交替地**按 x、y 坐标划分——深度偶数层取 VERTICAL（竖直线），奇数层取 HORIZONTAL（横线）；分割线取当前点集在该维度上的**中位数**（findMedian 线性$O(n)$可完成）；每个矩形区域约定左闭右开、下闭上开以避免歧义；递归至每个区域至多一个点，成为叶。

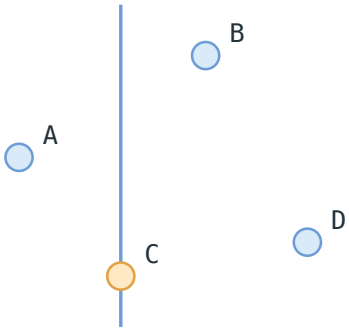
```
buildKdTree(P, d):
   if |P| < 2: return createLeaf(P)                    //递归基
   Root = createKdNode()
   Direction = even(d) ? VERTICAL : HORIZONTAL
   Line = findMedian(P, Direction)                      //O(n)
   (L, R) = divide(P, Direction, Line)                  //分而治之
   Root.lc = buildKdTree(L, d+1);  Root.rc = buildKdTree(R, d+1)
```

四个点的划分实例：

kd树：平面交替二分 第 1 / 4 步

1. 第1层：按x轴中位点划分

kd树：交替按x、y轴二分平面（左闭右开）



竖线x=150把平面分成两半。

四个点的x中位数取C.x=150，画竖直分割线；C归右侧（左闭右开）。

为什么：偶数层取VERTICAL，奇数层取HORIZONTAL。

图 28　逐步演示：kd 树：平面交替二分

查询（kdSearch）：对每个节点试探其区域与查询矩形 R 的关系——

- $region(v) \subset R$：整棵子树输出（递归基 #1）；
- $region(v)$ 与 R 不相交：整枝剪除（递归基 #2）；
- 部分相交：深入两个子树。

剪除是效率的来源：大量子树因区域不相交被整体跳过。2 维的复杂度为$O(\sqrt{n}+r)$，d 维一般地$O(n^{1-1/d}+r)$——仍优于蛮力的$O(n)$，且支持在线查询。

14.4 四叉树与空间递归

同一思路在图像与空间数据上的延伸是**四叉树**（quadtree）：把正方形区域四分为田字格，非均匀子区域继续递归细分，直至均匀或达到粒度下限——图像压缩、碰撞检测常用。三维对应**八叉树**（octree），按 x、y、z 逐层合并的思想也可推广出更高维的变体。kd 树与四叉树是同一哲学的两种切法：前者按中位数交替维度二分（区域大小自适应数据分布），后者按固定四分（区域规整、索引计算简单）。

14.5 本章要点

- 穿刺查询的离散化：EI 段内输出相同，二分$O(\log n + r)$；最坏$O(n^2)$空间爆炸。
- 区间树：中位点分三类、横跨者存根、作用域$R(v)$与$Int(v)$；查询$O(\log n + r)$、空间$O(n)$。
- kd 树：交替维度中位数二分，左闭右开消歧义；查询三分（包含则整树输出、相离则整枝剪除、相交则深入）；d维$O(n^{1-1/d}+r)$。
- 四叉树/八叉树：固定四分（八分）递归细化，图像压缩与空间索引。

15 散列：循关键码访问、冲突排解与重散列

向量靠秩寻址、BBST 靠比较寻路，散列提出第三种访问方式：**循关键码访问**（call-by-key）——只要拿到键，直接算出它的存储位置，期望$O(1)$，与数据规模无关。代价是键空间与桶空间通常不等大，冲突不可避免；本章的全部内容，可以概括为"两问"：**如何让冲突尽量少**（散列函数的设计），以及**冲突来了怎么办**（排解策略与重散列）。

15.1 散列原理：桶、装填因子与生日悖论

散列表（hashtable）是容量为 M 的桶（bucket）数组；**散列函数** `hash(key)`把任意键确定地映射到$[0, M)$内的桶号。词条键空间可达天文数字而桶数组有限，冲突必然发生——把在座同学按生日（月/日）散列到$M = 365$个桶，装填因子

$$\lambda = N/M = \text{在场人数}/365$$

哪怕只有 23 人，"至少两人生日相同"的概率已过半：冲突是常态而非意外。于是散列策略的选取至关重要：**完美散列**（perfect hashing，两级散列模式、键集固定时可做到无冲突且$O(1)$）可期而不可求；一般情形下要降低冲突概率、并准备好冲突的排解预案。

15.2 散列函数：设计原则与几种配方

好散列函数的四条原则：**确定**（同一键总映射到同一地址）、**高效**（期望$O(1)$）、**满射**（充分利用整个散列空间）、**均匀**（每个键落到每个桶的概率接近$1/M$，有效避免聚集）。

除余法（division method）：`hash(key) = key % M`。对理想随机的键，每个桶有$1/M$的均等概率；"M 取素数分布最均匀"的流传说法，需要辨析——对理想随机序列，表长是否素数无关紧要（判断标准是序列的 Kolmogorov 复杂度：生成它最短需要多少行代码）。真实数据远非理想随机：算术级数（单调线性）、循环级数（周期性）、自然语言（局部性）的规律普遍存在。以周期为 p 的一组键为例：它们只能入住间隔为$gcd(p, M)$的桶，同一桶内的键周期为$lcm(p, M) = gcd(p, M) = 1$时最好；为应对某一个这样的数据集可取 M 与 p 互素，为应对所有这样的数据集，唯有取 M 为素数。

MAD 法（Multiply-Add-Divide）：`hash(key) = (a·key + b) % M`。除余法有个不动点——无论表长多少，`hash(0)=0` 总成立，且相邻的键总是映射到循环相邻的桶，引发**初级聚集**（键连续的词条占据一段区间，区间内新冲突还会加剧）与**二级聚集**（连锁引发本不该有的冲突）。MAD 法以线性变换打散原有次序；再配合 a、b 的随机选取，即得**通用散列**（universal hashing）：随机化保证任意两个键冲突的概率不超过$1/M$。

更多配方：平方取中（取key^2的中间若干位——平方把高位信息搅进中位）、折叠法（等宽分段求和，如 `hash(123456789) = 123 + 456 + 789 = 1368`，或往复折返$123 + 654 + 789$）、数字分析（抽取某几位）。

15.3 闭散列：四个桶寻找下一个家

闭散列（开放定址）：冲突发生时按既定策略在表内寻找下一个空桶。实现要维护两个计数——N个正在使用的桶、U个曾使用过的桶（$N \le U \le M/2$，靠装填因子上限保证）。

线性试探（linear probing）：冲突就依次向后一个桶找。优点是" cache 友好"；缺点是**初级聚集**：

线性试探：冲突怎样连成一片 第 1 / 5 步

1. 空表

闭散列·线性试探：M=7，hash(k)=k % 7

None

7个桶全部为空。
为什么：hash(k) = k % M 定位首选桶。

图 29 逐步演示：线性试探：冲突怎样连成一片

M=7 时依次插入 12、26、19（都映射到桶 5）：12 落[5]；26 试探[6]；19 被[5][6]连环挡驾、落到[7%7]=[0]。三桶连成一片后，任何 hash 值落在这段区间的键都要连环试探。这引出**懒惰删除**（lazy deletion）的必要性：闭散列表里直接把桶清空，会切断后续键的试探链——remove 只做标记，查找越过标记继续。于是 Hashtable 需要 `used[]`标记数组与 N、U 双计数：

```
template <typename K, typename V> struct Hashtable : Dictionary<K,V> {
   Entry<K,V>** ht;  Rank M;          //容量为M的桶数组
   bool* used;  Rank N, U;            //N个在用，U个曾用：N <= U <= M/2
   Entry<K,V>* get( K );  bool remove( K );  bool put( K, V );
   void rehash();                     //重散列：必要时扩充，保证装填因子低于阈值
};
```

平方试探（quadratic probing）：冲突后按偏移量$1^2, 2^2, 3^2, \ldots$跳跃，缓解初级聚集；但仍可能**二次聚集**（同义键走同一条试探链）。只要 M 为**素数**且装填因子$\lambda \leq 50\%$，双向平方试探（交替取$\pm j^2$）必能终止于某个空桶——这也解释了 `N ≤ U ≤ M/2` 上限的来历。**再散列**（rehashing）：λ超过阈值时，开辟更大的表（通常翻倍、且仍取素数），把全部词条重新散列——虽是$O(n)$的大动作，但均摊到之前省下的每次$O(1)$操作，无伤大雅。其余变体：**再散列双函数**（冲突时换用第二个散列函数计算步长）、**公共溢出区**（冲突词条统一另开一表）。

15.4 拉链法：每个桶接一条链

独立链（separate chaining, 拉链法）：每个桶不存词条本体，而挂一个列表；冲突的词条直接接进同一桶。优点直观——删除是真正的删除（无懒惰标记）、装填因子可以超过 1、实现简单；代价是指针的额外空间与内存访问的局部性损失。工程上的取舍常在闭散列（空间紧、cache 友好）与拉链（删除干净、λ 自由）之间按负载特征定夺。

15.5 词典接口与本章要点

散列表对外提供词典（Dictionary）三接口：`get(k)`按线性试探第一阶段定位（途经的第一个空桶记入_s，终点记入_r）；`put(k, v)`先查后插、必要时 rehash；`remove(k)`懒惰删除并维护 N、U。要点收拢：

- 散列 = 循关键码访问；λ=N/M 度量拥挤，冲突是常态（生日悖论：λ 过半即大概率冲突）。
- 散列函数四原则（确定/高效/满射/均匀）；除余法取素数 M 是对抗一切周期性数据的唯一通用解；MAD 法消不动点、通用散列给概率保证；平方取中/折叠各有巧思。
- 闭散列：线性试探快但初级聚集；懒惰删除保试探链（N≤U≤M/2）；平方/双向平方试探消聚集（M 素数 +λ≤50% 必终止）；λ 超阈值即 rehash（均摊无伤）。

- 拉链法：桶挂列表，删除干净、λ 自由，牺牲空间与 cache 局部性。
- 词典三接口 get/put/remove 全部期望$O(1)$——这就是散列的承诺。

16 图：遍历、拓扑排序与双连通

图（graph）$G=(V,E)$由顶点与联边组成——地图上的城市与道路、课程与先修关系、任务与依赖，都是图。前几章的树其实是图的特例（连通无环）；本章从图的表示出发，讲两遍遍历——BFS 与 DFS——以及它们的三个直接产出：距离与中心（Erdös 数）、拓扑排序（依赖调度）、双连通分量与关节点（网络鲁棒性）。第 10 章的层次遍历，正是本章 BFS 在树上的特例。

16.1 图的表示：邻接矩阵与邻接表

邻接矩阵：$n\times n$矩阵，`E[i][j]`记录 i 到 j 的边（无向图对称）。优点：直观、判断两点是否有边$O(1)$、顶点度数$O(1)$，尤其适用于**稠密图**。缺点：空间$O(n^2)$，与边数无关——平面图（可嵌入平面的图）由 Euler 公式$n-e+f=2$可推出$e\le 3n-6$，空间利用率仅$\approx 1/n$，对**稀疏图**（sparse graph）极不经济。

邻接表：每个顶点挂一个邻接顶点列表。空间$O(n+e)$，与实际边数成正比；递增式构造$O(n+e)$；判断 u 是否为 v 的邻居需遍历 v 的表，$O(1+deg(v))$。折中方案：邻接表+散列表存边——判边期望$O(1)$、空间$O(n+e)$两全。各种图算法共同的底层需求是枚举 v 的所有邻居（$O(1+deg(v))$），两种表示都能胜任。

16.2 广度优先搜索：图的层次遍历

始自顶点 s 的 BFS：访问 s；依次访问 s 所有尚未访问的邻接顶点；再依次访问它们尚未访问的邻居……直到没有新顶点——策略与过程完全等同于第 10 章树的层次遍历（FIFO 队列），且确实构造出原图的一棵**支撑树**（BFS tree）。BFS 树有个漂亮的解释：树上每个节点的 `dTime` 即 s 到它的**最短距离**（按边数计）——社交网络里的 Erdös Number（与 Erdös 合写过论文者为 1，与 1 合拍过……）与 Chow Number（周星驰合作网络：吴孟达为 1、葛优经吴倩莲为 2）正是 BFS 距离；对无向图跑一遍 BFS，即得图的**离心率**（某点到最远点的距离）、**半径**（最小离心率）、**直径**（最大离心率）与**中心**。

```
template <typename TV, typename TE> void Graph<TV, TE>::BFS( Rank v, Rank& dClock ) {
   Queue<Rank> Q;  status(v) = DISCOVERED;  Q.enqueue( v );  dTime( v ) = dClock++;
   while ( !Q.empty() ) {
      v = Q.dequeue();                          //取出队首顶点v
      for ( Rank u = firstNbr( v ); -1 != u; u = nextNbr( v, u ) )
         if ( UNDISCOVERED == status( u ) ) {       //u尚未被发现
            status( u ) = DISCOVERED;  Q.enqueue( u );
            dTime( u ) = dClock;  type( v, u ) = TREE;  parent( u ) = v;    //树边
         } else
            type( v, u ) = CROSS;                   //跨边
      status( v ) = VISITED;  fTime( v ) = ++fClock;    //v访问完毕
   }
}  //O(n + e)：邻接表下每顶点入队一次、每边至多查两次
```

复杂度$O(n+e)$。每条边在遍历中被归类：发现的边为**树边**（TREE），指向已在队列或已访问顶点的为**跨边**（CROSS）——无向图的 BFS 里不会出现反向边。

16.3 深度优先搜索：一条路走到黑

DFS：访问 v；若 v 尚有未访问邻居，任取其一递归 DFS；否则回溯；全图遍历时，若还有未访问顶点，任取其一重新启动。对树而言等效于先序遍历；对图则导出一个 **DFS 森林**。递归实现配 `dTime`（发现时间）与 `fTime`（访问完毕时间）两个时钟：

```
template <typename TV, typename TE>
void Graph<TV, TE>::DFS( Rank v, Rank& clock ) {
   dTime( v ) = ++clock;  status( v ) = DISCOVERED;
   for ( Rank u = firstNbr( v ); -1 != u; u = nextNbr( v, u ) )
      if ( UNDISCOVERED == status( u ) ) {
         type( v, u ) = TREE;  parent( u ) = v;  DFS( u, clock );
      } else if ( VISITED == status( u ) )
         type( v, u ) = BACKWARD;               //指向自己的祖先：回路存在的证据
      else
         type( v, u ) = ( dTime( v ) < dTime( u ) ) ? FORWARD : CROSS;
   fTime( v ) = ++clock;  status( v ) = VISITED;
}
```

DFS 最锋利的副产品是**括号序列**：把 dTime 记作(、fTime 记作)，整幅图的括号序列中任意两对括号要么完全嵌套、要么完全分离——由此可$O(1)$判定祖先关系（u 是 v 的祖先当且仅当$[dTime(u), fTime(u)]$包含$[dTime(v), fTime(v)]$），并把边精确分为四类：树边 TREE、回溯边 BACKWARD（指向祖先——**有向图存在回路当且仅当遍历中出现回溯边**）、前向边 FORWARD、跨边 CROSS。

16.4 拓扑排序：DAG 的线性次序

任务之间有依赖（A 须在 B 之前），求一个不违反任何依赖的执行顺序。依赖图若有环则无解；**有向无环图**（DAG）则必然可解——拓扑排序对应于把 DAG 的偏序（POSET）扩充为全序（TOSET）。正确性的钥匙：**有限的偏序集必有极值元素**——DAG 中必有零入度顶点 v 可作序列起点，且$G \setminus \{v\}$仍是 DAG——减而治之：

```
template <typename TV, typename TE> Stack<TV>* Graph<TV, TE>::topoSort( Rank s ) {
   Stack<TV>* S = new Stack<TV>;            //用栈最终逆序输出
   while ( 0 < ( s = zeroInDegree() ) ) {   //策略1：循序输出零入度顶点
      status( s ) = VISITED;
      for ( Rank u = firstNbr( s ); -1 != u; u = nextNbr( s, u ) )
         InDgree( u )--;                    //后继入度减一
      S->push( s );                         //零入度者压栈
   }
   if ( ( Rank ) G.size() > S->size() ) S->push( nullptr );   //残留非空：存在环路
   return S;
}  //O(n^2)；改用反向邻接表+零入度队列，可至O(n+e)
```

残留顶点非空即说明图中有环（所有剩余顶点入度均大于零）。策略二借助 DFS：**第 k 个回溯（fTime）的顶点，恰是拓扑序的倒数第 k 个**——逆序输出 fTime 序即得拓扑序；两种策略导出的序列可以不同，但都合法（拓扑排序通常不唯一）。

16.5 双连通分量与关节点

无向图中删除后使连通性被破坏的顶点称**关节点**（articulation point）；不含关节点的极大连通子图称**双连通分量**（biconnected component）——网络鲁棒性的基本度量：关节点倒下，系统分裂。DFS 以$O(n+e)$找出全部关节点，核心是为每个顶点维护 **hca**（highest connected ancestor，最高可达祖先）：$hca(u) = \min\{dTime(w) \mid (u,w)$**为回溯边或** $w = u\}$沿子树取最小。判定规则：

- **树根**是关节点，当且仅当它在 DFS 森林中有**两个以上的孩子**（各有独立子树，无横跨边相连）；
- **非根 u** 是关节点，当且仅当存在孩子 C，其子树（含 C）通过回溯边最高只回到 **u 或更浅**——即$hca(C) \geq dTime(u)$，该子树与祖先之间没有旁路。

借助 hca 还能顺带标出每个双连通分量的边界：$hca(C) < dTime(u)$时子树可上溯更高祖先，u与 C 之间恰好切出一个双连通分量。整个算法一遍 DFS 完成，$O(n+e)$。

16.6 本章要点

- 邻接矩阵宜稠密（判边$O(1)$、平面图$e \leq 3n-6$故稀疏图浪费）；邻接表宜稀疏（$O(n+e)$）；表+散列判边期望$O(1)$两全。

- BFS=队列=层次遍历推广，BFS 树给出最短距离（Erdös 数、离心率/半径/直径/中心），$O(n+e)$，边分类 TREE/CROSS。
- DFS=递归=先序，DFS 森林；dTime/fTime 括号序列$O(1)$判祖先；边四分类，**回溯边⇔有向回路**。
- 拓扑排序：DAG 必有零入度顶点，减而治之；或逆 DFS 回溯序；残留非空⇔有环。
- 关节点：根（两棵以上孩子）与非根（$hca(C) \geq dTime(u)$）两条判据；双连通分量一遍 DFS 全部标出。

17 最短路径与最小生成树

带权图上的两类经典优化。**最短路径**（shortest path）：从顶点 s 出发，到每个顶点的最短路径及长度——SPT（最短路径树）；**最小支撑树**（MST）：连通网络上挑选$n-1$条边，让连通保持、总权重最小——运输、电信、VLSI 布线的共同模型，也是众多 NP 难问题的近似解跳板。两者形似（都借优先级队列贪心生长）而质异：前者最小化“路径长度”，后者最小化“边权总和”，答案不可互换。本章依次给出 Dijkstra（单源）、Floyd（每对顶点）、Prim 与 Kruskal（MST）。

17.1 最短路径树与贪心的通行证

WLOG 设每条最短路径唯一，则所有最短路径的并构成一棵树（SPT）。Dijkstra 算法赖以成立的两条性质：**最短路径的每个前缀也必是最短路径**（否则把坏的段替换掉，总长更短）；**沿任何最短路径，各节点到起点的最短距离单调递增**（边权非负）。于是从 S 出发，每次从候选中取出“当前 dist 最小且未确定”的顶点——它不可能再被绕行改进，可以放心“盖章”；随后用它的边松弛邻居。

17.2 Dijkstra：优先级搜索

教材把 Dijkstra 统一进图的 `pfs()`优先级搜索框架——与 Prim 共用骨架，只是优先级更新器不同：

```
g->pfs( 0, DijkPU<char, Rank>() );   //从顶点0出发启动Dijkstra
template <typename TV, typename TE> struct DijkPU {    //优先级更新器
   virtual void operator()( Graph<TV, TE>* g, int uk, int v ) {
      if ( UNDISCOVERED == g->status( v ) )             //尚未发现的邻居v
         if ( g->priority( v ) > g->priority( uk ) + g->weight( uk, v ) ) {
            g->priority( v ) = g->priority( uk ) + g->weight( uk, v );   //更新优先级
            g->parent( v ) = uk;                        //更新父节点
         }
   }
};
```

以四顶点图走一遍松弛过程（$S\to A=10$、$S\to B=3$、$B\to A=4$、$A\to C=2$、$B\to C=8$）：

Dijkstra：S/A/B/C的松弛过程 第 1 / 5 步

1. 初始化

Dijkstra：每步取出dist最小且未定的顶点，松弛其邻居

`dist: S=0 A=∞ B=∞ C=∞`

已确定：（无）

等待取出S。

dist[S]=0，其余∞。

为什么：最短路径树从S生长；每个前缀本身就是最短路径。

图 30　逐步演示：Dijkstra：S/A/B/C 的松弛过程

结果：dist = S:0、B:3、A:7、C:9。复杂度：优先队列实现下$O((n+e)\log n)$；$O(n^2)$的朴素数组版本对稠密图反而更快。**负权边时 Dijkstra 的贪心失效**（已盖章顶点可能被负权绕行改进）——单源需 Bellman-Ford（$O(ne)$，可检出负环）；每对顶点的全源问题另有更省的 Floyd。

17.3 Floyd–Warshall：每对顶点

需要所有顶点两两之间的最短距离时，逐点跑 Dijkstra 要$O(n \cdot (n+e)\log n)$；Floyd-Warshall 以动态规划$O(n^3)$、$O(1)$额外空间完成（允许负权、不许负环）。递推只问一件事：**中途点是否经过 k**？

$$d^{(k)}[i][j] = \min\big(d^{(k-1)}[i][j],\, d^{(k-1)}[i][k] + d^{(k-1)}[k][j]\big)$$

$d^{(k)}[i][j]$表示“允许途经$\{0..k\}$中顶点”时 i 到 j 的最短距离；k 一维一维地放开，三轮循环写完即得全表。路径重建借助 `next[i][j]`（i 出发去 j 时的下一站）随更新同步维护。

17.4 最小支撑树：割定理与两个贪心

MST 总是存在（支撑树虽多毕竟有限，权重可比），且未必唯一。两个算法共享同一块基石——**割定理**（cut property）：把顶点集分成两部分，横跨割的最轻边必属于某棵 MST（否则把它换进树里，总权下降）。

Prim（Jarník 1930 / Prim 1957）：从单个顶点出发生长支撑树，每步取**横跨“树内/树外”割**的最轻边、把对应顶点拉进树内。实现与 Dijkstra 共用 `pfs()`框架，优先级更新器换成 PrimPU——优先级不再是“到源点的距离”，而是“**到已生长部分的最短边权**”。每次$O(\log n)$，总体$O((n+e)\log n)$。

Kruskal（1956）：视角从“顶点生长”换成“边排序”。所有边按权重递增排序，逐边考察——两端点尚不连通则接受，否则舍弃，直至选满$n-1$条。判连通用第 09 章的散列与并查集思路：find 查询、union 合并，每次近$O(1)$，总体$O(e\log e)$（排序主导）。Kruskal 的正确性同样出自割定理：每次被接受的边恰是当时某个割的最轻边。

延伸：这一支算法家族历史久长——Boruvka（1926）、Jarník（1930）、Prim（1957）、Kruskal（1956）、Karger-Klein-Tarjan（1995，线性期望）、Chazelle（2000，软堆$O(e\alpha(e,n))$）；近亲问题里 Proximity Graphs 可$O(n\log n)$，而 Steiner MST（允许新增 Steiner 点）则是 NP-hard——同族问题的难度可以差一个数量级。

17.5 本章要点

- SPT：最短路径的前缀也是最短路径；距离沿最短路径单调递增——Dijkstra 贪心的通行证。
- Dijkstra = PFS+DijkPU（priority = 到源距离），$O((n+e)\log n)$；负权失效，Bellman-Ford 补位。
- Floyd-Warshall：$d^{(k)}[i][j] = \min(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j])$，$O(n^3)$、原地滚动，全源最短。
- MST 割定理：横跨割的最轻边必在 MST 中；Prim 顶点生长（PrimPU）、Kruskal 边排序+并查集判环，分别$O((n+e)\log n)$与$O(e\log e)$。
- 家族谱系从 Boruvka 到 Chazelle；Steiner MST 已 NP-hard——模型之差，难度天壤。

18 优先级队列：完全二叉堆、Floyd 建堆与左式堆

按任务优先级调度：insert 随时插入，delMax 随时取走最高者。排序是一次性的总序，无序数组每次取最大要扫全表——优先级队列（priority queue, PQ）只要一种**较弱的序**：堆序。本章实现三种：完全二叉堆（向量上的隐式完全二叉树）、Floyd 自底向上建堆（$O(n)$批量建堆），以及支持高效合并的左式堆。PQ 此前已经两度登场：Huffman 森林（第 11 章）与 Dijkstra/Prim 的优先级搜索（第 17 章）——本章补齐它们的底层。

18.1 优先级队列 ADT 与完全二叉堆

PQ 的接口极简：`getMax()`查询、`delMax()`摘除最大、`insert(e)`插入（第 11 章 Huffman 森林的 delMax/insert 即此）。**完全二叉堆**（complete heap）借助完全二叉树的**层次次序**把树形压缩进向量：节点 i 的孩子是$2i+1$与$2i+2$，父节点是$\lfloor (i-1)/2 \rfloor$；**堆序性**：处处满足$H[i] \le H[\text{Parent}(i)]$——父不小于子，但兄弟之间无序。

insert：上滤。新词条先接到向量末尾，然后 `siftUp`：与父节点比较，更大则交换上移，直至父不小于己。最坏$O(\log n)$；平均意义下只需$O(1)$步——随机键期望只上升常数层（习题）。

delMax：下滤。最大值必在根。摘除后把末元素搬到根位，`siftDown`：与较大的孩子交换下移，直至不小于两个孩子或到达叶。“割肉补疮”：末元素填补空洞，逐层与孩子换位。

18.2 Floyd 建堆：自底而上的下滤

给定 n 个词条（无序向量），如何批量建成堆？逐个 insert 要$O(n \log n)$。Floyd（1964）观察到：**若左、右子树已是各自的堆，只需对根做一次下滤**即可合并成整体堆。于是自底而上、对每个内部节点（秩$n/2-1$直至 0）逐个下滤：

```
template <typename T>   //Robert Floyd, 1964
void heapify( T* A, Rank n ) {          //Floyd建堆算法
   for ( Rank i = n/2 - 1; -1 != i; i-- )   //自下而上，对每个内部节点
      siftDown( A, n, i );                  //逐个下滤
}  //可理解为子堆的逐层合并，堆序性最终必在全局恢复
```

复杂度为什么是$O(n)$？关键在一句：每个内部节点所需的调整时间正比于其**高度**而非深度——下滤的成本取决于节点到叶的距离。WLOG 考查满树：所有内部节点的高度总和为

$$\sum_{d=0}^{h-1} 2^d (h-d) = 2^{h+1} - h - 2 < 2^{h+1} - 2 = O(n)$$

（实测：$h=4$时和为 26、$n=31$，比值 0.84；$h=7$时 247/255≈0.97——收敛于2^{h+1}。）逐个 insert 的$O(n \log n)$与此$O(n)$的差距，正来自“上滤看深度、下滤看高度”这一视角的颠倒。课堂留有两问：insert 的平均$O(1)$如何证明？heapify 把构造次序颠倒后，为什么复杂度会有实质降低？

18.3 堆排序与优先级队列应用

完全堆在向量上就地运行，直接催生**堆排序**（heapsort，第 12 章亦被排序器随机选用）：`heapify(A, n)`建堆$O(n)$，此后反复 `delMax`（与末元素交换+下滤），n次共$O(n \log n)$——最坏情况仍是最优的排序算法之一，且完全就地。PQ 的应用清单在此收拢：Huffman 森林（第 11 章：delMax+insert，列表实现$O(n^2)$、堆实现$O(n \log n)$、栈+队列$O(n)$）、Dijkstra 与 Prim 的优先级搜索（第 17 章）、事件驱动的银行模拟（第 09 章）、以及操作系统的任务调度。

18.4 左式堆：为合并而生

完全堆的合并（heap-merge）必须搬动全部元素，$O(n)$——当应用需要频繁 heap-merge 时（如可并优先队列），需要新的形态。**左式堆**（leftist heap）：仍满足堆序性，但不再完全；给每个节点记录 **NPL**（null path length，从该节点到其后代中最近外部节点的路径长度；外部节点 NPL=0）。**左式堆的定义**：任何内部节点 x 都左倾——

$$NPL(x.lc) \ge NPL(x.rc)$$

由NPL的归纳性质可得：从 x 出发的右单侧路径长度$NPL(x)$（即右链上的节点数）不超过$O(\log n)$——若右链长为d，则以 x 为根的子树至少含$2^{d+1}-1$个节点（左倾保证每层左侧都有“备份”）。
合并（二路归并）是左式堆的核心操作，递归且只沿右链进行：

```
template <typename T> BNP<T> leftistHeapMerge( BNP<T> a, BNP<T> b ) {   //a、b均为左式堆
   if ( !a ) return b;                     //递归基
   if ( !b ) return a;
   if ( a->data < b->data ) swap( a, b ); //确保a为大顶
   a->rc = leftistHeapMerge( a->rc, b );  //a的右链与b归并
   if ( !a->lc || a->lc->npl < a->rc->npl )
      swap( a->lc, a->rc );               //必要时交换，恢复左倾
   a->npl = ( a->rc ? a->rc->npl : 0 ) + 1;   //更新NPL
   return a;
}  //只可能经过最右侧藤上的节点，O(logn)
```

归并过程中只有最右侧藤上的节点可能暂时失去左倾，逐层回溯时顺手交换恢复；时间正比于两条右链之和，$O(\log n)$。insert = 单节点堆与自堆归并；delMax = 摘根后左右子堆归并——两者都化为 merge。

18.5 本章要点

- PQ 三接口 getMax/delMax/insert；完全二叉堆=向量化的完全二叉树，堆序性“父不小于子”。
- insert 上滤$O(\log n)$（期望$O(1)$）；delMax 末元素补根+下滤$O(\log n)$。
- Floyd 建堆：子堆逐层合并，自底而上对内部节点逐个下滤；成本正比于高度总和$< 2n$，故$O(n)$——上滤/下滤、深度/高度的视角颠倒是一切差别之源。
- 堆排序：建堆$O(n)$+n次 delMax，就地$O(n \log n)$最坏情况最优。
- 左式堆：NPL 左倾保证右单侧路径$O(\log n)$；merge 沿右链递归，insert/delMax 皆化归 merge，$O(\log n)$——可并优先队列的标准解。

19 字符串匹配：KMP 与 next 表

串匹配（string matching）：在文本$T[0,n)$中定位模式$P[0,m)$。蛮力算法 match 逐对齐位置从头比对，最坏$O(n\cdot m)$——比如$T=0^{n-1}1$、$P=0^{m-1}1$，每个对齐都几乎比对到末尾才失败。KMP 算法（Knuth-Morris-Pratt, 1977）做到$O(n+m)$，它的洞察只有一句话：**失配之前的每一次成功比对都是信息，失配后正文指针绝不回退。**

19.1 失配后知道什么：最长自匹配

设失配发生在正文i与模式j处——即前缀$P[0,j)=T[i-j,i)$已经匹配成功。朴素法把i回退到$i-j+1$、j清零重比；KMP 问：利用已知的$P[0,j)$，能把模式**右移多少而不漏解**？答案是“$P[0,j)$的最长真自匹配前缀”：找到最长的$t<j$使

$$P[0,t)=P[j-t,j)$$

模式右移$j-t$位后，这t个字符自动匹配（它们在模式内已经互相对齐验证过），比较直接从i、t继续——**正文指针i永不回退**。这份预案完全由模式自身决定，可以**事先构造**：记作 next 表，$next[j]=t$；约定$next[0]=-1$（失配于 j=0 时模式整体右移一位、j 归零）。

19.2 查询：化腐朽为神奇

```
int match( char* T, int n, char* P, int m ) {        //KMP主算法
   int* next = buildNext( P );                         //构造next表：O(m)
   int i = 0, j = 0;                                   //正文指针、模式指针
   while ( j < m && i < n )
      if ( 0 > j || T[i] == P[j] ) { i++;  j++; }     //若匹配（或j已到最左），携手前进
      else j = next[j];                                //失配：模式右移，i不动
   delete [] next;
   return i - j;                                       //j==m则命中于i-j；否则n-m+1示意失败
}  //O(n + m)
```

每一轮要么i前进（匹配），要么j下跳——j的下跳累计不超过i的前进累计（每次下跳后必有一次匹配使i前移），故总比对次数$O(n)$，加上构造$O(m)$，总体$O(n+m)$。

19.3 构造 next 表：减而治之

next 表自身用 KMP 的思想构造——**模式与自己做匹配**。已知$next[0..j]$，如何得$next[j+1]$？设$t=next[j]$：若$P[j]=P[t]$，则自匹配延长一格，$next[j+1]=t+1$；若不等，则按“自匹配的传递链”回退$t=next[t]$再比——下一候选恰是当前的 next 值，直至命中或-1终止：

```
int* buildNext( char* P ) {                          //构造模式P的next表
   size_t m = strlen( P ), j = 0;                    //"制表"指针
   int* N = new int[m];                              //next表
   int t = N[0] = -1;                                //模式串最左对齐重启
   while ( j < m - 1 )
      if ( 0 > t || P[j] == P[t] ) {                 //匹配
         j++;  t++;
         N[j] = t;                                   //改进版此处另作处理（见下）
      } else                                         //失配
         t = N[t];                                   //传递链回退
   return N;
}  //O(m)：j至多前进m步，t的回退总量不超过前进总量
```

以讲义模式$P=$ MAMMIA逐步构造（复算验证$next=[-1,0,0,1,1,0]$）：

KMP next表：MAMMIA逐步构造 第 1 / 5 步

1. 约定

next表构造（模式P=MAMMIA）：当前算到j=0，t=−1

M	A	M	M	I	A
next[0]=−1	next[1]=0	next[2]=0	next[3]=1	next[4]=1	next[5]=0

起点：next[0]=−1。

next[0] = −1：失配时模式串从最左重启。
为什么：next[j]：P[0,j)中最长真自匹配前缀的长度——失配时j直接跳到next[j]。

图 31　逐步演示：KMP next 表：MAMMIA 逐步构造

三处关键回退都能看清“传递链”：j=4 时$P[1]=A$与$P[3]=M$失配，回退$t=next[1]=0$，$P[0]=M$与$P[3]=M$命中，$next[4]=1$；j=5 时A对I失配回退到 0，M对I仍失配退至-1，$next[5]=0$；j=6 同理得 0。

19.4 next 的改进：一石二鸟

next 表还有一处可打磨：失配于j跳到t后，若$P[t]=P[j]$，则同一正文字符会再次失配——这次跳转白做。改进版在构造时顺手补刀：命中（$P[j]=P[t]$）时令

$$N[j]=N[t]\quad\text{（而非 }t\text{）}$$

把“必然再次失配”的跳转链一次折叠到位。改进只增加一行，查询效率进一步提升；这正是 KMP 作为“化腐朽为神奇”的范本被反复称道之处——**预处理换查询**，空间$O(m)$换时间$O(n)$。

19.5 后继者一瞥：BM 与 Karp–Rabin

讲义还铺开了两条进阶路线。**BM 算法**（Boyer-Moore）：模式**自右向左**比对，坏字符（bad character）与好后缀（good suffix）两张启发式表允许大步跳跃，实际运行常常亚线性。**Karp-Rabin**：为模式与各正文窗口计算滚动指纹（散列，第 15 章的哈希思想），指纹不等直接跳过、相等再精确验证——把匹配问题转成算术。三者的共同哲学与 KMP 一致：**预先计算，运行时不再回头**。

19.6 本章要点

- 蛮力匹配最坏$O(n\cdot m)$；KMP 靠“失配信息”$O(n+m)$。
- next[j] = $P[0,j)$最长真自匹配前缀长度；失配跳转$j=next[j]$，正文指针永不回退。
- next 表以模式自匹配构造，减而治之：命中则+1，失配沿传递链$next[t]$回退；总体$O(m)$。
- 改进 next：$P[t]=P[j]$时取$N[j]=N[t]$，折叠必然失效的跳转。
- BM 自右向左+坏字符/好后缀跳跃；Karp-Rabin 滚动指纹+验证——共同哲学：预处理换运行时。

20 快速排序：轴点、划分与希尔排序

归并排序先分后合：本层便宜、合并是实质；快速排序（quicksort）反过来——**先花本层时间做划分，后层各自自然有序**。划分之后轴点（pivot）各就各位，左段全部不大于它、右段全部不小于它，递归两段即可。平均$O(n \log n)$、就地（$O(1)$辅助空间）、内层循环紧凑，使它成为工程中最快的通用排序；最坏$O(n^2)$与不稳定则是需要正视的代价。本章末尾的希尔排序则以“增量序列”为钥匙，把插入排序改造成平均高效的亚平方算法。

20.1 轴点与划分

轴点的定义即划分的目标：选取候选 pivot 后，把区间重排成$L \le pivot \le G$三段，pivot 落到它的最终秩上。一次划分中轴点参与$n-1$次比较、仅 2 次移动——划分本身线性。

20.2 LUG 版划分：两端交替收缩

讲义给出的 LUG（Less-Unknown-Greater）版本维持不变式$L=[0,lo)$、$U=(lo,hi)$、$G=[hi,n)$，两端交替向内拓展：

```
template <typename T> Rank Vector<T>::partition( Rank lo, Rank hi ) {   //lo < hi
   swap( _elem[lo], _elem[ dice( lo, hi ) ] );   //随机化：任取候选轴点换到首位
   const T pivot = _elem[ lo ];                  //候选轴点
   for ( bool G = true; lo < hi; ) {             //两端交替向内，直至相遇（lo == hi）
      if ( G )   //向左拓展后缀G：遇到阻挡者则归入L，并换向
         { if ( pivot > _elem[ --hi ] ) { _elem[ lo ] = _elem[ hi ];  G = false; } }
      else       //向右拓展前缀L：遇到阻挡者则归入G，并换向
         { if ( _elem[ ++lo ] < pivot ) { _elem[ hi ] = _elem[ lo ];  G = true; } }
   }
   _elem[ lo ] = pivot;  return lo;              //轴点就位，返回其秩
} //LUG版
```

要点有三：随机化把最坏情形的概率摊薄（固定取首元素时，有序输入正是$O(n^2)$陷阱）；空穴在 L 与 G 之间来回搬运，累计移动$O(n)$；lo/hi 反向移动，故**不稳定**（相等的元素可能颠倒次序）。以$[6,3,8,2,9,1]$、pivot=6 走一遍：



图 32　逐步演示：LUG 划分：6 怎样各就各位

划分完成：$[1,3,2]\ 6\ [9,8]$，轴点就位秩 3——左边全≤ 6、右边全>6，递归两段。

20.3 平均 O(nlogn)：期望分析的骨架

最坏情形（轴点总取到极值，划分出空段）递推$T(n) = (n-1) + T(n-1) = O(n^2)$。随机化后，轴点秩 k 在$\{0, ..., n-1\}$上均匀，期望递推

$$E(n) = (n-1) + \frac{1}{n}\sum_{k=0}^{n-1}\big(\,E(k) + E(n-1-k)\big)$$

解得$E(n) = O(n \log n)$（常系数约$2\ln 2 \approx 1.39$，优于同阶算法的平均常数）。比较操作的期望次数与划分中移动操作的期望占比，课件逐项核算过：每一次快速划分中，随机元素的期望占比使移动总量保持在$O(n)$。

20.4 希尔排序：增量的学问

插入排序对基本有序的输入接近$O(n)$（第 07 章：$\Theta(n+I)$，I为逆序对数）。希尔排序（Shellsort, 1959）放大这一点：按增量序列$h_1 > h_2 > \cdots > 1$，逐轮把向量分成h_i列、对每列分别做插入排序（逐列排序）；最后一轮$h = 1$即普通插入排序——此时输入已“基本有序”，收尾很轻。

```
template <typename T> void Vector<T>::shellSort( Rank lo, Rank hi ) {   //0 <= lo < hi <= n
<= 2^30
   for ( Rank d = 0x3D5CCDD1; 0 < d; d >>= 1 )          //PS2序列：{1, 3, 7, 15, 31, ...}
      for ( Rank j = lo + d; j < hi; j++ ) {            //对各列
         Rank k = j;  T e = _elem[j];
         for ( ; ( lo <= k - d ) && ( e < _elem[k - d] ); k -= d )
            _elem[k] = _elem[k - d];                    //列内插入排序
         _elem[k] = e;
      }
}
```

全部效力系于增量序列。Shell 原始序列2^k（如 1024, 512, ..., 2, 1）各项不互素，甚至相邻项也非互素——奇偶位置的词条在前几轮里互不相见，最坏仍需$\Omega(n^2)$。改进序列层出不穷：PS2 序列（$\{1, 3, 7, 15, 31, ...\}$，形如$2^k - 1$的连续项）实测表现良好；Hibbard、Sedgewick 等各有界界。希尔排序至今没有紧的确界——它是“输入敏感”的最好例子：同样$O(n^2)$上界下，实际耗时随初始乱度浮动几个数量级。

20.5 List 排序的归宿

列表上的排序在第 07 章已经完成：选择排序（getMax+swap，比较$O(n^2)$移动$O(n)$）、插入排序（$\Theta(n+I)$输入敏感、稳定、在线）、归并排序（$O(n \log n)$搬指针）。向量本章补齐快排与希尔之后，教材§2D-2E 与§14 的排序版图就此合拢：比较模型下界的$\Omega(n \log n)$（第 11 章）说明归并/堆排/快排（平均）已到尽头；超出比较模型的桶排、基数排，则留在习题解析的视野里。

20.6 本章要点

- 快排=先划分后递归；LUG 划分线性、就地、不稳定；随机化使最坏$O(n^2)$的概率可忽略。
- 轴点秩均匀 $\Rightarrow E(n) = O(n \log n)$，常系数约$1.39 \log_2 n$优于同阶均值。
- 希尔排序：逐列插入排序，效力系于增量序列；Shell 序列不互素最坏$\Omega(n^2)$，PS2 等互素序列实测良好；输入敏感、确界未决。
- 列表排序三例已见于第 07 章；比较模型下界$\Omega(n \log n)$（第 11 章）为排序算法版图划界。

22 PS01：把“会背复杂度”变成“自己能数出来”

教师原始习题单：第 01 次习题共 32 题。下面保留原题号，重新解释题目要你做什么，再把能落在纸上的计算、证明或实验办法写出来。编译、测时间和个人经历类问题没有一个替所有人编好的“标准经历”；这些题给出可照着做的操作和检查结果的办法。

读这一章时，准备一张纸。遇到程序，先写变量下一步变成多少；遇到证明，先试三个小输入；遇到复杂度，先决定到底数什么。答案里的上界、下界、平均值和实测时间是四种东西，不能相互顶替。

22.1 1 · 计算工具：算法不一定写在电脑里

算盘、尺规和纸笔都能执行计算。区别在于，它们允许的基本动作不同。算盘可以拨珠，尺规可以画直线、画圆、取交点，纸笔可以逐位写数。一个过程要说清楚，首先要说明输入如何放进去。例如用算盘算 $27+35$，先用十位和个位的珠位表示 27，再加 3 个十和 5 个一；个位从 7 加 5 得到 12，要留下 2 并向十位进 1，最后读出 62。只有“把数加起来”这句话还不是给执行者的完整步骤。

可以把十进制竖式加法写成程序：从最低位起，把两位数字和进位相加；结果对 10 取余作为本位，除以 10 取整作为新进位；移动到高一位，直到两串数字和进位都用完。每次处理一位，长度为 d 的输入需要 $O(d)$ 个这样的逐位动作。

C++把动作写成有精确语法的语句，变量、条件和循环都有规定的含义；自然语言描述更容易夹带“适当处理”之类的空白。反过来，尺规的“取两个圆的交点”是它的原语，在普通电脑里却要另外表示坐标、处理精度、求方程。比较两种算法快慢以前，需要把基本动作放在同一个计算模型里。

可以自己试一个检查：让完全没参与设计的人照步骤算 $99+1$。若他不清楚最后的进位要不要额外写一位，说明描述漏了边界，而不是他“应该懂”。

22.2 2 · 曹冲称象与刻舟求剑：保存的究竟是什么

曹冲称象需要的量是“大象使船增加的排水重量”。把大象放上船，在船舷记录水线；移走大象，再逐块放石头，直到同一条水线重新与水面重合。在船本身、船中其他载荷、水的密度等条件保持一致时，相同吃水意味着相同排水体积，因此石头总重量等于大象重量。最后称石头就把一个难称的大物体转成了若干容易称的小物体。

这里的记号标在船上有用，是因为水线要表示船相对水面的吃水状态。刻舟求剑要找的却是剑相对河床的位置；船移动后，刻线跟着船走，剑没有跟着走。记号保存了“剑落水时相对船的位置”，而人想恢复“现在相对河床的位置”，中间缺了船的位移。若剑在落水后还受水流影响，甚至连固定的河床位置也要另作假设。

用算法语言说，输入信息必须足以确定输出。设落剑时船在地面坐标 x_0，记号相对船的位置为 r，则剑的初始地面坐标是 x_0+r。船后来到 x_1，拿同一记号推断的位置是 x_1+r，误差正是 x_1-x_0。把坐标写出来，就能指出错误发生在哪一步。

这个题也提醒我们：程序可以按预定步骤顺利结束，却算错问题。有穷性保证会结束，正确性还需要输入、模型和推理都对。

22.3 3 · 死循环：局部能查，万能检查器做不出来

先看最常见的一类错误：

```
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] < target) lo = mid; // 长度为1时不再缩短
    else hi = mid;
}
```

若 `lo=4, hi=5` 且 `a[4]<target`，就有 `mid=4`，更新后还是 `[4,5)`，此后完全重复。修正通常是 `lo=mid+1`，但仍要检查这是否符合想要的返回语义。调试时把 `lo,hi,mid` 打印出来，比一直盯着循环体更容易看见状态没变。

证明一个循环终止，可以找“非负整数、每轮严格减少”的量。这里取 $hi - lo$。它不能降到负数，每次又至少减 1，因此轮数有限。只写“区间变小”不够，要检查每一条分支，特别是长度 0 和 1。机器整数溢出还可能让本应下降的量突然变负或绕回去，实际程序必须另查类型。

为什么不能写一个检查任意程序的万能算法?这里需要先允许“程序也能作为数据输入”：源代码可以存成字符串或文件，一个程序既能读数字，也能读另一个程序的编码。P(P)并非一段程序同时执行两次；括号外的 P 是执行者，括号内的是提供给它的那份编码。

假设存在总能结束的检查器 halts(P,x)，对任意程序 P 和合法编码 x，都能准确回答“P 以 x 为输入时会不会结束”。用它构造另一个程序 D：

```
D(程序编码 P):
    如果 halts(P,P) 回答“会结束”:
        永远循环
    否则:
        立即结束
```

检查器声称能处理任意程序，因此也必须能处理 D。现在把 D 自己的编码交给 D，只有两种可能：

检查器对 halts(D,D) 的回答	D 按自己的定义接着做什么	矛盾在哪里
会结束	进入永远循环	检查器说错了
不会结束	立即结束	检查器还是说错了

矛盾来自“对所有程序都准确且检查本身总能结束”这一假设。检查器如果只处理一类受限制的程序，D 未必属于它的输入范围，不能硬套这个反证。

这不妨碍我们检查具体程序、有限状态程序，或要求循环带下降量的受限语言。正确的实践是组合使用不变量、边界测试、超时和审查；超时发现“这次没在限定时间结束”，本身不能证明永远不结束。

22.4 4 · Failstone：别因为长得像冰雹序列，就停止证明

原题递归在 p=q 时返回 1；否则变换参数并把返回值加 1。下面明确讨论正整数 $p, q \geq 1$，并假定算术精确、没有溢出。若允许 p=0,q>0，奇数 q 时会原样重复 (0,q)，结论就变了。“自然数”是否包含 0 必须先约定。

可以不用深递归，而是直接数经过多少个状态：

```python
def failstone(p, q):
    if p <= 0 or q <= 0:
        raise ValueError("本实现只接收正整数")
    states = 1
    while p != q:
        if p > q:
            p, q = p - q, p
        elif q % 2:
            p = 2 * p
        else:
            q //= 2
        states += 1
    return states
```

例如从 (1,3) 开始，依次得到 (2,3)、(4,3)、(1,4)、(1,2)、(1,1)。共有 6 个状态，所以返回 6。要把末尾的基础情形也数一次；只数变换次数会少 1。

困难在于，单步最大值不总下降：(2,3) 变成 (4,3) 时还变大了。我们把有限的几步捆成一组。若 p<q 且 q 为偶数，减半 q 后，新的最大值小于原 q。若 p<q 且 q 为奇数，就不断将 p 加倍。若中途相等立即结束；否则一定能到某个 $p^* < q < 2p^*$，因为正整数反复加倍会超过固定 q，而且奇数 q 不可能等于偶数 $2p^*$。接下来三步是

$$(p^*, q) \to (2p^*, q) \to (2p^* - q, 2p^*) \to (2p^* - q, p^*).$$

最后一步合法，因为第二项 $2p^*$ 是偶数，且 $0 < 2p^* - q < p^*$。这组变换后两项都小于原来的 q。若起初 p>q，先走一步得到 (p-q,p)，再进入刚才的小项在前的情形，后续一组同样让最大值降到原 p 以下。
因此，每组都在有限步内结束，并让正整数最大值严格下降；这样的组不可能无限多。这就证明了原过程对正整数一定结束。它与尚未普遍解决的另一道递推问题相似，并不能把那个问题的未知状态搬过来。

22.5 5 · 几何分布：要数到第一次成功为止

每轮独立，成功概率固定为p，其中$0 < p \le 1$；直到第一次成功的轮数记为K，成功那轮也计数。例如成功、失败后成功、失败两次后成功，分别对应$K = 1, 2, 3$。同一组动作有长有短，我们要算的是这些长度按各自发生概率加权后的平均值，记作期望$E[K]$，不是最长长度。
连续前$k-1$轮失败、第k轮成功的概率为

$$P(K = k) = (1-p)^{k-1}p, \quad k = 1, 2, \ldots$$

因为各轮独立，才可以将前面的失败概率连乘。先不用待证明的平均值列递推，改从每一轮是否执行来数。第 1 轮必做，第 2 轮只有第 1 轮失败才做，第 3 轮只有前两轮都失败才做。因此

$$E[K] = P(K \ge 1) + P(K \ge 2) + P(K \ge 3) + \cdots$$

$$= 1 + (1-p) + (1-p)^2 + \cdots = \frac{1}{p}.$$

第一行为什么成立？一次实际运行若做了 3 轮，就向前 3 项各贡献一个“1”，后面的项贡献 0；所以这次运行的总贡献恰好是 3。先把轮数截在有限的m，求这些指示量的平均，再让m增长，非负项的和就给出上式。几何级数收敛也说明这个期望确实有限；之后才可以用$E[K] = 1 + (1-p)E[K]$作简短复核：先花 1 轮，失败后剩余过程与起点相同。
取$p = 1/2$，第 1 轮成功的概率是$1/2$，先失败再成功是$1/4$，先失败两次再成功是$1/8$，它们的成本分别是 1、2、3。如果最多做 3 轮，实际执行轮数是$\min(K, 3)$：第 1 轮必做，第 2、3 轮分别以$1/2, 1/4$的概率做，期望为$7/4$。其结果还必须区分“第 3 轮成功”和“3 轮全失败”，后者的概率是$1/8$。限制次数改变了程序及随机变量，不能继续报平均 2 轮。
例如独立地掷公平硬币直到第一次正面，平均需要 2 次；成功率为 0.2 的独立尝试平均需要 5 次。平均 5 次不表示第五次一定成功，前 20 次都失败的概率仍是 $0.8^{20} > 0$。
符合模型的计算例子是：从固定集合均匀独立抽样，直到抽到某个目标；或者每轮独立选择随机候选，命中合法集合的比例恒为 p。网络重试常有服务器状态变化、限流、相关故障，不能只因为“也在重试”就套几何分布。若允许最多重试 m 次，随机变量还变成了截断版本，期望也要重新算。

22.6 6 · 分解大整数：先忠于题目的蛮力算法

原流程依次试素数 2、3、5、7.....；找到因子就除掉，直到剩余数为 1。这里没有写“试到平方根便停止”，所以不能在分析中悄悄加上这项优化。
最坏例子可以取接近 10^{100} 的素数 N。每个小于 N 的候选素数都试不出因子，直到 N 本身才整除成功，测试量约为素数计数函数 $\pi(N)$。用素数分布的渐近估计 $\pi(N) \sim N/\ln N$，在这个量级约为 $10^{100}/(100\ln 10) \approx 4.34 \times 10^{97}$ 次。这个数只是数量级估计，不能当成对某一个具体 N 的精确素数数量；光生成所有候选素数还另有成本。
最好让待分解数是很小素数的幂。取 $N = 2^k$，每次都能除以 2，约经过 $k = \log_2 N$ 次成功除法。10^{100} 附近的 2 的幂需要约 332 或 333 次，具体取决于选的是哪一个幂。“同样约 100 位十进制数字”，计算量能相差几十个数量级。
若采用平方根停止并在最后输出剩余因子的改进版，素数输入只需试到 $\sqrt{N}$。那是另一个更好的算法，回答时应先分析原流程，再单独说明改进。大整数除法本身也不必是单位成本；若按位模型收费，还要乘上相应位长的运算成本。

22.7 7 · 图灵机递减：借位时到底改了哪几位

设纸带上是最高位在左的二进制正整数，读头先移到最低位。做减 1 时：读到 0 就把它改成 1 并向左，继续借位；读到第一个 1 就把它改成 0，借位结束。之后根据题目约定删去多余前导 0、复位读头、进入停机状态。

用 1000 减 1 逐步看：最低位 0 改 1 得到 1001，再向左把 0 改 1 得到 1011，再改一位成 1111，最后把最左的 1 改 0，得到 0111，去掉前导 0 即 111。中间的 1111 不是结果，因为“仍在借位”的状态还没有结束。

1011 减 1 只改最低位 1 为 0，得到 1010；1 减 1 得到 0。输入 0 若属于允许集合，不能继续向左无限找 1，应该单独输出边界错误或按明确规定返回 0。减法所需改写次数等于尾部连续 0 的数量再加 1，最坏与数字位数成正比。

图灵机的状态名不是装饰。至少要区分“去最低位”“借位”“返回起始处”“结束”，否则同一个符号 1 在不同阶段应执行什么动作就说不清。复位读头产生的移动也属于时间成本。

22.8 8 · 允许读头原地停留，会不会多出一种计算能力

只允许 L 和 R 时，每走一步位置的奇偶性都会改变。若读头最后回到起点，向左和向右的步数相等，移动总次数是两者之和，因此为偶数。这是对位移计数的结论，不是说程序中所有操作都天然成双。

加入 N 表示头不移动。设某条可达转移在读到某符号、处于某状态时，既不改符号也不改状态，且动作为 N，那么执行后整个配置与执行前完全相同。确定性的机器下一步还会选同一转移，于是永久重复。要让一条有效计算路径最终结束，这类转移必须改变符号或状态，或者本身就是停机规则。

时间应该按转移次数数，而不是只数头移动了多远。连续 100 次改状态但不移动仍做了 100 步，不能算零时间。N 也能由原模型模拟：先向一侧移动，进入一个记住待返回状态的临时状态，再向另一侧移回；邻格读到什么就原样写回。这样只增加常数倍转移，没有改变哪些问题可计算。

这项模拟说明模型的可计算能力相同，但不保证逐步时间完全相同。比较复杂度时要保留“至多常数倍模拟开销”这一层说明。

22.9 9 · RAM：只用加减，怎样把乘除真的做出来

RAM 可以通过地址直接访问寄存器或数组格，图灵机则只能读写头下的格。原语集合决定一次动作的成本。本题要自己实现乘除时，就不能在循环里先偷用一个没提供的除法，再宣布已经实现了乘法。

最直接的非负整数乘法是加b次a。算13×11，累计值依次是 0、13、26、39、52、65、78、91、104、117、130、143。每轮只做加法、计数减 1 和判断，共$\Theta(b+1)$时间。循环不变量是“已经累计的和+还没执行的次数乘a，始终等于原乘积”。余次从b降到 0，所以会结束。

还可以在允许 RAM 随机访问数组时，只用加法、减法和比较构造倍增表。先同时倍增权重和对应的a倍数，权重超过b后停止；再从大到小选能容纳的权重。这个实现没有整数除法或乘法：

```
def ram_multiply(a, b):
    if a < 0 or b < 0:
        raise ValueError("nonnegative integers required")
    weights, values = [], []
    w, v = 1, a
    while w <= b:
        weights.append(w)
        values.append(v)
        w, v = w + w, v + v
    answer, remaining = 0, b
    for i in range(len(weights) - 1, -1, -1):
        if weights[i] <= remaining:
            remaining -= weights[i]
            answer += values[i]
    return answer
```

对 13 与 11，建表后为：

权重	对应倍数
1	13
2	26
4	52

权重	对应倍数
8	104

倒着读：8≤11，选 104，剩 3；4>3，跳过；2≤3，选 26，剩 1；1≤1，选 13，剩 0。答案 104+26+13=143。建表期间每行满足“倍数=a乘权重”；选择期间满足“已累计乘积+a乘剩余权重=原乘积”。处理完某个权重后，余数小于该权重，最终余数小于 1 且非负，只能为 0，证明没有漏掉还需累计的部分。表长$\lfloor \log_2 b \rfloor + 1$（$b > 0$），时间和辅助空间都是$O(1 + \log(b+1))$；若 RAM 已有按位指令，可省掉表并用常数个字完成，但那要另写原语假设。

除法也建一张倍增表。以$37 \div 5$为例，表是`(倍数,商贡献)=(5,1),(10,2),(20,4)`，40 已经超过 37，停止。倒着逐行：

试减	试前余数	能减吗	试后余数	已得商
20	37	能	17	4
10	17	能	7	6
5	7	能	2	7

每一步保留等式$37 = 5q + r$，其中q是已得商、r是余数。倒着扫完后$0 \le r < 5$，因此$q = 7$就是向下取整商；若还要向上取整，余数非 0 时再加 1，得到 8。

```
def ram_divide(a, b):
    if a < 0 or b <= 0:
        raise ValueError("a >= 0 and b > 0 required")
    multiples, powers = [], []
    v, w = b, 1
    while v <= a:
        multiples.append(v)
        powers.append(w)
        v, w = v + v, w + w
    quotient, remainder = 0, a
    for i in range(len(multiples) - 1, -1, -1):
        if multiples[i] <= remainder:
            remainder -= multiples[i]
            quotient += powers[i]
    return quotient, remainder
```

若$a < b$，表为空，直接返回商 0、余a；除数 0 不合法。假定精确机器字加减，时间与表空间为$O(\log(a/b + 1) + 1)$。定长机器上倍增前还要检查下一次加法是否会溢出，不能靠回绕后的负数结束循环。任意长整数的加减又需按位长付费。

C++还规定类型转换、对象生命期、资源释放等语言行为。把 C++中的 `string` 一次复制映射成 RAM 的一条整数赋值，会改变成本模型；回答模型差异时，应举出这样能实际改变计算量的例子。

22.10 10 · 小 o 与小 ω：比“同阶”更强的判断

$f(n) = o(g(n))$ 表示 $f/g \to 0$，即 f 相对 g 越来越可忽略；$f(n) = \omega(g(n))$ 表示 $f/g \to \infty$。这里先假定 g 最终为正。大 O 只要求比值最终有一个固定上界，小 o 则要求不管选多小的正上界，最终都能低于它。例如 $n/(n \log n) = 1/\log n \to 0$，所以 $n = o(n \log n)$，也就是 $n \log n = \omega(n)$。但 $3n$ 与 n 的比值恒为 3，所以 $3n = \Theta(n)$，却不是 $o(n)$。

再看一个容易误判的例子：$n/1000$ 在很多输入上比 n 小很多，但二者仍同阶，因为“1000”是固定常数。反过来 $n/\log n$ 与 n 的相对差别会随 n 增长，前者才是 $o(n)$。这些符号描述极限趋势，不表示某个小规模输入上哪一个实际更快。

22.11 11 · 乘法、除法能不能当$O(1)$

若课程明定“机器字内的整数乘除”属于 RAM 原语，分析时就可以算一次基本操作。但这个约定含有一个前提：操作数能放进规定字长，而且不把任意大整数悄悄塞进一个寄存器。

用只支持加减、移位的指令实现乘法，第 9 题展示的算法要按乘数位数做多轮；用小学竖式乘两个 L 位二进制数，最朴素需要 $O(L^2)$ 个逐位动作。这和“CPU 有一条乘法指令”的单位成本说法并不冲突，它们在不同模型里计费。

因此写复杂度报告时可以明确说：“以下 n 个元素均为固定宽度整数，比较和算术按单位成本计。”若题目让你处理有百万位的数，就必须把位长作为规模的一部分。用一句“乘法都是常数”会抹掉题目真正的难处。

22.12 12 · 取对数：正向可保留上界，反向只得到多项式关系

原题问的是：由 $T(n)=O(f(n))$，是否有 $\log T(n)=O(\log f(n))$，反向又怎样。先设 $T(n), f(n)\ge 1$，且 f 最终至少为某个大于 1 的常数；更常见的充分条件是 $f(n)\to\infty$。这样对数非负，分母不会一直为 0。

若 $T\le Cf$，两边取对数得到 $\log T\le \log C+\log f$。在 $\log f$ 最终有正下界时，常数 $\log C$ 能被另一个常数倍的 $\log f$ 吸收，所以正向成立。若 f 恒为 1、T 恒为 2，虽然 $T=O(f)$，但 $\log f=0$、$\log T>0$，就不能写成后一关系。这说明前提不是可省略的装饰。

反向通常不成立。取 $T(n)=n^2, f(n)=n$：有 $\log T=2\log n=O(\log f)$，但 $T/f=n\to\infty$，所以 $T\neq O(f)$。对数把幂次从指数位置拉成了乘法常数，而大 O 恰好会忽略这个常数。

从 $\log T\le C\log f$ 真正能推出的是 $T\le f^C$，即 T 受 f 的某个固定次幂约束；不能擅自把这个幂 C 改回 1。核对方法很简单：每次把两边指数化，看看常数究竟跑到哪里去了。

22.13 13 · 指数枚举：输出数量本身可能就很大

n 个元素的每一个子集都可用长度 n 的 0/1 串编码：第 i 位为 1 表示取第 i 项。每位有两个选择，候选共有 2^n 个。枚举 n 个布尔变量的所有真值赋值、逐一检查所有子集的重量，也都有这层枚举成本。

n 从 20 增加到 40 时，候选量从约一百万增加到约一万亿；输入只翻倍，候选却平方。这正是先估算复杂度再写 PA 的理由。若每个候选还要重新扫描 n 项，时间是 $O(n2^n)$；若用增量维护，单个候选成本可能下降，但候选数本身没有消失。

要区分两个结论：“这个算法枚举了 2^n 个候选”与“这个问题的所有算法都必须指数时间”。前者直接数流程就能证明，后者是问题下界，通常难得多。对于特定子集问题，折半枚举还能把主要枚举规模降到 $2^{n/2}$，这已经足以推翻“只能按原算法那么慢”的断言。

22.14 14 · 32 位、64 位无符号整数有几位十进制

正整数 x 的十进制位数是 $\lfloor \log_{10} x\rfloor+1$。因为 d 位数恰好落在 $10^{d-1}\le x<10^d$ 之间，两边取对数即可得到这个式子。

w 位无符号整数最大值为 2^w-1。32 位最大值是 4294967295，共 10 位；64 位最大值是 18446744073709551615，共 20 位。估算时用 $\log_{10}2\approx 0.30103$，32 乘它约 9.633，64 乘它约 19.266，分别向下取整再加 1。

这里问的是“十进制字符要写几位”，不是“数有多大就需要多少字节”。64 位数在机器的定长二进制表示里仍占 8 字节，而把它打印成十进制文本最多要 20 个数字字符，若用 C 字符串还要给末尾零字符留位置。

22.15 15 · 时限预算：给自己的程序做一个能被反驳的估计

先从题面记下 n 的上限、时限、内存限额、编译器和允许的库，再写出候选算法的操作量。例如 n 为 20 万时，二重循环约 4×10^{10} 对，排序约 $n\log_2 n\approx 3.5\times 10^6$ 次比较；即使常数有差异，这个数量级对比也足够先淘汰朴素二次方案。

不能把“每秒一亿次”当硬件定律。一次比较可能是两个整数，也可能比较长字符串；访问连续数组和随机追指针的缓存代价也不同。正确做法是在目标环境用与题目相似的数据做小型基准，确认随 n 增长的趋势，并保留一定余量。

内存也要算。例如两个长度 20 万的 64 位数组约占 $2\times 200000\times 8=3.2\times 10^6$ 字节；若递归每层复制数组，或每个元素另建一个带指针的节点，实际占用会增加。记录的是你实际运行的配置和结果，不把本书的示例数字当作 OJ 实测记录。

22.16 16 · 典型循环：先给每个位置写“收费项目”

官方《习题解析》1-32 中的 16 段程序已经在第 32 章逐段展开。这道题练的是数法，建议先遮住答案，把每段程序分成“外层取哪些值”“每个外层值的内层做多少次”“条件判断本身贵不贵”三列。

例如 `for(i=1;i<=n;++i) for(j=1;j<=i;++j)` 的收费是 $1+2+\cdots+n=n(n+1)/2$。把内层改成 `j*=2`，第 i 轮就只有 $\lfloor \log_2 i \rfloor + 1$ 次，累计才是 $\Theta(n\log n)$。外层同样写着 n 轮，并不能决定整体是几次方。

特别留意循环条件中的函数：`while (i < n/G(i))` 每检查一次条件都可能重新计算 G。若 G 本身递归，不能因为它没写在花括号里面就免费。第 32 章的开方递归例正是把条件成本漏掉后容易答错的题。

22.17 17 · 分数级数怎样估整数次数

运行次数当然是整数。具体取 `for (j=i; j<=n; j+=i)`，访问的是 $i, 2i, 3i, \ldots$，所以真实次数是 $\lfloor n/i \rfloor$。起点和端点不能省：若换成 `j=0; j<n; j+=i`，次数就成为 $\lceil n/i \rceil$。以下分析前一个循环，把它夹住：

$$\frac{n}{i} - 1 < \left\lfloor \frac{n}{i} \right\rfloor \le \frac{n}{i}.$$

对 i 从 1 加到 n，得到 $nH_n - n < \sum_i \lfloor n/i \rfloor \le nH_n$。上下界只差至多 n，而 $nH_n = \Theta(n\log n)$，因此真实整数和也为 $\Theta(n\log n)$。

拿 n=5 手算，次数为 5、2、1、1、1，总计 10；分数和为 $5(1+1/2+1/3+1/4+1/5) \approx 11.42$。我们没有声称程序做了 11.42 次操作，只是用一个更容易计算的数给真实次数定界。

22.18 18 · 调和级数：把一块块矩形放在曲线上

记 $H_n = 1 + 1/2 + \cdots + 1/n$。函数 $1/x$ 单调下降，所以在区间 $[k, k+1]$ 上，函数面积不超过高度为 $1/k$ 的矩形；把这些区间拼起来，得到

$$\int_{1}^{n+1} \frac{dx}{x} \le H_n.$$

另一方面，从第 2 项起，每一项 $1/k$ 不超过区间 $[k-1, k]$ 下的面积，所以 $H_n \le 1 + \int_1^n dx/x$。积分就是自然对数，于是

$$\ln(n+1) \le H_n \le 1 + \ln n.$$

上下都是对数增长，故 $H_n = \Theta(\log n)$。换对数底只乘一个固定常数。若只记住“调和级数发散”还不够：发散表示最终能超过任意常数，这道题还要知道它慢到什么程度。

22.19 19 · 为什么 $\log(n!)$ 是 $n\log n$ 量级

把乘积取对数：$\log(n!) = \sum_{i=1}^{n} \log i$。上界很直接，每项至多 $\log n$，所以和至多 $n\log n$。下界只拿后半段，至少有 $\lfloor n/2 \rfloor$ 项，每项至少 $\log(n/2)$，因此和至少约 $(n/2)\log(n/2)$。这已经证明了 $\Theta(n\log n)$。

Stirling 公式给出更细的近似：

$$n! \sim \sqrt{2\pi n}(n/e)^n, \qquad \ln(n!) = n\ln n - n + \frac{1}{2}\ln(2\pi n) + o(1).$$

例如 n=10 时，$\log_2(10!) \approx 21.79$。这表示区分 10 个互异元素的所有排列，二元比较树至少需要足够深，不能用十次左右的比较普遍完成排序。精细近似能估常数；只求渐近阶时，前面的上下夹逼更短，也更不容易记错。

22.20 20 · 递归空间：数同时没回家的调用，不数一共来过几次

线性递归 `sum(a,n)=sum(a,n-1)+a[n-1]` 在最深处会同时保留 n 层左右的调用帧。每层若只存固定几个下标、返回地址和局部量，辅助空间是 $\Theta(n)$。把输入数组本身排除以后，这些栈帧仍然是额外空间。

二分求和先算左半，再算右半。递归树总节点仍有 $\Theta(n)$ 个，因此总时间为 $\Theta(n)$；但任一时刻只在一条根到叶的路径上，深度为 $\Theta(\log n)$，辅助空间只有 $\Theta(\log n)$。顺序执行的左右子树不会同时保留各自所有内部调用。

取 4 项 `[2,5,7,1]`，二分求和在最深处的存活链可以逐时刻画成：

```
进入sum[0,4)
  进入sum[0,2)
    进入sum[0,1)，返回2
    进入sum[1,2)，返回5
  返回7，sum[0,2)的调用帧消失
  进入sum[2,4)
    进入sum[2,3)，返回7
    进入sum[3,4)，返回1
  返回8
返回15
```

在 sum[2,3)执行时，左半的各个子调用已经结束，只剩根帧保存左结果 7。总共调用 7 次，最多同时 3 层；把 7 误当最大栈深，就把“累计出现”与“同时存在”混在一起。对 8 项最多 4 层，对 16 项最多 5 层，因此是对数。

迭代求和只用一个累计值和一个循环下标，空间为 $O(1)$。若每次递归都先复制子数组，再调用函数，复制出的数组可能还没释放，空间就不能只按“每层常数”算了。若并行启动左右递归，也要重新检查同时存活的任务和缓冲。

时间和空间有联系，但不必同阶。二分求和就是时间 n、空间$\log n$；缓存所有子问题可能多占空间却大幅省时间。判断时画“某个时刻哪些对象仍活着”，比把递归树所有节点直接相加更可靠。

22.21 21 · reverse：交换放在递归前后，为什么结果一样

采用闭区间 [lo,hi]。当 lo>=hi 时不做事；否则交换两端，再反转内部 [lo+1,hi-1]。交换的两个位置和内部递归处理的位置不重叠，因此可以调换这两步。

例如 [1,2,3,4]：先交换外端得到 [4,2,3,1]，再交换内部得到 [4,3,2,1]。先做内部则先得到 [1,3,2,4]，再换外端，同样得到 [4,3,2,1]。这不是“所有两行代码都能交换”，而是这里两步对互不重叠的数据起作用。

每层做一次常数时间交换，规模减 2，层数约 n/2，所以两种顺序时间都为 $\Theta(n)$。普通递归实现保留 $\Theta(n)$ 层帧。递归放最后时有尾调用形式，某些编译器可能优化；C++不保证一定消去栈帧，因此若必须保证常数辅助空间，明确写双指针循环更稳妥。

22.22 22 · Master 定理：从每层的账单证明，而不是背三行结论

先固定$n = 8$，分别画出每次分成 4 个一半、2 个一半的递归树。节点上的n或n^2指当前节点自己的工作；不是拿最初的 8 给每个节点重复收费。每个叶收费 1，逐层相加得到：

递推	第 0 层	第 1 层	第 2 层	第 3 层叶子	总量
$4T(n/2)+n$	8	16	32	64	120
$2T(n/2)+n$	8	8	8	8	32
$2T(n/2)+n^2$	64	32	16	8	120

第一行越往下越贵，第二行每层一样，第三行根附近最贵。两个 120 相同只是$n = 8$的数值巧合：第一行总量随n^2增长，第三行也随n^2增长，但它们的钱花在不同层；第二行是n乘层数。层成本计算依赖的是“节点数×每节点工作量”，不要只看一条路径上规模变小。

有了这三笔账，再把同样的数法写成一般式。先处理 $T(n) = aT(n/b) + g(n)$，其中 $a \geq 1, b > 1$ 为常数，基础规模的成本为正常数。为看清结构，先取 $n = b^L$，省掉取整；常用取整版本还需保证递归规模确实下降。记 $d = \log_b a$。

第 i 层有 a^i 个节点，每个规模 n/b^i，本层非递归成本为 $a^i g(n/b^i)$；第 L 层叶数是 $a^L = n^d$。所以

$$T(n) = \Theta(n^d) + \sum_{i=0}^{L-1} a^i g(n/b^i).$$

若 $g(n) = O(n^{d-\varepsilon})$，代入第 i 层，有 $a^i g(n/b^i) = O(n^{d-\varepsilon} b^{i\varepsilon})$。这是向叶层增长的几何级数，和为 $O(n^d)$，叶层本身又提供 $\Omega(n^d)$，故 $T = \Theta(n^d)$。例如 $4T(n/2) + n$ 得 $\Theta(n^2)$。

若 $g(n)=\Theta(n^d)$，每层都为 $\Theta(n^d)$，共 L 层，故 $T=\Theta(n^d\log n)$。例如 $2T(n/2)+n$ 得 $\Theta(n\log n)$。
若 g 比 n^d 多一个固定正幂，并且存在 c<1 使 $ag(n/b)\le cg(n)$，第 i 层成本至多 $c^ig(n)$，整棵树的非递归成本至多 $g(n)/(1-c)$。根层又必须支付$g(n)$，叶成本可被 g 吸收，因此 $T=\Theta(g(n))$。例如 $2T(n/2)+n^2$ 得 $\Theta(n^2)$。
注意课程把根主导情况放在“第一种”，有些书把它编号为第三种。应按条件认定情形，不能只记编号。这个证明也没有覆盖 $n/\log n$ 一类恰在边界附近的所有函数，遇到它们要回到层成本和。

22.23 23 · 正则条件已经包含增长下界吗

假设 $ag(n/b)\le cg(n)$，其中 0<c<1，且基础区间中的 g 有正下界。把式子改写为 $g(n)\ge(a/c)g(n/b)$。重复 k 次，有

$$g(n)\ge(a/c)^kg(n/b^k).$$

选 k 使 n/b^k 落到固定基础区间，k 约为 $\log_b n$。因此 $g(n)=\Omega(n^{\log_b(a/c)})$。由于

$$\log_b(a/c)=\log_b a+\log_b(1/c),$$

后面多出的 $\varepsilon=\log_b(1/c)>0$，正是所需的多项式间隔。若只讨论 $n=b^k$，证明直接沿这条序列成立；要推广到所有 n，需要基础区间统一下界等常规条件。
反过来，光有增长下界不能约束相邻尺度的起伏。取 a=b=2，在 $n=2^k$ 上规定 g：k 为偶数时 $g(n)=n^2$，k 为奇数时 $g(n)=n^3$。无论哪种都有 $g(n)\ge n^2$，已比 $n^{\log_2 2}=n$ 多一幂。
但当 k 为偶数时，$2g(n/2)/g(n)=2(n/2)^3/n^2=n/4$，随 n 变大，连小于 1 都做不到，更别说统一小于某个固定 c。这个明确的反例说明，“整体长得够快”与“向下一层时有规律地衰减”是两个不同要求。

22.24 24 · 带对数幂的递推：别漏掉 p=−1

以底数 2、$n=16$、$k=1$、$p=-1$为例，离叶子还有 4、3、2、1 层时，本层成本分别是16/4、16/3、16/2、16。相加得到$16(1+1/2+1/3+1/4)$，不是只有一个 16。把n换成2^L，括号内就变成H_L，所以多出的是 $\log L=\log\log n$。基础层的工作另外计入$\Theta(n)$，避免在叶子上算$1/\log 1$。
这笔账提示我们：幂次恰好抵消后，剩下的对数因子还要逐层加，不能删掉。下面统一计算。设 $g(n)=\Theta(n^k(\log n)^p)$，在足够大 n 上取对数，避免基础规模出现 log1=0。仍写 $n=b^L$。第 i 层的主要大小为

$$n^k(a/b^k)^i(L-i)^p,$$

其中换底常数已经吸收到 Θ 里。令 j=L-i，就能看见几何权重和幂次怎样叠加。
若 $a<b^k$，从根向下的几何因子衰减，任何固定对数幂都无法推翻固定多项式间隔，所以 $T=\Theta(n^k(\log n)^p)$。若 $a>b^k$，叶层主导，$T=\Theta(n^{\log_b a})$。
若 $a=b^k$，几何部分恰好抵消，剩下 $n^k\sum_{j=1}^{L}j^p$。当 p>−1，和为 $\Theta(L^{p+1})$；当 p=-1，和是调和级数，给 $\Theta(\log L)$；当 p<−1，和有常数上界，叶成本也同阶。对应地：

$$T(n)=\begin{cases}\Theta(n^k(\log n)^{p+1}), & p>-1,\\ \Theta(n^k\log\log n), & p=-1,\\ \Theta(n^k), & p<-1.\end{cases}$$

例如 $2T(n/2)+n/\log n$ 是 $\Theta(n\log\log n)$，不能机械套成 $\Theta(n)$。对于恰为平滑函数 $n^k(\log n)^p$，根主导时的比值最终小于 1；若只知上下常数包住它，原函数可能起伏，正则条件未必逐点成立，但可用平滑上下包络和递推比较得到同一渐近界。不要把“结论可由夹逼得到”写成“所有起伏函数都自动满足逐点正则条件”。

22.25 25 · Akra–Bazzi：不等长的递归树，怎样加出积分式

先看具体树，再进入一般证明。对$T(n)=T(n/3)+T(2n/3)+n$，取 n=27，前两层是：

```
                27                  本层工作27
          /              \
         9                18        本层工作9+18=27
       /    \           /    \
      3      6         6      12     本层工作3+6+6+12=27
```

每个框里是该节点处理的规模，也是它自己要付的非递归工作。左支占原规模 1/3，右支占 2/3。若只沿一条路径走，为了代表整树，就不能把两种支路一律各算一半；下面会用 1/3、2/3 作为归一化后的路径权重。具体例题的$n\log n$结论也可先用第 27 题的“完整浅层+最长路径”证明。以下一般证明用到加权平均、换元和期望，可以先读第 28 章对应推导，再回来看。

现在证明一个覆盖本课常用多项式与对数幂成本的版本。设固定基础阈值$x_0>0$，在$x>x_0$时

$$T(x)=\sum_{i=1}^{m} a_i T(b_i x)+g(x),\qquad a_i>0,\quad 0<b_i<1.$$

m是子问题种类数，a_i是该类的系数，b_i是规模缩小比例；它们都是与x无关的常数。基础区间的T有统一的正常数上下界。$g(x)$是当前节点的非递归成本，先设其为正，并满足以下“不要在一个固定比例区间剧烈振荡”的条件：令$b_{\min}=\min_i b_i$，存在常数K，当u,v充分大且$b_{\min}u\le v\le u$时，$g(u)/u^p$与$g(v)/v^p$之比都不超过K。常见的$x^q(\log x)^r$在远离$x=1$后满足此条件。若$g\equiv 0$，下面只剩基础项，单独得到$T=\Theta(x^p)$。

指数p由

$$\sum_{i=1}^{m} a_i b_i^{p}=1$$

确定。左侧关于p连续且严格递减，$p\to-\infty$时趋无穷，$p\to+\infty$时趋 0，所以有且只有一个实数解。这里p不必是整数。对$T(n)=T(n/3)+T(2n/3)+n$，代入$p=1$，两项是$1/3+2/3=1$。

要证明的结论是

$$T(x)=\Theta\!\left(x^p\left[1+\int_{x_0}^{x}\frac{g(u)}{u^{p+1}}\,du\right]\right).$$

下面把积分从递归树里推出来，而不是只报公式。

第一步，把每条分支变成一个总和为 1 的权重。 设$w_i=a_i b_i^p$，由p的定义有$\sum_i w_i=1$。再设

$$F(x)=\frac{T(x)}{x^p},\qquad h(x)=\frac{g(x)}{x^p}.$$

原式两边除x^p，得到

$$F(x)=\sum_i w_i F(b_i x)+h(x).$$

右边的第一项是较小规模F的加权平均，第二项是本节点归一化后的成本。我们可以用一条随机路径计算这个平均：从$X_0=x$出发，每一步独立以概率w_i选择第i类分支，令$X_{t+1}=b_i X_t$；第一次不超过x_0时停下，停下步数记为τ。记号$E[Z]$表示把每条可能路径上的数值 Z，按该路径被选中的概率加权平均；它不是取最大，也不是假设每条路径等可能。沿递推展开，有

$$F(x)=\mathbb{E}\left[F(X_\tau)+\sum_{t=0}^{\tau-1}h(X_t)\right].$$

这条等式只是“平均的平均仍按所有路径的概率加权”。在本来并不随机的递推分析中引入一条随机路径，是为了方便给每棵子树分配正确权重，不是在假设原算法随机。

第二步，基础项是常数量级。 停止时，X_τ落在固定区间$[b_{\min}x_0,x_0]$。这个区间不随初始x增长，基础T和x^p都有正常数上下界，因此$F(X_\tau)=\Theta(1)$。公式外面的“1”就是从这里来的，不能随意删掉；若节点工作极小，总成本仍有叶子贡献。

第三步，把一条路径上的和换成面积。 对规模取自然对数，记$S_t=\ln X_t$。每步向下走的距离是$-\ln b_i$，介于两个正常数c和C之间。因此路径把从$\ln x$到基础区间的一段数轴分成若干长度介于c,C的区间。

在区间$[S_{t+1},S_t]$上，e^s与X_t至多相差固定比例。前面给的正则条件保证$h(e^s)$与$h(X_t)$至多相差固定倍数。因此面积

$$\int_{S_{t+1}}^{S_t} h(e^s)\,ds$$

被两个固定正常数乘$h(X_t)$夹住：区间长度既不会缩到 0，也不会无限大，函数值也没有突然跳高或降为 0。把各段相加，就得到对每一条路径都成立的

$$1+\sum_{t<\tau} h(X_t)=\Theta\!\left(1+\int_{\ln x_0}^{\ln x} h(e^s)\,ds\right).$$

跨过基础点的最后一小段只带来常数，可以放进前面的 1。夹逼常数对所有路径相同，因此取期望后仍成立。最后令$u=e^s$，则$ds=du/u$，代入$h(u)=g(u)/u^p$，积分恰好变成$\int g(u)/u^{p+1}\,du$。再乘回x^p，定理得到证明。

这个证明说明了为什么分母是$p+1$：p来自按树的规模权重归一化，多出的 1 来自对数尺度的换元$ds=du/u$。它也说明为什么需要成本函数的正则条件：不能未经控制就把节点上的离散采样当成连续区间的面积。

现在计算两例。对$T(n)=T(n/3)+T(2n/3)+n$，$p=1$、$g(u)=u$，积分为$\int du/u=\ln n+O(1)$，答案$\Theta(n\log n)$。若节点工作改成n^2，积分为$\int 1\,du=\Theta(n)$，外面再乘n，答案$\Theta(n^2)$。同一棵不齐递归树，工作分布不同，主导位置就不同。

完整文献还有更一般的成本正则条件、$b_i x+h_i(x)$扰动、向上向下取整版本。本段证明的是已明确写出的固定比例、无扰动版本；不能据此把$n-1$当作某个固定$b_i n$，也不以“Akra–Bazzi 适用”跳过实际条件核对。对本课取整后的固定比例线性合并，还可直接用每层总规模上界和完整浅层下界证明，与下一题一起互相检查。

22.26 26 · 最大和区间：把“哪个区间”也放进状态

原题要给四种算法补上区间位置。先把课程接口固定下来，否则两个程序虽然都返回最大和，端点却可能互不相同。依据本地 119 页《01 算法概论》PDF 第 94 页定义、第 99 页扫描代码，主规则是：**允许空区间；和越大越好，和相同时长度越短越好，长度也相同时越靠右越好**。下标从 0 开始，区间写成$[l,r)$，长度为$r-l$。因此可以把每个候选统一写成比较键

$$(\text{sum},-(r-l),r),$$

按字典序取最大：第一项相同才比第二项，前两项相同才比第三项。空区间都长 0、和 0，其中最右的是$[n,n)$。全负数组和全零数组都应返回它；这不是“未找到答案”的特殊标记，而是合法的最佳区间。

沿老师扫描方向，从右向左处理`[-2,3,-1,3,-5]`。当前状态是一段已经贴住右侧的候选：`total`保存和，`end`保存右端；读到下标k时把$a[k]$接到它前面。若接完后的和非正，下一轮不再保留这段：负和只会拖累更左的区间；零和虽不拖累总和，却使它更长，按本题规则也该丢掉。

读入下标k	元素	接上后候选$[k,end)$	候选和	全局最好	给下一轮留下什么
初始	—	空	0	$0,[5,5)$	空，右端 5
4	−5	$[4,5)$	−5	$0,[5,5)$	丢掉；右端改 4
3	3	$[3,4)$	3	$3,[3,4)$	保留和 3、右端 4
2	−1	$[2,4)$	2	$3,[3,4)$	保留和 2、右端 4
1	3	$[1,4)$	5	$5,[1,4)$	保留和 5、右端 4

读入下标k	元素	接上后候选$[k, end)$	候选和	全局最好	给下一轮留下什么
0	−2	$[0, 4)$	3	$5, [1, 4)$	扫描结束

```
def max_interval_course(a):
    n = len(a)
    best_sum, best_l, best_r = 0, n, n
    total, end = 0, n
    for k in range(n - 1, -1, -1):
        total += a[k]
        candidate = (total, -(end - k), end)
        current = (best_sum, -(best_r - best_l), best_r)
        if candidate > current:
            best_sum, best_l, best_r = total, k, end
        if total <= 0:
            total, end = 0, k
    return best_sum, best_l, best_r
```

这里的状态足以覆盖所有区间，理由不是“负数都可以删”。固定一个左端k，最佳区间要么只到$k+1$，要么接上从$k+1$出发的最佳后段。后段若有正和，接上它会改善总和；若和为 0，空后段更短；若为负，更应取空。归纳地，处理k以前留下的正和后段正是所需状态。每个左端的最好候选都参与全局比较，任何没被保留的同左端候选都不可能胜过它，因此全局答案不会漏。循环下标严格递减，经过n轮结束；只用常数个数和端点，时间$\Theta(n)$，辅助空间$O(1)$。

再用三个专门卡并列规则的输入检查：`[2,0]`返回$2, [0, 1)$，不会多带右边那个 0；`[3,-9,3]`返回$3, [2, 3)$，两个长度 1 的最大区间中选最右；`[-4,-1]`返回$0, [2, 2)$。把代码中的`<=0`改成`<0`，会把零和后段留下，`[2,0]`便可能错误地带上尾 0。这一行取舍来自题意，而不是每一种最大子段算法都必须这样写。

其余三种算法也要使用同一个比较键。

- 三重枚举：先把最佳初始化为空$[n, n)$；枚举$l < r$，第三重循环重新累加$a[l]$到$a[r-1]$，连同端点一起比较。候选有$\Theta(n^2)$个，总累加量$\Theta(n^3)$。
- 二重枚举：固定l，先令和为 0；r从$l+1$逐项增加，每次只加新进入的$a[r-1]$，立即比较。每个候选只做常数工作，时间$\Theta(n^2)$。
- 分治：分别求左右子区间的最佳答案，再求贴住分界的左后缀与右前缀，将两段拼成跨界候选。例如按中点 2 切`[-2,3,-1,3,-5]`，左后缀选$[1, 2)$、和 3，右前缀依次有和−1、2、−3，选$[2, 4)$、和 2，合成$[1, 4)$、和 5。每一侧允许空段；相同和先取短段，同一后缀的右端、同一前缀的左端固定，长度确定后端点也确定。三个候选最后仍用统一比较键取最好。扫描后缀、前缀共线性时间，平衡分治总计$\Theta(n \log n)$。

跨界候选不能由“左右各自的任意最大区间”拼接，因为它们可能都不贴分界，拼起来会漏掉中间元素。空段还使边界递归自足：无元素子问题返回其最右空区间，单元素若为正则取它，否则取右端空区间。

如果另一题明确要求非空，必须改初始化；如果规定最左优先，则比较键也要改。例如在`[0,0]`上，本课程返回$[2, 2)$；“非空、先最左、再最短”返回$[0, 1)$；“非空、先最短、再最右”返回$[1, 2)$。这些都是不同接口，不能拿其中一个答案否定另一个，也不能在同一个实现里混用规则。第 03 章保留了课程完整原页和另一方向的等价实现，可用同一输入交叉核对。

22.27 27 · 最大区间按 25% 与 75% 切，为什么没有变成二次

每个子问题在线性时间内求跨越分界的最佳区间，再加上左右递归。固定按 1/4 和 3/4 切，递推是$T(n) = T(n/4) + T(3n/4) + \Theta(n)$。一层各节点代表互不重叠的输入区间，总长度约 n，所以合并工作每层线性。最长路径沿 3/4 那边下降，约经过$\log_{4/3} n$层；这虽然比$\log_2 n$深，但仍是常数倍的$\log n$。先由“每层至多线性、最多对数层”得到$O(n \log n)$。下界也要交代：最短路径沿 1/4 下降。在深度不超过约$\log_4 n$的完整浅层中，各节点都尚未到基础规模，每一层的子区间仍覆盖整个输入，所以这一层的跨界扫描总计$\Theta(n)$。这样的完整层有$\Theta(\log n)$层，已经支付$\Omega(n \log n)$；与上界合并才得到$\Theta(n \log n)$。取整只改变接近基础规模的常数层和常数误差。

对任意固定 $\alpha \in (0,1)$ 都一样。但“固定”不能删掉：若每次切成 1 与 n-1，最长路径有 n 层，层成本依次 n、n-1……，合计 $\Theta(n^2)$。看见两个递归分支，不能自动认定是 nlog n。

22.28 28 · 记忆化：缓存的键必须能决定整个子问题

以 Fibonacci 为例，直接递归 `F(n)=F(n−1)+F(n−2)` 会反复计算同一个 `F(3)`、`F(4)`。把已求得的 Fk存起来，再次遇到同一个 k 就直接读，只有 n 个不同状态，每个状态合并两项，因此单位字模型下时间$O(n)$、缓存$O(n)$。

可以手画 `F(5)`：左分支 F4里会计算 F3，右分支又要 F3。不缓存时重新展开，缓存后第二次只取已有结果。缓存节约的是重复子问题，不会神奇地消除互不相同的所有分支。

对 `F(5)`，约定 `F(0)=0,F(1)=1`，先递归左分支时，第一次计算并写回缓存的顺序是：

```
F2 = F1 + F0 = 1，缓存 {0:0,1:1,2:1}
F3 = F2 + F1 = 2，缓存再添3:2
F4 = F3 + F2 = 3，F2直接读缓存
F5 = F4 + F3 = 5，F3直接读缓存
```

`F3` 的第二次调用不再展开 `F2+F1`。若用自底向上，直接按 2、3、4、5 的顺序算同一批格；若只求最终值，又只需保留前两项，因为每次用完更早项后没有其他依赖。缓存所有答案与滚动两个变量是两种空间选择，不应混说成记忆化自动只占常数空间。

一个常见错误是求“从位置 i 出发、不许用已经选过的元素”的答案，却只用 i 作为键。两个调用虽然位置相同，已选集合不同，剩余合法选择也不同，不能共享答案。正确键可能是 `(i,used_mask)`，状态数也随之大增。先问“相同键是否保证以后面对完全相同的问题”，再写缓存。

22.29 29 · LCS 恢复：表里存的是长度，路径带出字符位置

设 $D[i][j]$ 表示 A 前 i 个字符与 B 前 j 个字符的 LCS 长度。若末字符相同，取 $D[i-1][j-1]+1$；否则取上格和左格的较大值。第一行、第一列为 0，因为有一串为空。

以 A=`ABC`、B=`AC` 为例，完整表是：

A 前缀 / B 前缀	空	A	AC
空	0	0	0
A	0	1	1
AB	0	1	1
ABC	0	1	2

从右下角 `(3,2)` 回溯：C 相同，记录 A 中下标 2、B 中下标 1，走左上到 `(2,1)`；B 与 A 不同，上格大于左格，向上到 `(1,1)`；A 相同，记录 `(0,0)`，再走左上。得到的是逆序的 C、A，把记录反转就是 A、C 及两串中的位置。

相等的候选格可能产生多条最优路径。规定“并列优先上格”便能得到一个确定答案，但它不自动保证字典序最小的 LCS；那是更强要求。保存完整长度表，计算时间$O(mn)$，回溯$O(m+n)$，额外输出长度不超过$\min(m,n)$。若原先只保留滚动两行求长度，直接回溯所需的信息已经丢掉；不能声称不增加信息就自然恢复路径。更省空间的恢复算法需要另讲分治技巧。

22.30 30 · 自顶向下和自底向上：同一个依赖图的两种走法

自顶向下从 `D[m][n]` 开始，需要哪个依赖就递归求哪个，算过的格子缓存。自底向上按行或列填写，保证填某格时左、上、左上已存在。两者使用同一递推关系，差别是访问顺序和实现开销。

LCS 的密集最坏情形中，两个方法都可能碰到 $\Theta(mn)$ 个状态。自底向上用连续数组，容易滚动行，且没有递归栈；自顶向下贴近定义，并可能避开没用到的状态。例如两串每个末字符都相等时，沿对角线就能一路下降，不必访问整个矩形。

但若依赖图是稀疏、不规则的搜索空间，先把所有可能状态铺成表可能很浪费；反过来，状态几乎全用到而且规则，逐格填表通常更便于控制常数。先比较访问了多少个不同状态，再比较每个状态的递归、查表与内存成本，不用“递归差、循环好”概括所有情况。

22.31 31 · 读源码：把接口的约定也读进来

打开课程代码包后，不必第一遍就追进每个模板细节。先选一个最小调用，例如对 `[2,5,7]` 求和，找到入口、参数、递归基和返回值。把调用链写成 `sum(3)→sum(2)→sum(1)→sum(0)`，再沿返回方向累计。

对每个接口写四件事：合法输入是什么、输出保证什么、哪些对象被修改、谁负责释放资源。例如 `[lo,hi)` 中 hi 不属于区间；若错按闭区间传参，复杂度公式完全正确也会越界。再用空数组、单元素、全相等、最大合法下标和非法边界分开测试。

如果本机平台无法直接打开 Visual Studio 工程，可以读取源文件并用合适编译器编译选定项目，并分别记录所用编译器、选取项目和运行结果。测试通过意味着这些输入符合预期；正确性证明还要覆盖其他合法输入。

22.32 32 · 看演示：先预测下一步，再按播放

演示最有价值的用法是让自己暴露误解。以反转或二分查找为例，先写当前数组、lo、hi，预测下一次交换或轴点；只播放一步，再比较。若预测错了，要指出自己把哪个区间端点或更新顺序理解错。

可以做一次具体记录：数组 `[1,4,6,9]`，目标 5，用返回最后一个不大于目标的位置的 C 版二分。第一次看 6，应把 hi 改到 2；再看 4，应把 lo 改到 2；此时 lo=hi=2，返回 1，对应值 4。返回值是前驱位置，不是“没找到，所以算法失败”。

动画只展示所选输入的一条执行轨迹。要把观察变成普遍结论，还要提炼不变量和终止量。纸笔预测、单步核对、写出理由，才是把演示变成可复用理解的过程。

22.33 本章练习：离开答案再做一次

1. $T(n)=n^3, f(n)=n^2$，判断 $T=O(f)$ 与 $\log T=O(\log f)$。说明两个判断为什么能不同。
2. 用 Failstone 规则追踪 `(3,5)`，数清返回值，并指出哪几步组成最大值下降的一组。
3. 按本课程规则计算`[2,0]`、`[3,-9,3]`和`[0,0]`，分别给出和与半开区间。为什么扫描时应丢掉和为 0 的待接后段?

第一题：T/f=n 无界，故第一个不成立；对数比值为 3/2，第二个成立。第二题依次为 `(3,5),(6,5),(1,6),(1,3),(2,3),(4,3),(1,4),(1,2),(1,1)`，返回 9；前三次变换后最大值从 5 降到 3，是一组。第三题依次为 $2,[0,1)$，$3,[2,3)$，$0,[2,2)$。零和后段不能改善将来的和，却会增加将来区间的长度；在允许空的本题中，替换成空后段才符合先取短的规则。全零输入中还要按最右规则选末尾空区间。

23 PS02：向量里的每一次搬动，都要知道搬了谁

教师原始习题单：第 02 次习题共 43 题。统一把有效区间写为 `[0,n)`，含 0、不含 n；容量是能容纳多少项，规模是已经存在多少项。元素复制、比较暂按单位成本计；大对象的构造、移动和析构需要另计。

23.1 1 · 面向对象：从一个能自己维护状态的小类开始

先做一个具体的小程序：保存两个数，求和；再让一个对象记住“每次加 3”；最后从同一种基类接口查询不同图形的角数。这三个要求分别用到模板、函数对象和虚函数，先看它们各自解决什么问题，避免把名词混成一团。

```
#include <iostream>

template<class T>
class Pair {
    T first, second;
public:
    Pair(T a, T b) : first(a), second(b) {}
    T sum() const { return first + second; }
};
struct AddTo {
    int delta;
    void operator()(int& x) const { x += delta; }
};
struct Shape {
    virtual int corners() const { return 0; }
    virtual ~Shape() = default;
};
struct Triangle : Shape {
    int corners() const override { return 3; }
};
int main() {
    Pair<int> p(2, 5);
    Pair<double> q(0.5, 1.25);
    std::cout << p.sum() << ' ' << q.sum() << '\n';
    int x = 4;
    AddTo plus3{3};
    plus3(x);
    std::cout << x << '\n';
    Triangle t;
    Shape& s = t;
    std::cout << s.corners() << '\n';
}
```

输出依次是 7 1.75、7 和 3。从第一行开始看：`Pair<int>` 把 T 确定为 int，两个成员存的是 2 和 5；`Pair<double>` 则存 0.5 和 1.25。模板是按类型生成相应代码的规则，`Pair` 本身并不是一个已经装了两个数的对象。`p` 与 `q` 才是实际存在的对象，各自保有数据。

`Pair(T a,T b) : first(a),second(b)` 是构造函数，冒号后在进入函数体前初始化两个成员。这里成员不向外公开，外部不能随手更改它们；`sum() const` 表示这次求和不修改该对象的普通成员。课程 Vector 同样要用构造函数建立 `size<=capacity` 等合法初态，不过它还拥有动态资源，需要进一步处理复制和析构；本例没有自己申请数组，不能拿它代替资源管理例子。

`plus3` 中 `delta` 存 3。表达式 `plus3(x)` 调用的是 `operator()`，形参 `int&` 直接引用原来的 x，因此把 x 由 4 变成 7。若写成 `int x` 而没有 `&`，改的将是函数内的副本，主函数中的 x 仍为 4。方法结尾的 `const` 只说明不会改 `plus3.delta`，不妨碍通过引用修改另外的 x。这是两个不同对象，不能见到 `const` 就断言所有东西都不能改。

最后，s 的静态类型是 Shape&，引用的实际对象却是 Triangle 的 t。corners 是虚方法，所以运行时选择 Triangle 的实现，得到 3。若把基类的 virtual 和派生类的 override 一起去掉，通过 Shape& 调用将选基类方法而返回 0。override 的作用是要求编译器核对“我确实在重写基类虚方法”，拼错签名会被报错。

虚析构保证将来通过基类指针释放派生对象时有合适的析构行为；它不是求角数所需的步骤，但与这个可派生接口的资源安全有关。重载是在同一名称下提供不同参数签名，重写则是派生类提供基类虚方法的实现。模板在编译期按类型选代码，虚调用根据实际对象选择实现；读 Vector 模板时，应分别看清类型、对象、引用和资源，而不是用一句“多态”盖过它们。

23.2 2 · std::vector：reserve 以后，为什么仍不能写 v[0]

空向量的 size 为 0。reserve(10) 只保证容量至少 10，不创建 10 个元素，所以 v[0]=7 仍越界。push_back(7) 才创建第一个有效元素；resize(4) 则把有效规模改成 4，新建整数按该接口的初始化规则得到相应初值。

照着做：空表→reserve(10)→push_back(7)→push_back(8)→erase(begin())。规模依次 0、0、1、2、1，最后元素为 8。打印每一步 size、capacity 和 data 地址；容量除标准明确的下界等保证外，不能把某个实现的数值照抄成普遍规律。

若保存元素指针，然后扩容迁移，旧指针不会自动指向新数组。释放后继续解引用违反有效性要求，即使偶尔还能读到旧数字也不能据此判断安全。只比较地址数值作观察，不去访问已经失效的对象。

标准原文：vector 容量与 reserve/resize，可核对规模不变、容量保证与重分配失效规则。

23.3 3 · 课程 Vector：测试报告必须对照哪个不变量

通常有 _size、_capacity、_elem 三项核心状态。首先检查 $0 \le size \le capacity$，其次检查有效对象恰好落在 [0,size)，最后检查谁拥有并释放底层数组。

从容量 4、元素[2,4,6]开始，依次头插 1、尾插 8、删中段、复制、修改副本。头插后应是[1,2,4,6]；尾插可能扩容；副本的修改不应改变原表。若改变了，复制很可能只复制了地址，没有建立独立资源。

把这条操作序列的内容、规模都补齐，就能写出不会猜答案的断言。设删去半开区间[1,3)，结果如下：

操作	有效数组	size	应检验的性质
初态	[2,4,6]	3	size≤capacity
头插 1	[1,2,4,6]	4	后缀顺序保持
尾插 8	[1,2,4,6,8]	5	新数组若迁移，内容仍相同
删[1,3)	[1,6,8]	3	删除的是 2、4，不是 6
复制到 w	v 与 w 均[1,6,8]	各 3	内容同，资源独立
令 w[0]=99	v 为[1,6,8]，w 为[99,6,8]	各 3	原表不受修改

capacity 只在确实取得所用实现的扩容规则后才能填写精确值；表里故意不猜某个标准库的增长倍数。

把空表、单元素、刚好满、自赋值、空删除、删除到触发缩容分别作为测例。随机通过 100 次只说明这些轨迹没有暴露错误。阅读测试主程序的每条断言，确认它查的是内容、规模、地址失效，还是资源泄漏；同一个“通过”不代表所有性质都已验证。

23.4 4 · 批量接口：同一后缀只搬一次

向长度 1000 的表头插 100 项。一次批量插入，原 1000 项整体右移 100 格，再写 100 个新项，主要约 1100 次写入。反复在同一位置单项插入，搬移量为 $1000 + 1001 + \cdots + 1099 = 104950$，还要加 100 次新值写入，而且新项次序会反转。

若每次递增插入位置以保留新项顺序，之前新插的项可能不用再搬，但原 1000 项仍各搬 100 次，共 100000 次。具体公式随调用次序改变，重复移动的浪费却很清楚。

因此先做区间接口，再把单元素视作长度 1 的特例，可以共享一次搬移和边界处理。删除也一样：一口气删除头部 100 项，只需压缩一次后缀；逐个删会重复压缩。批量设计的收益来自减少真实工作，不来自函数名字更漂亮。

23.5 5 · insert：插入多少项，原后缀又有多长

令 $k = hi - lo$，原规模 n，在秩 r 前插入。没有扩容时，旧后缀 [r,n) 的 n-r 项右移 k 格，再复制 k 项，故成本为 $\Theta(1 + k + n - r)$。非平凡情况常省掉 1，写成 $\Theta(k + n - r)$；触发扩容还要计入旧数组迁移。

原表[10,20,30,40,50]，r=2，插入[7,8]。先从末尾搬：50 到 6、40 到 5、30 到 4，再写 7 到 2、8 到 3。结果为[10,20,7,8,30,40,50]，搬 3 项、写 2 项，对应公式里的 $n - r + k = 5$。

逐次赋值也可以画出来。下划线表示已申请但还未写入结果的格：

```
原态        [10,20,30,40,50, _, _]
50: 4→6     [10,20,30,40,50, _,50]
40: 3→5     [10,20,30,40,50,40,50]
30: 2→4     [10,20,30,40,30,40,50]
写7到2      [10,20, 7,40,30,40,50]
写8到3      [10,20, 7, 8,30,40,50]
```

中间还出现旧 40、50 没有关系，因为当前在构造结果，尚未宣布新的有效区间完成。循环不变量是：已搬过的右端元素已在最终位置，尚未搬的原后缀仍完整。倒序使下一次读取总在未被覆盖的区域。

若源区间来自同一个向量，扩容可能让源地址失效，移动又可能覆盖尚未复制的项，必须遵守接口的别名约束或暂存源。不能在证明里假定源独立，实现里却把自身的一段直接传进去。

23.6 6 · 移动方向：右移倒着来，左移正着来

把[A,B,C,D]的 B、C、D 右移一格。若先搬 B 到 C 的位置，原 C 就丢了；下一轮读到的已经是 B，后面可能连着复制同一个值。正确顺序是先 D，再 C，再 B，让每次读到的源都还没被覆盖。

错误的正序搬移也值得走完一次。若把 B、C、D 右移一格，先做 `a[2]=a[1]`得到`[A,B,B,D,_]`；再做 `a[3]=a[2]`读到的已经是 B，变`[A,B,B,B,_]`；最后再写一个 B。C、D 都被覆盖了，后面再小心也找不回来。正确顺序则是 `D:3→4`、`C:2→3`、`B:1→2`，最后`[A,B,B,C,D]`，留出的下标 1 才用于新项。

删除后的后缀左移则从前向后，因为目的地在源左侧，不会破坏更靠右的未读项。源和目的完全不重叠时，方向可以不同；判断依据是区间重叠关系。

`memcpy` 要求不重叠，`memmove` 允许重叠。但是含字符串等非平凡 C++对象不能简单按字节搬来搬去：构造、析构、移动和所有权都有类型规则。画整数格子帮助理解方向，写模板容器还要把对象生命期处理正确。

23.7 7 · remove：没删东西，却可能搬完整个表

删除 [lo,hi) 后，旧后缀 [hi,n) 左移 hi-lo 格。整数搬移模型的成本为 $\Theta(1 + n - hi)$，固定接口成本不能在 n=hi 时解释成“完全零时间”。

原循环如果不判空，lo=hi=0 时不删任何项，却让每个元素做一次自赋值，正好仍是线性最坏例。加 `if(lo==hi) return 0;`，便能让空区间常数返回。真实删除首项也要搬 n-1 项，仍为线性。

通用对象还可能需要析构被删项，所以仅以 n-hi 计费隐含了元素处理模型。多一次分支值不值，要看空删除频率与元素移动代价；能给出实验设计，不能凭空报提速百分比。

23.8 8 · 几何扩容：一次昂贵迁移由很多次追加分担

容量每次乘固定 $\alpha > 1$。从空表一路追加到 N 项，旧容量迁移量形成几何级数，累计小于约 $\alpha N/(\alpha - 1)$，加上 N 次新项写入仍为 O(N)，所以按单元素追加均摊 O(1)。

倍增时搬过 1、2、4、8 项，合计 15，不是每次都搬 16 项。$\alpha = 1.1$ 仍满足常数倍增长，但复制常数大致随 $1/(\alpha - 1)$ 增加；增长后装填率约 $1/\alpha$。更小的闲置空间通常意味着更频繁的迁移。

以初始容量 1、单项追加为例，逐次迁移账单是：

第几次追加	追加前 size/capacity	本次搬旧项	追加后容量
1	0/1	0	1
2	1/1	1	2
3	2/2	2	4

第几次追加	追加前 size/capacity	本次搬旧项	追加后容量
4	3/4	0	4
5	4/4	4	8
6、7、8	5/8、6/8、7/8	各 0	8
9	8/8	8	16

前 9 次共搬 15 项，另外写 9 个新项，总 24 次元素写入。单次第 9 次确实很贵，摊还分析没有把它改成便宜；它说的是这 9 次合计只花了常数倍 9。

一般增长率$\alpha > 1$固定。最后一次迁移的旧容量小于N，前面的容量至多依次除以α。所以总迁移小于$N(1 + 1/\alpha + 1/\alpha^2 + \cdots) = \alpha N/(\alpha - 1)$。分母越小，复制常数越大；若增长率随规模趋近 1，就不能再把这个因子当固定常数。

批量插入不能按“调用一次”无条件均摊 O(1)：一次插百万项，光写新项就得线性；在一个已有巨大满表上触发扩容，也可能有大迁移。应按完整操作序列和插入元素总数算账。标准库没有规定每个 vector 实现都恰好倍增，具体倍数要查所用库版本并打印 capacity 核对。

23.9 9 · 为什么不能只平均真正扩容的调用

倍增到 N，实际迁移约 log N 次，总迁移量为 $\Theta(N)$。只对这些昂贵调用取平均，结果是 $\Theta(N/\log N)$；把所有追加检查算进来，才是每次摊还 O(1)。两者换了分母，没有矛盾。

两次扩容之间的便宜操作为下次“搬家”积攒费用。若把这些操作从统计集合删除，自然不能继续引用原来的均摊结论。摊还也不要求随机输入；对任何遵守接口的合法序列，总账都受控制。

写分析时明确列出你平均的是所有 push_back、所有 expand 调用，还是仅有搬迁的 expand。先把样本集合讲清楚，后面的公式才有具体含义。

23.10 10 · 半满就缩会抖动，四分之一才缩留出了间隔

容量 C、规模刚低于 C/2 就缩成 C/2，接着只需插很少几项就再次满；删一点又缩。对手用常数次插删，就能迫使约 C/2 项反复迁移，昂贵操作之间没有足够便宜操作来分摊。

取“严格小于半满才缩”的规则，从 `size=8,capacity=16` 开始：

```
删除1项：size=7 < 8，搬7项，容量缩到8。
追加1项：size=8，容量8，未搬迁。
再追加1项：原来已满，搬8项，容量扩到16，size=9。
删除1项：size=8，不满足严格小于8，暂不缩。
再删1项：size=7，再搬7项缩到8。
```

从第一次缩容后的状态起，反复“加 2、删 2”，每 4 次普通插删就搬约原规模的项。若阈值用≤半满，抖动甚至能更频繁。不能用“原来追加时已经证明过均摊”来覆盖这条新操作序列。

改成低于 C/4 才缩为 C/2，缩后约半满。要再扩容得插入约 C/4 项，要再缩还需删掉一批项。这个间隔使累计收费能够覆盖搬移。

仍从容量 16、规模 8 开始，固定采用严格不等式$n < C/4$，用同一串“加 2、删 2”就看得出差别：

操作结束时的规模	8→7→8→9→8→7	每步是否触发迁移
半满就缩	容量 16→8→8→16→16→8	搬 7、0、8、0、7 项
四分之一才缩	容量一直 16	全部不迁移

继续一直删除时，规模 4 恰好等于 16/4，仍不缩；删到 3 才搬 3 项，容量改成 8。此时再追加 5 项可使规模依次 4、5、6、7、8，都不迁移，第 6 次追加才先扩成 16 再写第 9 项。若改为继续删，容量 8 的阈值是 2，规模 2 仍不缩，删到 1 才再缩。昂贵操作之间隔着与当前容量成比例的一批普通操作，才是摊还成本可控的原因。阈值用le会挪动边界那一步，分析时必须统一；最小容量还要有固定下限，空表不应反复分配零长数组。

23.11 11 · 去重里的哨兵：当前项本来就在停止位置上

查 `_elem[k]` 是否出现在 `[0,k)` 时，目标就在下标 k。若 find 靠第 k 格的哨兵让扫描最终停下，就不必再保存、写入和复原同一个目标，因为它已经在那儿。

例如`[5,2,5]`处理下标$k=2$。要查目标 5 在前缀`[0,2)`里是否出现，哨兵查找可从 0 顺扫到相等即停。即使前缀没有 5，走到下标 2 也一定停，因为那一格本来就是目标。所谓省掉设置与恢复，就是省去“暂存第 2 格、把 5 写入第 2 格、查完写回”的三次多余动作；它没有允许搜索结果把第 2 格本身当作前缀命中。返回位置若等于 2，仍应解释为前缀中不存在。

若处理`[(5,A),(2,B),(5,C)]`时命中前面的 A 并删除它，最后保留的是 C。改删当前 C 就保留 A；两者 key 相同，记录身份却不同，必须先决定接口承诺。

每轮省常数次操作，n 轮省 O(n)。如果整体仍有二次比较或移动，总阶没有改变。常数优化值得解释，但不能说它把算法改成了线性。

查到前面的 i 时删 i，会保留较新的 k；改删 k 则保留先出现的一份。附加字段可能不同，所以这不只是下标替换。删 k 以后，下一个未处理项移到 k，不能还无条件 k++，否则会漏项。

23.12 12 · 无序去重：先减少移动，再讨论比较

若保留最后一次出现，可以让 i 扫描原表，find 检查未读后缀 `[i+1,n)` 是否还有同值项；有则跳过，没有才把整条记录写到 w 并令 w++。w 始终不超过 i，不会破坏未读后缀。

例：[A1,B1,A2,C1,B2]，A1 和 B1 后面还有同值记录，跳过；A2、C1、B2 依次写入，得到[A2,C1,B2]。每项最多写一次，移动不超过 n。已写前缀始终是已经确认不再有后继重复的记录。

压缩过程可以保持旧数组尾部不动：

读下标 i	当前记录	后缀还有同 key 吗	写前 w	本轮动作	已确定输出前缀
0	A1	有 A2	0	跳过	`[]`
1	B1	有 B2	0	跳过	`[]`
2	A2	无	0	写 a[0]=A2，w=1	`[A2]`
3	C1	无	1	写 a[1]=C1，w=2	`[A2,C1]`
4	B2	无	2	写 a[2]=B2，w=3	`[A2,C1,B2]`

输出写指针从未超过读指针，故不会覆盖还要搜索的后缀。处理结束才令 size=w；不能每跳过一项就缩容，否则原读下标的含义会改变。比较次数仍取决于后缀查找遇到相等多早，写入次数则至多 n，这两笔账分开。

全相等时，若 find 见到相等项立即返回，可每轮很快确认重复，整体 O(n)；全不同时仍要扫描各后缀，比较为 $n(n-1)/2$。减少移动没有自动减少所有比较。

若允许排序，可 O(nlog n)后线性去重，但顺序可能改变；允许散列表时可在相应散列模型下得到期望 O(n)，空间也增加；小整数值域可用位图。对于任意元素只提供相等测试的模型，不应承诺同一改进。题面所说移动比比较贵，是优化动机；通用临界长度还取决于元素类型和机器，不能编一个数字。

23.13 13 · 函数对象：累加到的是外部对象还是内部副本

求和访问器保存累加状态。要判断结果保存在哪里，必须把访问器的传参方式一起写出来，而不能只看到 `operator()`就认为外部对象会变化：

```
#include <iostream>
#include <vector>
struct Sum {
```

```
    long long value = 0;
    void operator()(const int& x) { value += x; }
};
template<class Visitor>
void traverse_ref(const std::vector<int>& a, Visitor& visit) {
    for (const int& x : a) visit(x);
}
template<class Visitor>
Visitor traverse_copy(const std::vector<int>& a, Visitor visit) {
    for (const int& x : a) visit(x);
    return visit;
}
int main() {
    std::vector<int> a{2, -1, 5};
    Sum s;
    traverse_ref(a, s);
    std::cout << s.value << '\n';
    Sum t;
    Sum result = traverse_copy(a, t);
    std::cout << t.value << ' ' << result.value << '\n';
}
```

第一种在调用点把 s 直接交给引用，访问后的状态是 0→2→1→6，外部 s 也为 6。第二种先把 t 复制成形参 visit，内部副本同样变成 6，原 t 仍为 0；返回的副本由 result 接住，因此输出 `0 6`。两种接口都能实现求和，调用者取结果的位置不同。课程 `traverse` 究竟采用哪一种，应读它的签名再调用；不能拿这段演示函数名当作课程已有接口。

空表求和保持初值 0；整数元素多了以后可能超过 int，所以累加器范围仍要算。函数对象能把“访问时顺便保存状态”组织起来，不能免掉数值范围和复制语义。

23.14 14 · 用遍历做冒泡：访问器要记住前一项

一趟冒泡比较相邻元素。访问器保存上一项地址与交换标志；访问当前项时，若前项更大就交换，然后把上一项地址更新到当前项。对[3,1,2]，先交换 3 和 1 得[1,3,2]，再交换当前 3 和 2 得[1,2,3]。

若访问器只收到值的副本，就不能修改原表；要限制未排序区间，还需区间遍历或计数器。把循环换成 traverse 只是换了表达方式，邻接关系和边界状态仍然要维护。

对`[3,1,2]`，访问器的状态应是：第一次访问下标 0，只记住地址 0；访问下标 1，比较 `a[0]=3` 与 `a[1]=1`，交换后数组`[1,3,2]`，把前项地址改为 1；访问下标 2，比较 3 与 2，交换后`[1,2,3]`，前项地址改为 2。若第二步交换后仍让前项地址停在 0，第三步会比较 1 与 2，错过 3 与 2。保存的是“刚经过的格子地址”，不是“当初见到的那个对象值”。

一趟线性，最坏 n-1 趟，比较仍约 $n(n-1)/2$，总时间 $\Theta(n^2)$。接口复用与渐近效率要分别评价。

23.15 15 · 判等与比较：同分数的两条记录算相同吗

记录(score,id)按分数排序，(90,A)与(90,B)互不小于，属于同一排序等价类；若相等运算符还比较 id，它们又不相等。去重按分数还是按完整记录，要由业务定义。

严格弱序比较器的等价可写为 `!less(a,b)&&!less(b,a)`。这不一定与 `a==b` 一样。排序按 score、去重按全字段，并不会自动得到“每个分数只留一条”。

比较规则还必须在整个算法中一致。若同一个 a 满足 less(a,a)=true，或排序过程中临时改变顺序标准，二分与归并的证明都失去了前提。

23.16 16 · 缩容后：整数秩还在，旧地址不会自己更新

旧底层数组在 P，缩容后迁到 Q。秩 k 只是一个整数；再调用当前 `_elem[k]` 会用 Q 寻址，只要 k 仍合法，就能访问新数组。

旧指针 P+k 却不会自动改成 Q+k。原数组释放后它悬空；引用和某些迭代器也一样。删除还会改变秩：旧 k 可能现在指向原来的下一项，即使没越界，逻辑也可能错。

检查去重循环时分开问：“保存的是秩还是地址？”“删后下一项移到了哪里？”它们分别对应地址失效和扫描漏项，不能混成一句“扩容后小心”。

23.17 17 · 懒惰 uniquify：前面没重复时，为什么还写自己

read 向前读，write 指向最后保留位置；新值与保留值不同才推进 write 并写入。遇到首个重复以前 read=write，赋值只是自己写回自己。

可先找第一对相邻相等项；若没有便直接返回，找到了再开始压缩。例[1,2,3,3,4]的前三项不用写，最后 4 才移到下标 3。始终维持“输出前缀等于已读部分的去重结果”。

具体下标是：先比较 `a[0],a[1]`，再比较 `a[1],a[2]`，都不同，尚不写；比较 `a[2],a[3]`发现两个 3，令 write=2、read=4；4 与保留的 3 不同，把 write 增为 3，执行 `a[3]=4`，最后 size=4。结果`[1,2,3,4]`，只发生一次有效写入。若数组`[1,2,3]`没有重复，检测完直接返回，无需写 3 次自己；若空或单元素，则在读取相邻格以前就返回。

这减少写操作，却增加了首段检测或分支。对无重复、大量重复和昂贵对象，实际收益不同；总读次数仍线性，所以时间阶不变。优化不能以更难验证的边界代码为代价。

23.18 18 · 本题保留最后一份：按组扫描就能做到

原题把稳定约定为每组相等记录保留最后一项。它与排序稳定性的通常定义相关，但要求不完全相同，回答时要忠于这条题面约定。

有序表逐组扫描：i 是组起点，j 一路走到组末，把整条 a[j]写到输出位置。[(1,A),(1,B),(2,C),(2,D)]最终得到[(1,B),(2,D)]，不能只改 key 而丢掉附加字段。

组内最后项的定位可用如下边界：从 i 开始令 j=i+1，当 j<n 且 key(a[j])等于 key(a[i])时一直加 j。循环退出后，本组是`[i,j)`，最后项在 `j-1`。写 `a[w]=a[j-1]`，再令 w++、i=j。例中第一组`[0,2)`选 B，第二组`[2,4)`选 D。每轮至少吃掉一项，j 总向前，所以终止；整个记录一起复制才保留最后一项的所有附加字段。

每项被访问常数次，总时间 O(n)，几个下标只需 O(1)空间。从右向左也可选最后记录，但要把输出恢复到正常顺序，并把那部分移动计入成本。

23.19 19 · 分支次序：大 O 相同，不等于比较次数相同

一次查找要区分小于、大于、等于轴点三种情况。用两个 if 和末尾 else 表达，排在第一种的情况只用一次关键字比较，其余用两次。若本层三类概率为p_L, p_R, p_E，三者之和为 1；设放第一的类别概率为p，则成本为$1cdotp + 2cdot(1-p) = 2-p$。所以在这套收费且各类都能用一次测试辨别的模型下，应把概率最大的类别放第一，最小期望为$2 - max(p_L, p_R, p_E)$。实际比较器若只提供<，要先确认能否真的用一次操作判断所选类别；判等可能需要两次<，这时得重列成本。

无论怎么排，只要每次把区间按固定比例缩短，最坏迭代轮数仍是 $\Theta(\log n)$。但相等命中若极少，把相等测试放第一，就可能每层几乎都要多做一次比较。取百万个互异 uint32_t 关键字、目标在全部 2^{32} 个数中均匀抽取，总命中概率约 $10^6/2^{32} \approx 0.000233$，即 0.0233%。

这还不能推出每一层左右概率恰好各半。若关键字和查询都分布平衡，可粗估相等优先约每层 2 次，方向优先约每层 1.5 次；约 20 层时分别接近 40 和 30。若值集中、查询偏斜或比较代价不对称，估计会变。CPU 分支预测也会影响真实时间，所以这里估的是关键字比较，不能把它直接当纳秒。

23.20 20 · 左转 1、右转 2：课程为什么忽略循环下标的比较

原 A 版先测试 `e<S[mid]`，为真就左转；否则再测试另一方向，不等于才右转。按关键字比较计费，左转 1 次、右转 2 次。`lo<hi`、计算 mid 和更新下标也做工作，但课程把它们作为统一的控制开销，或把昂贵的关键字比较作为单独的分析对象。

若元素是长字符串，关键字比较可能远贵于一次整数下标运算，这个模型很有意义。若所有元素是短整数，要估实际运行时间，必须把这些成本和内存、分支都纳入。没有哪条物理定律保证右转恰好耗时左转两倍。

所以不是随意把 1:2 改成 2:3 或 3:4；那相当于在每轮额外加入了另一种固定收费。模型可以换，但模型换后，黄金分割最优常数的推导也应跟着换。

23.21 21 · 中点溢出：最终合法，计算途中也可能先出事

在 `0<=lo<=hi<=INT_MAX` 下，`hi-lo` 非负且不超过 INT_MAX，半差也在范围内，`lo+(hi-lo)/2<=hi`，所以这条中点公式不会因正整数加法超上界而溢出。

而 `(lo+hi)/2` 先算和。例如 32 位有符号数里 lo=1500000000、hi=2000000000，两者分别合法，总和 3500000000 已经超范围，不能指望最后再除 2 救回来。

若改用无符号类型，算术可能按模回绕，仍不等于算出了正确中点；若 lo、hi 来自不同符号类型，转换也要另查。库代码不应因为“当前机器放不下那么多元素”就省掉几乎不增加复杂度的安全写法；不过本题的具体小规模实测可能根本触不到这个边界。

23.22 22 · 重复项：“随机返回”其实是确定轨迹

A 版遇到等于轴点就返回。数组[2,2,2,2]、区间[0,4)第一次 mid=2，立即返回 2；区间换成[0,3)，mid=1，立即返回 1。同样的代码、输入和区间会重复返回同一个位置，所谓“随机”只是接口没有承诺首个还是末个。

需要第一项时寻找第一个不小于 e 的位置；需要最后一项时寻找第一个大于 e 的位置，再减 1 并检查是否等于 e。相等时继续向哪边收缩，决定了边界语义。

做测试时给重复元素附不同 id，才能看出返回的是哪一条。只打印 key，全是 2，很容易误以为测试已经验证了“保留哪份”。

23.23 23 · Fibonacci 切分：便宜的一边为什么分得更大

原 A 版左转收费 1，右转收费 2，轴点在 lo 加 $\lambda(hi-lo)$ 处。左边约占 λ，右边约占 $1-\lambda$。若最坏比较数近似 $C\log_2 n$，两种路径分别要求

$$C \ge \frac{1}{\log_2(1/\lambda)}, \qquad C \ge \frac{2}{\log_2(1/(1-\lambda))}.$$

要把两者的最大值压到最低，使两项相等，即 $\log(1-\lambda) = 2\log\lambda$，得到 $1-\lambda=\lambda^2$，正根为 $\lambda=(\sqrt{5}-1)/2\approx 0.618$。较大的子区间交给便宜的左转，最坏比较主项系数约 1.4404。

若镜像取 0.382，昂贵右转反而分到约 0.618，最坏系数约 $2/\log_2(1/0.618)\approx 2.8808$，并不仍然最优。这个最优性依赖 1 与 2 的收费模型，若使用三向比较器或不同硬件成本，结果要重新推导。

两条不等式可以从一个分支直接推出。若左转一次后规模约λn，剩余最坏成本约$C\log_2(\lambda n)$，加本次 1 次比较，应不超过$C\log_2 n$；消去公共项，得到$1\le C\log_2(1/\lambda)$。右转同理收 2 次，于是得到第二条。λ增加时，左子问题变大、第一约束变紧，右子问题变小、第二约束变松；两条相等处才使较大者最小。

Fibonacci 切分可以不用每轮算浮点数。若区间长度接近F_k，取左跨度F_{k-1}，余下约F_{k-2}；因为$F_k=F_{k-1}+F_{k-2}$，转到哪一边，就把“当前 Fibonacci 尺度”改成对应较小项。譬如长度 13 附近用 8 与 5，转左后用 5 与 3，再转右后用 2 与 1。实际轴点还需扣除轴点本身并夹到合法下标，不能把 8/5 比例当成每个长度都精确可分的整数公式。

Fibonacci 数满足相邻两项比值趋向这个比例，可以用递减的 Fibonacci 跨度定位轴点，避免每层浮点乘法。但若关键字比较很便宜，维护额外状态的成本未必值得；“比较次数更少”是理论结论，是否更快还应实测。

23.24 24 · B 版为什么先判 e 小于最小值

课程 B 版维持至少一个候选，循环到区间长度 1，然后直接返回 lo 作为“不大于 e 的最后一项”。若 e 连最小值都不到，却不提前排除，最后仍会留下下标 0，错误地声称它是合法前驱。

例如[3,7,9]查 2。候选不管怎么缩都不可能出现合法元素，所以应返回起始 lo 减 1 的失败哨兵；这就是开头判断的作用。A 版查相等项，可以一直缩到空区间而报告失败；C 版允许空区间，并从统一边界减 1，故不需要同样的最小值特判。

原题 b 行写着 A、C“都需要如此特判”，与这两版实际代码的处理不同；应把源码行为讲清楚，不能因为题面一句话便给它们强加多余分支。B 版读取 `s[lo]` 前还必须保证非空，这又是另一项前置条件。

23.25 25 · 空区间：返回哨兵，不代表可以解引用哨兵

C 版求 upper_bound 后减 1。若目标小于最小项，所有比较都向左，最终 lo 停在起始位置，返回 lo-1。若起初 lo=hi，循环根本不执行，也返回起始 lo-1。

这只是表示“本区间没有不大于目标的项”。若对全表[0,n)得到-1，不能接着访问 `S[-1]`。若 Rank 改为无符号类型，减 1 会回绕，所以接口还需改用可表达失败的类型或显式结果。

A 版空区间自然不进入循环；B 版如果一开始就读 `S[lo]` 则越界，必须先判空或把非空写成前置条件。把“返回值有定义”和“该值可以当元素位置访问”分开，是这组题的重点。

23.26 26 · C 版的另一种证明：按区间长度归纳

C 版每次选 mid，若 e 小于轴点，则 `[mid,hi)` 全部大于 e，不可能包含前驱，剩下左区间；否则 `[lo,mid]` 全部不大于 e，它们已经可以归入确定的左侧，未知部分只剩 `[mid+1,hi)`。

先把被证明的程序写出来，返回语义才不会漂移：

```
def predecessor(a, target, lo, hi):
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if target < a[mid]:
            hi = mid
        else:
            lo = mid + 1
    return lo - 1
```

例如`[2,2,5,8]`查 2：`(lo,hi,mid)=(0,4,2)`，5 太大，hi 改 2；再`(0,2,1)`，等于 2 也走右，lo 改 2；未知区间变空，返回 1。已确定左侧是两个 2，右侧是 5、8，前驱就是左侧最后位置。若相等立即返回，就不再是本题的 C 版。

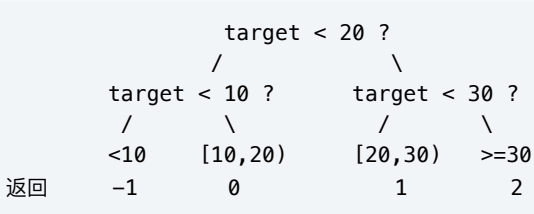
以初始区间长度为归纳参数。长度 0 时，未知区域已经空，边界 lo 左侧全不大于 e、右侧全大于 e，返回 lo-1 正确。长度大于 0 时，两个分支都只删除已确定的一侧，并保留同样的边界含义，且剩余长度严格更小，因此可用归纳假设。

这个证明同时需要有序性、边界语义和严格缩短。只说“规模小了所以对”没有证明删掉的那一半不会藏着答案。若存在重复项，相等时也把 mid 归到左侧，正好确保最后找到最后一个不大于目标的项。

23.27 27 · 平均查找长度：先讲“平均哪些输入”

令 n 个关键字互异，成功时各关键字等概率，失败时 n+1 个间隙等概率；计费是 C 版循环中的关键字比较。若 $n = 2^k - 1$，决策树的 n+1 个终点间隙都在深度 k，所以任何查询均做 k 次比较，两个平均值恰好都等于 k。

先把$n = 3$列全，键为 10、20、30。每次先看下标 1 的 20，下一步无论左右都会再做一次比较。成功目标 10、20、30 分别返回 0、1、2，比较数都 2；失败 4 个间隙`<10`、`(10,20)`、`(20,30)`、`>30` 的比较数也都是 2。这是$k = 2$的完整小树：



图中叶子写的是整个有序数轴区间。等于某个键与紧挨它右边的失败间隙走相同的 C 版路径，所以成功查询并不在内部节点提前结束。

对一般n，令$N = n + 1$表示间隙数。最浅叶深$k = \lfloor \log_2 N \rfloor$，叶深只会是$k$或$k + 1$；设浅叶数$u$、深叶数$v$。叶数给$u + v = N$，完整二叉决策树的容量给$u/2^k + v/2^{k+1} = 1$。两式相减解出$v = 2(N - 2^k)$，再算深度总和$D = ku + (k + 1)v = kN + v = N(k + 2) - 2^{k+1}$。这就是后面公式中每个量的来源。

一般 n 就不能把“同一主项”说成精确相等。n=2 时，间隙依次小于首项、两项之间、大于末项，比较次数为 2、2、1，失败平均为 5/3；目标等于两个关键字时为 2、1，成功平均为 3/2。两者已经不同。

沿用刚才的N, k, D，失败均值是D/N，因为这次把所有N个间隙等概率平均。成功等于第 j 个键的轨迹与其右侧间隙相同，成功查询覆盖除最左间隙外的所有叶子，因此成功均值是$(D - d_0)/n$。其中$d_0 = lceillog_2(n+1)ceil$是最左间隙深度。例如$n = 2$有$N = 3, k = 1, D = 5, d_0 = 2$，代回正好得到失败$5/3$、成功$(5-2)/2 = 3/2$。$n = 0$没有成功键，成功平均本来就未定义；失败查询则 0 次。

这些式子共同给出 $\log_2 n + O(1)$，即主项系数为 1。题面用大 Theta 串起的表达应按渐近意义理解；若要求任意 n 时两个精确均值相等，上面的 n=2 就是反例。对数底与概率模型也必须说明。

23.28 28 · 比较 A、B、C：三版返回语义的分界

n 个互异有序键把数轴切成 n+1 个失败间隙，因此“失败类比成功类多 1”来自顺序结构，而不是某份二分源码的偶然。A 版成功是遇到相等项，B/C 版主要返回前驱；若需回答是否命中，还要在返回位置上再判等。

对 $n = 2^k - 1$，A 版成功键所在层 d 有 2^{d-1} 个。若只数访问层数，平均深度为

$$\bar{d} = \frac{\sum_{d=1}^{k} d\,2^{d-1}}{2^k - 1} = k - 1 + \frac{k}{2^k - 1}.$$

但按原代码关键字比较，左 1 右 2，成功末层还需判等，平均约 $1.5\bar{d} + 0.5$；失败间隙等概率时平均约 1.5k。访问一次节点与做一次关键字比较不能混算。

C 版在这个规模上每次固定 k 个方向比较，成功与失败都 k。B 版循环到长度 1，另有最小值特判；大多数路径同样为 log n 量级，但其精确平均应按课件源码把开头特判和最终命中检查分别计入。第一个失败间隙常能在特判处结束，它与其他间隙并不对称。先列清收费表，再求平均，才能解释几版常数为何不同。

23.29 29 · 终止：为什么 lo 不会一脚跨过 hi

C 版在 lo<hi 时，取 `mid=lo+(hi-lo)/2`，总有 $lo \le mid < hi$。左分支令 hi=mid，仍 lo≤hi，长度从 m 降到 $\lfloor m/2 \rfloor$；右分支令 lo=mid+1，仍 lo≤hi，剩余长度也严格小于 m。

所以长度是非负、严格下降的整数，循环必停；出口因为不能 lo>hi，又不再 lo<hi，只能 lo=hi。长度 1 时左分支把 hi 落到 lo，右分支把 lo 落到 hi，两种都一步结束。

B 版的持续条件为长度大于 1，更新 lo=mid 时不会卡住，因为长度至少 2 保证 mid>lo；出口长度 1。若误把同样的更新配上 C 版 lo<hi 条件，长度 1 时就会死循环。三版更新规则与持续条件必须成套理解。

23.30 30 · 插值查找：一个极端值就能把猜测拉歪

取序列[0,1,2,...,n-2,M]，其中 M 远大于 n，例如 M=n³，查找靠近 n-2 的值。按端点线性插值，估计位置大致为 $(n-1)e/M$，常常接近区间起点；一次仅排掉一两个小值，反复执行就可能线性。

把$n = 6, M = 216$代入，数组为`[0,1,2,3,4,216]`，查 4。闭区间端点插值第一次猜$\lfloor 5 \times 4/216 \rfloor = 0$，只排除 0；第二次在`[1,5]`猜$1 + \lfloor 4 \times 3/215 \rfloor = 1$，只排除 1；第三次又猜 2，再猜 3，最后才到 4。每次猜的位置合法，正确性仍在，但进展退化成顺序查找。极端值 216 把其他值的秩距离挤得几乎看不见了。

序列确实有序，坏的是“数值距离能预测秩距离”的假设。二分不依赖值分布，每次按位置砍半，因此最坏仍对数。插值实现可设置进展检查或回退二分，避免长期猜在边缘。

还要处理端点值相等的分母 0、乘法溢出、目标超范围和重复值。理论上在均匀模型下很快，不表示可以省略边界检查。

23.31 31 · 非均匀分布：用累计概率猜秩，而非硬套直线

若已知值服从分布 F，$F(x)$ 表示随机值不大于 x 的概率。长度 n 的独立样本里，不大于 x 的数量期望约 nF(x)，因此可用累计概率替代原始数值做插值。

对高斯分布，先标准化 $z = (x - \mu)/\sigma$，再用其 CDF 估计累计比例。中心很密、尾部很稀，直接按 x 的线性距离猜位置会失真，CDF 映射才反映这些密度差别。

但参数估错、有限样本起伏、离群点和重复值都会使预测不准。CDF 只负责提出一个候选轴点，仍需把它限制在合法区间，并靠有序比较维持正确性。不能把统计模型的期望当作每份数据的精确秩。

23.32 32 · 二分何时切顺序：做一个不会骗自己的实验

固定编译优化、元素类型、比较器和查询集合；分别测试长度 8、16、32、64 等小表，既有成功也有失败，既有均匀也有聚集值。每种重复足够多次，防止单次计时噪声淹没差异。

结果应累计到校验值并输出，避免编译器删除“算了却没人使用”的循环。把数据生成时间移出计时区，改变测试次序，报告中位数和波动，再选在代表数据上稳定的切换阈值。

顺序扫描虽然比较多，但控制简单、内存连续；小表可能胜过多分支二分。长字符串、缓存状态和 CPU 改变后，阈值可能变。切换阈值应来自相同接口、相同数据上的本机实测。

23.33 33 · 冒泡提前停：一整趟没交换才证明已经排好

每趟开始 sorted=true，只要交换一次就设 false。若整趟结束仍 true，所有相邻项非降，因此整个序列有序，可以结束。

[1,2,3]第一趟无交换；[2,1,3]第一趟交换得到有序，但标志已经 false，若没有别的逻辑，还要下一趟确认；[2,3,1]中的 1 每趟只能向左移动一格，需要更多趟。全相等也应不交换，否则既损害稳定性又失去提前停机会。

最坏逆序仍二次，最好有序为线性。平均省多少趟，需要指定随机排列模型和是否把最后确认趟算入；只挑一个近乎有序样本，不能推出总体平均提速。

23.34 34 · last 跳跃：最后交换之后的区域，为什么可以不再碰

一趟从左向右比较，把最后一次交换的右端位置记为 last。本趟结束后，last 之后没有逆序交换需求，且大项已向右传递，因此下一趟可把活动右边界缩到 last。

例如[3,2,1,4,5,6]的乱序只在前三项，第一趟最后交换发生在前部，后面 4、5、6 可以直接排除。若只按 n-1、n-2 缓慢缩小，会重复比较这些已经安稳的后缀。

把它走到结束，设活动区间为`[0,hi)`，last 存最后一次交换的右端下标：

```
hi=6：[3,2,1,4,5,6]
  (0,1)交换 -> [2,3,1,4,5,6]，last=1
  (1,2)交换 -> [2,1,3,4,5,6]，last=2
  后三对均不换，本趟结束令hi=2。
hi=2：只比较(0,1)，交换 -> [1,2,3,4,5,6]，last=1，令hi=1。
hi=1：不足两项，结束。
```

下一轮只需处理下标 0、1；下标 2 上的 3 已经是原活动区间前三项的最大值，且后面 4、5、6 也都不小于它。若把 last 理解成“下次还要比较到的右端下标”，会多比较或漏比较，必须与半开 hi 的约定配套。

last 初值和边界含义要配套：若 last=0 表示未发生交换，就可以结束；若存左端下标，下一轮界限又差 1。画清“还比较哪些相邻对”，比套一个神秘赋值更可靠。逆序最坏仍二次。

23.35 35 · 非对半归并：固定比例仍是 nlog n

设固定 $\alpha \in (0,1)$，递推为 $T(n) = T(\alpha n) + T((1-\alpha)n) + \Theta(n)$。每层区间总长度不超过 n，最长路径 O(log n)，上界 O(nlog n)；前常数倍 log n 层都尚未触到基础规模，层成本为 Θ(n)，给出匹配下界。

例如 1/4 与 3/4 切分比对半树更深，但深度只差固定常数。若比例随 n 变化成 1 与 n-1，就变成 $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$。

固定比例偏斜通常使常数变差，还可能改变实际缓存表现；渐近同阶不代表实际一样快。先分清常数比例与每次只减固定个数。

23.36 36 · 复用归并缓冲：不必每个节点都 new 一遍

排序入口一次申请长度 n 的辅助数组，递归时把区间边界传下去，每次归并使用对应片段，退出整个排序后统一释放。顺序递归的兄弟区间不会同时使用同一片段，可以复用。

原来每个归并都申请、释放缓冲，分配调用次数可能达到线性数量；统一缓冲把这部分调用降为常数次。元素比较和迁移总量仍为 O(nlog n)，峰值辅助空间仍 O(n)，不能把少做分配说成空间阶从 n 降成常数。

若改成并行归并，要检查不同任务的片段是否重叠。C++还需选择缓冲中哪些元素已经构造、哪些需移动赋值，不能只把裸指针传来传去。计时应对同一输入副本比较两版，并把复制测试输入放在计时区之外。

23.37 37 · 一边耗尽：右半剩下的值为什么可能不用再复制

课程常把左半复制到缓冲，右半仍在原数组，写入位置从原区间左端向右移动。左半先耗尽时，写指针与右半读指针会重合，右半剩余项已经处在正确位置，无需原地复制。

例如左[1,2]、右[3,4]，前两次把 1、2 写回，随后读右半的位置恰好就是接下来输出的位置。若右半先耗尽，缓冲中的左半还没在正确位置，必须继续写。

再用交错例核对写指针为何追得上但不会超过右读指针。原数组`[1,4,7 | 2,3,8]`，左缓冲`B=[1,4,7]`；写指针 k=0，右读指针 j=3。依次写 1、2、3、4、7，对应来源为左、右、右、左、左，此时 k=5、j=5、左缓冲耗尽，原下标 5 的 8 已经在最终位置。任何时刻“尚未输出的左缓冲项数”恰是 j−k，因此左侧不空时 k<j，不会写坏未来还需读的右项；左侧为空时 k=j，后缀自然就位。

如果实现把两半都复制到了外部缓冲，那么两边的剩余项都要拷回。答案取决于源、目的在哪以及读写指针的关系，不是“归并永远只处理一边耗尽”。

23.38 38 · 已经整体有序：只检查分界那一对就够吗

左右分别有序且非空。若左半最大值不大于右半最小值，即`a[mid-1]<=a[mid]`，那么任意左项都不大于任意右项，整段已经有序，可以直接返回。

这个检查多一次比较；在已经有序输入上，每个递归节点都能跳过整段归并，总成本可降到线性量级的节点检查。普通输入若经常不满足，只是多付一个小常数，不保证所有机器都一定更快。

要先处理空段和长度小于 2，避免 mid-1 越界；比较器若只提供 less，用`!less(a[mid],a[mid-1])`表达非降关系。相等时不必交换，稳定性也能保留。

23.39 39 · inplace_merge：接口名字没有保证完全不申请缓冲

C++的`inplace_merge`表示结果仍放回原区间，不等于所有实现都禁止临时存储。要以标准规定的语义、复杂度条件和手头库源码为准，不能照题面一句“彻底避免辅助空间”推广到所有库。

一种低元素缓冲的思路是：在两段中选切点，把中间错位的两块通过旋转交换，再递归处理两个较小归并。旋转可用反转实现，能省大块元素缓冲，却增加移动和控制工作；递归本身还可能用 O(log n)栈空间。

例如[1,4,7]与[2,3,8]，可先找 4 在右段应插入的位置，把[4,7]与[2,3]旋转成[2,3,4,7]，再处理更小边界。要证明稳定，切点的 lower_bound 或 upper_bound 取法也必须正确。空间紧张、大对象移动昂贵、数据规模等因素共同决定是否值得，不能只看“原地”两个字。

上例的块旋转可用三次反转实现：原中段`[4,7 | 2,3]`，先分别反转得`[7,4 | 3,2]`，再整体反转得`[2,3 | 4,7]`。块内相对次序最终恢复，所以这个旋转本身不打乱各块内部的相等记录；但二分切点若把左右相等记录分错边，整个递归归并仍可能不稳定。真正的通用算法还要递归处理两侧的新归并任务，这个特例旋转后碰巧全部有序，不代表任意两块转一次就完成排序。

标准对非并行 inplace_merge 明确区分额外内存是否足够：有足够内存时比较至多 N−1，否则允许 O(Nlog N)。核对合并接口与复杂度。

23.40 40 · 位图复位：清零所有格子，还是宣布旧记录失效

第 k 位在字节`k/8`，偏移`k%8`；置位用掩码或，清位用掩码取反再与。若值域大小 U，普通位图整体清零要处理约 U/字长个存储字，成本依赖 U。

例如值 13 在字节下标 1、位偏移 5。若字节当前是二进制`00000100`，掩码是`00100000`，或以后得到`00100100`；清 13 则与`11011111`相与，恢复`00000100`。字节中的低位约定必须统一，跨字节时 13 不是“第 13 个字节”。位图扩容也要拷贝旧字节、初始化新增字节为 0；若只追加未初始化内存，随机旧位可能被误认成存在。

`std::bitset<N>`的 N 是编译期固定大小，提供按位逻辑、set/reset/test 等接口；课程可扩 Bitmap 通常运行时扩容。二者不只名字不同，大小何时确定、越界时接口保证、能否整体位运算都影响用法。

Hopcroft 结构用稠密表 dense 保存当前存在的值，稀疏表 sparse 给出候选位置，另存逻辑长度 m。判断 x 存在，需要同时满足 `sparse[x]<m` 与 `dense[sparse[x]]==x`。只把 m 设为 0，就让所有旧候选位置失去校验效力，逻辑清空 O(1)。

另一种方法给每格存时间戳，清空时增加当前世代号，格子时间戳等于当前世代才有效。世代号有限宽，回绕时需要真正重置；首次分配、初始化和空间都不能消失。C++读取未初始化整数也不合法，所以“逻辑清空 O(1)”不等于首次创建一个巨大数组也完全免费。

23.41 41 · 普通位图去重并排序：时间里有值域这一项

把 n 个值的对应位设为 1，耗 O(n)；再从 0 到 U-1 扫描置位项并输出，耗 O(U)，合计 O(n+U)。扫描按数值递增，所以同时完成去重与排序。

若 U≤cn，便可简写为 O(n)。若 n=1000、U=十亿，扫描十亿位范围的需要处理的范围由 U 决定，不能只拿 1000 估计；位图还可能占约 125MB，具体单位和额外对象另计。

可以只在读输入时检查并保留首次出现来避免全域输出扫描，但那保留的是输入顺序，不再自动排序。输出要求不同，复杂度公式也不同；不能拿去重的改进偷偷替代“去重且排序”的原任务。

23.42 42 · 校验环删除：拿最后一项填洞

设 dense=[4,9,2]、m=3，sparse[4]=0、sparse[9]=1、sparse[2]=2。删 9 时，位置 1 会留下洞；把末尾 2 搬到 1，令 sparse[2]=1，再 m−，有效 dense 变成[4,2]。

旧 sparse[9]即使还等于 1 也没关系，因为 dense[1]现在是 2，双向校验失败，9 被判为不存在。若删的本来就是末项，直接 m−即可，交换自身的写入也可省掉。

将“查不到的旧值”也走一次：删除 9 前，查询 9 读 `sparse[9]=1`，1<3 且 `dense[1]=9`，两关都过；删除后 m=2，旧 `sparse[9]`仍为 1，第一关 1<2 还过，但第二关 `dense[1]=2!=9`，因此拒绝。若只检查位置在有效范围，就会把 9 错判为仍存在。后来再插入 9，写 `dense[2]=9,sparse[9]=2,m=3`，双向关系重新建立。

每次只改常数个格子，因此删除 O(1)。前提是先确认 x 确实存在；若拿一个无效 sparse 位置直接搬，会破坏其他记录。删后 dense 的顺序可能变化，这也是它与保持插入顺序容器的差别。

这三个接口可以放在同一个完整结构里验证。为避免读取未初始化位置，下面在创建时初始化 sparse；“常数时间清空”只指创建完成后把逻辑长度改成 0。

```
class SparseSet:
    def __init__(self, universe):
        if universe < 0:
            raise ValueError("值域大小非负")
        self.sparse = [0] * universe
        self.dense = [0] * universe
        self.size = 0

    def contains(self, x):
        return (0 <= x < len(self.sparse)
                and self.sparse[x] < self.size
                and self.dense[self.sparse[x]] == x)

    def add(self, x):
        if not 0 <= x < len(self.sparse):
            raise ValueError("元素超出值域")
        if self.contains(x):
            return False
        self.dense[self.size] = x
        self.sparse[x] = self.size
        self.size += 1
        return True

    def remove(self, x):
        if not self.contains(x):
            return False
```

```
        hole = self.sparse[x]
        last = self.dense[self.size - 1]
        self.dense[hole] = last
        self.sparse[last] = hole
        self.size -= 1
        return True

    def clear(self):
        self.size = 0

    def values(self):
        return self.dense[:self.size]
```

例如依次加入 4、9、2，删除 9，`values()`得到`[4,2]`，再加入 9 得到`[4,2,9]`。`clear()`不擦数组，但随后所有 `contains` 的第一关都失败；重新插入时会逐项建立新的双向对应。`remove` 删除末项时让它写回自身也合法，只是多做两次常数赋值。`values()`为了返回一个独立列表要复制 size 项，耗$O(size)$，这不影响内部单项查、增、删和逻辑清空的常数时间结论。

23.43 43 · Hopcroft 位图：免掉全域扫描，付出什么代价

扫描 n 个输入，每个用双向校验判断是否已存在，不存在就加入 dense，并更新 sparse。已有结构的逻辑清空 O(1)，每个输入处理 O(1)，因此这一轮去重 O(n)，不用把整个值域逐格清零或扫描。

但直接寻址的 sparse 仍随 U 占空间；相较一位表示一个值，它常需要一整个整数位置，空间明显增加。首次建立可合法读取的存储仍有初始化成本，若把 U 也计入本次从零开始的总成本，不能声称完全与 U 无关。

dense 保存的是处理顺序或交换后的顺序，去重结果不再天然按数值排序。若还要求有序输出，需要另排序，或者重新扫描值域。节约的是反复清空和稀疏访问的时间，代价包括空间、顺序与初始化条件。

23.44 本章练习与核对

1. 容量 8、规模 8 的表从头插 1 项，为什么“追加均摊 O(1)”不能推出这次头插 O(1)?
2. 对[2,2,5]查 2，用 C 版列出 lo、hi、mid，返回哪一个 2？若查 1 又怎样?
3. dense=[6,1,8,3]，删除 1 后，至少需要改哪几项？旧 sparse[1]为什么可以暂时不擦?

第一题：几何扩容只控制整条追加序列的迁移账；头插还需要移动原后缀，单次本来就是 Θ(n)，扩容也可能单次 Θ(n)。第二题：起初[0,3)，mid=1，2 不小于 2，lo=2；再 mid=2，2 小于 5，hi=2，返回 1，即最后一个 2。查 1 先 hi=1，再 hi=0，返回-1，不可解引用。第三题：把 3 写到 dense[1]，sparse[3]改成 1，m 由 4 减到 3；旧 sparse[1]=1，但 dense[1]不再是 1，所以校验拒绝。

24 PS03：节点身份、接线顺序与排序成本

教师原始习题单：第 03 次习题共 37 题。这里把节点身份、逻辑次序和数据值分开：移动节点改变链接；交换 data 改变值属于哪个节点；删除并新建还会改变对象生命期。它们有时得到相同的数值序列，却不一定对外部指针、稳定性和资源管理产生相同效果。

24.1 1 · 静态与动态：看逻辑结构，不能只看地址动没动

静态操作查询现有数据，动态操作改变集合内容或结构。游标链表把节点放在固定数组格子里，用 succ 下标说明后继；把链接从 2⟶5⟶7 改成 2⟶7⟶5，物理格子都没挪，逻辑顺序已经改变，因此仍是动态操作。

反过来，底层把整个存储搬到新地址，只要抽象集合与顺序保持，用户看到的逻辑内容未必变化。接口分析应先说明以哪一层为准，而不是把“动态”误解成“指针数值变了”。

可以拿三个卡片做演示：卡片固定贴在桌上，只改“下一张编号”，输出顺序也能改变。这就是列表逻辑链接与数组物理邻接的差别。

24.2 2 · 秩与位置：写成 p+100，也还是走了 100 步

秩是第几个，位置是某个节点句柄。向量从首地址加秩乘元素大小可直接寻址；列表一般只能从一个已知节点沿链接走。给列表重载 `p+k`，可以把走 k 步封装得好看，不能因此让它变成常数时间。

在长度 n 的链表上循环访问 `L[i]`，若每次从头找第 i 项，总访问量是 $0+1+\cdots+(n-1)=\Theta(n^2)$。改为保存当前节点 p，每轮 p=p->succ，总共只走 n 次，就成线性。

因此课程不建议频繁用模仿数组的重载语法。阅读代码时主动问“这一行内部要走几条边”，不要让一个加号把复杂度藏起来。

24.3 3 · 两节点距离未知方向：两边同时试，不必先找头

假设 p、q 是同一合法双向链表内节点。若已知 p 在 q 前面，从 p 沿 succ 走，数到 q 即可。距离 d 按边数计算，p=q 时返回 0，整体可写 O(1+d)。

方向未知时，可以从 p 同时向前、向后扩展，每轮各走至多一步，任一方向碰到 q 就结束；碰到哨兵的方向停止，另一边继续。最多各探索 d 步，所以仍 O(1+d)。也可从 p、q 分别向后探索，看谁先遇到对方。

例如链 `H↔A↔B↔C↔D↔E↔T`，p=C、q=A。第 0 轮两探针都在 C；第 1 轮向前到 B、向后到 D，都不是 A；第 2 轮向前到 A，返回距离 2，不再继续。若 q=E 则同样在第 2 轮从另一边找到。p 恰在首项时，向前探针第 1 轮碰到 H 就冻结，向后探针仍继续；不能让探针穿过哨兵去读无效 pred。每轮付至多 2 条边，找到后至多付 2d，所以是$O(d+1)$。

这需要同表与有效节点的前提。若 q 不属于该表，走到边界也找不到，必须返回错误，不能把哨兵当正常数据继续走。为了防御不可信句柄而先全表验证，可能另付 O(n)，与在已保证前提下的距离算法分开计费。

24.4 4 · 倒数第 k 项：双指针省的是重复定位，不是神奇少读所有节点

单链无长度时，先遍历数 n，再从头走 n-k 步，总边访问大致 $n+(n-k)$。双指针先让 fast 领先 k 步，再让两者同步；fast 到末尾时，slow 到倒数第 k 项。

以[10,20,30,40,50]找倒数第 2 项：fast 先越过 10、20，到 30；同步三轮：`(slow,fast)`从`(10,30)`依次变成`(20,40)`、`(30,50)`、`(40,tail)`，这时 slow 到 40。初始化究竟把 fast 放在“第 k 项”还是“越过 k 项”取决于尾哨兵约定，必须按所用表示画一遍。

对应代码假定空链的 first 就是 tail，fast 每领先一步都先检查是否已到 tail：

```
slow = first; fast = first
重复k次：若fast==tail则k过大；否则fast=fast.succ
当fast!=tail：slow=slow.succ；fast=fast.succ
返回slow
```

两指针之间始终相隔 k 条链接。fast 到尾后，slow 后面恰好还剩 k 个真实节点（包括 slow 自己），所以 slow 是倒数第 k 项。k=n 时 fast 初始化后已经到 tail，slow 不动，返回首项；k=1 时两者最后分别在末项与 tail。

两种方法都 O(n)，双指针仍可能合计走近 2n 条边；其优势是一次向前流程、避免重新从头定位，也可能有不同缓存行为。k≤0 或 k>n 时必须拒绝。实验要固定内存布局和 k 的比例，而不是只比较一条很短链的偶然耗时。

24.5 5 · 哨兵：空表也有能接线的邻居

双向链表保留 head、tail，空表为 head↔tail。把 x 插入两者之间，用与一般位置同样的四条连接：x.pred=head，x.succ=tail，head.succ=x，tail.pred=x。删除唯一元素 x，再把 head.succ 指回 tail、tail.pred 指回 head，就恢复空表。

这消除了“首节点没有前驱”“尾节点没有后继”的多套分支，但并不把所有 O(n)接口改成 O(1)。它主要统一边界并减少错误机会，精确运行时间仍应实测。

要测试空表插入、删唯一项、连续清空两次、首尾移动和复制空表。哨兵不保存普通元素，不参与查找比较，也不该被普通 remove 删掉。若遍历到 tail 还读取 data，哨兵并没有替你保证安全。

24.6 6 · 四条链接的顺序：先保存旧关系，再改图

把新节点 x 插在 p 与 q 之间，先记住 q=p->succ，再写 x 的两条边，最后改 p 与 q 反向指向 x。若一上来 `p->succ=x`，随后又用 `p->succ` 当旧 q，就把旧后缀丢掉了。

令原图 `P↔Q`，新节点 X 刚分配好，还不在表中。按下面顺序写，每行需要的旧信息都仍保留：

步	写入	这一步之后已确定的边
0	`q=p.succ`	局部变量 q 记住 Q
1	`x.pred=p`	X 向左认 P
2	`x.succ=q`	X 向右认 Q
3	`p.succ=x`	P 向右改认 X；Q 仍由 q 保存
4	`q.pred=x`	Q 向左改认 X，最终 P↔X↔Q

第 1 与第 2 行可互换，第 3 与第 4 行在前两行准备好后也可互换；第 0 行若拖到改写 `p.succ` 之后，就会把 q 错记成 X。中间图尚未双向一致不表示这段单线程、无回调代码必错，但在一致性恢复之前不能把半成品暴露给可能遍历的代码。

删除 x 时先保存 p=x->pred、q=x->succ，再令 p->succ=q、q->pred=p，最后释放 x。释放后再读 x->succ 违反生命周期要求。两条互不依赖的写入可以交换，读旧边之前覆盖旧边则不行。

检验顺序的办法是每写一行就画出所有箭头，标出下一行还会读取哪条旧边。局部最终图相同不够，执行途中也不能读取已经丢失或释放的对象。

24.7 7 · 把节点移动到自己前面：这次应当什么都不做

若 `this==s`，通常把 move 定义成无操作，直接返回。若没有检查，先把 this 摘链，再试图插到 s 之前，s 恰好就是刚摘下的节点，后继和前驱已不再代表原位置，容易接成自环或断链。

相邻情形也要检查：x 本来就在 s 前面，再移到 s 前面应当保持原样；从另一个表转移，则还要同步两表的 size 和所有权。若接口不允许跨表或自身移动，应把它写成前置条件并断言，而非含糊地期待调用方永远猜对。

真正移动 B 到 A 前时，先摘 B：保存 `oldLeft=A,oldRight=T`，写 `A.succ=T,T.pred=A`，此时表为 `H↔A↔T`，B 独立；再保存目标 A 的前驱 H，写 `B.pred=H,B.succ=A,H.succ=B,A.pred=B`，得到 `H↔B↔A↔T`。必须先判断 B 不是目标自己。对 A 移到 B 前的相邻无操作，也可直接早退；若仍用通用摘接，要在摘掉后重新取得目标的当前前驱，不能使用已过时的相邻关系。

一个很小的测试表 head↔A↔B↔tail 就够，分别移动 A 到 A 前、A 到 B 前、B 到 A 前，逐次核对正向和反向遍历是否一致。

24.8 8 · insertFirst 复制：反着读，才能正着留下

原表 A→B→C，从头顺序读并每次 insertFirst，目标变化为空→A→B,A→C,B,A，结果反序。若从尾向前读，先 C、再 B、再 A，目标最终 A,B,C，顺序保持。

双向链表能直接沿 pred 反向遍历。单链若每次重新找前驱，会退化；可先用辅助栈保存节点或采用尾插，别为了使用 insertFirst 而制造无谓二次扫描。

独立复制 n 个节点需要 Θ(n)时间、Θ(n)新存储。即使额外工作空间是常数，新表本身的 n 个对象也必须分配；回答空间复杂度时说明是否把输出存储排除。

24.9 9 · 单链析构：先记住下一家，再拆当前这家

```
Node* p = first;
while (p != tail) {
    Node* next = p->succ;
    delete p;
    p = next;
}
```

单链没有 pred 不妨碍这个流程，因为销毁时只向前走。每个节点读一次后继、释放一次，时间 Θ(n)。若 tail 也是动态分配的哨兵，循环后再按所有权规则释放它；不能当普通节点提前删。

错误写法是先 delete p，再读 p->succ；另一个低效写法是每轮找最后节点并删除，每次都从头走，成本 n+(n-1)+…，变成 $\Theta(n^2)$。保存一个 next 就解决的问题，不需要用重新遍历补救。

24.10 10 · 链表无序去重：删一个节点，不必搬整个后缀

维护已经处理的无重复前缀。当前节点 p 在此前缀里找到同值 q，可以删 q 保留较新的 p；若改删 p，则必须先保存 p 的后继，再释放，随后从保存的节点继续，不能再访问 p。

全相等输入，每轮很快找到相等项，且删除只改常数条链接，最好 Θ(n)。全互异输入，第 k 项要查 k 个前缀节点，累计 $\Theta(n^2)$。课程原 Vector 版本即使比较少，也可能不断移动后缀，所以“同样的去重思路”在不同存储上有不同最好情况。

原题还问第一次 find 查空前缀是否安全。合法的计数式查找先检查“还需比较的节点数 r 是否大于 0”，r=0 时直接返回前边界，根本不读节点 data。若接口约定先取 pred 再循环，返回 h 仍可合法，但绝不能把 h.data 当普通元素判等。

对 `H↔A1↔B1↔A2↔T` 保留最后一份：A1 查空前缀，留下；B1 查 A1，不等，留下；A2 查 B1 再查 A1，命中后摘掉 A1，得到 `H↔B1↔A2↔T`。若改成删当前 A2，必须先保存 next=T，摘并释放 A2 后令 p=T，不能继续用 p 取 succ。两个版本输出不同但都可实现，先声明保留哪一份。

若改维护无重后缀，可从尾向前扫描，在后缀找重复；双向表能够对称实现。空表、单项、全等、首尾重复都要测。用哈希额外索引可改变期望比较成本，但已经增加了另一数据结构，不能不计其空间与假设。

24.11 11 · 遍历：访问器有没有资格修改链接

求和、计数、打印、验证有序性都可用访问器。计算平均值时让访问器保存 sum 与 count，遍历完再除；空表 count=0 时不能直接除 0。

若访问器仅修改 data，普通遍历一般还能沿 succ 继续；若它删除当前节点，遍历框架随后读取当前 succ 可能已经访问释放内存。需要安全删除协议：预存 next，允许回调返回新的位置，或明确禁止改链接。

因此“都能用遍历实现”不表示任何回调都能塞进同一个接口。先读参数是值、引用还是节点位置，再判断它能读写什么。

24.12 12 · 用 traverse 写 find/search：接口可能让你无法提前停

find 访问每个元素，若第一次相等就保存命中节点；search 在有序表里保存最后一个不大于 e 的节点。后者遇到第一项大于 e，本来就可以结束。

但若 traverse 的回调只返回 void、框架必定扫完整表，访问器虽然可以不再做比较，仍然发生后续指针移动。要真正提前停，可设计返回 bool 的回调或单独提供短路遍历。

例如[2,5,9,11]查 3，前驱是 2，看到 5 即可确认；通用遍历若继续走 9、11，只是功能正确而未达到同样的常数优化。接口能力和算法成本需要一起检查。

24.13 13 · 有序去重：不必为了少一行特判去冒险读哨兵

少于两项，没有一对真实相邻项可比较。先判 size<2 可以直接返回，避免构造第二节点位置后读到 tail。也可以令循环显式要求当前 p 和其后继 q 都是真实节点。相等时删除其中一项，不等时推进；若删除 q，p 不动并重新取新后继，才能处理[1,1,1]连着三个相等值。

对 `H↔1A↔1B↔1C↔2D↔T` 保留首项时，p 先在 1A，q 在 1B，相等删 q；p 仍在 1A，重新令 q=p.succ 即 1C，再删；下一次 q 变 2D，不等，才把 p 推进到 2D。若每次删除后也推进 p，可能跳过 1C，留下两个 1。停止条件应先保证 q 不是 T，再读 q.data。最终链接为 `H↔1A↔2D↔T`，真实节点 3 次相邻判定，删除 2 个节点。

测试空、单项、两项同值、两项不同、整表全等、首尾各一组重复。无论哪种改写，证明都应是“已经越过的前缀无重复，且保持指定的保留规则”，而非单纯说代码行数少了。

24.14 14 · search 返回前驱：返回了一个位置，不等于找到了相等值

在有序区间(h,t)里，search 找最后一个不大于 e 的项。若 e 比所有项小，结果是头边界 h；插在它后面，就让 e 成为首项，顺序仍正确。

例如[2,5,9]查 7，返回 5 的位置。接着在其后插 7 得到[2,5,7,9]，这就是前驱接口的用途。若只想判断 7 是否存在，需要先确认返回值不是 h，再用一致的等价关系比较返回元素与 7。

查 2 返回 2、查 1 返回 h、查 12 返回 9，三个例子可以把“精确匹配”“前驱”“插入位置”区分清楚。把任意非空指针当命中会把后两种误判。

24.15 15 · 有序单链搜索：少比较，也未必少走路

只有相邻链接、无额外索引时，要到中间节点就得先经过前面的链接。即使关键字有序，对手也能把答案或最后确认位置放在很远处，最坏需要 Ω(n)次边访问。

在链表上做二分，比较次数可能降成 O(log n)，但找各次中点的总步数仍可为 O(n)，不会像数组那样获得 O(log n)总时间。若每次又从头定位中点，甚至可能更差。

取 8 项，第一次中点在第 4 项，需要从首走 3 条边。若目标在后半，下一次在余下 4 项中定位中间，再走约 2 条；再下一次约 1 条。总步数是$4+2+1=7$量级，虽然只比较 3 次，仍与 n 同阶。即使刻意把每层的走路量做得最省，几何级数$n/2+n/4+\cdots$也有$\Theta(n)$，省比较并没有省掉第一次抵达中点的路程。

增加跳表层、树索引或按秩定位结构能改进，但这已改变数据结构和维护成本。题目问原链表能不能做到，答案必须带着这个访问模型，不能用另一个结构的能力冒充。

24.16 16 · 选择排序的交换、平移和删建，是三种不同的动作

课程《03.列表》PDF 第 34 页的主代码是 `swap(getMax(h,t)->data,x->data)`，交换的是两个节点中的 data；同页注释另给 `getMax(h,t)->movePred(t)`，移动的是原节点。再加上本题提出的 remove 后 insert，一共三种动作。它们都能把最大值放到后缀前端，但保留了什么，并不相同。

先固定真实节点的身份为 A、B、C、D，它们当前装的值分别为 4、1、3、2。外部还握着指向 A 的句柄 p。只看第一轮，将最大值 4 放到末尾：

```
初态：H ↔ A:4 ↔ B:1 ↔ C:3 ↔ D:2 ↔ T，p指A。
```

交换 data：A 与 D 交换内容，链接完全没变，得到

```
H ↔ A:2 ↔ B:1 ↔ C:3 ↔ D:4 ↔ T。
p仍指A，但通过p读到的值已由4变为2。
```

移动同一节点：先从 H 与 B 之间摘下 A，再把 A 接到 D 与 T 之间，得到

```
摘下后：H ↔ B:1 ↔ C:3 ↔ D:2 ↔ T，A单独保存。
接上后：H ↔ B:1 ↔ C:3 ↔ D:2 ↔ A:4 ↔ T。
p仍指A，读到的仍是4；A的邻居改变了。
```

删除后新建：若 remove 返回值 4 并释放 A，insert 再建节点 E 保存 4，结果为

```
H ↔ B:1 ↔ C:3 ↔ D:2 ↔ E:4 ↔ T。
旧A已结束生命期，p失效；不能因为新节点也装4就继续使用p。
```

具体的分配器甚至可能复用相同的地址数值，也不意味着旧句柄因此自动继续合法；接口若承诺节点身份，必须按对象生命期处理。反之，若某个名为 remove 的接口只摘链而不释放，返回原节点，那么它与 move 接近，不能仅凭函数名下结论。

三种方式都使最大值 4 就位，但未排序前缀并不相同。上面交换法得到`[2,1,3,4]`，移动和删建得到`[1,3,2,4]`。后一轮从哪个状态继续，会影响原地次数与稳定性，这正是第 20、21 题不能混用逐输入环数等式的原因。

交换 data 每轮只做常数次值移动，不申请新节点；对便宜、可交换的数据很直接。移动原节点每轮也只改常数条链接，能保留数据身份，并可通过选择最后一个最大值保持稳定。删建通常多付分配、释放、构造或复制，还可能破坏外部句柄，所以课程更倾向于前两种。若数据是大对象或不可复制对象，应另外比较值交换与接链成本，而不是断言交换对所有类型都便宜。

24.17 17 · 最后一个最大值：能帮助稳定，但不能救任意交换

从后向前找最大，初始化为最右项，只在遇到严格更大值时更新，就自然保留最后一个最大值。相等项不会更新；某个值即使多次出现，也最多在它第一次严格超过现有最大值时触发一次记录更新。

若从前向后扫描，为了仍取最后一个最大值，要在大于或等于时更新。全相等长度 r 就会更新到每一项，最多 r-1 次（不计初始化）。扫描方向与相等处理必须配套。

把最大记录更新过程单独走一次，前缀`[4A,7B,7C,2D,7E]`从右扫描：初始化 m=7E；遇 2D 不换，遇 7C 相等不换，遇 7B 相等不换，遇 4A 更小不换，最终保留 7E。若从左扫描并用≥更新，则初始化 4A，依次更新到 7B、7C、7E，三次；同一个最大 key 造成多次更新。全相等 r 项时达到 r−1 次更新。

“每组相等元素最多使 m 更新一次”依赖的是从右向左、严格大于才更新：某组的最右成员若第一次刷新了最大值，之后同值成员都不再严格大；若它没能刷新，后面的最大值只会更大，这一组以后也不可能刷新。

选最后最大再把整个节点稳定地摘到后缀，可保留等值次序；但任意交换 data 仍可能破坏稳定。对[(2,A),(1,B),(2,C),(1,D)]，先将 2C 换到末尾，再将 2A 换到剩余末尾，可使 1D 跑到 1B 前面。因此要审查整套排序，不把 getMax 的一个好性质扩展成整体保证。

24.18 18 · 向量选择排序：二次比较，线性移动确实能同时成立

长度 r 的无序前缀找最大需要 r-1 次比较，r 从 n 降到 2，合计 $n(n-1)/2$。每轮最多做一次交换，而交换由常数次元素赋值组成，所以元素移动 O(n)。

例如[3,1,4,2]：先选 4 与末项 2 交换得[3,1,2,4]；再选 3 与 2 交换得[2,1,3,4]；最后换 2 与 1，得到有序。比较扫描很多，真正交换仅 3 次。

它适合需要控制移动次数的模型，但若比较也很贵，二次比较仍是明显缺点。普通交换版不稳定；若改成数组区间平移来保序，可能引入二次移动，不能同时保留原来的全部优点。

24.19 19 · 交换与轮换：先把排列画成箭头

将每个位置连向其元素的最终位置，整个排列分成若干不相交的环。已经就位的元素是长度 1 的环。交换两个位置，只改变这两个位置发出的归属关系，其余箭头保持。

若两位置在同一环上，交换会把环切成两个；若在不同环上，交换会把两环接成一个。可以用[2,3,1]：原箭头 1⟶2⟶3⟶1，交换位置 1 与 3 后成为[1,3,2]，拆成固定点 1 与环(2,3)。

再补“不同环”的情况。排列[2,1,4,3]有两个环(1 2)和(3 4)。交换位置 2 与 3，数组变[2,4,1,3]，箭头依次为 1→2→4→3→1，两个环合成一个。把这次交换再做一次，当然又把一个环拆回两个。由此可见一次任意交换不会保持环数不变，而是准确改变 1；下界论证才能按每次最多增 1 收费。

因此一次交换让环数改变 1，奇偶性也改变。环不是“数组里相邻的一段”，两节点在环中相邻与在物理序列中相邻完全不同。

24.20 20 · 交换选择排序：为什么有效交换数是 n 减环数

无序前缀末位置 x 应该放当前最大值 m。因为 m 的目标就是 x，m 所在位置的箭头直接指向 x 所在位置，所以两者属于同一环且沿一个方向紧邻。交换后 m 就位成为固定点，原环少一个成员。

若初始有 c 个环，每个长度 l 的环需要 l-1 次这种有效交换，合计 n-c。课程若连最后长度 1 的步骤也做，每个环最终留下的那个固定点对应一次原地交换，所以无效交换共 c 次；若省去最后一步，计数相应少 1，必须统一约定。

在[2,3,1,5,4]上可以把“每个有效交换拆一个固定点”全部看见：

```
初始环：(1 2 3)、(4 5)，c=2。
r=5：最大5在位置4，交换4与5 -> [2,3,1,4,5]；环(4 5)拆成(4)(5)。
r=4：最大4已在位置4，原地动作。
r=3：最大3在位置2，交换2与3 -> [2,1,3,4,5]；三环拆出(3)。
r=2：最大2在位置1，交换1与2 -> [1,2,3,4,5]；拆出(1)(2)。
r=1：原地动作。
```

有效交换 3 次，等于$n-c=5-2$；含 r=1 的原地动作 2 次，等于初始环数。若课程循环在 r=2 之后结束，就不执行最后一次原地动作，此时原地次数为 c−1。计数前先说明最后一步是否执行。

互异均匀随机排列中，当前 r 项的最大值等概率位于任一位置，在末端的概率 1/r。对 r 从 1 到 n 求和，原地交换期望为 $H_n=\sum_{r=1}^{n}1/r$。由逐输入“原地次数=环数”，得到期望环数也是 H_n，因而 Θ(log n)。

这不是假设每次事件独立。期望可相加，不需要独立；但最大位置均匀来自输入排列的随机模型，不能无条件套到人为构造数据。

24.21 21 · 换成平移：逐输入的环数等式已经失效

取[3,1,2]，原排列是一个三环。平移最大 3 到末尾后变成[1,2,3]，随后长度 2 与长度 1 的最大项都已经在末尾，发生两次原地动作。初始只有 1 个环，因此“原地次数恰好等于初始环数”不再成立。

仍在[3,1,2]上分别画两种状态：交换法先把 3 与 2 互换得到[2,1,3]，还要交换 2 与 1；平移法摘掉 3，把后面的 1、2 向前接起来，再把 3 放到末尾，立即得到[1,2,3]。二者剩余排列不同，所以不能沿用同一条输入的环数计数。

为什么平均数还能相同？固定 r−1 个较小元素的一种相对次序，把最大元素插入 r 个不同位置，就得到 r 个等可能的原排列；删掉最大项都回到同一剩余次序。所以每一种剩余次序恰好有同样多的来源，均匀性保留。交换法也能用“指定最大原位置后逆向交换”做相同的来源计数。

因此，在这套均匀随机模型下，每轮删去最大项以后，下一轮的剩余次序仍均匀；当前 r 项最大值处在末端仍以 1/r 概率发生。原地次数期望依旧 H_n（若从 r=2 计则 H_n-1）。这个结论来自来源计数，而不是把“看起来没有偏向”当成证明。

两个方法能有相同的平均数，却对同一个具体排列给出不同的次数。证明平均结论应重新基于剩余相对顺序的均匀性，不能拿已经失效的逐输入等式继续推。

24.22 22 · 在线与输入敏感：抓一张牌就插进去

插入排序看见新牌，就在已排好前缀里找位置并插入，任何时刻已收到的牌都有序，所以适合“还会继续来牌”的在线维护。用平衡搜索树接收新键也能在线维护顺序，但结构和输出方式不同。

输入敏感指成本还取决于某种输入难度。插入排序可按逆序对数 I 写成 O(n+I)的常见数组移动界；已经有序时线性，完全逆序时二次。带提前终止的冒泡也对某些接近有序输入更省。

选择排序每轮仍要扫描剩余集合，比较数不因输入已序而减少。它的优势是总交换 O(n)。因此不能只问“哪个排序更好”，应说清比较、移动、在线响应和稳定性分别要求什么。

24.23 23 · 数组插入：二分找到了位置，格子还是要腾开

保存当前值 x，把比 x 大的前缀元素依次右移，最后把 x 填入空位。例[2,4,7,3]：先存 3，7 右移，4 右移，再把 3 写到 2 之后，得到[2,3,4,7]。

这一轮的完整数组状态如下，空洞只在逻辑上标记，x 已经存到临时变量：

```
x=3，洞在3：[2,4,7,_]
7>3，7右移：[2,4,_,7]，洞移到2
4>3，4右移：[2,_,4,7]，洞移到1
2<=3，停止：[2,3,4,7]
```

循环不变量是：洞右侧到当前轮末尾已经是应当位于 x 之后的元素，原前缀尚未被覆盖；每次把洞左边较大项搬到洞里，洞向左走 1。保留 x 直到最后填洞，避免把它在第一次右移时丢掉。

逆序输入时，第 i 项要越过前 i 项，移动数是 1+2+...+(n-1)，所以 Θ(n²)。不能像链表那样只改常数条边，因为数组的逻辑连续性正依靠连续格子，必须给插入位置腾空间。

二分能把查位置的比较降到 O(log n)，却不能消除 Θ(n)后缀移动。链表则相反：按秩查位置不便宜，但已知位置后接入节点可以常数改链。每种结构省的工作不同。

24.24 24 · move 保留对象，remove 加 insert 可能重建对象

节点转移先摘下现有节点 x，再接到目标位置；x 的 data 对象可以完全不复制，外部持有的节点地址也保持。remove 若销毁节点再返回值，insert 又新建节点，则增加分配、构造和析构。

对于大字符串、不可复制对象或有身份的记录，这些差别比几条指针赋值大。对于只有几个字节的简单对象，差别可能主要是分配器常数，也应通过实际实现确认。

例如 H↔2A↔4B↔7C↔3D↔8E↔T，待插节点 x=3D，原后继 next=8E。已找插入前驱 2A：先摘 3D 使 7C 连到 8E；再把 3D 接到 2A 与 4B 之间。得到 H↔2A↔3D↔4B↔7C↔8E↔T。下一轮必须处理保存的 8E，若沿移动后的 3D.succ 走，会回到 4B，把已经排好的节点再处理一次。

移动后扫描指针必须指向原后继，而不是沿 x 的新后继回到已经排好的前缀。先保存 next，再移动 x，是避免重复处理和无限循环的关键。

24.25 25 · 插入排序：原地不动几次，比整体有序容易得多

互异均匀随机排列中，完全有序只有 1 种，概率为 1/n!。但某次插入不移动，只要求新项是当前前缀最大值；在前 k 项中，每个位置等概率容纳最大值，所以该事件概率 1/k。

若排序从第 2 项开始，原地插入的期望次数为 $\sum_{k=2}^{n} 1/k = H_n - 1$；若把首项初始化也计一次，就是 H_n。n=4 时，期望为 1/2+1/3+1/4=13/12，不需要接近 1/n!那么罕见。

平均逆序对为 n(n-1)/4，标准插入排序每次右移消除一个相应逆序，所以平均仍 Θ(n²)。偶尔许多项无需移动，与总体二次工作并不矛盾。

24.26 26 · 为什么链表插入排序不直接二分

有序只说明比较后能排除一半候选，不保证可以常数时间跳到中点。链表从当前边界走到中点已经需要一段指针移动，所有层累计仍可能线性；每次从头重新找位置还会增加重复工作。

要真正像数组那样实现 O(log n)定位，需要高效随机按秩访问，或增加可跳跃的索引。同时插入后还要高效维护这个索引，否则定位省下的时间会在更新时付回。

即便对数组采用二分插入，移动仍可能二次；对链表普通顺序找位置虽然比较多，却能常数接链。先把“比较次数”“走链接次数”“移动数据次数”三张账分开，再决定该优化哪一个。

24.27 27 · 逆序对：每一对位置各自交一张账单

互异元素中，逆序是 i<j 却 a[i]>a[j]。共有 n(n-1)/2 对位置，完全递减时全部逆序，这是最大值。

随机排列中任意一对大小方向对称，成为逆序的概率 1/2。令每对的指示变量为 1 或 0，期望相加得 $E[I] = n(n-1)/4$，无需假设所有对之间独立。

例如[3,1,2]逆序为(3,1)、(3,2)，共 2；相邻交换 3 和 1 后剩(3,2)，再交换便有序。普通冒泡每次只消一个逆序；插入右移也能用这些对逐项收费，所以随机平均 Θ(n²)。

24.28 28 · 冒泡只跑 n 趟：先查清最后一趟是不是确认趟

原题长度为 2n，采用课件第 75 页提前终止版。该版在最后一次有交换之后，还要执行一趟“完全没交换”的确认，才让外循环停止。下面把这趟也计入 n，因此有交换的最多左移深度为 $d = n - 1$。

一个元素每趟最多向左移动一格。若需要 d 趟有交换，至少存在一个元素要越过 d 个较大前项，所以逆序至少 d。取前 d+1 项为[2,3,...,d+1,1]，其他值递增，恰有 d 个逆序，并需要 d 次有交换加一次确认。

每个有交换的第 i 趟至多比较并交换 2n-i 对，所以逆序总数至多

$$\sum_{i=1}^{d}(2n-i) = 2nd - \frac{d(d+1)}{2}.$$

取前 d 项为最大的 d 个数、递减排列，后面其余数递增：每趟都能把一个大项推到末端，达到这个上界。把较小的$n = 3, d = 2$走全，最少逆序构造为`[2,3,1,4,5,6]`：

```
第1趟：[2,1,3,4,5,6]，1只左移1格，有交换。
第2趟：[1,2,3,4,5,6]，1再左移1格，有交换。
第3趟：不改变数组，无交换，确认结束。
```

最多逆序构造为`[6,5,1,2,3,4]`：

```
第1趟：[5,1,2,3,4,6]，6一路右移，5次交换。
第2趟：[1,2,3,4,5,6]，5一路右移，4次交换。
第3趟：无交换。
```

总逆序分别 2 和 9。最大构造中，前 d 个大元素各自与后面 m−d 个小元素逆序，另有大元素彼此的$d(d-1)/2$个逆序，总数为$d(m-d) + d(d-1)/2 = md - d(d+1)/2$。这既核对了逐趟上界，也证明构造确实达到上界。

n=15 时，d=14，最少 14、最多 315。最少例为[2,3,...,15,1,16,17,...,30]；最多例为[30,29,...,17,1,2,...,16]。若采用另一计数习惯，把“n 趟”仅指有交换的扫描，便应取 d=n，得到最少 15、最多 330。不要把两种趟数口径混写；课件这份源码可以实际打印 sorted 来确认。

24.29 29 · 交换远处逆序：能多消一些，但距离不是充分条件

互异数 a>b 位于 i<j。交换后，它们这对逆序消失；中间每个值 x 若满足 b<x<a，又会同时消掉两对逆序。因此逆序减少量为 $1 + 2k$，k 是中间落在两值之间的元素数。

距离远意味着可能跨过更多合适的 x，却不保证一定多消。例如中间值全大于 a 或全小于 b，仍可能只减少 1。若有重复值，还要单独计与端点相等的情况，不能照搬奇数公式。

把三种中间元素各算一次，能看到公式的来源。端点$a > b$交换后：若$b < x < a$，原来有 a>x 和 x>b 两对，交换后都消失，贡献 2；若$x < b$，端点与 x 的逆序只是从 a 转到 b，净减少 0；若$x > a$同理为 0。端点本身的 a>b 再减少 1。若允许重复，$x = a$或$x = b$会各净减少 1，所以完整式为

$$\Delta I = 1 + 2\#\{x : b < x < a\} + \#\{x : x = a\} + \#\{x : x = b\},$$

其中只数两端位置之间的元素。交换端点外的某个元素与这两个端点的相对比较，总数不变，因此不用额外计入。

任意交换模型下，环分解给出 n-c 次就能排序，至多 n-1。选择排序保证 O(n)次交换，堆排序和快排一般不保证这个线性界；冒泡只用相邻交换，完全逆序需 n(n-1)/2 次。允许交换的位置不同，最少次数也不同。

24.30 30 · 不等长二路归并：仍以两边总长度计上界

两有序段长 m、n。比较两头，输出较小者并推进对应一边；每次至少消耗一项，直到一边空。关键字比较至多 m+n-1，输出和访问总量 O(m+n)，并不要求 m 与 n 接近。

例如[1,100]与[2,3,4,5,6]，先出 1，连续出 2 到 6，最后接 100，长的一侧并没有被反复从头扫描。若链表剩余尾段可整段接上，某些输入的真实工作还可更少。

线性上界仍成立；但不能因此说所有输入都必须逐节点复制尾段。要按接口是否维护 size、尾指针、统计信息，判断一次接链之外还有没有必须访问的工作。

24.31 31 · 节点转移归并：保存 next，稳定地从左边先取

遇到右段 x 小于左段 y，把 x 摘到 y 前；先存 x 在原段的后继，再改链接，后续扫描从保存位置继续。若直接用移动后的 x->succ，它已经指向 y，可能反复处理错误位置。

相等时让左段先出，保留来自原左半的记录在前，才能稳定。对左[2A,4]、右[2B,3]，输出应为 2A、2B、3、4。

将例子画成带身份的两条链：

```
L: H_L ↔ 2A ↔ 4C ↔ T_L
R: H_R ↔ 2B ↔ 3D ↔ T_R
```

比较 2A 与 2B，相等先保留左 2A，左指针到 4C；比较 4C 与 2B，先存右后继 3D，摘 2B 并接到 4C 前，左链成 `2A↔2B↔4C`；再比较 4C 与 3D，保存右后继 T_R，摘 3D 接到 4C 前，得 `2A↔2B↔3D↔4C`。右链为空，结束。A 仍在 B 之前，且 A、B、C、D 的节点地址都没变。若相等先从右取，就会变成 2B 在 2A 前，稳定性在第一次比较已经破坏。

remove 加新 insert 也能造出正确值序，但会分配、复制并改变身份。原节点转移的优势是复用资源；如果还有外部节点句柄，语义差别尤其需要写清。

24.32 32 · 向量与列表归并：同为 nlog n，代价落在不同地方

向量连续，扫描常有较好的局部性，按秩分半方便；常见稳定归并需要 O(n)元素缓冲，并多次复制或移动值。列表可以拆链、重接原节点，避免大对象复制，额外元素存储可以很少。

列表节点却可能散在内存，走 succ 会发生缓存失效，节点分配本身也占额外指针空间。找中点若每层扫描，仍能并入每层线性成本，整体 Θ(nlog n)，但常数不同。

比较时使用同一元素类型、相同数据与实际内存布局。只有少量大对象和很多小整数两种场景，得到不同胜负很正常；不能用渐近符号决定所有实测结论。

24.33 33 · 固定 15/16 与每次 n−1：看似都偏斜，结论却不同

按 2/3 与 1/3、15/16 与 1/16，或任意固定 $\lambda : (1-\lambda)$ 切分，两边仍以固定比例缩小，每层总长度 O(n)、深度 O(log n)，整体 Θ(nlog n)。

15/16 那条长路径深约 $\log_{16/15} n$，常数比对半大，却仍是对数。若改成 n-1 与 1，路径 n 层，递推为 $T(n) = T(n-1) + \Theta(n)$，累计二次。

把单项不断与已排序的大段合并，效果接近逐项插入排序。是否从前还是后剥离，决定具体移动方向；判断渐近时关键是每次只减少 1，而非减少一个固定比例。

24.34 34 · 归并数逆序：右项先出时，整批左项一起记

左[2,5,7]、右[1,4,6]。先出 1，左边剩 3 项都比 1 大，增加 3；出 2 不增；出 4 时左边剩 5、7，增加 2；出 5 不增；出 6 时剩 7，增加 1，总跨段逆序 6。

递归先数两半内部逆序，再加跨段逆序，每层只扫线性，总 O(nlog n)。相等时先取左项而不加逆序，因为逆序要求严格大于。总数最大约 $n^2/2$，累加器要有足够位宽。

把计数与输出状态同时记下：

左尚未输出	右尚未输出	本次输出	新增逆序	累计跨段逆序
2,5,7	1,4,6	1	3	3
2,5,7	4,6	2	0	3
5,7	4,6	4	2	5
5,7	6	5	0	5

左尚未输出	右尚未输出	本次输出	新增逆序	累计跨段逆序
7	6	6	1	6
7	空	7	0	6

“左边剩多少就加多少”成立，因为左半已经有序：右头小于左头，便小于左半所有剩余项。左头先出时，它不大于右头，也就不大于右半所有剩余项，不新添跨段逆序。两半内部逆序在递归中已各算一次，因此没有重计。

下面把“左半还剩多少”落实为下标，返回排序副本和原输入的逆序数。递归返回的每一半已经有序；`i` 是左半已输出的项数，所以右项先出时应增加 `len(left)-i`。

```
def sorted_with_inversions(a):
    if len(a) < 2:
        return list(a), 0
    mid = len(a) // 2
    left, count_left = sorted_with_inversions(a[:mid])
    right, count_right = sorted_with_inversions(a[mid:])
    merged, i, j = [], 0, 0
    count = count_left + count_right
    while i < len(left) and j < len(right):
        if right[j] < left[i]:
            merged.append(right[j])
            count += len(left) - i
            j += 1
        else:
            merged.append(left[i])
            i += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged, count
```

对`[2,5,7,1,4,6]`，左半和右半内部都没有逆序，所以返回数正是表中的 6。对`[2,1,2,1]`，两个子问题分别贡献 1，顶层只有右半的 1 越过左半尚余的 2，再贡献 1，总数 3；相等的 1 和 1、2 和 2 从左边先出，不计逆序。空数组返回空与 0，单项也为 0。

这份Python演示用了切片，每层切片总量仍为线性，因此总时间仍$O(n \log n)$；顺序执行时，当前调用保留的输入切片、两个有序结果与归并结果沿一条递归路径形成几何规模，总辅助存储为$O(n)$，另有$O(\log n)$调用栈。若要减少分配常数，可以像第 23 章第 36 题那样预分配缓冲并传边界，计数规则不变。

还可压缩秩后用树状数组：从右向左读，每次查询已有多少个更小秩，再插入当前秩，同样 O(nlog n)。若小整数值域允许更专门的操作，复杂度可变化；在一般比较模型里不应无条件宣称线性。

24.35 35 · 游标链表：数组下标充当地址

节点放进数组，每格保存 data、pred、succ，succ 是下一个节点的下标。另用一条 free 链管理空闲格。申请时从 free 头摘一格，释放时把该格接回 free 头。

例如有效链 2⟶5⟶1，物理下标既不递增也不连续。删除 5 只改 2.succ=1、1.pred=2，再回收 5；不需要把数组第 6 格以后整体前移。

把池的有效链接和空闲链接分开保存。例如有效 2→5→1→空，free 为 3→4→6→空。插入新项 X 到 2 之后：从 free 头取 3，free 改成 4；令格 3 的数据为 X，succ=5、pred=2，再改 2.succ=3、5.pred=3。删除 5：先让 3.succ=1、1.pred=3，然后把格 5 接到 free 头，即 5.succ=4、free=5。一个格只属于有效链或空闲链之一，这就是分配器必须维护的不变量。

它适合固定池、需要序列化引用或不便动态分配的场景。回收后的下标若被外部继续使用，可能指向后来新建的另一个对象；可以加代际号或限制句柄生命期。游标替代地址，不会自动解决悬空身份问题。

24.36 36 · Java：垃圾回收不会帮你接正确的链

interface 规定可调用能力，implements 表示类提供这些方法。Java 变量常持有对象引用，不像 C++某些写法那样直接复制整个节点对象；模板与泛型的具体机制也应按语言规则区分。

摘掉节点后，若没有任何可达引用，垃圾回收最终可释放它；若你仍无意保留一条引用，它可能继续存活。GC 不会自动把前驱接到正确后继，也不会修复环、错误 size 或逻辑重复。

对照课程版本时，把插入后前后遍历结果、size、外部引用指向对象身份逐项比较。运行测试只能报告实际运行的平台与版本，不能把“语言有 GC”当作算法正确性的证明。

24.37 37 · Python list：名字叫 list，底层却可能是数组

Python 内置 list 是动态数组，尾部追加或弹出通常有摊还常数表现，头部插删需要移动后缀；课程自己实现的链式 List 是另一类型。先确认 reverse.py 操作的是哪一种对象。

双指针原地反转只交换首尾，再向中间收缩，时间 O(n)、额外空间 O(1)。若递归版本每层切出剩余子串或子列表，再与首项拼接，每层复制长度 n、n-1……，就会出现二次时间和大量临时对象。

对`[A,B,C,D]`，双指针版只做`(0,3)`交换得到`[D,B,C,A]`，再做`(1,2)`得到`[D,C,B,A]`，两次交换。若写成`reverse(a[1:]) + a[:1]`，第一层先复制 BCD，下一层复制 CD，再下一层复制 D；返回时又拼接长度 2、3、4 的列表。切片与拼接并不因为写在一行里就成为常数，累计复制数量形成$1+2+\cdots+n$，所以会是二次量级。

同样是“递归反转”，若仅传左右下标、不切片，时间仍可线性，只增加递归栈。解释性能差距时要读具体切片、拼接和访问成本，不能把原因缩成“Python 慢”或“递归一定慢”。

24.38 本章练习与详细答案

1. 对[2,3,1]画出置换环；任选一次把元素放回最终位置的交换，环数怎样改变？
2. 交换[5,2,4,1,3]首尾的逆序 5 与 3，会消掉几个逆序？先用中间值判断，再枚举核对。
3. 为长度 6、含最后确认趟共 3 趟的提前终止冒泡，各构造最少和最多逆序的输入。

第一题：位置 1→2→3→1，一个三环。交换位置 1 与 3 得[1,3,2]，拆成固定点 1 和环(2,3)，环数增加 1。

第二题：端点之间仅 4 严格位于 3 与 5 之间，故减少 1+2=3；原逆序为 7 个，交换成[3,2,4,1,5]后剩 4 个。

第三题：d=2，最少[2,3,1,4,5,6]含 2 逆序；最多[6,5,1,2,3,4]含 9 逆序，分别两趟交换后再一趟确认。通过逐趟打印数组，可检查自己有没有漏数确认趟。

25 PS04：先把栈里的东西写出来，再谈递归、表达式和单调结构

教师原始习题单：第 04 次习题共 35 题。表达式题采用课件第 41–42 页的实际优先关系表来解释原程序；特别提醒：原表的 ^ 对 ^ 是先归约，因此原程序是左结合。数学惯例通常右结合，修改版要明确说明，不能把两者混成同一个答案。

25.1 1 · 向量做栈：把栈顶搬到头部，哪些成本跟着变

原 Vector 尾端作为 top，push 就是尾插，pop 就是尾删；没有扩容时只改末项，几何扩容下 push 摊还 O(1)，top 读末项 O(1)。

若把首端作为 top，每次 push 必须把旧 n 项右移一格，每次 pop 把剩余项左移，单次 Θ(n)。连续压入 n 项，移动量 0+1+…+(n-1)，总 $\Theta(n^2)$。top 仍是读首项，仍 O(1)。

栈的“后进先出”规则没有变，改变的是底层接口代价。实现一种 ADT 不只要输出顺序正确，还要检查承诺的时间复杂度。可以用[10,20,30]各压弹一次，画出每次真正移动的格子。

25.2 2 · 列表做栈：双向与单向不能混答

双向表有首尾哨兵和尾指针，两端插入、删除都能常数改链，交换 top 与 bottom 的角色不会改变渐近阶。节点分配和缓存常数仍可能不同。

单链若有尾指针，尾插可以 O(1)，但尾删还要找到前驱，一般 O(n)；首插、首删则 O(1)。所以“列表两端对称”需要双向链接或额外结构，不是所有叫 List 的类型都成立。

读示例项目时找 push、pop 实际调用哪个接口，再看那个接口需要多少次走链接。先确认表示法，才知道对称性是不是实质的。

25.3 3 · 迷宫里的粉笔：调用栈不会记住所有来过的地方

调用栈保存“当前还没做完的路径”。函数返回后那一帧就消失，所以以后从另一路走到同一图节点，栈未必知道它曾被处理。

要避免图中的重复或环，可以把 visited 放在节点字段、外部数组或哈希集合里。C++并没有自动防止递归环的内建机制；静态或全局变量能跨调用保存状态，但也可显式传入一个共享的访问表，并非只能靠全局变量。

把“栈记路径”和“标记记历史”放在同一个小图上比较：边为 A→B、A→C、B→D、C→D。从 A 先走 B，再到 D；D 返回、B 返回后，调用栈里只剩 A。接着走 C 到 D 时，D 不在栈中，但它确实已处理过。若任务是可达性搜索，保留一个永久 visited 就能跳过第二次 D，避免重复展开；若任务是枚举从 A 到 D 的所有简单路径，第二条 A→C→D 也是答案，不能因为 D 曾经出现便抹掉它。

后一任务应维护“当前路径中是否已有此节点”：进入时设为真，退出这条路径时恢复为假。追踪这张图时，走完 A→B→D 后恢复 D、B 的路径标记；接着 A→C→D 仍可进入。若另加 B→A，走到 B 时发现 A 已在当前路径，便拒绝这条回边，防止无限环。永久 visited、当前路径标记、处理完毕标记解决的问题不同，不能只给数组起名 visited 而不定义含义。

只在返回时才标“处理完成”，又不区分正在访问的灰色状态，在 A→B→A 这类环中会还没返回就再次递归 A。反过来，判断有向环要看目标是否仍在当前递归路径，不能把“连到了任一曾访问节点”都判为环；上面 C→D 只是汇合，不是环。第 16 章的灰色/黑色状态把这两类情况分开。

25.4 4 · 每帧减半，理想规模为什么可能平方

设调用栈预算 B，每帧 s 字节，允许深度约 B/s。固定比例分治若深度 $h = c\log n$，在同一栈预算下帧减半，允许深度翻倍，故 $c\log n' \approx 2c\log n$，得到 $n' \approx n^2$。

减到十分之一则允许约 10 倍深度，理想上 $n' \approx n^{10}$。这不是“内存容量变成原来的平方”，只是对数深度与输入规模之间的指数换算。

若算法实际是线性递归 h≈n，则帧减半只能把规模约翻倍。若输入数组、输出和堆缓冲占用了主要空间，也不能按这个理想算式承诺真实机器能力；题目已明确暂忽略这些消耗。

25.5 5 · 递归都能消去，但未必能消掉需要保存的信息

普通递归可以用显式栈模拟：每个帧保存参数、尚需使用的局部量、返回后该接着执行哪一段。while 循环读取栈顶状态，遇到递归调用就压一个新帧，遇到返回就弹出并把结果交给父帧。

例如二叉树后序遍历，帧需说明“左边还没做”“左边已做、等右边”“左右都完成”；只保存节点地址而不保存阶段，就不知道返回后该去哪。

例如计算树 `A(B,D)`的后序，叶子 B、D 没有孩子。把帧写为`(节点,下次阶段)`，阶段 0 准备左、1 准备右、2 准备访问自己，完整栈从底到顶为：

```
[(A,0)]                          进入根
[(A,1),(B,0)]                    父帧先改成“等左返回后做右”，再压B
[(A,1),(B,1)]                    B没有左孩
[(A,1),(B,2)]                    B没有右孩
[(A,1)]，输出B                   弹B
[(A,2),(D,0)]                    A保存“等右后访问自己”，再压D
[(A,2),(D,1)] -> [(A,2),(D,2)]
[(A,2)]，输出D                   弹D
[]，输出A                        弹A
```

如果只在栈里存 A、B，B 返回以后看到 A 却不知道左子树已经完成，就可能反复压 B。阶段就是递归调用返回地址原本替你保存的信息。

这消除了语言层面的递归调用，却仍可能使用 O(h)辅助栈。一般程序需要记住哪些待办任务，是问题本身的信息需求，不会因为把函数改成 while 就自动变成 O(1)。

25.6 6 · 第二个递归是尾调用：可以省那条等待链

若某函数先递归处理左半，左半返回后直接返回右半的递归结果，右半调用是真尾位置，可以把当前帧参数改成右半并跳回循环起点，第一支仍正常递归。

这能减少“父帧等第二支结束”的堆叠，但第一支的深度仍存在。对快排一类过程，常见办法是递归较小一侧、循环处理较大一侧，可控制同时活跃的递归深度；需要证明每次递归那侧至多减半。

以“访问根、处理左子树、处理右子树”的先序为例，右递归在最后，可改为：

```
preorder(v):
    while v不是空:
        visit(v)
        preorder(v.left)
        v = v.right
```

树 `A(B(D,E),C)`的执行是：外层访问 A，递归进入 B；B 访问后递归 D，D 访问结束，B 的循环改成 E 并访问；B 整个调用返回后，A 的循环改成 C，访问 C。输出 A、B、D、E、C。右边的一条长链现在由循环走，不会一层层保留旧帧；左边的递归仍会保留父节点，最坏左链仍线性深。

如果右递归返回后还要合并左右结果，它就不是尾位置，不能直接覆盖父帧；必须保留左结果和后续合并任务。看源码中的“调用以后还有没有实际需要的工作”，而非只看它是不是第二个调用。

25.7 7 · 阶乘：把待完成乘法装进一个累加器

普通阶乘 `fac(n)=n*fac(n-1)` 返回后还要乘 n。引入 acc，保持不变量“acc 乘上尚未处理的 1 至 n，等于原目标阶乘”，便能迭代：

```
def factorial(n):
    if n < 0:
        raise ValueError("阶乘输入须为非负整数")
    acc = 1
    while n > 1:
        acc *= n
        n -= 1
    return acc
```

n=4 时，(acc,n)依次(1,4)→(4,3)→(12,2)→(24,1)。固定宽度模型下只用常数个变量；若使用任意精度整数，保存 n!本身需要的位数仍在增长，不能叫实际字节 O(1)。

推广要检查剩余操作能否以固定大小状态组合，顺序能否改变、副作用是否允许。浮点乘法重排可能改舍入，打印输出的顺序也不能随意交换。“递归后只剩一行”不等于可以自动塞进一个标量。

25.8 8 · 头递归：参数推进也算计算

进制转换先调用 `convert(n/base)`，返回后输出 n%base。它的主要输出在递归之后，所以常被归为头递归；但实参中的除法已经使问题缩小，并不是调用之前完全没有计算。

如果所谓“纯头递归”是状态与参数完全不变，就再次调用同一个实例，那么确实不可能靠自己接近基例。例如 `f(n){if(n==0)return; f(n);}` 对 n>0 不断压栈。若参数改成 n-1，便已经有了推进，不能套这条断言。

去递归时，只需保存将来输出所需的余数，而不必保存编译器帧中的所有机器细节。例如十进制 13 转二进制，依次保存余数 1、0、1、1，反向弹出 1101。保存什么，取决于返回后还要用什么。

25.9 9 · 单链析构：后序删与顺序删都能线性

头递归可先销毁后继，再 delete 当前；它保证每个节点只删一次，但长 n 的链会同时压 n 层帧。调用前必须先判断 p 是否为空或尾哨兵，实参 p->succ 的读取本身也是工作，不属于“毫无状态推进”。

迭代版先保存 next，再释放当前，再 p=next。它不用返回旧节点处理剩余事，所以可常数辅助空间，时间仍 Θ(n)。释放前保存后继，释放后绝不能再从 p 读链接。

如果数据元素的析构必须遵循某种业务次序，头到尾与尾到头可能有语义差别；普通独立节点可选迭代，但不能把所有带副作用的析构顺序视为无关。

25.10 10 · 自定义括号：先分清字符串，再谈栈

词法阶段把输入识别成左记号、右记号与普通内容，栈保存(类型,起始位置)。遇左记号压栈；遇右记号，若栈空就报告多余关闭，类型不同就报告当前位置与期待的关闭类型；输入结束还剩左记号则报告未关闭的起点。

例如 `<a>[x)</a>`，读到右圆括号时，栈顶是方括号起点，应说明“这里期待]，它与位置 3 的[对应”。这比单纯返回 false 更能帮助修改。

固定一个只支持`()`、`[]`的简单语言，输入`([x)]`，用从 0 开始的位置：

位置	字符	动作	栈（底→顶）
0	(	压入类型与位置	`[(圆,0)]`
1	[	压入类型与位置	`[(圆,0),(方,1)]`
2	x	普通内容，不动栈	同上
3	)	与栈顶方括号冲突，停止	报错位置 3，期待]以匹配位置 1

不能在失配时先把栈弹掉再继续假装合法。若输入是`([`，扫描结束后剩`(圆,0)`，错误应报告为“位置 0 的左圆括号尚未关闭”，与中间遇错右括号是不同情况。

在这个明确受限的语言里，算法可以完整写成下面这样。返回 `None` 表示合法，否则返回含位置的错误说明；普通字符都忽略，不声称识别 JSON 或 HTML。

```python
def bracket_error(text):
    opens = {'(': ')', '[': ']'}
    closes = {')', ']'}
    stack = []
    for pos, token in enumerate(text):
        if token in opens:
            stack.append((token, pos))
        elif token in closes:
            if not stack:
                return f"位置{pos}多出右括号{token}"
            left, start = stack[-1]
            if opens[left] != token:
                return f"位置{pos}期待{opens[left]}，以匹配位置{start}的{left}"
            stack.pop()
    if stack:
        left, start = stack[-1]
        return f"位置{start}的左括号{left}尚未关闭"
    return None
```

每个左括号保存一次，配对时弹一次；下一项要配的是最后一个尚未配对的左括号，正好对应栈顶。匹配失败时不修改栈即可立即报告，扫描结束还剩左括号则报告最近一个未闭合处。空串没有待配项，所以合法。时间$O(N)$，最坏保留N个左括号而用$O(N)$空间。

JSON 字符串中的 "[" 是内容，不是结构括号，反斜杠转义也影响引号结束位置。HTML 还涉及空元素、属性、注释与复杂语法，简单标签栈不等于完整 HTML 解析器。扩展算法要先说支持哪一种受限语言，不能只按字符匹配就宣称解析所有格式。

25.11 11 · 加法结合方向：表中同类相遇的那一格决定顺序

课件 + 对 + 为大于，意味着栈顶先归约。2+3+4 读到第二个+时，先算 2+3=5，再把+压入等待 4，得到(2+3)+4。要改成 2+(3+4)，把这格改为小于，让新+先入栈，之后从右边归约。对只含合法加法的表达式，主循环不需换框架；若同时有减法，则必须检查相同优先级的交叉格，不能只改一个角落却声称整个语法都一致。

先看完整教师原表，横向是“刚读到的运算符”，纵向是“栈顶等待的运算符”。>表示先处理栈顶，当前输入先不前进；<表示压入当前符号并前进；匹配格表示消去相应括号或结束哨兵。

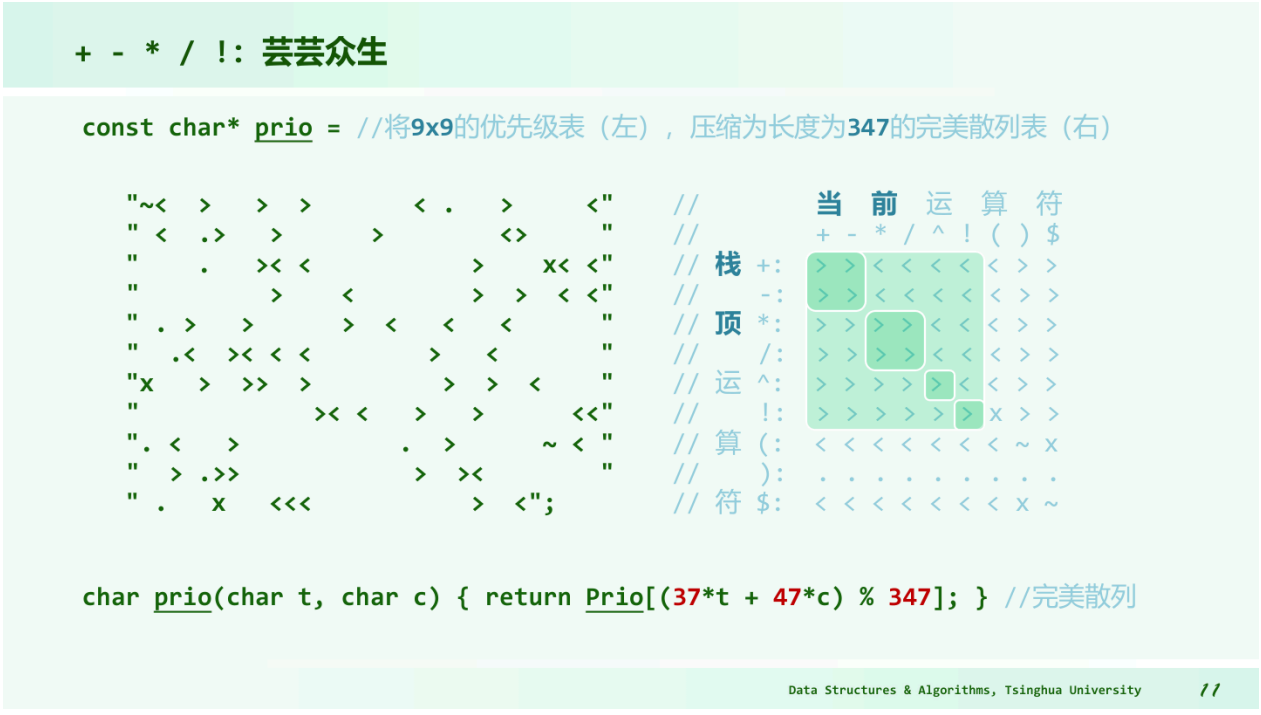


图 33　课程《04.栈与队列》分章 PDF 第 42 页完整原图（幻灯片页脚 11）。例如栈顶乘号、当前加号应读第*行第+列，得到先归约；乘方对乘方也是先归约。

对 2+3*4，让 # 在本章表示原图的结束哨兵，逐步求值如下。每行只做一个动作，同一个当前符号可能被连续看多次：

当前输入	查表或动作	运算符栈	数值栈
2	数字入数值栈	#	[2]
+	# < +，压+	# +	[2]
3	数字入栈	# +	[2,3]
*	+ < *，压*	# + *	[2,3]
4	数字入栈	# + *	[2,3,4]
结束 #	* > #，弹*并算 3×4	# +	[2,12]
仍为 #	+ > #，弹+并算 2+12	#	[14]
仍为 #	哨兵匹配，结束	空	[14]

最容易漏的是后 3 行：归约只弹栈顶，没有消费当前输入。若每归约一次也把输入指针前移，最后的加法可能根本没机会执行。

精确整数无溢出时两者同值，浮点却不保证结合律。时间仍线性，但连续右结合运算符会积在栈里，某些输入的峰值空间可由常数增加到线性。只说“时间空间都没变”会漏掉这一输入族上的变化。

25.12 12 · 原程序的乘方是左结合，修改后才符合通常数学惯例

原表 ^ 对 ^ 为大于，因此 2^3^4 先算 2^3=8，再算 8^4=4096。通常数学写法解释为 $2^{(3^4)} = 2^{81}$，两者差很多。必须先报告原表实际做什么，再说要怎样修改。

用较小的 2^3^2 核对两个版本，不用算很大的数：

时刻	原表左结合：符号栈/数值栈	修改后右结合：符号栈/数值栈
刚读完 2、第一乘方、3	# ^ / [2,3]	相同
遇第二乘方	先弹旧乘方算 8，当前仍未消费	直接压新乘方，变 # ^ ^ / [2,3]
消费第二乘方并读 2	# ^ / [8,2]	# ^ ^ / [2,3,2]
输入结束	8 的平方=64	先 3 的平方=9，再 2 的 9 次方=512

两个版本都遵守“先弹右数、再弹左数”，差别出在何时决定旧乘方已经拥有完整的右子表达式。

把这格改为小于，连续^先入栈，最后从右向左归约。读取、弹出和计算框架不必改变，但测试应同时包含括号、阶乘与多个乘方；C++里的字符^本身是按位异或，这里只是自定义表达式语言的乘方 token。

总时间依然 O(N)，最坏总空间依然 O(N)；但无括号纯乘方链原先可即时归约，改右结合后会积压 O(N)个运算符。复杂度类别与具体输入上的峰值，应分别说明。

25.13 13 · 右括号那一行为什么全空

当前右括号遇到左括号时只匹配并消去，不作为待执行运算符长期压栈。因此“栈顶已经是右括号，再看下一个 token”的状态不应出现，表中对应整行为空。

另一些空格表示语法不合法，例如没有左括号却遇到右括号，或输入结束还有未配对左括号。它们不是“优先级差不多，随便处理”，而是应报告的错误。

区分三种表项：小于表示移进，大于表示归约，匹配标记表示相消；空白表示不允许。若程序默认输入合法，可以断言；面向真实输入的解析器应返回位置和原因。

25.14 14 · 双阶乘：6!!不是(6!)!

定义 $n!! = n(n-2)(n-4)\cdots$，到 1 或 2 停止，并约定 0!!=1。6!!=6×4×2=48；(6!)!完全是另一个大得多的数。

间接方案把一个新 token 映射成一元后缀双阶乘，计算器按步长 2 累乘。它与阶乘一样只弹一个操作数，必须检查非负整数定义域和溢出。直接读!!则需词法分析的最长匹配：先看下一个字符是否也是!，一次识别双阶乘。

若语法还允许两个连续普通阶乘，必须额外规定用括号如 `(6!)!` 表示，否则同一字符流有歧义。3!!!也要说明是 3!!后接!，还是别的扩展；不能依靠“人看着懂”让程序猜。

25.15 15 · 操作数顺序：先弹右边，后弹左边

读完 3、8 后，栈底 3、栈顶 8。遇减号先弹 b=8，再弹 a=3，算 a-b=-5；如果反过来做 b-a，就变 5。加法乘法的交换律会掩盖这个错误，所以测试要包含减、除和乘方。

一元阶乘只需一项；二元运算需两项；栈不足必须在执行前报告。除数为 0 也要先判断，不应偷偷压 0 继续算，因为后面看似合法的结果会隐藏真实失败。

整数截断、精确有理数、浮点是三种数值语义。第 30 章的表达式核对采用精确有理数；若课程 double 求值，最后小数应允许相应舍入，不把两种执行结果机械按字符串判等。

25.16 16 · 运算符种类有限，栈为什么仍能任意深

相邻待处理符号能同时入栈，是因为前一符号相对后一符号为移进关系。这个表关系不传递：+对(为小于，(对+也为小于，但+对+为大于。因此不能说“每层严格升高，符号只有九种，所以最多九项”。

构造很多尚未闭合左括号，栈就能线性深；运算数栈也能用嵌套的 `1+(1+(1+...))` 积累待算左值。输入结束前某个时刻的峰值需要计入，不能只看最终剩一项。

若问原优先表下“达到峰值 n 最少多少对括号”，计入底部结束哨兵时，一层括号内最多可暂留+、*、^、!四个不同优先级运算符；!后不能直接接(，所以每增加一个嵌套层最多增加四个待留项。无括号最多 5 项（含哨兵），故下界是 $\max(0, \lceil (n-5)/4 \rceil)$ 对，利用 `0+0*0^(0+0*0^0!)` 这类递归嵌套可达到相应峰值，其余 n 用较短运算链补齐。

这里把“补齐”实际做一遍。含底部哨兵时，下列四个式子都只用一对括号：

表达式	读到最内层!、尚未归约时的运算符栈（底→顶）	峰值
`(0+0*0^0!)`	`# ( + * ^ !`	6
`0+(0+0*0^0!)`	`# + ( + * ^ !`	7
`0+0*(0+0*0^0!)`	`# + * ( + * ^ !`	8
`0+0*0^(0+0*0^0!)`	`# + * ^ ( + * ^ !`	9

每加一个完整外层，外层可留下+、*、^三个运算符，再加左括号，共增加 4 项。!是后缀符号，原表不允许!后直接移进左括号，不能让每层都多留一个!再继续往里嵌套。对更深的目标，重复完整外层后，在最外层选上表对应的短前缀，就能补齐余数；例如峰 10 可在峰 9 的式子外再包一对括号。

若不计底部哨兵，最大峰变成 4P+4，最少括号相应为 $\max(0, \lceil (n-4)/4 \rceil)$；若先把^改成右结合，纯乘方链就能不加括号积到任意深，刚才的最少括号式不再适用。回答本题必须带上表版本与栈计数口径。

25.17 17 · 转 RPN：每个运算符都应等自己的孩子先输出

把表达式看成树，数字是叶，运算符是内部节点。后序遍历先输出左子树、右子树，再输出父运算符，就是 RPN。按树规模归纳：两个更小子表达式各自输出正确，接上运算符，就恰好表示它们的组合。

把 `2*(3+4)` 画成树，两个孩子的顺序也标清：

```
    *
   / \
  2   +
     / \
    3   4
```

树的后序是：访问叶 2；进入右子树，访问 3、4 后访问+；最后访问*，所以 RPN 为 `2 3 4 + *`。归纳命题必须说“输出恰好对应这棵子树的完整 RPN”，而不能只说“运算符在数字后面”：后者不足以保证左右孩子归属正确。

在线栈算法还没看到右操作数完整范围时，先保存运算符；只有后续 token 说明该子表达式已闭合，才归约输出它。因此数字顺序不变，运算符输出顺序遵从相同的树依赖。

手工画树是整体分析，栈法则逐 token 保存尚未确定的工作；两者依据同一子树依赖，才会得到同一后序，而不是只在这个数值例子中碰巧相同。

25.18 18 · 转换也要线性，RPN 还有什么价值

只计算一次时，中缀解析加 RPN 求值仍是线性，并没有把 O(N)降到更低阶。它的收益是将“理解括号与优先级”同“执行运算”拆开。

如果一个带变量表达式需要对一万组输入求值，语法只解析一次，后面反复执行简单指令序列；虚拟机也可以让每条指令注明弹入几个值。错误位置和类型检查可在转换阶段先完成。

这是一种可复用表示，而非复杂度魔法。若表达式只用一次、程序很小，直接求值也可能更简单；应根据任务选择，不必把 RPN 描绘成所有情形都更快。

25.19 19 · 21 个长表达式：用短片段先确定读法

完整 21 式的 RPN、数值与逐步栈追踪在第 30 章。这里先拆第一式：括号内 0+1+2-3+4+5+6 按左结合依次输出 `0 1 + 2 + 3 - 4 + 5 + 6 +`；另一个括号输出 `7 8 +`；两段相乘后输出，再读 *9*，最后输出。

合起来是 `0 1 + 2 + 3 - 4 + 5 + 6 + 7 8 + * 9 *`。数值第一括号 15，第二括号 15，所以 15×15×9=2025。

转换与求值要分别核对：一个是 token 顺序，一个是算术结果。

含!!的式子不能逐字符当两个!；含连续减除的式子要按左结合；第 30 章使用同一套原表约定；这 21 式也没有无括号连续^造成的冲突。若另加 `2^3^4` 作为测试，必须标明使用原表还是修改表。

25.20 20 · 前缀 PN：从右往左读也能用栈

前缀 `- 8 * 2 3` 表示 8 减 2×3。倒序扫描 3、2 入栈，遇*弹出的第一个是左操作数 2、第二个是右操作数 3，压 6；再读 8 入栈，遇-弹 8 与 6，压 2。

这和 RPN 从左读时“先弹右操作数”的顺序恰好不同，因为扫描方向变了。固定元数让程序知道每个运算符要几个孩子；单目、双目应在 token 定义里标明。

每个 token 进出常数次，时间 O(N)，最坏栈 O(N)。也可递归按“运算符后接几个子表达式”解析，仍需检查 token 耗尽、操作数不足和最后剩余多份结果。

25.21 21 · 禁形：晚读入的大数，把中间数压在了小数上面

统一按输入 1、2、…、n 依次压入，输出按实际弹出顺序记录。设 i<j<k，目标输出却含 k…i…j。为了先弹 k，必须已经把 i、j、k 按输入顺序读入；若 j 尚未弹出，它就在 i 上面，因此 i 不能先于 j 弹出，矛盾。

特例 j+1…i…j 只是令 k=j+1，机制相同。它们说的是三个元素的相对次序，不要求三个在输出中紧挨着。

例如想输出 3,1,2：压 1、2、3，弹 3 以后栈顶是 2，要得到 1 必须先弹 2，却又要求 2 在 1 后面，卡住。输出 3,2,1 则可行。若教材图把最终输出画成另一只栈，视觉阅读方向可能反着，需要先换成实际弹出顺序，再使用禁形名称。

25.22 22 · 依最后或最先输出分类：两种视角不必有同样简单的公式

设最后输出 k。所有小于 k 的元素必须在 k 入栈前弹完，否则被 k 压住以后就不可能赶在最后 k 之前弹出。大于 k 的元素在 k 上方完成另一套独立栈过程，因此输出分为“小于 k 的混洗、大于 k 的混洗、最后 k”。两块分别有 C_{k-1}, C_{n-k} 种，分类互斥且穷尽，组合可逆，于是

$$C_n = \sum_{k=1}^{n} C_{k-1} C_{n-k}, \qquad C_0 = 1.$$

用$n = 3$列出按最后一项的三组：最后 1 是 231、321（前两项都属于大于 1 的子问题），最后 2 只有 132（小于 2 的 1 必须先于大于 2 的 3 完成），最后 3 是 123、213。组数为 2、1、2，恰是C_0C_2、C_1C_1、C_2C_0。

卷积的可逆性也需要说清：任取左子问题的合法操作，再压入并暂存 k，执行大于 k 的那部分，最后弹 k，得到且只得到这一组输出。不同 k 由最后项区分，不会重复计数。

按第一个输出 k 分类时，要先压 1 至 k 再弹 k，剩余栈有 k-1 项，还有 n-k 个输入没读。后续不是两块简单相接，应该数从高度 k-1 回到 0、始终非负的入出栈路径。

令 u=n-k、h=k-1，可用完成数 F(u,h)递推：允许再压入时加 F(u-1,h+1)，栈非空允许弹时加 F(u,h-1)，u=0 时只能依次弹完，完成数为 1。也可由反射法得

$$F(n-k,k-1)=\binom{2n-k-1}{n-k}-\binom{2n-k-1}{n-k-1},$$

差式里的两项分别在数什么？首弹 k 后，初始高度$h=k-1$，剩$u=n-k$次入栈、$u+h$次出栈，总步数$L=2u+h$。不管会不会弹空，任意排列这些步共有$\binom{L}{u}$种。对会从高度 0 走到−1 的坏路径，保留它首次到−1 的前缀，把这之后的所有上下动作互换。原来后缀从−1 回到 0，反射后改为从−1 到−2，整条新路径比原来少 1 个上步、多 1 个下步，故只有$u-1$个上步。反过来，任何从高度h结束在−2 的这种步串都必经过−1，在首次到−1 后反射即可恢复原路径。所以坏数为$\binom{L}{u-1}$。两者相减就是合法完成数。这个反射计数需要保留初始高度，不能默认每次从空栈开始。

越界二项式视为 0。n=3 时按首项 1、2、3 分别为 2、2、1，总 5。不能为了得到熟悉的 Catalan 卷积，硬把这次分类也拆成并不存在的两块。

25.23 23 · CSTDSA：两个 S 不能直接让 Catalan 数除以 2

先给两个 S 编号，输入顺序 C、Sa、T、D、Sb、A。最后弹 Sa 时，C 不可能留在 Sa 下面等最后才出，所以 C 必须先出；中间 T、D、Sb、A 可独立混洗，故这类有 $C_4=14$ 种。

最后弹 Sb 时，A 在它上面，必须紧挨末尾先弹 A；更早四项已经完成，所以这类也是 14 种。再要求以 C 开头、末两项为 A 和 Sa 或 Sb，分别留下 3 项自由混洗，各 $C_3=5$ 种。

把原题 e、f 两类各 5 个可见串完整列出来，交集就不必靠猜：

以 C 开头、以 A Sa 结尾	以 C 开头、以 A Sb 结尾
CDSTAS（左类独有）	CSTDAS（右类独有）
CDTSAS	CDTSAS
CSDTAS	CSDTAS
CTDSAS	CTDSAS
CTSDAS	CTSDAS

每列内部的 S 仍分别来自有标记的合法操作，只是此表省去标记显示。四个相同字符串可能由不同 S 身份的路径产生，计“可见串”时应合并；计“有标记路径”时不能合并。

去掉 Sa、Sb 身份，只显示 S 后，这两组会有重叠。交集的可见输出是 CDTSAS、CSDTAS、CTDSAS、CTSDAS，共 4 种；于是末项为 S 的不同可见字符串为 14+14-4=24。两个特定 5 元素集合的对称差有 2 种，交集 4 种；各自独有 1 种。

可以用递归枚举全部 132 条六元素合法栈操作，再分别放入集合核对。去重的作用发生在输出字符串层面，不是在操作路径层面。因为重复字符的两种标号不一定都对应合法过程，直接 132 除 2 没有依据。

25.24 24 · 栈混洗判定：当前目标不在顶上，只有一个可能的补救

按目标输出逐项处理。如果栈顶就是目标，立刻弹；否则继续从输入压入，直到目标成为栈顶。若所有输入已压完仍不匹配，之后没有新动作能把埋在下面的目标提前取出，判失败。

贪心正确性来自动作被迫：想输出当前目标，栈顶若是其他值，不能把它先弹到输出；唯一可尝试的是继续压尚未读入的值。如果目标仍在输入里，压到它会成功；如果它已被压在错误的上层元素下面，过程就会卡住。

成功与失败各走一次，输入都为 1、2、3：

目标输出	当前要的数	连续压入	弹出后栈	结果
2,1,3	2	压 1、2	[1]	输出 2
同上	1	无	[]	输出 1
同上	3	压 3	[]	输出 3，成功
3,1,2	3	压 1、2、3	[1,2]	输出 3
同上	1	输入已耗尽	顶仍是 2	不能先弹 2，失败

失败返回当前尚未匹配的输出下标，故小于 n；全部匹配才返回 n。若目标含重复或漏项，不再是“互异输入的一个排列”，应先验证输入规格或让模拟按相同定义拒绝。

虽然外 for 最多 n 轮、某次 while 也可能压 n 项，但每个输入只压一次，每个输出只弹一次，内循环跨所有外层的总次数最多 n，整体 O(n)。二重循环不能不看变量变化就相乘。

25.25 25 · 括号与混洗：对应的是合法操作路径

入栈写(，出栈写)。任何前缀里出栈次数不能超过入栈次数，所以左括号数不少于右括号数；结束时各 n 个。这恰好是 n 对合法括号串的条件。

反向读一个合法括号串，遇(就压下一个输入，遇)就弹栈，非负前缀保证不会从空栈弹出。这个过程唯一，因此合法括号串与合法入出栈操作路径一一对应。

对互异且输入顺序固定的元素，可行输出也唯一决定其被迫的入出过程，于是输出混洗数等于 Catalan 数。两个字符相同后，可见输出可能由多条路径得到，刚才的 CSTDSA 正说明此时不能再把输出字符串与括号串一一对应。

25.26 26 · 枚举、第 k 个、均匀抽样：先知道每条分支下面有多少串

设已用 o 个左括号、c 个右括号。o<n 时可加左括号；c<o 时可加右括号。完成数 A(o,c)满足这两个合法后继的和，终点 A(n,n)=1，其余非法状态为 0。用表或记忆化计算，共 $O(n^2)$个状态。

按约定的字典序“(在)前”深度优先枚举，n=3 依次得到((()))、(()())、(())()、()(())、()()()。取从 1 编号的第 k 个时，先看左分支完成数 L：k≤L 就走左，否则 k 减 L 再走右。例如第 4 个跳过左分支的前三个，进入以()开头的部分，最终为()(())。

“第 4 个”的决策不要跳过。n=3 时第一字符只能(；到状态$(o,c)=(1,0)$，再写(的分支共有 3 个完成串((()))、(()())、(())()，写)的分支有 2 个。k=4 先跳过前 3 个，改 k=1，走到前缀()；接着只能写(，在前缀()(之后先试(，有 1 个完成串()(())，k=1 命中。每一步都减的是整棵尚未进入的分支的完成数，不是每跳一个符号就减 1。

完成数表也可手填 n=3 的几个格：$A(3,3)=1$，$A(3,2)=1$，$A(2,2)=A(3,2)=1$，$A(3,1)=1$，$A(2,1)=A(3,1)+A(2,2)=2$，$A(2,0)=A(3,0)+A(2,1)=1+2=3$，$A(1,0)=3+2=5$。状态只允许$c\le o\le n$，非法格为 0。

先填表，再按表中块大小选路，可以直接写成三个共用状态定义的接口。`ways[o][c]`就是$A(o,c)$；从大 o、大 c 向回填，才能保证两个后继已经存在。

```
def bracket_counts(n):
    if n < 0:
        raise ValueError("括号对数非负")
    ways = [[0] * (n + 1) for _ in range(n + 1)]
    ways[n][n] = 1
    for o in range(n, -1, -1):
        for c in range(o, -1, -1):
            if o == n and c == n:
                continue
            if o < n:
                ways[o][c] += ways[o + 1][c]
            if c < o:
                ways[o][c] += ways[o][c + 1]
```

```
    return ways

def kth_brackets(n, k):
    ways = bracket_counts(n)
    if not 1 <= k <= ways[0][0]:
        raise ValueError("k须落在1到合法串数之间")
    o = c = 0
    result = []
    while o < n or c < n:
        left_count = ways[o + 1][c] if o < n else 0
        if k <= left_count:
            result.append('(')
            o += 1
        else:
            k -= left_count
            result.append(')')
            c += 1
    return ''.join(result)

def all_brackets(n):
    if n < 0:
        raise ValueError("括号对数非负")
    def walk(o, c, prefix):
        if o == n and c == n:
            yield prefix
            return
        if o < n:
            yield from walk(o + 1, c, prefix + '(')
        if c < o:
            yield from walk(o, c + 1, prefix + ')')
    yield from walk(0, 0, '')

def random_brackets(n, rng):
    # rng.randrange(m)必须在0,...,m-1中无偏选整数。
    ways = bracket_counts(n)
    return kth_brackets(n, rng.randrange(ways[0][0]) + 1)
```

`kth_brackets(3,4)`沿前面的决策返回`(()())`；`all_brackets(0)`产生唯一的空串，`kth_brackets(0,1)`也返回空串。选第 k 个的循环不需要在右分支再试非法路径：表只给合法完成计数，合法 k 跳过左分支以后，必然落在右分支的计数内。每步总长增加 1，最终恰为$2n$，所以会结束。

计数表有$O(n^2)$格，单次选路走$2n$步；若把表留给同一 n 的多次查询，可复用填表成本。列出全部串则至少要输出$2nC_n$个字符，不能把它也叫$O(n^2)$；演示生成器还复制前缀字符串，这是实现的额外字符处理成本。大整数的加法成本也会随计数位数增长，上述“每格常数”是单位整数模型。

均匀随机抽样也按完成数分支：在总数 L+R 里均匀取整数，小于 L 选左，否则选右。每一整串的路径概率相乘后恰为 1/Cn。若每次在可选的左右之间直接各取一半，不同串经过的自由选择次数不同，概率会不等；n=3 的((()))可有 1/4，而(()())仅 1/8。

大数需要大整数计数和无偏随机整数，不能用溢出的机器整数或有偏取模。这里给出一套可验证的动态规划方案，解决原题三项任务；它不是把 Knuth 所列 P/U/W 算法的原始实现逐行复刻，阅读原书时还需比较其不同优化目标。

25.27 27 · 循环向量队列：出队只移动头，不搬整队

保存 head 和 size，容量 C>0 时，第 i 个逻辑元素位于 `(head+i)%C`。入队写到 `(head+size)%C` 后 size++；出队读 head，再令 head=(head+1)%C 并 size−。

容量 4，依次入 A、B、C，出 A、B，再入 D、E，物理数组可能是[E,旧 B,C,D]，逻辑顺序却是 C、D、E。旧格中留下的字节不等于仍有有效元素，size 决定有效范围。

把本例的每次物理写也列出来，_表示本来为空，带旧字样的内容已经失效：

```
初态      [_,_,_,_]      head=0,size=0
入A       [A,_,_,_]      head=0,size=1
入B       [A,B,_,_]      head=0,size=2
入C       [A,B,C,_]      head=0,size=3
出A       [旧A,B,C,_]    head=1,size=2
出B       [旧A,旧B,C,_]  head=2,size=1
入D       [旧A,旧B,C,D]  写(2+1)%4=3, size=2
入E       [E,旧B,C,D]    写(2+2)%4=0, size=3
```

逻辑第 0、1、2 项的物理下标分别为 2、3、0。若这时扩到 8 格，应依次复制 C、D、E 到新下标 0、1、2，再令 head=0；原数组从物理 0 直接往后复制，会把逻辑队首误改成 E。

扩容时按逻辑顺序把两段复制到新数组 0 起点，head 重置 0。记录 size 可区分 head==tail 时究竟空还是满；也可以保留一个空位，但容量含义改变。容量 0、恰好满、绕回和扩容后顺序都要测。

25.28 28 · FIFO 应用：按到达次序处理的真实任务

BFS 把刚发现的节点加入队尾，从队首取下一个处理，因而先处理距离近的一层。打印任务、生产者消费者缓冲、离散事件里同一优先级的等待队列，也常使用先进先出。

但“最早到达”与“最紧急”不同。需要先处理最高优先级时应使用优先队列；需要先处理时间戳最小事件时，时间顺序不等于消息到达顺序，也可能要堆。

设计应用例子时写清入队事件、出队事件、满时如何处理。只报“操作系统用队列”没有说明它在保存哪种待办任务。

25.29 29 · 双栈当队：三种证明的常数为何不一定一样

in 接收入队，out 负责出队；只有 out 空时才把 in 逐项弹出并压进 out。一个元素最多入 in 一次、从 in 弹出一次、入 out 一次、最后从 out 弹一次，总共常数个栈动作，所以任意 m 次合法队列操作总 O(m)。

势能取 $\Phi = 2|in|$：入队实际 1、势能增 2，摊还 3；out 不空出队实际 1、势能不变，摊还 1；倒 k 项再出队实际 2k+1，势能减 2k，摊还仍 1。

对“入 A、入 B、出、入 C、出、出”，两只栈从底到顶的实际状态为：

操作	in	out	栈动作数	返回
入 A	[A]	[]	1	—
入 B	[A,B]	[]	1	—
出	[]	[B]	5：倒 2 项各 2 次，再弹 1 次	A
入 C	[C]	[B]	1	—
出	[C]	[]	1	B
出	[]	[]	3：倒 1 项再弹	C

第五步 out 还有 B，必须直接弹 B，不能因为 in 里有新 C 就再次倒入 out，否则新元素会越过旧元素。势能依次为 0、2、4、0、2、2、0，实际总 12，摊还成本 3、3、1、3、1、1 也合计 12。

记账法若每次入队收更多费用，会得一个更松的常数；聚合法也可能把尚未出队元素按全流程预收。只要是上界，它们不必精确相等。比较常数前先统一 push/pop、空检查、返回值复制是否计费，而非看到 2 与 3 就认定证明互相矛盾。

25.30 30 · 最大直方图矩形：面积相同，左边界也要比

每个柱宽 1、高非负。矩形覆盖半开区间[l,r)，高度不能超过区间最矮柱，面积为 $(r-l)\min h$。逐根柱子向两边扫描直到更矮处，单调输入可使总扫描 $\Theta(n^2)$；枚举所有区间又每次重扫最小值的另一种朴素写法甚至 $O(n^3)$，不要混为同一算法。

“最靠左”应明确定义为同面积时 l 最小；若 l 也相同，再规定 r 最小或最大，本章取 r 最小。更新答案时比较(-area,l,r)的字典序，或写等价的逐项条件。

例如[2,0,2]，左右两个单柱面积都为 2，应选[0,1)。有的栈实现先弹右边候选，仅用面积严格大于更新就会留错；有的扫描顺序恰好先见左侧，又碰巧符合，不能把偶然顺序当证明。核对课件 Excel 时，可以输入同一个`[2,0,2]`，记录它最后返回的左端是否为 0；只看面积 2 不能区分左右两个候选。

25.31 31 · 单趟比两趟多做了什么即时工作

两趟方案先算每根柱左侧最近更矮位置，再反向算右侧边界，最后统一算面积。它便于把所有边界画在表上，但需要存储相应数组或中间状态。

单趟保持递增栈，当前更矮柱一到，就确定被弹柱的右边界，立即结算面积。对[2,1,5,6,2,3]，读到后面的 2 时，先弹 6 得面积 6，再弹 5 得宽 2、面积 10，不必等整份右边界数组建完。

这个例子的单趟完整过程如下，栈保存柱下标，只在新高度严格更小时弹出，末尾补高度 0：

当前下标 i/高度	本轮弹出及面积	压入后栈（下标:高度）
0/2	无	`0:2`
1/1	弹 0，左边空记−1，宽 1，面积 2	`1:1`
2/5	无	`1:1,2:5`
3/6	无	`1:1,2:5,3:6`
4/2	弹 3，左 2，宽 1，面积 6；弹 2，左 1，宽 2，面积 10	`1:1,4:2`
5/3	无	`1:1,4:2,5:3`
6/0	弹 5 得 3；弹 4 宽 4 得 8；弹 1 宽 6 得 6	`6:0`

弹出柱 p 时，右边界就是当前 i，不包括 i；左边界为弹后栈顶 left 的后一格。覆盖`[left+1,i)`，宽度为$i-left-1$。等高柱在本版本中暂时保留，left 可能等高而非严格更矮；较右等高柱先结算窄矩形，较左等高柱以后获得完整宽度。例如`[2,2,0]`先结算右柱面积 2，再结算左柱面积 4，最大值仍正确。

单趟可流式前进、较早给出已结算候选，但仍可能把一整段递增输入存进栈。流式不等于常数空间，也不表示未来不可能出现更大的全局答案。

下面按本题第 30 题的并列约定，返回`(面积,左端,右端)`。空输入单独约定返回`(0,0,0)`；非空输入选非空区间，因此全零时应选最左、最短的`[0,1)`，先把它作为合法初始答案。代码中的`<`只在当前柱更矮时弹栈，与上表保留等高柱的规则一致。

```
def largest_rectangle(heights):
    if any(h < 0 for h in heights):
        raise ValueError("柱高不能为负")
    if not heights:
        return 0, 0, 0
    best = (0, 0, 1)
    stack = []
    n = len(heights)
    for i in range(n + 1):
        current = heights[i] if i < n else 0
        while stack and current < heights[stack[-1]]:
            p = stack.pop()
            left = stack[-1] + 1 if stack else 0
            area = heights[p] * (i - left)
            candidate = (area, left, i)
            if (-area, left, i) < (-best[0], best[1], best[2]):
                best = candidate
        if i < n:
            stack.append(i)
    return best
```

末尾的 0 只触发正高柱结算，不作为真实下标压入；零高柱即使留在栈中也不影响任何正面积候选，全零时已有初始答案。对`[2,0,2]`，第一柱在 i=1 时得到`(2,0,1)`，末柱结束时得到`(2,2,3)`，比较左端后仍选

前者。对[2,2]，右柱先得到宽 1、面积 2，左柱随后得到宽 2、面积 4，所以返回(4,0,2)。这两次分别检验并列位置和等高宽度，不只检验面积。

25.32 32 · 单趟线性：一次 while 很长，不代表每根柱都很长

聚合法：每根真实柱恰入栈一次，最多出栈一次，所以总 push≤n、总 pop≤n；外扫描 n 步，合计 O(n)。结尾追加高度 0 只负责把未结算部分清出，还需明确相等高度的保留规则。

记账法：每次入栈收两份，一份支付本次 push，一份存给未来 pop；从未弹出的柱保留信用，也不影响上界。每根柱收费常数，所以总线性。

势能法取栈大小。一步弹 k 根再压当前根，真实栈成本 k+1，势能变化 1-k，摊还为 2；再加常数比较等控制成本，仍常数。若把一次弹栈的比较也逐项收费，改变固定系数即可。这个证明完全不需要随机高度。

25.33 33 · 单时刻平均与全程峰值：不能把期望搬进 max

严格递增输入会让 n 根柱同时留在栈，最坏 Θ(n)；严格递减时每次弹掉旧项，最好峰值 O(1)。整个运行空间是 $M = \max_{1\le r\le n} S_r$，其中 Sr 是第 r 根真实柱处理后的栈长，最后清空之前的状态也必须算。

假设高度互异、相对顺序为均匀随机排列。栈恰保存此前缀从右往左看到的记录最小值；一个后缀长度 j，其中最左项成为最小值的概率 1/j，所以 $E[S_r] = H_r$。

r=3 时可以不带公式列全。6 种相对排列处理完后，栈内后缀最小记录分别为：

输入前缀	留下的高度	S_3
1,2,3	1,2,3	3
1,3,2	1,2	2
2,1,3	1,3	2
2,3,1	1	1
3,1,2	1,2	2
3,2,1	1	1

平均$11/6 = 1 + 1/2 + 1/3$。同一条输入 2,3,1 的前缀栈长却是 1、2、1，峰值 2 而最终只有 1，说明“最后栈长”和“运行空间峰值”连在同一个样本上都可能不同。

要证明峰值期望也为 Θ(log n)，还需控制“某一步偶然特别大”的概率。先补上乘积公式所需的独立性：从右向左读一个长 r 的均匀随机排列，把第 j 个读到的数在此前 j 个数中的名次记为 R_j，最小者名次为 1。每个 R_j 可取 1 到 j。

为什么这些名次独立？给定一组 $R_1, \dots, R_r$，从 j=r 往前恢复：先在尚未使用的 r 个数中取第 R_r 小者，作为最后读入的数；删掉它，再在剩余 r−1 个数中取第 R_{r-1} 小者，依此继续。每组合法名次恰好恢复一个排列。名次组合一共 $1 \cdot 2 \cdots r = r!$ 组，而每个排列等可能，所以联合概率为 $1/r! = \prod_{j=1}^{r}(1/j)$。这就证明各名次独立且分别均匀；只证明单个事件概率 1/j，本来不足以得到独立性。

令 $B_j = 1$ 表示 $R_j = 1$，即这次读到了新的最小记录，否则为 0。于是这些 B_j 独立，且 $P(B_j = 1) = 1/j$；栈长 $S_r = \sum_{j=1}^{r} B_j$。独立变量的指数期望可以相乘，单项为 $E[e^{B_j}] = 1 + (e-1)/j$，因此

$$E[e^{S_r}] = \prod_{j=1}^{r}\left(1 + \frac{e-1}{j}\right) \le e^{(e-1)H_r} \le C r^{e-1}.$$

这里e是自然指数的底，约为 2.718；C是不随 r 增长的正常数，来自$H_r \le 1 + \ln r$。若$S_r \ge t$，就有$e^{S_r} \ge e^t$，因此$E[e^{S_r}] \ge e^t P(S_r \ge t)$，移项便得到所用的 Markov 上界：

$$P(S_r \ge t) \le C r^{e-1} e^{-t}.$$

峰值$M \ge t$意味着至少有一个前缀的栈长达到 t。对这些事件求并集，概率至多各自概率之和，不要求不同前缀独立。又有$r^{e-1} \le n^{e-1}$，总共 n 项，得到$P(M \ge t) \le C n^e e^{-t}$。

取一个固定$A > e$，令$t_0 = \lceil A \ln n \rceil$。因为非负整数的平均值等于尾概率之和，

$$E[M]=\sum_{t=1}^{n}P(M\ge t)\le t_0+\sum_{s=1}^{\infty}Cn^e e^{-(t_0+s)}\le t_0+C\sum_{s=1}^{\infty}e^{-s}.$$

最后一步用到$n^e e^{-t_0}\le n^{e-A}\le 1$。尾部是收敛的几何级数，所以只多一个常数，得到$O(\log n)$。另一方面$M\ge S_n$，故$E[M]\ge H_n=\Omega(\log n)$，两边合起来才是$\Theta(\log n)$。$n=0,1$可直接处理，上面的$\ln n$渐近式针对增长的 n。

这些步骤解释了为什么答案成立，也暴露它依赖的随机模型。重复、相关或刻意排序的高度不能自动引用此概率界；最坏保证仍是线性。

25.34 34 · MaxQueue 摊还时间：每项只可能离开一次

影子队列按值单调递减。新项 x 到来，把尾部不可能在未来胜过 x 的较小项删掉，再把 x 接尾；原队列出队时，若影子头正对应离开的项，也删头。用索引或计数处理重复值，避免删错同值的另一项。

每项进影子一次，之后要么从尾被更大新值淘汰，要么从头因过期离开，至多一次。m 项总操作 O(m)，所以摊还 O(1)，不需要题名 Expected 暗示的随机假设。

取原队列依次入 3、1、4、2，再依次出队，影子保存(下标,值)。严格更小才从尾淘汰：

```
入(0,3)：影子[(0,3)]，最大3
入(1,1)：影子[(0,3),(1,1)]，最大3
入(2,4)：尾1、尾3依次淘汰，影子[(2,4)]，最大4
入(3,2)：影子[(2,4),(3,2)]，最大4
原队出下标0、1：它们已不在影子里，影子不动
原队出下标2：影子头就是2，删头，剩[(3,2)]，最大2
原队出下标3：影子清空
```

被新 4 淘汰的旧 3、1 即使还在原队列中，也不可能再成为最大值：4 更大而且更晚离开。这个“更大且更晚离开”同时保证淘汰安全，单有更大却马上过期并不够。用下标区别相同数值，也能准确判断离开的到底是哪一份。

记账法给每次加入预存一次未来删除费；势能取影子长度，一步删 k 项再加 1 项，实际 k+1 加势能 1-k 为 2。若维护原队列 X，其常数入出也另加，仍不改变摊还界。

25.35 35 · MaxQueue 空间：一扇窗口的平均，与长时间的最大值

固定窗口长 w，窗口内值独立连续同分布时，影子保存后缀最大记录。距窗口右端 j 项的位置成为记录的概率 1/j，影子在这一固定时刻的期望长度为 Hw，即 Θ(log w)。严格递减的一窗却会保留 w 项，最坏为 Θ(w)。

若观察越来越长的整段运行，不能仍把峰值期望写成 Hw。取互不重叠窗口，每窗恰好递减的概率 1/w!；观察 q 窗至少一次递减的概率为 $1-(1-1/w!)^q$，q 增加时趋近 1，因此固定 w 下的长期峰值趋近 w。

另外，若原队列 X 显式保存全部 w 项，它本身就占 Θ(w)，即使影子平均仅 log w，整个数据结构也不是总共 log w 空间。需要明确题目是在分析额外影子，还是包含原始元素的完整队列。

25.36 本章练习与答案

1. 原优先表和右结合修改表分别如何转换 2^3^2？写 RPN 与数值。
2. 高度[2,1,2]的最大矩形覆盖哪里？把每次弹栈对应的宽度写出来。
3. n=3 按第一个弹出元素分类，为什么不是三个类别都一样大？

第一题：原表 2 3 ^ 2 ^，值 64；修改表 2 3 2 ^ ^，值 512。第二题：最后 0 清栈时，中间高度 1 的柱可以扩到[0,3)，面积 3；高度 2 的左右单柱面积各 2，所以最佳是整个区间，高度 1。第三题：先弹 1 后剩 2 项自由混洗有 2 种；先弹 2 后还有两种合法完成；先弹 3 时栈中剩 2 压着 1，只能 2 再 1，所以 1 种，总 5。分类计数要由剩余状态决定，不能凭对称感猜。

26 PS05：树的箭头、遍历顺序和编码成本，一步一步核对

教师原始习题单：第 05 次习题共 37 题。路径长度按边数，根深度 0，叶高度 0，空树高度约定为-1。先序为根–左–右，中序为左–根–右，后序为左–右–根。“真二叉树”指每个内部节点恰有两个孩子，不等于每层都满。

26.1 1 · 路径按边计：空走一步都没走，就是 0

节点到自己的路径长 0。u 到 v 长 a，v 到 w 长 b，若可沿树中同一路径接起来，总长就是 a+b，不需要因为连接点 v 被数了两次再减 1。

例如根 R 到 A 一条边，A 到 B 一条边，R 到 B 深度 2。若按节点数，前两段各有 2 个节点，连接后却是 3 个，公式总要额外修正；按边计数与“每过一条链接深度加 1”一致。

计数约定本身都可以定义，但后续高度递推、路径拼接、复杂度边界必须统一。看到别的书叶高 1 时，先做整体偏移换算，再判断公式是不是矛盾。

26.2 2 · 深度加高度不超过全树高度

设 v 深度 d，从根到 v 有 d 条边；v 的最深后代距 v 为 h(v)，拼起来就是根到某个叶的路径，长 d+h(v)。全树高度 H 是所有根叶路径的最大值，因此 $d + h(v) \le H$。

若 v 的子树中含有全树某条最深路径，取等；若 v 在较短的侧枝，便可能严格小于。例：根左链深 4，右孩子就是叶。右孩子 d=1、h=0，总 1<4；左链中任一节点的深度加剩余高度都等于 4。

不能把全树最长路径误当“必须经过每个节点”。这条不等式是在比较 v 所在的那条候选路径与所有候选的最大值。

26.3 3 · 三种树表示：多存一条边，省下哪种搜索

固定同一棵有序树，再分别只保留三种目录：

```
        A
      /   \
     B     C
    / \
   D   E

父目录：A→空，B→A，C→A，D→B，E→B。
孩子目录：A:[B,C]，B:[D,E]，C:[]，D:[]，E:[]。
```

只有 parent 时，判断 B 是不是 E 的祖先，可以从 E 向上：E⟶B，命中；问 C 是不是 E 的祖先，则 E⟶B⟶A⟶空都没有 C。每步沿一条父边，最坏$O(h+1)$，其中 h 是树高。这个过程并不需要先找 B 的孩子。

现在把新节点 F 接到 C 下面，设置 `parent[F]=C` 便表达了父关系。但要列出 C 的所有孩子，父目录里没有“指向 C 的人”的反向名单，可能必须扫描全体节点。若要释放 B 整棵子树，也不能只删除 B：D、E 仍指向已经失效的对象。一个直接办法是对每个节点沿 parent 检查是否会到 B，收集出 B、D、E，再统一释放，最坏$O(nh)$；这只是一个正确但不一定最优的办法，不应把“设置一条父边常数时间”推广到“整棵子树删除常数时间”。

只有孩子列表时，B 的目录直接给出 D、E，所以从 B 下行就能访问它的整棵子树；问 E 是否属于它，DFS 最坏访问$size(B)$个节点。但若现在只拿到待删节点 B，想把它从全树接线中摘掉，还需找到父亲 A，以及 B 在 `A.children` 中的位置。没有父目录或外部给定位置，普通办法从根搜索，最坏$O(n)$。找到后，在数组孩子表中删去 B 还要移动后面的 C；在有已知位置的双向孩子链表中，摘掉可以常数改链。不能只写“孩子列表插删$O(1)$”而不说存储形式和已知句柄。

两种目录都保留时，接入 F 就要同时执行 `C.children` 加入 F、`F.parent=C`。删除 B 整棵子树时，先从 A 的孩子表摘 B，再沿 B 的孩子表处理 D、E，最后处理 B；所有权若要求逐个释放这 s 个对象，至少要$\Omega(s)$。

移动 B 到 C 下面则不同：D、E 不必逐个改 parent，因为它们的直接父亲仍是 B；只改 A、C 的孩子列表与 `B.parent`，并检查 C 不是 B 的后代，否则会把树接成环。

两侧目录的同步也可用同一图检查：把 B 移到 C 下面以后，父目录只有 B 由 A 改为 C，孩子目录由 `A:[B,C],C:[]`改为 `A:[C],C:[B]`，B 仍为`[D,E]`。若漏改任一侧，向上与向下遍历就会对“B 在哪里”给出相互矛盾的答案。

求 D 和 C 的最近公共祖先 LCA，已有 parent 时先算深度：D 深 2、C 深 1；把 D 移到 B，使两边同深 1。B 与 C 不同，再一起上移到 A，得到答案。两者每次都向根前进，时间$O(h+1)$。只有孩子列表时，可从根分别找到通往两点的路径 `A,B,D` 与 `A,C`，比较最后一个相同节点，得到 A；普通搜索最坏$O(n)$时间，保存路径需$O(h+1)$空间。若需要大量 LCA 查询，再考虑额外预处理，不能把它的成本省掉后用于本题的原结构比较。

26.4 4 · 叶子总比双分支多一个：同一条边数两遍

非空树 n 个节点有 n-1 条边。另一方面，每个单分支节点向下贡献 1 条，双分支贡献 2 条，所以

$$n_1 + 2n_2 = n_0 + n_1 + n_2 - 1.$$

消去 n1，得到 $n_0 = n_2 + 1$。再写 $e = n - 1 = n_1 + 2n_2$，可看见单分支越多，边数里越多是单链贡献。真二叉树 n1=0，于是 $n = 2n_2 + 1$、$n_0 = (n+1)/2$、$e = 2n_2$。完全二叉树可以有一个单孩子节点，不能不看结构就套真树公式。

例：根有两个孩子，其中左孩子还有一个孩子。n0=2、n1=1、n2=1，边 3=1+2，叶 2=双分支 1+1。手数一次，再对照公式，比直接背三个字母更稳。

26.5 5 · 长子—弟弟编码：一棵二叉树可能代表一片森林

约定左指针表示第一个孩子，右指针表示下一个兄弟。多叉树的根没有兄弟，因此它编码后的根右指针为空。任意二叉树若根有右孩子，按这套含义就代表根还有兄弟，更自然地对应森林，而非单棵树。

例如二叉根 A 的右指针指 B，左指针指 C，解码为两棵树：第一棵根 A、孩子 C，第二棵根 B。不能把 B 同时当 A 的普通孩子，那会混用左右指针的含义。

把整树根在某些接口里视为虚拟父节点的右孩子，是为统一“指向该子树的入口引用”的约定；它不让真实根凭空多一个父亲，也不意味着根在长子–弟弟编码里有真实的右兄弟。

26.6 6 · 有序森林与二叉树：正反变换都写出来才叫一一对应

森林各根按从左到右用右兄弟指针串起来；每个根的孩子列表，首孩子接左指针，后续孩子也用右指针串起来。所有节点只保留这两类指针，就得到二叉结构。

具体把森林 `A` 的孩子为`[B,C]`；`B` 的孩子为`[D]`和另一根 E 编码：

```
原有序森林：              二叉编码（左=首孩，右=弟弟）：
    A       E                    A --右--> E
   / \                          /
  B   C                        B --右--> C
 /                            /
D                            D
```

真实的原树边有 A→B、A→C、B→D；编码中的 B→C 却是兄弟边，不表示 B 是 C 父亲。解码 A 的孩子时，从 A.left=B 开始沿右链收集 B、C；解码 B 的孩子时，从 B.left=D 开始，只收 D。根 A 的右链给森林另一根 E，不应把 E 放进 A 的孩子列表。

解码时，从二叉根沿右指针拆出森林的根序列；每个节点的左指针给第一个孩子，再沿这条孩子的右链列出其余孩子，逐个递归。顺序没有丢，层级也由左指针与兄弟链的位置唯一决定。

正变换后立刻反变换会恢复每个原孩子列表，反过来也恢复每一条二叉指针，所以既不会两个森林编码成同一结构，也不会遗漏某个二叉结构。无序树若本来没有兄弟顺序，就不具备这项同样的一一对应前提。

26.7 7 · 原题要求的真树：每个空位置都要补，包括旧叶子的两边

以下先设原树非空，n=n0+n1+n2，n0=n2+1。原题要求新树双分支数 $2n_2 + n_1 + 1 = n$，叶数 $2n_2 + n_1 + 2 = n + 1$。正确转换是：把每个原节点都保留为内部节点，原来缺失的左、右孩子各补一个带标记的外部空叶。

原叶有两个空位置，要补两个；原单分支补一个；原双分支不用补。新外部叶总数 $2n_0 + n_1 = 2n_2 + n_1 + 2$；每个原节点如今都有两个孩子，新双分支数正是 n，单分支为 0。

例：只有一个原根 A。补后是 A 连两个外部叶，内部 1、外部 2。若只给单分支节点补另一边，A 一开始没有孩子，完全不补，这时仍只有一个叶，达不到题面要求的两个外部叶。只给单分支补孩子也是一种变成真树的方法，但它的节点数与本题指定的转换不同。

原树 A 只有左孩子 B 时，转换可以全部画出来。圆点 # 是带标记的外部叶，不是原数据节点：

```
原树                    扩充后
  A                        A
 /                        / \
B                        B   #
                        / \
                       #   #
```

原来有 2 个节点、1 条边、4 个孩子指针槽，其中 1 个非空、3 个空。一般 n 个原节点有 2n 个孩子槽，非空槽恰好对应 n−1 条边，所以空槽为$2n - (n - 1) = n + 1$。补每个空槽一个外叶，正好得到 n+1，不依赖原树是否有单分支。这是与$n_0 = n_2 + 1$推导独立的第二次核对。

保留外部叶标记，逆变换删除所有外部叶即可唯一恢复原树。标记不能与普通数据值混用，否则真实节点恰好也存“空”字样时会有歧义。

26.8 8 · 高度更新：某处没变，往上的高度也不会变

高度由两个孩子高度决定，$h(v) = 1 + \max(h_L, h_R)$。一次局部修改只影响一条祖先路径；若更新到某祖先 a，发现 h(a)与旧值相同，那么 a 的父亲看到的这个孩子高度没变，另一孩子又没改，因此父高也不变，往上同理。

插叶、删叶、接入整棵子树、分离子树都适用这个逻辑，但必须从真正受影响的入口向上，并在改完孩子链接后重算。高度不变不表示 size 没变；若还维护节点数、和或其他统计，不能一并提前停。

需要把其他缓存与高度分开：原节点 A 的左子树高度 2、右子树高度 0，所以 A.height=3。向右子树插一个叶，让右高变 1，A.height 仍为 3，可以停止高度传播；但 A.size 已经加 1，所有祖先的 size 也都应加 1。若把 `if(newHeight==oldHeight) break` 放在同时更新 height 和 size 的循环里，会少更新上层 size。提前停止只适用于已经证明不变的那一项统计。

例：根两边高度分别 3 和 1，在较短一边增加到 2，根高度仍 4，继续往更高处传播高度就是多余。最坏插在唯一最长链末端，高度一路变到根，优化仍 O(h)，只是某些修改能提前结束。

26.9 9 · 勤奋缓存与懒惰计算：用操作序列算总账

若每次求 height 都遍历子树，单次可能 O(n)；缓存 height 后查询 O(1)，但修改时需向上维护。size 也一样：即时维护使 size 查询常数，代价是每次接入、删除、旋转等都正确更新。

例如一天修改 100 次、查询一百万次，缓存常常合适；一天改一百万次、只查一次，按需遍历可能更简单。要把树高、子树规模和查询频率代入，而不是认为“勤奋一定更好”。

维护多个缓存还会增加一致性风险：旋转后必须先更新低节点，再更新高节点，使父读到的新孩子信息正确。测试除了最终值，还要在每次修改后用独立遍历结果核对缓存。

26.10 10 · 中序在单侧链上：空调用忽略后，看剩余工作

全右链中序先遇空左树，然后访问当前，再递归右树，右递归之后没有剩余访问工作，可按尾递归化。全左链则先递归左边，再返回访问当前，访问工作都积在返回路上，是头递归形状。

一般树左右都有非空分支，不能同时把两条路径都压成同一种单链循环。空子树调用仍有固定检查成本，讨论形状时可以忽略它，但实际计费不要说从未执行过。

以 A 右连 B 右连 C，中序就是 A、B、C，可直接循环；A 左连 B 左连 C，中序应为 C、B、A，要记住祖先或借助结构回溯，不能从根一路输出。

26.11 11 · 单链先序：只向前走，不需要 parent

每节点最多一个孩子时，先访问当前，再把 p 设为唯一的非空孩子，直到空。因为没有另一棵尚未处理的侧树，不需要返回祖先寻找分支。

时间 Θ(n)，辅助空间 O(1)，即使没有 parent 也完全可行。若递归版编译器没有尾调用优化，它会保留 n 层；显式循环才把空间保证写进实现。

这项简化依赖“最多一个孩子”。一般树中若先深入左边，右边还没处理，需要保存任务；删去 parent 并不会删去这些待办任务。

26.12 12 · size 访问器：只要不重不漏，哪种遍历都能数

定义 Count 含 value，visit 每到一个真实节点就加 1。先序、中序、后序、层序都访问每个节点一次，所以最终都为 n；空树为 0，外部空叶是否算节点必须按所求结构定义。

求所有 data 总和、检查每项是否在某范围，也可这样写。若要计算“每个节点子树大小”，单纯给全局计数器加 1 不够，需要把左右子结果汇回父节点，后序更自然。

函数对象按引用还是值传递也要核对，避免最后外部计数仍 0。遍历的抽象复用建立在任务真正只需要一次单节点访问的前提上。

26.13 13 · clone：中序、后序都能复制，但要保存结构位置

后序做法：先复制左子树得到 L，再复制右子树得到 R，然后新建根并接上 L、R。先序则先建根，再递归填两边。二者都能保持形状，且每个原节点对应一个独立新节点。

中序也可以先复制左边，保存结果 L，创建当前根，再复制右边并连接；不能因为“根在中间创建”就说不可能。要保存孩子身份，而非仅把中序值输出，再当作 BST 插入，那会改变原形状。

用一个最小节点类实现它。这里把 parent、height、size 一起维护，后面的重建也使用相同节点表示。`make_tree` 接上的两棵子树应当原本互不相交；它不是随意移动现有共享子图的通用接口。

```
from dataclasses import dataclass

@dataclass(eq=False)
class TreeNode:
    data: object
    left: object = None
    right: object = None
    parent: object = None
    height: int = 0
    size: int = 1

def make_tree(data, left=None, right=None):
    node = TreeNode(data, left, right)
    if left is not None:
        left.parent = node
    if right is not None:
        right.parent = node
    hl = left.height if left is not None else -1
    hr = right.height if right is not None else -1
    node.height = 1 + max(hl, hr)
    node.size = 1 + (left.size if left is not None else 0) \
                  + (right.size if right is not None else 0)
    return node

def clone(node, copy_data=lambda x: x):
    if node is None:
        return None
```

```
    left_copy = clone(node.left, copy_data)
    right_copy = clone(node.right, copy_data)
    return make_tree(copy_data(node.data), left_copy, right_copy)
```

左右递归都返回以后，两个孩子的 height、size 已经算好，当前节点才能据此更新。克隆树的根 parent 默认为空，接到更大的克隆子树时再由新父亲更新；不会从原树照抄 parent 地址。默认的数据处理适合整数、字符串等不需要另复制可变资源的标签；如果 data 里有可变列表，默认会共享那个列表，调用者应另传合适的 `copy_data`，例如深复制函数。树节点独立与节点内业务资源独立，是两层不同承诺。

对原树 A 左 B 右 C，先创建新 B'，再 C'，最后 A'，连接 A'.left=B'、A'.right=C'。根数据相同但地址不相同，且 B'、C'都不指回原 A。只复制中序 `B,A,C` 并逐项插入 BST，可能得到根 B、右 A、A 右 C，与原形状不同。复制值序列不能代替复制孩子位置。

无论哪种都需 Θ(n)新输出存储，递归栈最坏 O(h)。若构造中途失败，还要释放已经复制的部分或使用资源管理对象，避免只在完全成功的路径上没有泄漏。

26.14 14 · 打印树：缩进只说明深度，还需要说明哪条边

若每层缩进两格，只能看出节点深浅，不一定知道它是哪个父亲的左或右孩子。完整打印还需保存左右分支身份，以及哪些祖先层还有未打印的兄弟，用来决定竖线是否继续。

可以先打印右子树、再根、再左子树，让树向侧面展开；也可以先序打印带"L/R"的路径。选择一种后把图例写清，避免读者把上方节点误当左孩子。

先序带路径打印适合最早的自检。例如树 A 左 B 右 C，B 只有右孩子 D，应输出：

```
root: A
L:    B
L/R:  D
R:    C
```

`L/R` 表示从根先走左边、再走右边，所以 D 是 B 的右孩子。若只缩进，`D` 出现在深度 2，读者仍可能误以为 B 有左孩子。路径文字与图中空位置一起保留，就能验证分支方向。

用一棵左右高度不同、有单孩子且有空位的小树核对，比用满树更能暴露错误。打印是显示接口，不能通过交换真实孩子指针来"画得好看"而改变树。

26.15 15 · 清空不只允许后序，但绝不能释放后再读

后序先清左右，再删根，最容易保证仍能找到未处理节点。先序也可先保存 left、right，再 delete 根，再用保存的地址清左右；删除根后不能再读它的字段。

中序可先清左，保存右指针，再删根，然后清右。对象析构若会自动递归释放孩子，就不能同时再手工递归，否则双重释放；所有权策略必须统一。

各种顺序都能做到 Θ(n)，差别在于保留哪些地址、析构副作用和代码可验证性。正确性由生命周期与剩余对象可达性决定，不是某个遍历名字天然授予安全。

26.16 16 · 从 RPN 建树：栈里放的从数字变成子树

遇操作数就建叶节点入栈；遇二元运算符，先弹右子树 R，再弹左子树 L，新建运算符根并接上 L、R；遇一元后缀运算则只弹一棵树。

例如 `2 3 4 + *`：先有叶 2、3、4，+弹 4 和 3，形成根+的子树；再弹这棵右子树和叶 *2*，得到 *2*(3+4)。最后栈恰有一棵树，便是答案。

栈里每项是一整棵已经建好的子树，不能把"栈中多个节点"误连成一棵链：

读 token	操作	栈（底→顶；用括号表示子树）
2	建叶 2	[2]
3	建叶 3	[2,3]

读 token	操作	栈（底→顶；用括号表示子树）
4	建叶 4	`[2,3,4]`
+	弹右 4、左 3，建`+(3,4)`	`[2,+(3,4)]`
*	弹右`+(3,4)`、左 2，建`*(2,+(3,4))`	`[*(2,+(3,4))]`

归纳不变量是：已经消耗的 RPN 前缀被栈中这些互不相交的完整子表达式恰好分解。遇运算符合并的就是最近完成的几个子表达式，顺序由先左后右的后序决定。每个 token 创建一个节点、每条孩子边写一次，故时间线性，输出树也必须占线性空间。

执行前检查元数，结束检查剩余棵数；`2 +` 操作数不足，`2 3` 留下两棵，都不合法。固定 token 与元数后，树结构可唯一恢复；若同一符号有不明确的一元、二元含义，还需词法或语法信息。

26.17 17 · 沿右藤的“对称版”，会不会把先序变成根右左

直接访问根后一路深入右边、把左子树暂存，得到的是根–右–左，并非要求的根–左–右。若只是把 lc 与 rc 交换，必须同时承认遍历定义变了。

沿右藤分解时，可以把基本工作改成“访问藤上当前节点，完整处理它挂着的左子树，再沿右边走一步”。左子树仍必须先完成，不能把整条右藤预先访问完。直接写出来是：

```
def preorder_right_vine(node, visit):
    while node is not None:
        visit(node)
        preorder_right_vine(node.left, visit)
        node = node.right
```

while 负责沿当前右藤前进；递归负责侧面的整棵左子树。没有左孩子时，空调用立即返回；有左孩子时，调用结束前不访问当前节点的右孩子。这正好保持根–左–右。

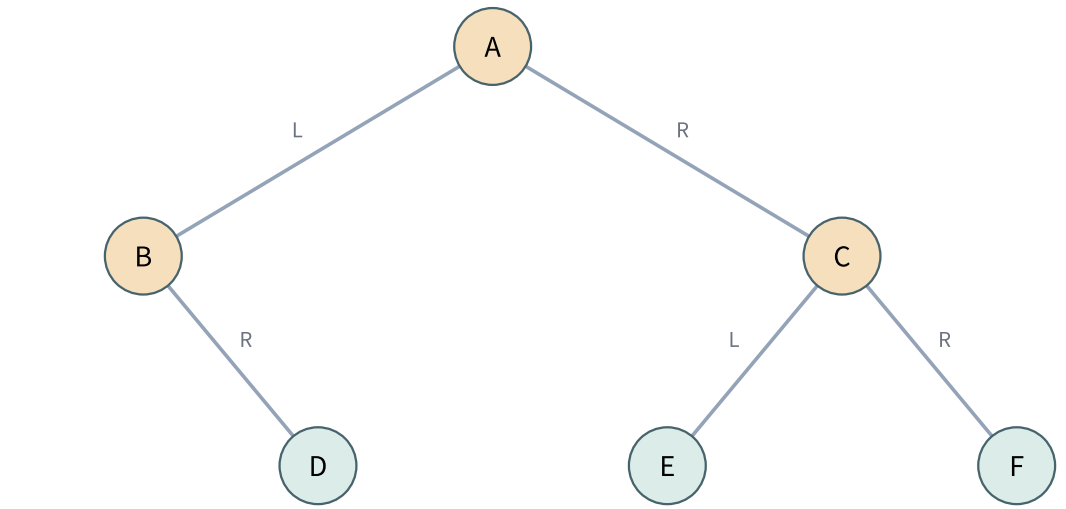


图 34　后面逐步走读的小树。L/R 标在真实边上，B 没有左孩子；图下四种访问次序可以先遮住，用来核对自己的栈和队列。

取 A 左 B、右 C，B 右 D，C 左 E、右 F。访问 A 后，先进入左子树 B；访问 B，左边空，沿 B 的右藤访问 D。整个 B 子树完成，才回到 A 那一层沿右藤到 C；访问 C，递归访问 E，再沿右藤访问 F。输出为 A、B、D、C、E、F。若先扫 A、C、F 再补 B、D、E，先序就被破坏了。

这个版本把右方向改成循环，递归栈只为尚未处理完的左侧调用保存返回位置。全右链只需常数层调用，全左链却要 Θ(n)层；时间仍 Θ(n)，一般辅助空间 O(h)。沿左藤的迭代版则显式保存待办右子树，两者保存的位置不同。递归版在长链上还可能超过语言的调用深度限制。

若要把上述左递归也改成显式栈，离开当前节点进入左边之前，就保存它尚未继续的右藤起点；左侧完成再恢复：

```
def preorder_right_vine_iter(root, visit):
    resume = []
    node = root
    while node is not None or resume:
        if node is None:
            node = resume.pop()
            continue
        visit(node)
        if node.left is not None:
            if node.right is not None:
                resume.append(node.right)
            node = node.left
        else:
            node = node.right
```

栈内每项都是“当前左侧工作完成后，接着处理这段右藤”的任务，最后存入者最先恢复。它与常见的“先压右、后压左”任务栈在访问顺序上等价，但这里不必把立即要走的左孩子也压栈。两种写法都没有更好的普遍时间阶；优缺点要比较具体树形上的待办数量、调用开销与实现清楚程度。

26.18 18 · 先序正确性：按整树规模归纳，侧枝也要包括

空树无输出；单节点访问一次正确。对更大树，沿左藤访问根及各左后代，并把每处右子树压栈。最深处的右侧任务最后压入，最先弹出，正好符合先做完当前左子树内部，再返回更高祖先处理右树的顺序。

每个被压的右子树规模小于原树，且彼此不重叠，归纳假设保证它们各自按先序完成。藤节点与这些侧树恰好覆盖原树，所以不漏、不重、次序正确。

若只归纳藤长度，外挂侧树可能比整条藤大得多，原归纳假设未必覆盖它们。可以增强命题，让它对任意合法侧树也成立；不能只把参数名字从“规模”换成“藤长”就算证明。

26.19 19 · 不压空右树：减少无效栈动作，但加了一次判断

原先无条件压右孩子，可能压入 nullptr；弹出后又立刻发现为空。改成右孩子非空才压，就消去这些无效 push 和 pop。

对全左链，若原版每层都压空右树，会积累很多空任务；去掉后待办栈可以显著缩小。对几乎每个节点都有右孩子的树，新增判断很少省动作。

总时间仍 Θ(n)，在不同编译器与数据下实际净收益需要测量。能严格说少做了哪些栈操作，不应直接保证所有机器上运行时间都按同一比例下降。

26.20 20 · 先序栈空间：高度是上界，待办右树数更贴切

沿左藤版栈保存还没访问的右子树根。当前路径每经过一个“走向左孩子、右边还有任务”的祖先，最多增加一个待办，因此栈长不超过相应左祖先数，进而不超过 O(h)。底部哨兵或当前节点另加常数。

全右链高度 n-1，几乎没有左祖先，待办栈可常数；全左链若不压空右树，也可以常数。若长左链每个祖先还挂一个真实右叶，走到底时会同时积累约 h 个右任务，达到高度阶。

原题 c、d 要求构造极端树，不只报一个$O(h)$。若只统计非空右任务：全右链长 n，高度 n−1，最大左祖先数 0，显式待办栈峰值 0（连当前指针一起计也只多常数）；高度上界可比实际大线性倍。要让左祖先数与高度相同，可取每一步都走左的深链；若还要求实际栈任务达到高度阶，就在左链每个非末节点挂一个右叶，下降时分别留下这些右叶，走到底同时有 h 个待办。

```
      A
     / \
    B   a
   / \
  C   b
 / \
D   c
```

访问 A、B、C、D 时，栈中的非空右任务依次为[a]、[a,b]、[a,b,c]、[a,b,c]，栈顶任务 c 最先恢复。树高 3，待办峰值 3。左链若完全不挂右叶，左祖先数仍大，实际非空任务却为 0，说明这个数量是上界而非每棵树的精确等式。

题目问两个上界之比的数学期望，必须说明树形与选点分布。左右方向独立对称的一条固定路径，其左转次数期望是路径长一半；但“全树最大左祖先数/全树高度”的期望不是因此自动一半。max 与期望以及比值都不能随意交换。没有随机模型，不能给出普遍数字。

26.21 21 · 中序归纳：左藤只是拆解入口，不是全部规模

中序先沿左藤把祖先压栈，直到最左节点；弹出当前访问，再把其右子树按同样方式展开。整棵树规模归纳可以直接覆盖左右两个更小子树。

对文中的弯曲树，完整中序栈过程是：

```
A左B，B右C，C左D。
沿左压A、B，栈[A,B]，当前到空。
弹B并访问，输出B；转B.right=C。
沿左压C、D，栈[A,C,D]，当前到空。
弹D访问，输出B,D；D.right为空。
弹C访问，输出B,D,C；C.right为空。
弹A访问，输出B,D,C,A；结束。
```

每次弹出的节点，其左子树已经完成，自己访问后再转右子树。等待中的 A 不会在 B 结束时被马上访问，因为 B 的右子树 C 已经被安排在 A 之前。这就是栈的后进先出与中序要求相符的具体位置。

若以左藤长度归纳，需要把“藤上每个节点挂着任意右子树时，算法仍正确处理它们”写进命题，并独立保证这些侧树的正确性。只讨论藤本身的访问顺序不足以证明整棵树。

用根 A、左 B、B 右 C、C 左 D 的小树测试：中序应 B、D、C、A。它很容易揭穿只按一条直链推理、漏掉弯曲侧枝的证明。

26.22 22 · 最后一个中序节点：一路向右后，再向上越过根

中序最后访问整树最右节点。它没有右子树，否则右子树中还有更晚节点。求 succ 时，若有右子树应取其最左节点；没有就向上，直到第一次从某祖先左侧返回，该祖先才是后继。

对于整树最右节点，向上始终处在祖先右侧，最终越过根得到 nullptr。因此算法应在访问 parent 前判空，不能再从根去读不存在的父节点。

最小树只有一个根，它也是最后节点，succ 直接为空。测试这个例子能检查代码是否把“根无父亲”的边界遗漏。

26.23 23 · 没有 parent，也能后序：自己保存返回阶段

栈保存(node,stage)。stage=0 时准备处理左边，改成 1 后压左孩子；stage=1 时左边已完成，改成 2 后压右孩子；stage=2 时左右都完成，访问当前并弹出。

```
def postorder(root, visit):
    stack = [(root, 0)] if root is not None else []
    while stack:
        node, stage = stack.pop()
        if stage == 0:
            stack.append((node, 1))
            if node.left is not None:
                stack.append((node.left, 0))
        elif stage == 1:
            stack.append((node, 2))
            if node.right is not None:
                stack.append((node.right, 0))
        else:
            visit(node)
```

以 A 左 B 右 C 走程序：栈先`[(A,0)]`；处理阶段 0 后变`[(A,1),(B,0)]`；B 没有孩子，依次改为 B 的阶段 1、2，然后访问 B 并弹掉；恢复 A 阶段 1，改存 A 阶段 2 并压 C；C 同样完成三个阶段、访问 C；最后弹 A 阶段 2 并访问。输出 B、C、A。每个父帧在压孩子以前已改好下一阶段，所以孩子返回后不会再次从阶段 0 开始。

这相当于把语言调用帧的程序位置显式存出来，时间 Θ(n)、辅助空间 O(h)。parent 指针是获得返回方向的一种办法，不是唯一办法。

26.24 24 · 后序正确性：访问条件就是“两棵子树已完成”

对每个帧，阶段只按 0→1→2 向前推进。进入阶段 1 前左子树任务已执行完，进入阶段 2 前右子树也已执行完；因此访问根时，恰好符合左–右–根。

每个非空孩子只由唯一父帧压一次，树没有共享孩子或环，所以不会重复。空孩子直接跳过，不影响阶段含义。按子树规模归纳，两个子任务各自输出正确，接上当前根便正确。

如果输入其实是共享子图或有环，这份树遍历可能重复或不结束，需要 visited 和另一种语义。先把“是合法树”写成前提，证明才不会偷换对象。

26.25 25 · 迭代遍历总时间：一次下藤走 h 步，不能给每个节点都收 h 步

先序每个节点访问一次，其右任务至多压弹各一次；中序每个节点压一次、弹一次；阶段式后序每个节点只经历固定三阶段。合计都 Θ(n)。

内层 while 偶尔走很长一条藤，但这条藤上的节点随后不会在别处再次作为新的未访问节点下藤。把所有内循环迭代相加是 n 量级，不能看到嵌套循环就相乘。

若算法每次求 succ 都从根重新搜索，或每次重算整棵子树信息，已经不是这里被分析的迭代版本，可能增加成本。复杂度结论应绑定到真实的节点进出次数。

26.26 26 · 层序遍历：队列里为什么只会混着相邻两层

先把根入队。每次弹一个节点 v 访问，再按左后右把其孩子入队，孩子深度仅比 v 大 1。FIFO 保证当前层尚未处理节点在前，刚发现的下一层孩子排在后，所以队列深度非降，最多差 1。

归纳可知根先到，每个非根节点恰由唯一父亲发现一次。每个节点入队一次、出队一次，因此总 Θ(n)。同层左先右后来自孩子入队顺序；若调换左右，深度顺序仍正确，但同层阅读方向会变。

用原题下一问的 5 节点例列全队列：根 R 左 A 右 B，A 左 C 右 D。

本次弹出访问	新加入的孩子	操作后的队列（左为队首）	深度序列
尚未开始	先放 R	`[R]`	`[0]`
R	A、B	`[A,B]`	`[1,1]`
A	C、D	`[B,C,D]`	`[1,2,2]`
B	无	`[C,D]`	`[2,2]`

本次弹出访问	新加入的孩子	操作后的队列（左为队首）	深度序列
C	无	[D]	[2]
D	无	[]	[]

处理 A 以后当前层还剩 B，下一层 C、D 已经到尾部，两层同时存在；但深度差从不超过 1。每一轮一定弹 1 个，加入可以是 0、1、2 个，所以队列长度并不单调。

“左边更早”只能对同层按该有序树定义解释。一个很浅的右节点会早于深处左节点，不能把全树横向位置当成绝对处理次序。

26.27 27 · 层序空间：队列可能比某一层还稍宽

设最大层宽 w。队列可能同时包含当前层剩余部分与下一层已发现部分，故不一定恰好≤w，但最多是相邻两层的总宽，仍 O(w)。单链每次队列最多 1，完全树可线性大。

更精确地，队列每个节点代表一棵互不相交的尚未处理子树，每棵至少包含一个不同叶子，所以队列长度不超过叶子总数。非空二叉树叶子最多 $\lfloor (n+1)/2 \rfloor = \lceil n/2 \rceil$，完全二叉树可以达到这个数量级，且这种 BFS 队列也能达到该上界。

例如 n=5 完全树，根孩子 A、B，A 有 C、D。处理 A 以后队列为 B、C、D，长度 3；最大单层宽却只有 2。这一小例解释了为何空间 Θ(w)不表示每时每刻精确等于某层宽。

26.28 28 · 完全树放进数组：父子下标怎么算

零基存储时，节点 i 的左孩子 2i+1、右孩子 2i+2；i>0 的父亲为 $\lfloor (i-1)/2 \rfloor$。存在与否只需检查孩子下标是否小于 n，不能先访问再判断。

根 0，下一层 1、2，再下一层 3、4、5、6。这些区间连续，因此层序扫描局部性很好；沿一条深路径，下标却按倍增跳跃，不能保证每对父子都物理紧挨。

完全性保证中间无空洞；任意稀疏树硬放这种数组，深单链可能需要指数大的空位置。适用条件与缓存收益都要按访问模式说明。

26.29 29 · 真树先后序重建：第二个先序项给出左子树范围

假设标签互异，且给的是同一棵真二叉树。先序首项和后序末项必须是同一个根；若不止一个节点，先序第二项就是左子树根。到后序中找它，可得到整个左子树的末端与长度，从而把两个序列都切成左右两块递归。

例如先序 A,B,D,E,C，后序 D,E,B,C,A。根 A，左根 B 在后序下标 2，左块长度 3；递归左块得到 B 的孩子 D、E，右块只有 C。真树保证根不可能只有一支，消除了“这段到底左子树还是右子树”的歧义。

把两次递归切分写得更具体。下标采用从 0 开始的半开区间：

```
pre = [A,B,D,E,C]       post = [D,E,B,C,A]
根A: pre[0] == post[4]
左根B在post[2]，左子树长度=2-0+1=3
左: pre[1:4]=[B,D,E]，post[0:3]=[D,E,B]
右: pre[4:5]=[C]，     post[3:4]=[C]
左根B: 其左根D在左post的第0位，左长度1
于是B.left=D，B.right=E；C是叶。
```

得到的形状是：

```
      A
     / \
    B   C
   / \
  D   E
```

若不是叶，真树要求左右两块都非空，且每块节点数为奇数；例如只剩 2 项却试图构造真树，应报告输入不合法。保留四个边界下标而不反复切片复制，才能达到线性总工作；用 Python 切片直接实现会额外复制子数组，不能仍把这些复制当免费。

用标签到后序位置的字典可使定位期望常数，总体 O(n)；若每层线性找标签，退化树形可二次。还要验证左右块均合法、根标签一致、标签集合一致，不能把不一致序列强行拼成树。

下面的完整实现不复制子数组，递归参数分别指定先序和后序的半开边界。标签要求可哈希且互异，用字典定位后序位置。它先核对集合，递归中再核对对应的根和左右块；集合一样只是必要条件，并不自动保证遍历相容。

```
def full_tree_from_pre_post(pre, post):
    n = len(pre)
    if len(post) != n or len(set(pre)) != n \
            or len(set(post)) != n or set(pre) != set(post):
        raise ValueError("两序列须含相同的互异标签")
    position = {label: i for i, label in enumerate(post)}

    def build(pl, pr, ql, qr):
        size = pr - pl
        if size != qr - ql:
            raise ValueError("对应子树的节点数不同")
        if size == 0:
            return None
        if pre[pl] != post[qr - 1]:
            raise ValueError("先序首项与后序末项不是同一根")
        if size == 1:
            return make_tree(pre[pl])
        if size % 2 == 0:
            raise ValueError("非空真二叉树节点数必须为奇数")
        left_end = position[pre[pl + 1]]
        if not ql <= left_end < qr - 1:
            raise ValueError("左根不在当前后序块内")
        left_size = left_end - ql + 1
        right_size = size - 1 - left_size
        if left_size % 2 == 0 or right_size <= 0 or right_size % 2 == 0:
            raise ValueError("内部节点必须有两棵非空真子树")
        left = build(pl + 1, pl + 1 + left_size, ql, left_end + 1)
        right = build(pl + 1 + left_size, pr, left_end + 1, qr - 1)
        return make_tree(pre[pl], left, right)

    return build(0, n, 0, n)
```

对 `pre=ABDEC, post=DEBCA`，第一次参数是`(0,5,0,5)`；字典找到 B 在后序 2，切成`(1,4,0,3)`与`(4,5,3,4)`，恰对应上图。单标签是叶；两份空序列约定恢复空树；`pre=AB,post=BA` 虽然能表示单孩子树，却不属于本接口的真树范围。又如 `pre=ABCDE,post=DBECA`，标签集合相同、根也同为 A，但 B 在后序 1 导致左块大小 2，立即拒绝，不能制造一棵不符合真树要求的结果。

在字典期望常数定位模型下，预处理和递归总计$O(n)$时间，新树$\Theta(n)$存储、调用栈$O(h+1)$。最后还可以独立遍历新树，把先后序与输入逐项比较，并检查每个内部节点恰有两个孩子、每条孩子的 parent 确实指回父亲；这些是比“能画出一棵树”更完整的核对。

26.30 30 · 增强遍历：空标记能救先序、后序，救不了中序

先序读一个 token，若是 # 返回空；否则建根，再依次读取左、右子树。对 `A B # # C # #`，先建 A，左建 B 并读两个空，右建 C 并读两个空，边界被递归结构唯一确定。

增强先序的反向恢复不用猜哪个子树在哪结束：读到 A 要继续恢复两个孩子；读到 B 也要继续恢复两个孩子；两个 # 分别让 B.left、B.right 为空，返回 B；再读 C，读完它的两个 # 返回 C；最后 A 的两个孩子都齐，返回 A。递归中的“还欠几个孩子”就是边界信息。

```
读取后：A            读取B,#,#后：  A       全部完成：  A
                                  /                 / \
                                 B                 B   C
```

反读增强后序时规则对称：最后一个真实 token 是根，接着最先读到的是右子树的末根，所以必须先建右、后建左。先后顺序弄反会得到镜像树，值集合一样，结构却错。

后序从末尾反读，遇根后先恢复右、再恢复左，同样唯一。每个真实节点和空位置读一次，时间线性，还要检查最后有没有多余 token。

中序增强不唯一：A 只有右孩子 B，与 B 只有左孩子 A，都会给 `# A # B #`。空标记本身不自动记录根在哪，所以“所有增强序列都能唯一重建”是错误推广。

增强先序的读取规则也能写成真正有错误出口的解析器。下面用 None 作空 token，因此真实数据不允许等于 None；数据中若要使用它，需另选带类型的标记。

```python
def tree_from_augmented_preorder(tokens):
    cursor = iter(tokens)

    def read():
        try:
            token = next(cursor)
        except StopIteration:
            raise ValueError("序列提前结束，仍欠孩子") from None
        if token is None:
            return None
        left = read()
        right = read()
        return make_tree(token, left, right)

    root = read()
    try:
        next(cursor)
    except StopIteration:
        return root
    raise ValueError("树已经完整，后面仍有多余token")
```

`['A','B',None,None,'C',None,None]`恢复上面的三节点树。若删去末尾 None，递归到 C 的右孩子时发现还欠一个 token 而报错；若尾部再加'D'，整树读取后发现多余 token 而报错。空树必须显式写成`[None]`，完全空的 token 流没有给出那棵空树的表示，因而属于截断输入。这个区别使解析器能报告文件损坏，避免把缺失数据默默解释成空孩子。

26.31 31 · 组合重建：每个“唯一”都带着前提

互异标签下，先序给根、中序给左右分界，故先+中唯一；后+中同理。普通先+后对单孩子方向无解：A 左 B 和 A 右 B 的先序都 A,B，后序都 B,A。

加上真树条件可排除单孩子歧义，如前题。层序加中序也能逐层选出各区间最早出现的根来重建，但单独层序只有访问顺序，不能决定哪些边存在。

层序+中序也走一个例：层序`[A,B,C,D,E]`、中序`[D,B,E,A,C]`。层序最早的 A 为根，中序把左边切成 D、B、E，右边为 C；在左集合中，层序最早出现的是 B，故左根 B，再由中序分出左 D 右 E；右边只有 C。得到与前题同一棵树。朴素每层筛一遍层序可能$O(n^2)$，唯一性成立不代表任意构造实现都已经线性。

标签重复时，连先+中也可能不唯一，必须为不同节点加身份或另外约束。先检查输入条件，再选重建方法，不能只拿两个遍历名称就保证有解且唯一。

26.32 32 · 交换子树：成本差由总权与深度差决定

两棵互不为祖先后代的子树根深 d1、d2，总权 W1、W2。交换后各叶内部相对深度不变，第一棵整体移动 d2-d1 层，第二棵反向移动，因此

$$\Delta C = (W_1 - W_2)(d_2 - d_1).$$

若较大权 W1 原来更深，即 W1>W2 且 d1>d2，则差为负，成本下降。权相等或深度相等只保证不变，不能说严格下降。

把差分逐项展开。交换前这两棵子树的每个叶深都可写成“子树内部深度+子树根深度”。内部深度不变，其对应成本会完全抵消；只剩根深度项：

$$C_{后} - C_{前} = W_1 d_2 + W_2 d_1 - W_1 d_1 - W_2 d_2 = (W_1 - W_2)(d_2 - d_1).$$

W_1是第一棵子树所有叶权之和，不是子树根原来的 data；d_1是交换前该根距整树根的边数。若把单个叶权与整棵子树总权混用，计算会少掉内部其他叶整体移动的贡献。

例 W1=10、W2=3，深度 3 与 1，交换差(10-3)(1-3)=-14。若一棵包含另一棵，直接交换会破坏树，不能把同一公式不加结构限制地套用。

26.33 33 · Huffman 最优：贪心选择要能嵌进某棵最优树

先取最小两权 a、b。在一棵最优真编码树里选最深的一对兄弟叶，把 a、b 交换到这两个位置。最小权放得更深不会增加成本，故存在一棵最优树让它们为兄弟。

把这对兄弟缩成权 a+b 的叶，原树成本恰等于缩小树成本再加 a+b。若缩小树不是最优，换成更优的小树再展开，就会改进原最优树，矛盾。因此在缩小问题上继续做同样贪心，按叶数归纳得到最优。

交换论证中，“最小权放深处”可以直接算：若$a \le w$而 a 原在深度$d_a \le D$，把它与深度 D、权 w 的叶交换，成本变化为$(a-w)(D-d_a) \le 0$。先放 a，再放 b；最深的两个兄弟位置深度相同，第二次不会增加已放好的 a 的成本。最深叶的兄弟必须也是叶，否则兄弟的后代更深，矛盾。于是找到的是**一棵同样最优且具有贪心第一步的树**，不需要断言所有最优树都只有同一个形状。

缩成一叶时，原 a、b 比合并叶都多 1 层，所以展开额外成本恰好是$a+b$。这也是“每次合并和的总和等于最后 WPL”的来由：每次合并为其子树的每个叶深加 1，累计到叶上就是权重乘最终深度。

对[2,3,7,8]，先合 2+3=5，再 5+7=12，再 12+8=20，WPL 为 5+12+20=37。展开树的叶深分别 3、3、2、1，直接算 2×3+3×3+7×2+8×1 也为 37。两个算法得到不同左右码字不一定有错，最优编码树可能不唯一。

26.34 34 · 框架与森林容器：换容器要重新带入成本

核心框架只有：取两个最小根、合并为新根、把新根放回，直到剩一棵树。它依赖优先队列能力，不必知道底层是无序数组、堆还是两个有序来源。

无序数组每轮线性找最小，n 轮可二次；小根堆每次取最小、插入均 O(log n)，总 O(nlog n)。初始建堆可以线性，但后面 n-1 轮合并仍在。

接口隔离让框架可复用；复杂度分析却不能对不同容器照抄一个数字。源码中找到森林类型、取最小和插回的位置，再把每项成本相加。

26.35 35 · 双队列 Huffman：线性的是已经有序之后的合并阶段

原始权重升序放在一个来源，新合并的权放第二个队列。每次只比较两个队首，从中取全局最小的两个。新合并权按生成顺序不减：下一轮取到的两个可用最小值，不会产生比上次合并和更小的新和。

原输入[1,2,4,8]：取 1、2 得 3，合并队列[3]；取 3、4 得 7；取 7、8 得 15。每个原项与合并项进出常数次，有序合并阶段 O(n)。

每次取出两项时都重新比较两个来源的队首，而不是先在一个来源拿齐两项。例如原权[1,2,4,8]：

轮	轮初原队列	轮初合并队列	依次取两最小	轮后合并队列
1	[1,2,4,8]	[]	1、2	[3]
2	[4,8]	[3]	3 来自合并队、4 来自原队	[7]
3	[8]	[7]	7、8	[15]

有序性的严格理由是：设本轮取$x \le y$，此前所有未取项都≥y；插回$x+y$也≥y，因为权非负。下一轮取的两个数都≥y，所以新和至少$2y \ge x+y$。因此合并队列尾部追加仍非降。若允许负权，这一步就失去依据，题目的非负频数条件不能省。

用栈存原权时，应让最小项在栈顶，即压入方向相反；换成升序队列也一样。若原权未排序，前面的排序一般 O(nlog n)，因此原题总复杂度仍 O(nlog n)，不能把后半线性当无条件总线性。

26.36 36 · 平均比较排序下界：少数短路径救不了所有排列

只讨论通过二元大小比较决定控制流程、对每个输入都正确的排序算法。互异 n 项共有$n!$个等概率排列，每个排列必须落在足以确定其顺序的决策叶上。令$N = n!$，叶i的深度d_i就是这个输入的比较次数，所有输入的平均深度为$\bar{d} = (d_1 + \cdots + d_N)/N$。只知道某片叶很深，只能约束最坏情况；这里必须把 N 片叶一起计算。

先看二叉决策树给叶子分配了多少可区分的空间：给二叉决策树根一个长度 1 的区间，每比较一次把当前区间平分给两个孩子。深度d_i的叶占长度2^{-d_i}，不同叶区间不相交，所以总长≤1。设$N = n!$，这 N 个长度的几何平均不超过算术平均，于是

$$\left(\prod_{i=1}^{N} 2^{-d_i}\right)^{1/N} = 2^{-(d_1+\cdots+d_N)/N} \le \frac{\sum_i 2^{-d_i}}{N} \le \frac{1}{N}.$$

两边取以 2 为底的对数再改符号，便得到平均深度$\bar{d} \ge \log_2 N$。叶区间总长不超过 1 的性质常称为 Kraft 界；上面的几何平均不超过算术平均，也等价于对凸函数2^{-x}使用平均值不等式。先有具体叶区间，再看这些术语，证明中就没有凭空多出的一条定理。最深叶只约束一个输入；这一平均数不等式把所有叶都算进来，因此才能回答本题。

也就是说，哪怕某些输入一眼就能认出，其他分支必须付出足够比较，均匀平均仍不能突破此界。若输入分布高度集中，平均下界应该用其信息量重新描述；若采用整数桶、位运算等超出比较模型的操作，也不能把这条下界照搬过去。

随机算法还需说明平均是在输入随机、内部随机，还是两者共同取期望。证明固定比较树的均匀平均后，对内部随机选择的树再平均，仍能得到相应下界。

26.37 37 · 编码构造下界：限定比较模型，再构造足够多的输出

这里讨论任意未排序正实权重、仅通过比较区分大小、输出带原字符标签的最优树。已排序权重的线性合并没有违反下界，因为排序信息已经作为输入给出了；受限整数模型也可能有其他手段。

可取一组增长极快的固定权重，后一权重远大于前面所有权重之和的 n 倍，再把字符标签任意置换。这时最大权必须在深度 1：让它多一层的损失，大于其他所有叶在至多 n-1 层内调整所能补偿的总量。移去最大叶后对剩余子树重复，最优树被迫呈梳状，依次用深度暴露各权重名次，只有最小两叶可互换。

例如权重 1、10、100、10000，四叶的梳形最优深度为 3、3、2、1。10000 若被放到深度 2，光它一叶就多付 10000；把它换到深度 1 后，其余权总和 111 即使多移动最多 3 层，也补不了这个损失。接着在剩下 3 叶里，100 也被迫比 1、10 更浅，最后 1、10 同深可以互换。把四个字符标签任意分配到这四个权重，树的叶深至少区分$4!/2 = 12$种名次信息。

一般构造要求每个更大权W大于n倍所有较小权之和 S。若最大权不在深度 1，成本至少$2W$；而存在一棵把它放深度 1、其余叶深不超过 n−1 的候选树，成本至多$W + (n-1)S < 2W$，矛盾。去掉这个最大叶后，另一侧子树必须也是剩余权的最优树，再递归使用相同条件。这个候选成本比较补上了“增长足够快为何强迫梳形”的必要一步。

所以至少有 n!/2 类必须区分的带标签输出，二叉比较树需 $\log_2(n!/2) = \Omega(n \log n)$ 深度。也可把问题看成：构造后在线性时间读出深度顺序，便得到这个受限排序族的排序信息。

Huffman 树是最优树的一种，因此同一组强分离输入也约束其构造。不能把“任意树的任意层序遍历一定按所有叶权排序”当成无条件事实；左右孩子可交换，同层叶也可能需要额外安排。下界应建立在明确可区分的信息上。

26.38 本章练习与核对

1. 原树仅 A 左孩子 B。扩充全部空位置后，内部、外部叶各几项？若只给 A 补右孩子，会漏什么？
2. 给先序 A,B 和后序 B,A，画出两棵可能树；加什么条件才能排除这种歧义？
3. 权重 1、2、4、8 做双队列 Huffman，每轮取哪两项，最终 WPL 多少？

第一题：原节点 n=2，扩后内部 2、外部叶 3；A 右边补 1 个，B 左右各补 1 个。仅给 A 补右边会让 B 仍是普通叶，达不到原题“每个原节点成为内部”的计数。第二题：A 左 B 与 A 右 B 均符合；要求真二叉树后，这个双节点输入本身不合法，并非随便选一棵就解决。第三题：1+2=3，3+4=7，7+8=15，WPL=3+7+15=25；叶深 3、3、2、1，直接 1×3+2×3+4×2+8=25 核对。

28 把证明里省掉的那一步写出来

读到“平均对数”“势能抵消”“按归纳假设”，很容易把结论记住，却不知道自己能不能重做。本章从实际状态和小样本开始：先说明数的是什么，再把符号放上去。前半的摊还与期望适合直接读；“全程峰值”一节用到指数、尾概率，可以在理解固定时刻的例子以后再进入。

28.1 摊还分析：先做完一段操作，再摊它的总账

摊还不需要掷硬币，也不假设输入随机。它要证明：无论合法操作怎样安排，一整段的总工作量都受控制。某一次很贵完全可以。

双栈队列有 in 和 out 两只栈，均从底到顶书写。in 接新项，出队只弹 out；out 空时才把 in 倒过去。把一次基本 push 或 pop 各计 1，空检查等固定开销暂时单列：

操作	in	out	实际成本
初始	[]	[]	—
入 A	[A]	[]	1
入 B	[A,B]	[]	1
出 A	[]	[B]	5：弹 B 压 B、弹 A 压 A、再弹 A
出 B	[]	[]	1

总成本 8，第三次单独花 5。每个元素一生最多做 4 个栈动作：进 in、离 in、进 out、离 out。所以 m 次合法队列操作总成本$O(m)$。元素如果还没出队，实际工作更少；即使预先给它收齐 4 份费用，也仍是上界。

28.1.1 为什么要人为定义“势能”

前两次便宜的入队，可以替第三次倒栈预留费用。给一个状态定义

$$\Phi = 2|\text{in}|.$$

竖线表示 in 中的元素个数，前面的 2 表示每个尚未倒过去的元素，还需要一次 pop 和一次 push。Φ是待付工作的记账量，不是额外真的执行这些动作。

设第i次操作花的实际成本为c_i，操作后势能为Φ_i，操作前为Φ_{i-1}。定义第i次摊还成本

$$\hat{c}_i = c_i + (\Phi_i - \Phi_{i-1}).$$

势能增加，表示这次除支付实际工作，还存下未来要用的费用；势能减少，表示这次从已有存款里支付了一部分。对上面 4 步，逐列算：

操作	c_i	操作前后Φ	势能变化	$\hat{c}_i$
入 A	1	0⟶2	+2	3
入 B	1	2⟶4	+2	3
出 A	5	4⟶0	−4	1
出 B	1	0⟶0	0	1

实际合计 8，摊还合计也 8。若做完“入 A、入 B”就停止，实际只花 2，摊还收 6，还剩势能 4；收得多一些不会破坏上界。

把 m 步的定义逐个相加，可以亲眼看到抵消：

$$\begin{aligned}\hat{c}_1 + \hat{c}_2 + \cdots + \hat{c}_m &= c_1 + c_2 + \cdots + c_m \\ &\quad + (\Phi_1 - \Phi_0) + (\Phi_2 - \Phi_1) + \cdots + (\Phi_m - \Phi_{m-1}) \\ &= \sum_{i=1}^{m} c_i + \Phi_m - \Phi_0.\end{aligned}$$

每个中间Φ_i有一次加、一次减，只剩初态与末态。若初态为 0、所有状态势能非负，实际总成本就不超过摊还总成本。如果允许势能跌到任意大的负数，它就像无限透支账户，不能用小额收费控制实际账。

把 3 类队列操作分别代入也能普遍核验：入队$c=1,\Delta\Phi=2$，摊还 3；out 非空出队$c=1,\Delta\Phi=0$，摊还 1；out 空、in 有 k 项时，$c=2k+1,\Delta\Phi=-2k$，摊还仍 1。这里 k 再大也消掉了，所以总界对任何合法序列成立。

这种从二进制计数器出发、依次用总和、记账、势能解释同一成本的组织，可对照Jeff Erickson Amortized Analysis，第 1–5 页。本章改用本课双栈队列和下面的向量状态计算，便于直接接上课程实现。

28.1.2 向量加倍：一个完整的非负势能

只考虑尾部追加，初始容量 1、规模 0，满时容量翻倍。令 n 是当前有效元素数、C 是当前容量，取

$$\Phi(n,C)=\max(0,2n-C).$$

这个定义在空表也非负。非扩容追加只写新项，成本 1；势能最多增加 2，摊还至多 3。扩容前 n=C，势能 C；搬 C 项、写 1 项，成本 C+1；扩容后规模 C+1、容量 2C，新势能 2。因此摊还为$(C+1)+(2-C)=3$。

第几次追加	追加后 n,C	本次实际写入	操作后势能	本次摊还
0	0,1	—	0	—
1	1,1	1	1	2
2	2,2	2	2	3
3	3,4	3	2	3
4	4,4	1	4	3
5	5,8	5	2	3

前 5 次实际 12，摊还 14，差 2 正好等于末态势能。势能函数不是猜出后便自动正确，必须检查初态、便宜操作、昂贵操作和下界。若允许头插、批量插入、缩容，这个只分析尾追加的证明不能直接覆盖那些新操作。

28.2 概率与期望：先列“可能发生什么”

随机变量是给每个可能结果算出的一个数。公平硬币抛两次有 4 个等可能结果：正正、正反、反正、反反。令 X 为正面数，四个结果对应 2、1、1、0，所以

$$E[X]=(2+1+1+0)/4=1.$$

E读作期望，意思是按各结果的概率加权平均，不是“最可能发生的值”，也不保证某次结果正好等于它。一般写成

$$E[X]=\sum_{x} x\,P(X=x),$$

即把 X 可能取的每个值 x 乘出现概率，再加起来。上例也可按数值合并：X=0 概率 1/4，X=1 概率 1/2，X=2 概率 1/4，得到$0/4+1/2+2/4=1$。

28.2.1 指示变量为什么省事，为什么不用先证明独立

令I_1表示“第一次正面”，发生记 1，否则 0；I_2同理。逐个样本上都成立$X=I_1+I_2$。对于任一 0/1 变量 I，

$$E[I]=0\cdot P(I=0)+1\cdot P(I=1)=P(I=1).$$

把四个样本与指标放在一起，再逐列取平均：

样本 ω	概率	I_1	I_2	$X=I_1+I_2$
正正	1/4	1	1	2
正反	1/4	1	0	1
反正	1/4	0	1	1

样本 ω	概率	I_1	I_2	$X=I_1+I_2$
反反	1/4	0	0	0

最后一列平均为 1；I_1、I_2两列各平均 1/2。一般有限样本空间中，对每个样本 ω 的概率记为p_ω，不要求这些概率相等。逐个样本都有$X(\omega)=I_1(\omega)+I_2(\omega)$，于是

$$\begin{aligned}E[X] &= \sum_{\omega} p_\omega \left(I_1(\omega)+I_2(\omega)\right)\\ &= \sum_{\omega} p_\omega I_1(\omega) + \sum_{\omega} p_\omega I_2(\omega)\\ &= E[I_1]+E[I_2].\end{aligned}$$

这只是将每个样本贡献里的加法展开，再把第一列、第二列各自汇总，没有用联合概率等于边缘概率乘积，因此不需要独立。这条性质称为线性期望，对有限个可积随机变量同样成立。若涉及无穷多个非负项，可先加前 m 项再令 m 增加；若正负无穷项混合，期望可能根本不存在，不能不加条件地交换和式。
相关的例子更能说明这一点：公平掷一颗六面骰，X 为顶面点数，Y 为底面点数，故 Y=7−X。它们显然相关，X 变大时 Y 一定变小；但$E[X+Y]=7=3.5+3.5$仍成立。不能因为“可以把期望相加”就误断独立。
乘积则没有同样的无条件拆分规则。独立是保证$E[XY]=E[X]E[Y]$的充分条件；只知道两变量的各自平均，通常不够。顶底骰子这里，$E[XY]=(6+10+12+12+10+6)/6=28/3$，而$E[X]E[Y]=49/4$，并不相等。反过来，即使某组相关变量的这两个数碰巧相等，也不能仅凭这一条等式推断独立；独立要求整个联合分布满足乘积关系。
这些基础定义与模型可对照 Jeff Erickson Discrete Probability，第 5–9 页。原讲义把几何变量定义成“成功前失败了几次”；下面按课程常见习惯数“包含成功的总轮次”，因此期望相差 1。

28.2.2 几何分布：平均 2 轮，为什么仍可能失败很多轮

每轮独立、成功概率固定$p>0$，K 表示首次成功的轮次，成功那轮也计入。K=k 意味着前 k−1 轮失败、第 k 轮成功，因此

$$P(K=k)=(1-p)^{k-1}p,\qquad k=1,2,\ldots.$$

例如$p=1/2$，最短三类运行是：成功，概率 1/2、成本 1；失败成功，概率 1/4、成本 2；失败失败成功，概率 1/8、成本 3。还存在更长的运行，不能在第 3 类截掉后便说列全了。
求平均轮数可以按“这一轮还需要执行吗”计数。任意整数 K=k 都可拆成 k 个 1；对所有样本取平均，便有

$$E[K]=\sum_{j=1}^{\infty} P(K\ge j).$$

第 j 轮会发生，当且仅当前 j−1 轮全失败，概率$(1-p)^{j-1}$。于是

$$E[K]=1+(1-p)+(1-p)^2+\cdots=\frac{1}{p}.$$

最后一步是等比级数：把整个和乘$(1-p)$再从原和减去，只剩 1，故 p 乘原和等于 1。$p=1/2$时平均 2 轮，但前 20 轮全失败仍有2^{-20}的正概率。期望有限没有给出统一最坏轮数。
若程序最多做 3 轮就停止，实际轮数是$\min(K,3)$，平均$1+(1-p)+(1-p)^2$，失败概率$(1-p)^3$；它保证有限结束，却允许报告失败。若每轮p变化或试验相关，也不能继续把失败概率写成同一个幂。

28.3 随机排列的环：为什么最少交换平均是$n-H_n$

排列`[2,3,1,5,4]`表示位置 1 的元素应该去位置 2，位置 2 去 3，位置 3 去 1，另有 4 和 5 互换。因此环为`(1 2 3)`、`(4 5)`，共 c=2 个。

```
[2,3,1,5,4]  交换4、5 -> [2,3,1,4,5]
               交换2、3 -> [2,1,3,4,5]
               交换1、2 -> [1,2,3,4,5]
```

共 3 次，正好$n-c=5-2$。任何一次交换最多让环数增加 1，目标排列有 n 个单点环，所以至少要 n−c 次；逐环拆又恰好能达到，故这是真正的最小值。

现在才加入随机模型：n 个互异标签的 n!个排列等可能。按标签 1、2、...构造环，加入第 k 个标签时有 k 种选择：让它单独成一个新环，或把它插入已有 k−1 个标签中的某个标签之后。每种最终排列可以唯一逆向恢复这些选择：删去最大标签 k，若它原来独立，就记“新环”；否则记下它原来的前驱，再把环接回。共有$1\cdot 2\cdots n=n!$组选择，与排列一一对应。

令J_k表示第 k 步是否选“新环”，是则 1、否则 0。k 种等可能选择中只有一种新增环，所以$E[J_k]=1/k$。最终环数是$J_1+\cdots+J_n$，线性期望给

$$E[c]=1+\frac{1}{2}+\cdots+\frac{1}{n}=H_n,\qquad E[n-c]=n-H_n.$$

H_n叫第 n 个调和数。n=3 时六个排列的环数分别为：恒等 123 有 3 环；213、132、321 各 2 环；231、312 各 1 环。平均$(3+2+2+2+1+1)/6=11/6=H_3$，最少交换平均为$3-11/6=7/6$。小样本可以核对一般推导，而不是替代一般证明。

28.4 单调栈某一时刻：从右往左看，谁还没有被淘汰

递增单调栈遇到更小的新高度就弹旧项。假定高度互异、相对次序均匀随机。处理完前 r 项后，一个旧项还在栈中，当且仅当它小于自己右边所有已处理项；否则右边的某个更小项早已把它弹掉。留下的是“从右往左看到的新最小值”。

例输入`[4,2,5,1,3]`：

```
读4：  [4]
读2：  弹4，剩[2]
读5：  [2,5]
读1：  弹5、弹2，剩[1]
读3：  [1,3]
```

从右看原数组，先看到 3，是记录；再看到 1，比 3 小，是新记录；5、2、4 都比已经见到的 1 大，不是记录。栈恰好留下 1、3。

在长度 j 的后缀中，最左那项成为最小值的概率是 1/j。将每一项“是否留下”写成 0/1 变量相加，得到固定时刻栈长S_r的期望

$$E[S_r]=H_r=\Theta(\log r).$$

这里不需要独立。r=3 时六种排列栈长为 3、2、2、1、2、1，平均 11/6；但严格递增的一份输入还是会留下 3 项，概率平均没有删除最坏输入。

28.4.1 全程峰值为什么还要另证

令$M=\max(S_1,\dots,S_n)$。它是每次完整运行中出现过的最大栈长，先在同一样本里取最大，再对不同样本取平均。不能把 E 随意搬到 max 里面。

一个只有两个样本的反例：一半概率$(X,Y)=(1,0)$，另一半为$(0,1)$。X、Y 各自平均 1/2，但每个样本里的最大值恒为 1，所以$E[\max(X,Y)]=1$。知道每个时刻平均小，还没有排除不同运行各自在不同时间出现大峰值。

下面给全程峰值的真正上界。若只需要最坏空间，看到$M\le n$已经足够；以下是随机模型中的更细结论。

28.4.2 第一步：先证明记录指标独立

把一个 r 项均匀排列从右向左读。第 j 次读到的项，在已经读过的 j 项中的名次记为R_j，最小名次是 1。成为新最小记录正好是$R_j=1$。名次变量不是原数值，只比较相对大小。

先对三个数从左读的版本列全（反向读仍同样均匀）：

排列	R_1,R_2,R_3
1,2,3	1,2,3

排列	R_1, R_2, R_3
1,3,2	1,2,2
2,1,3	1,1,3
2,3,1	1,2,1
3,1,2	1,1,2
3,2,1	1,1,1

所有“第 1 名只能 1、第 2 名可 1 或 2、第 3 名可 1 或 2 或 3”的组合恰好各出现一次。
一般也是如此：给定一组合法名次，从 j=r 倒着恢复。在尚未使用的有序标签中取第R_j小的，放到第 j 个读入位置，然后删掉它。剩下的标签相对次序不变，继续恢复$j-1$。例如名次`[1,2,1]`：从{1,2,3}取第 1 小即 1 放第 3 位；余{2,3}取第 2 小即 3 放第 2 位；最后 2 放第 1 位，恢复 231。
合法名次元组共$r!$个，排列也$r!$个，且一一对应、等可能，所以每个元组的概率是

$$\frac{1}{r!} = \frac{1}{1} \cdot \frac{1}{2} \cdots \frac{1}{r}.$$

这恰等于每个分量取其名次的概率之积，证明所有R_j相互独立、各自在 1 到 j 中均匀。于是$B_j = [R_j = 1]$也独立，概率 1/j。一个固定前缀从右读，栈长便是$S_r = \sum_j B_j$。仅说每项概率 1/j，本来不足以证明独立；这里的一一对应补上了缺口。

28.4.3 第二步：用“指数后的平均”控制大值概率

取自然常数$e \approx 2.718$。对 0/1 变量B_j，e^{B_j}只能为 1 或 e，因此

$$E[e^{B_j}] = (1 - 1/j) \cdot 1 + (1/j) \cdot e = 1 + (e-1)/j.$$

又因为$e^{S_r} = \prod_j e^{B_j}$，独立性允许乘积的平均拆成平均的乘积：

$$E[e^{S_r}] = \prod_{j=1}^{r} \left(1 + \frac{e-1}{j}\right).$$

使用$1 + x \le e^x$，每个因子不超过对应指数，乘起来就是

$$E[e^{S_r}] \le e^{(e-1)H_r} \le C r^{e-1}, \qquad C = e^{e-1}.$$

这里ln是自然对数。两个所用不等式都可以检验：对$x \ge 0$，函数$e^x - 1 - x$在 0 处为 0，导数$e^x - 1 \ge 0$，所以$1 + x \le e^x$。对$j \ge 2$，$1/j \le \int_{j-1}^{j} dx/x$，相加得到$H_r \le 1 + \ln r$。常数 C 不随 r 增长。于是所有大栈长按指数放大后，平均仍受一个多项式控制。
非负变量 Z 满足$P(Z \ge a) \le E[Z]/a$：发生事件的那部分样本每个至少贡献 a，所以整个平均至少$aP(Z \ge a)$。这就是 Markov 不等式的完整理由。取$Z = e^{S_r}$、$a = e^t$，得到

$$P(S_r \ge t) \le C r^{e-1} e^{-t}.$$

28.4.4 第三步：把所有时间加进来，再用尾和求平均

M≥t 表示至少一个时刻r有$S_r \ge t$。把这些事件的概率相加可能重复计算，却一定是上界；这一步不需不同时间独立。因此

$$P(M \ge t) \le \sum_{r=1}^{n} P(S_r \ge t) \le C n^e e^{-t}.$$

当然概率至多 1，实际取二者的较小值。设$n \ge 2$，选整数阈值

$$t_0 = \lceil (e+1)\ln n + \ln C \rceil.$$

那么$C n^e e^{-t_0} \le 1/n$。前t_0个尾概率每个最多 1，后面的概率每增加 1 格就至少乘e^{-1}，所以

$$E[M] = \sum_{t \ge 1} P(M \ge t) \le t_0 + \frac{1}{n} \sum_{s \ge 1} e^{-s} = O(\log n).$$

另一方面每份运行都有$M \ge S_n$，取平均仍得$E[M] \ge H_n = \Omega(\log n)$，上下合为$\Theta(\log n)$。n=1 单独为 1，不影响渐近结论。高度重复、相关或刻意排好时，独立相对排列模型不再成立；最坏峰值仍可以线性。

28.5 一扇固定窗口，与无限期运行的最大队列

宽度 w 固定，值独立、连续、同分布，单调最大队列影子留下的是后缀最大记录，和前面的最小记录对称。固定时刻平均候选数为H_w。

w=3 时窗口 `[3,2,1]` 会留下全部 3 项；这类严格递减顺序的概率为 1/6。取 q 个互不重叠窗口，它们独立。一次都没有递减的概率是$(5/6)^q$；至少遇到一次的概率是$1-(5/6)^q$。q 越大，后者越接近 1，因此长期峰值达到 3 的概率趋 1。

一般 w 的递减排列概率为$1/w!$，相同计算给$1-(1-1/w!)^q$。所以固定 w、观察时间越来越长，峰值趋向 w，而不是一直保持平均$\log w$。若原始队列本身也存了 w 项，整个结构更本来就需要$\Theta(w)$空间；“影子候选平均很少”只是在说额外索引。

28.6 平均比较排序下界：把每片叶子占的空间相加

n 个互异元素有$n!$种排列，若均匀出现，每种概率$1/n!$。比较决策树每次问一个两分结果，叶深d_i就是第 i 种输入的比较次数。给根一条长度 1 的区间，每往下比较一次，区间平分。第 i 片叶占2^{-d_i}长度，不同叶不重叠，所以

$$\sum_{i=1}^{n!} 2^{-d_i} \le 1.$$

这叫 Kraft 界，在这里就是“所有叶区间放得进长度 1”。令$N=n!$，这 N 个正长度的几何平均不大于算术平均：

$$2^{-\bar{d}} = \left(\prod_{i} 2^{-d_i}\right)^{1/N} \le \frac{1}{N}\sum_{i} 2^{-d_i} \le \frac{1}{N},$$

其中$\bar{d}=(d_1+\cdots+d_N)/N$。取以 2 为底的对数、再把负号移过去，得到$\bar{d} \ge \log_2 N = \log_2(n!) = \Omega(n\log n)$。

n=3 时，有一棵合法比较树的 6 片叶深为 2、2、3、3、3、3。平均是 8/3，约 2.667，大于$\log_2 6 \approx 2.585$；平均并不必等于最坏值 3。只证明某一片叶很深，约束不到平均；刚才的长度总和把所有叶一起控制住了。该结论限定一般比较模型与均匀互异排列。整数桶利用关键字位值，已超出仅比较模型；输入几乎总是某个已知顺序时，分布也不同，不能不改条件地引用同一个平均下界。

28.7 终止与归纳：选的量必须真的覆盖完整状态

“每次变小”要指出哪个非负整数变小。二分可用 hi−lo，树递归可用子树节点数。沿树的一条左藤归纳时，外挂右子树可能很大，只证明藤长减少，没有保证侧树的过程正确；按整树规模归纳更容易覆盖它们。

Failstone 甚至不是每一步最大值都减小。正整数状态`(3,5)`会先变`(6,5)`，最大值还增加了。把后续三步一起看：

```
(3,5) -> (6,5) -> (1,6) -> (1,3)
```

阶段端点的最大值由 5 降到 3。一般在 p<q、q 为奇数时，有限次翻倍会经过$p^* < q < 2p^*$，随后到$(2p^*-q, p^*)$，两项都小于 q；q 为偶数时减半可直接降低最大值。每个阶段有限，阶段端点的正整数最大值严格下降，才证明整个过程有限。p=0 时不能沿用“翻倍总能超过 q”，这正是要写正整数条件的理由。

28.7.1 对数尺度换元，为何会多出一个$1/u$

第 22 章 Akra–Bazzi 证明把规模 u 改成$s=\ln u$，这样每次乘固定比例就变成减固定距离。例如 u 从 27 到 9，s 减少$\ln 3$；从 27 到 18，s 减少$\ln(3/2)$，都与当前 u 无关。

连续微小变化满足$ds=du/u$：这来自$\ln u$的导数是$1/u$。若归一化后的工作为$h(u)=g(u)/u^p$，在 s 轴上累加面积就是

$$\int h(e^s)\,ds = \int \frac{g(u)}{u^p}\frac{du}{u} = \int \frac{g(u)}{u^{p+1}}\,du.$$

p来自递归树规模权重，多出来的 1 来自换元，不是定理中随手加的指数。例$p=1, g(u)=u$时，面积是$\int du/u = \ln u$，乘回规模 n 便给$n \log n$。这一步只负责解释积分形式，离散节点和面积之间的夹逼仍需第 22 章明确写出的正则条件。

28.8 遮住答案，再做几笔账

1. 双栈队列只做入 A、入 B、入 C 就停止，实际、势能和摊还总账分别多少？
2. 公平硬币抛两次，X 为正面数、Y 为反面数。X 与 Y 独立吗？期望相加是否仍成立？
3. 对排列`[3,1,2]`求环数、最少交换数、完整单调栈的各步长度和峰值。
4. 为什么固定时刻记录概率 1/j 还不足以直接把指数期望写成乘积？

答案：

1. 实际 3，末势能 6，摊还每次 3、合计 9；关系为$3 = 9 + 0 - 6$。未来即使永远不出队，这 6 份未用费用仍只是上界里的余款。
2. 不独立，因为 Y=2−X，X=2 会强迫 Y=0；但$E[X+Y] = 2 = 1 + 1$，线性期望不要求独立。
3. 箭头`1→3→2→1`是一个三环，最少$3 - 1 = 2$次交换。单调递增栈处理 3 得到`[3]`，处理 1 弹 3 得`[1]`，处理 2 得`[1,2]`，长度 1、1、2，峰值 2。
4. 单个概率只给每个变量的边缘分布，没有给联合分布。两个变量可以始终同时发生，也可以互相排斥，单个成功概率却一样。相对秩与排列的一一对应证明所有名次组合等可能，才提供拆乘积需要的联合独立性。

29 题目没有告诉你用什么结构时，先算一个小输入

下面八题都先确定操作和答案，再选择要维护的状态。读到一个步骤时，可以盖住下一行，自己画一次数组、树或队列；旁边的状态不是装饰，它应当能用来查出自己的第一处错误。

29.1 订单按到达顺序处理，还要随时知道最大金额

输入操作为：入 A 金额 5、入 B 金额 2、查最大、出单、入 C 金额 7、查最大、出单、查最大、出单。出单必须先进先出。若只用普通队列，每次查最大要扫当前所有订单，重复查询可能二次。

用两个栈组成队列，每个栈格保存(订单,金额,从栈底到此格的最大金额)。这样一只栈的最大就是顶格附带的摘要，弹出后新栈顶自动给出剩余部分的最大，无需重扫。

in 接收新单，out 负责出单，只有 out 为空时才倒栈。每个格以下简写为订单:金额/本格最大，均从底到顶书写：

操作	in	out	返回
入 A5	[A:5/5]	[]	—
入 B2	[A:5/5,B:2/5]	[]	—
查最大	同上	空	5
出单	[]	倒完为[B:2/2,A:5/5]，再弹 A，剩[B:2/2]	A
入 C7	[C:7/7]	[B:2/2]	—
查最大	同上	同上	max(7,2)=7
出单	[C:7/7]	[]	B
查最大	[C:7/7]	[]	7
出单	[]	C 倒入后再弹，清空	C

为什么 in 里新来的 C 不会越过旧 B？out 非空时始终先用 out，不因为新单到来而重倒。倒栈又把 in 的后进先出反转一次，使最早入 in 的单处于 out 顶。两只栈各自的最大取 max，恰好覆盖当前队列全部元素；空栈的摘要用“无值”表示，别用 0，否则全负金额模型会误报 0。

每项至多进 in、离 in、进 out、离 out 各一次，所以 m 次操作总$O(m)$，查询最大最坏$O(1)$，空间与当前订单数同阶。某一次出单倒很多项仍可能线性，这个方案给摊还保证；要求每次都严格常数延迟的系统，需要另外的逐步迁移方案。

29.2 成绩序列：两个 2 能不能都选，先看等号

记录按到达顺序为[3,1,2,2,4]，允许跳过，禁止重排。找最长不下降子序列时，1、2、2、4 合法，长度 4；找严格上升时不能连两个 2，答案长度 3。

先写慢 DP。设f_i表示必须以第 i 项结束的最佳长度，初值 1；每个前驱 j 需满足 j<i 且$a_j \le a_i$。把所有合法前驱的f_j+1取最大：

i / 值	合法前驱	当前f_i	一条达到它的链
0 / 3	无	1	3
1 / 1	无	1	1
2 / 2	下标 1 的 1	2	1,2
3 / 2	下标 1 的 1、下标 2 的 2	3	1,2,2
4 / 4	前面全部	4	1,2,2,4

每项扫前面全部，时间$O(n^2)$。若把≤改<，下标 3 不能接 2，得到$f=[1,1,2,2,3]$，精确体现题意差别。

加速时，tails[l-1]只保存“已经见过的、长度为 l 的不降子序列中，最小的末尾值”。长度相同，末尾越小，未来可接的新值至少不会更少。新值 x 应接在所有末尾≤x 的长度后面，因此找第一个**严格大于 x** 的位置，使用 upper_bound：

```
读3：tails=[]，位置0，追加        -> [3]
读1：首个>1是下标0，替换          -> [1]
读2：没有>2，位置1，追加          -> [1,2]
再读2：没有>2，位置2，追加        -> [1,2,2]
读4：没有>4，位置3，追加          -> [1,2,2,4]
```

位置 p 是零起下标，代表新项可以成为长度 p+1 的末尾。为什么？p 前面的末尾都≤x，所以长度 p 的代表链可以接上 x；p=0 时，x 自己就形成长度 1。另一方面，旧 tails[p]若存在就严格大于 x，说明任何长度 p+1 的旧链末尾都大于 x，不能接 x。因此新项不能一下产生长度 p+2 或更长的链。这个上、下两面一起证明了新长度恰为 p+1。

更新时，长度 p+1 原来的最小末尾若存在，就用更小的 x 替换；若不存在，就新增长度。其他长度不变。这既保留了每个长度的可行链，也保留了“同长度末尾最小”的含义。tails 仍然非降，可以继续二分。每项 O(log n)，总 O(n log n)。

29.2.1 恢复的是一条链，不能直接打印 tails

读[2,3,1]后 tails 为[1,3]，但 1 出现晚于 3，它们不是原输入中的子序列。tails 的每一格各自代表一种长度，格与格可能来自不同链。

要恢复一条答案，同时存 end[p]：长度 p+1 的代表链末项在原数组中的下标；parent[i]：以第 i 项结尾的这条链，上一项的原下标。新项落 p 时，先读取更新前的 end[p−1]作为前驱，再把 end[p]改成 i。−1 表示没有前驱。

i / x	二分位置 p	先保存 parent[i]	更新后的 tails	更新后的 end
0 / 3	0	−1	[3]	[0]
1 / 1	0	−1	[1]	[1]
2 / 2	1	1	[1,2]	[1,2]
3 / 2	2	2	[1,2,2]	[1,2,3]
4 / 4	3	3	[1,2,2,4]	[1,2,3,4]

最后从 end[3]=4 开始沿 parent 回退：原下标 4⟶3⟶2⟶1⟶−1，对应数值 4⟶2⟶2⟶1；反转后得到 [1,2,2,4]。以后某个 end 格被替换，不会改写已经保存的 parent 链接，因为 parent 记的是原数组下标，不是会变化的 tails 位置。

```python
from bisect import bisect_right

def longest_nondecreasing(a):
    tails, end = [], []
    parent = [-1] * len(a)
    for i, x in enumerate(a):
        p = bisect_right(tails, x)  # 第一个严格大于x
        if p > 0:
            parent[i] = end[p - 1]  # 先保存旧代表
        if p == len(tails):
            tails.append(x)
            end.append(i)
        else:
            tails[p] = x
            end[p] = i
    indices = []
    i = end[-1] if end else -1
    while i != -1:
```

```
        indices.append(i)
        i = parent[i]
    indices.reverse()
    return indices  # 用[a[i] for i in indices]取得数值
```

前驱下标严格小于当前下标，链不会成环，恢复次数不超过 n。空输入返回空下标列表。严格上升版把 bisect_right 改为 bisect_left，即找第一个≥x 的位置；这样等值项只替换旧代表，不增长长度。

29.3 道路网络：哪个路口一删，连通块就增多

无向简单图边为 1—2、2—3、3—1、3—4、4—5。前面是一个三角形，后面接一条链：

```
1 ----- 2
 \     /
  3 ----- 4 ----- 5
```

慢办法逐个删除顶点后重跑连通性。一次 DFS 可同时找割点与桥：`dfn[v]`记首次发现次序；`low[v]`记 v 子树沿树边向下、最多再用一条回祖先边，能到达的最小发现号。只看“有没有回边”不够，还要知道能绕到父亲之上哪一层。

固定邻居访问顺序，使 DFS 树走 1⟶2⟶3⟶4⟶5：

事件	dfn/low 发生的变化
发现 1、2、3	各自 dfn 为 1、2、3，low 先等于自己 dfn
3 看到已访问祖先 1	`low[3]=min(3,dfn[1])=1`
发现 4、5	dfn 为 4、5，low 初值 4、5
5 返回 4	没有回边，low5=5；5>dfn4=4，所以 4—5 是桥；5≥4，所以非根 4 是割点
4 返回 3	low4=4；4>dfn3=3，所以 3—4 是桥；4≥3，所以 3 是割点；low3 仍 min(1,4)=1
3 返回 2	low3=1<dfn2=2，子树可绕过 2 到 1，2 不是割点；low2 改 1
2 返回 1	根 1 只有一个 DFS 树孩子 2，所以不是割点

根虽然与 2、3 都相邻，但 3 在 2 的递归过程中已经被发现，不能把两条邻边当两个 DFS 树孩子。最终割点 3、4，桥 3—4、4—5。割点条件用≥，桥用>：若孩子能回到父亲本身，边不一定是桥，但删掉父亲仍可能断开子树。

每条无向边扫描常数次，时间$O(V+E)$。若有平行边，要按边 ID 识别“刚才来的父边”，否则把另一条回父边一并跳过会误报桥。这个简单例子的证明不能靠省掉多重边约定来冒充一般实现。

29.4 反复插入、删除以后，仍要能查第 k 小

依次插 8、3、10、1、6，重复值暂不允许，查询第 3 小；再删 3，再查第 3 小。只用普通散列表找不到全局秩；可以用平衡 BST，并给每个节点额外保存子树节点数 size。

插入后的合法 AVL 形状可以是：

```
        8(size=5)
       /         \
  3(size=3)     10(size=1)
   /     \
1(1)     6(1)
```

查询节点 v 时，左子树大小记 L。按中序，左边恰占前 L 名，v 自己是 L+1 名；目标 k≤L 就去左边，k=L+1 返回 v，否则去右边，并把 k 减去 L+1。

当前节点	左大小 L	当前 k	决定
8	3	3	3≤3，去左
3	1	3	越过左边 1 项和根，k 改 1，去右
6	0	1	1=L+1，返回 6

删 3 时用其后继 6 替换并删除原 6 节点，左子树变根 6、左孩子 1。原根 8 的左大小从 3 变 2，所以第二次查询 k=3，直接在 8 返回。全序也确实由 `1,3,6,8,10` 变为 `1,6,8,10`。

维护式是 `size(v)=1+size(left)+size(right)`，空子树 size=0。每次修改或旋转都要更新；旋转先更新变低的节点，再更新新根，否则新根会读到旧摘要。高度没变时可以停止高度传播，size 却可能继续变，不能一起停止。平衡树高度$O(\log n)$，所以四种接口均可对数；k 必须在 1 到当前 size 之间，空树没有第 1 小。

29.5 矩形内的点：计数摘要与输出全部是两种任务

点为(1,1)、(2,4)、(3,2)、(4,5)、(5,3)，查询闭矩形$[2,5]\times[2,5]$。逐点答案是后 4 个点，计数 4。

设 kd 树根点为(3,2)，左子树保存前两点，包围盒$[1,2]\times[1,4]$；右子树保存后两点，包围盒$[4,5]\times[3,5]$。每个节点保存 size 和包围盒：

要检查的对象	与查询的关系	计数操作
根点(3,2)	在闭矩形内，包括边界	+1
左子树包围盒	部分相交	继续分开检查
左点(1,1)	外部	+0
左点(2,4)	边界上	+1
右子树包围盒	整个包含	直接加 size=2，不逐点看

共 1+1+2=4。安全跳过的理由是包围盒确实包住所有后代：不相交则整棵无解，完全包含则每项都合法，部分相交才需要深入。若包围盒在插入后没更新，这个剪枝就不再安全。

若要求报告四个坐标，右子树即使完全包含，也必须把其中两个点输出，至少再付 2 步。一般输出 k 项有$\Omega(k)$成本。节点摘要能常数回答“这段一共有多少”，不会常数吐出任意多条数据。kd 树的最坏查询也可能访问许多节点，不能因为小例里跳了一棵就声称所有矩形查询都$O(\log n)$。

29.6 接线总价最低，与从源点出发路程最短

用能区分两种目标的三角形：A–B 权 2，A–C 权 3，B–C 权 2。

```
  A
2/ \3
B---C
  2
```

最小生成树只需连接全部顶点，总边价最小。Kruskal 按权排序：先选 A–B2，分量变`{A,B},{C}`；再选 B–C2，连成一棵树，总价 4；A–C3 加入会形成环，跳过。

A 作为源的最短路却应保留 A–B2、A–C3，距离分别 2、3，总边价 5。若走 MST 到 C，要 A→B→C 花 4，比直接 3 长。MST 优化“一次性买下哪些边”，最短路树优化“从指定源到每个点怎样走”，两个目标不同，不能只看它们都画成树。

权互异是 MST 唯一的充分条件；有重复却未必不唯一。例如图本来就是一条两边的链，两边都权 1，也只有这一棵生成树。回答“唯一吗”要给能区分的结构，不能把充分条件误读成必要条件。

29.7 数据流的第 3 大：保留集合和堆数组都画出来

输入依次 4、9、1、7、2、10，要求读满 3 项后随时报告第 3 大，重复值按记录计名次。保留容量 3 的小根堆，根是当前保留的前三大中的最小值，也就是第 3 大。普通存全部再排序可正确但空间随流增长，反复排序也浪费。

数组按完全树层序保存，根在下标 0；以下是一种合法的实际堆状态：

新值	动作	堆数组（不是排序数组）	第 3 大
4	入堆	`[4]`	未满 3 项
9	入堆，无须上滤	`[4,9]`	未满
1	入堆后与 4 交换上滤	`[1,9,4]`	1
7	替换根 1 得`[7,9,4]`，与较小孩子 4 换	`[4,9,7]`	4
2	不大于根 4，丢弃	`[4,9,7]`	4
10	替换根 4 得`[10,9,7]`，与 7 换	`[7,9,10]`	7

第 4 步后堆是根 4、左 9、右 7，父子关系合法，但数组没有排序。要输出前三名的降序顺序，需另处理，不能直接打印堆数组。

不变量是堆中恰好保存已读前缀最大的 3 条记录。新项不大于根时，已经有 3 项不小于它，它不会进入前三；较大时，旧根是这 4 个候选里该被淘汰的最小项。丢掉的项未来也不会重新变成历史前缀的前三，因为未来只会增加更大候选，不会删掉已有项。若题目改成滑动窗口、旧项会过期，这一保留规则就不够了。

对正整数 k，每项$O(\log(k+1))$，整个 n 项流$O(n\log(k+1))$、辅助空间$O(k)$。写 k+1 是为了包含 k=1：维护最大值仍需逐项看，整体$\Theta(n)$。k=0 应拒绝，未读满 k 项应报告未就绪；一边说流式只占 k 空间、一边另存全体输入，会违背自己的空间承诺。

29.8 同一张图，BFS 求的是边数，Dijkstra 求的是权和

有向边 S→T 权 10，S→A 权 1，A→B 权 1，B→T 权 1。BFS 第一次从 S 就发现 T，按最少边数取直接边；Dijkstra 不能把“第一次发现”当距离定值。

Dijkstra 确定的点	松弛以后 `dist[A],dist[B],dist[T]`	T 的前驱
初始 S 为 0	`∞,∞,∞`	无
处理 S	`1,∞,10`	S
处理最小候选 A1	`1,2,10`	S
处理最小候选 B2	`1,2,3`	改为 B
处理 T3	终止，路径 S→A→B→T	B

堆里如果还留着早先的 T10，懒删除版弹到它时应发现 10 不等于当前 dist[T]=3 并丢弃，不能再次当最终距离覆盖 3。维护一条记录/顶点的 decrease-key 堆与允许多份旧记录的懒堆，内存、时间口径也不同。若所有边权都为 1，最小权和就是最少边数，BFS 每点每边常数处理，O(V+E)更简单。

29.8.1 只有 0、1 边权时，双端队列如何代替堆

这是普通 BFS 的一个扩展。取有向边 S→A 权 1、S→B 权 0、B→A 权 0、A→T 权 1，按上述顺序扫描 S 的邻边。最短路应为 S→B→A→T，权和 1；若把第一次发现 A 时的距离 1 当作定值，就会得到错误的 2。队列保存`(候选距离,顶点)`，队头写在左。只有严格改善 dist 时入队；经 0 边得到同层距离，放队头，经 1 边得到下一层距离，放队尾。旧候选出队时核对是否过期。

出队与处理	距离变化	处理后的双端队列
初始化	dist[S]=0，其余∞	[(0,S)]
S	A 改 1 先放尾；B 改 0 再放头	[(0,B),(1,A)]
B	经 0 边把 A 从 1 改 0，放头	[(0,A),(1,A)]
A 的 0 候选	T 从∞改 1，放尾	[(1,A),(1,T)]

出队与处理	距离变化	处理后的双端队列
A 的 1 候选	1≠当前 dist[A]=0，过期，跳过	[(1,T)]
T	距离 1 确定，无出边	[]

```
from collections import deque

def zero_one_bfs(graph, source):
    n = len(graph)
    if not 0 <= source < n:
        raise ValueError("源点越界")
    if any(w not in (0, 1) for edges in graph for _, w in edges):
        raise ValueError("边权必须为0或1")
    dist = [float("inf")] * n
    dist[source] = 0
    queue = deque([(0, source)])
    while queue:
        d, u = queue.popleft()
        if d != dist[u]:
            continue
        for v, w in graph[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                if w == 0:
                    queue.appendleft((nd, v))
                else:
                    queue.append((nd, v))
    return dist
```

图以 0 到 n−1 为顶点下标，邻接表中的端点也须合法。为何能不用堆？丢弃过期项后，候选按距离非降；处理距离 d 时，队里只会剩 d 和 d+1 两层。新 0 边仍为 d，插到前面；新 1 边为 d+1，插到后面，都保持次序。因而每个未过期出队项都与 Dijkstra 一样取到当前最小距离，该点的距离随后不会再改善。每点的出边只在这次定值时扫描一次，每条边最多触发一次入队；连同过期项的弹出，总 O(V+E)。非负 0 边可有环，但相等距离不再次入队，所以不会沿 0 环无限重复。

若权值变成 0、2、7，新候选不再只有相邻两层，同样的头尾规则不能保持距离次序，应换回适合一般非负权的最小堆。

29.9 再做五题，先写判断依据

1. `ABCA→ACBA` 只允许插删，预算 2。给具体操作，并说明预算 1 为何不行。
2. 有人承诺：任意 n 个互异实数，只用两项比较，最坏不超过 3n 次。怎样用下界否定它对所有 n 成立？
3. 向量`[1,3,5,7]`在秩 1 插 2，未扩容。原下标 2 的物理格现在装什么？保存“指向原 5”的旧迭代器能否照用？
4. 随机快排对固定数组的内部随机轴取期望为$O(n \log n)$，是否保证每次运行都如此？
5. 有序数组允许任意点修改，同时求区间和与第一个≥x。一棵只存 sum 的线段树够不够？

解答：

1. 删原串下标 2 的 C，得到 ABA；在首 A 后插 C，得到 ACBA，共 2 步。长度相同、串不同，不可能只用一次插或删，因为任何一次都会改变长度；因此预算 1 不行。等价地 LCS 长 3，距离$4+4-2\times 3=2$。
2. 比较树至少 n!叶，最坏深度至少$\log_2(n!)$。取后半至少 n/2 项，每项不小于 n/2，得粗下界$(n/2)\log_2(n/2)$；对足够大 n（如偶数 n>128）已经超过 3n。这里不追求最小矛盾规模。整数位操作不属于“只比较”模型，不能把同一下界套在计数排序上。
3. 底层格子模型中结果`[1,2,3,5,7]`，原下标 2 的格装 3，原 5 移到下标 3。`std::vector` 插入点及其后的旧迭代器、引用会失效，不能通过旧句柄读取来“验证仍可读”；链表插入新节点通常保持其他未删节点的身份。物理地址、合法生命期和逻辑身份需要分别判断。

4. 不保证。连续选极端轴仍可能平方；期望是对每一固定输入的内部随机选择平均，不必假设输入本身随机。若实现固定选首项，则没有这份内部随机模型，不能借用同一个期望证明。
5. 任意修改可能破坏全局有序，原二分前提就不在了。只存 sum 也不能判断某区间是否含≥x：`[1,9]`与`[5,5]`和都 10，对阈值 8 却不同。需额外 max 摘要才能向左优先寻找满足阈值的位置；区间和仍由 sum 回答。若修改保证始终有序，可另对原数组二分，但应说明维护这份保证的方式。

30 PS04 的 21 个表达式：把每一次压栈、弹栈写全

本章对照 PS04 第 19 题的 21 式。每式先拆语法上的关键子表达式，再列 RPN，最后让每个 token 真实走过数值栈。表中栈从底到顶书写，最右项是下一次被弹出的项；数字的压入也单独列一行，因此可以从空栈逐行重做，不需要自行补隐含步骤。

这里!是阶乘，!!是双阶乘。例如 3!=6，3!!=3，4!!=8。双阶乘是一个 token，不能逐字符当两次普通阶乘。阶乘优先于乘方，乘方优先于乘除，乘除优先于加减。使用教师原表的左结合乘方约定；这 21 式没有连续无括号乘方链，故改右结合也不会改变它们。C++自身的^是异或，本章只是用这一字符表示自定义语言的乘方。

除法保留精确有理数，3/4 就存分数 3/4，不截成 0。阶乘只对非负整数执行。二元运算先弹**右值 b**，再弹**左值 a**，计算 a 运算 b。输入 8 3 -算 5，输入 3 8 -算−5；加法与乘法会掩盖左右弹反的错误，所以必须用减除检查。

30.1 先做一次从中缀到 RPN 的完整转换

取 2*(3+4)。数字直接输出；运算符暂存；遇更低或同级左结合运算时先弹旧运算符；左括号阻隔比较，右括号让内部运算符弹完，再消去左括号。# 表示结束哨兵，本章只用来画状态。

当前 token	本次动作	运算符栈（底→顶）	本步新输出
初始	放底哨兵	#	无
2	数字直接输出	#	2
*	哨兵让它入栈	# *	无
(	压左括号	# * (	无
3	数字直接输出	# * (	3
+	左括号内暂存+	# * (+	无
4	数字直接输出	# * (+	4
)	先弹+，当前右括号不前进	# * (	+
仍为)	消去左括号，消费右括号	# *	无
结束 #	弹*，当前结束符不前进	#	*
仍为 #	哨兵相遇，结束	空	无

从上到下拼接“本步新输出”，就是 2 3 4 + *。括号没有进入 RPN，因为它只负责决定子树边界，不执行数值运算。它对应的表达式树是：

```
      *
     / \
    2   +
       / \
      3   4
```

后序先 2，再 3、4、+，最后*，与栈转换相同。下面每式给出的 RPN 都可以按这个规则从中缀重建；数值表则独立验证“这个 RPN 到底算成什么”。表中偶尔连续出现多个数字，是在保存尚未凑齐右子表达式的左值，不表示该立即把它们相加。

30.1.1 阶乘在什么时候输出，乘除在什么时候弹出

再取第 2 式中的核心片段 2*3!/4!*5，完整处理一次。!是后缀一元运算，只作用于它前面已经构成的操作数；教师表把它暂压栈，遇到后续较低优先级运算时弹出。下面保留这种处理时机。

当前 token	本次动作	动作后运算符栈	本步新输出
初始	放结束哨兵	#	无
2	输出数	#	2
*	入栈	# *	无
3	输出数	# *	3
!	优先级高于*，暂存	# * !	无
/	先弹!；当前/还未消费	# *	!
仍为/	/同级且左结合，再弹	#	*
仍为/	压入/，消费它	# /	无
4	输出数	# /	4
!	暂存	# / !	无
*	先弹!	# /	!
仍为*	乘除左结合，弹/	#	/
仍为*	压入*	# *	无
5	输出数	# *	5
结束 #	弹*，再匹配底哨兵	空	*

拼起来为 `2 3 ! * 4 ! / 5 *`。3 与!先构成 3!，随后才与 2 相乘；处理/时先输出旧，所以算的是*(2*3!)/4!*。若把一直留到最后，结合关系就改了。

本题还出现!!。这是题面扩展出的双阶乘运算，例如 `4!!=4×2=8`，`5!!=5×3×1=15`，并约定 `0!!=1`；教师基础表中的单个!并不自动提供它。实现扩展时，词法分析应优先把连续两个感叹号识别成一个双阶乘 token，加入相应一元操作和优先关系，不能原样交给只识别!的程序。`3!!=3`，两次普通阶乘`(3!)!=720`，区别可直接用算值检验。

左结合也能画成两棵不同的树：

```
6−7−8，按左结合                6−(7−8)
       −                          −
      / \                        / \
     −   8                      6   −
    / \                            / \
   6   7                          7   8
RPN: 6 7 − 8 −                 RPN: 6 7 8 − −
值: (6−7)−8=−9                 值: 6−(7−8)=7
```

转换阶段决定的是这棵树，执行阶段才按树算值。下面 21 个表核对的是数值栈；如果树已经转错，即使每一次加减都算对，仍会得到另一道表达式的答案。

30.2 第 1 式

原式：`(0+1+2−3+4+5+6)*(7+8)*9`

先把左括号内的连加减从左到右算完，得到 15。另一个括号 7+8 也是 15，最后 15×15×9。两个独立子表达式只有在各自算完后才交给乘法。

```
0 1 + 2 + 3 − 4 + 5 + 6 + 7 8 + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / `0`	压入数字 0	`[0]`

步骤 / 读到 token	这一动作	动作后的完整数值栈
2 / 1	压入数字 1	[0, 1]
3 / +	先弹右 1，再弹左 0；(0) + (1) = 1	[1]
4 / 2	压入数字 2	[1, 2]
5 / +	先弹右 2，再弹左 1；(1) + (2) = 3	[3]
6 / 3	压入数字 3	[3, 3]
7 / −	先弹右 3，再弹左 3；(3) − (3) = 0	[0]
8 / 4	压入数字 4	[0, 4]
9 / +	先弹右 4，再弹左 0；(0) + (4) = 4	[4]
10 / 5	压入数字 5	[4, 5]
11 / +	先弹右 5，再弹左 4；(4) + (5) = 9	[9]
12 / 6	压入数字 6	[9, 6]
13 / +	先弹右 6，再弹左 9；(9) + (6) = 15	[15]
14 / 7	压入数字 7	[15, 7]
15 / 8	压入数字 8	[15, 7, 8]
16 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[15, 15]
17 / *	先弹右 15，再弹左 15；(15) * (15) = 225	[225]
18 / 9	压入数字 9	[225, 9]
19 / *	先弹右 9，再弹左 225；(225) * (9) = 2025	[2025]

最终值为 2025，数值栈峰值 3 项。

30.3 第 2 式

原式：(0*1+2)*3!/4!*5*6*(7+8)*9

先算 0×1+2=2，阶乘 3!=6、4!=24。乘除同级按左结合：2×6÷24=1/2，再乘 5、6、15、9。中间出现分数并不代表错误，不能在这里擅自做整数截断。

```
0 1 * 2 + 3 ! * 4 ! / 5 * 6 * 7 8 + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / 1	压入数字 1	[0, 1]
3 / *	先弹右 1，再弹左 0；(0) * (1) = 0	[0]
4 / 2	压入数字 2	[0, 2]
5 / +	先弹右 2，再弹左 0；(0) + (2) = 2	[2]
6 / 3	压入数字 3	[2, 3]
7 / !	弹 3，算 3!=6，压回	[2, 6]
8 / *	先弹右 6，再弹左 2；(2) * (6) = 12	[12]
9 / 4	压入数字 4	[12, 4]
10 / !	弹 4，算 4!=24，压回	[12, 24]

步骤 / 读到 token	这一动作	动作后的完整数值栈
11 / /	先弹右 24，再弹左 12；(12) / (24) = 1/2	[1/2]
12 / 5	压入数字 5	[1/2, 5]
13 / *	先弹右 5，再弹左 1/2；(1/2) * (5) = 5/2	[5/2]
14 / 6	压入数字 6	[5/2, 6]
15 / *	先弹右 6，再弹左 5/2；(5/2) * (6) = 15	[15]
16 / 7	压入数字 7	[15, 7]
17 / 8	压入数字 8	[15, 7, 8]
18 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[15, 15]
19 / *	先弹右 15，再弹左 15；(15) * (15) = 225	[225]
20 / 9	压入数字 9	[225, 9]
21 / *	先弹右 9，再弹左 225；(225) * (9) = 2025	[2025]

最终值为 2025，数值栈峰值 3 项。

30.4 第 3 式

原式：`0!+(1+2)!!*3-4-5!-6-7-8+9`

0!=1，括号 1+2=3，3!!=3，所以前面 1+3×3=10。另一个阶乘 5!=120，随后从左到右减 4、120、6、7、8，再加 9。注意双阶乘与连续两次阶乘的差别。

```
0 ! 1 2 + !! 3 * + 4 - 5 ! - 6 - 7 - 8 - 9 +
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / !	弹 0，算 0!=1，压回	[1]
3 / 1	压入数字 1	[1, 1]
4 / 2	压入数字 2	[1, 1, 2]
5 / +	先弹右 2，再弹左 1；(1) + (2) = 3	[1, 3]
6 / !!	弹 3，算 3!!=3，压回	[1, 3]
7 / 3	压入数字 3	[1, 3, 3]
8 / *	先弹右 3，再弹左 3；(3) * (3) = 9	[1, 9]
9 / +	先弹右 9，再弹左 1；(1) + (9) = 10	[10]
10 / 4	压入数字 4	[10, 4]
11 / -	先弹右 4，再弹左 10；(10) - (4) = 6	[6]
12 / 5	压入数字 5	[6, 5]
13 / !	弹 5，算 5!=120，压回	[6, 120]
14 / -	先弹右 120，再弹左 6；(6) - (120) = -114	[-114]
15 / 6	压入数字 6	[-114, 6]
16 / -	先弹右 6，再弹左-114；(-114) - (6) = -120	[-120]
17 / 7	压入数字 7	[-120, 7]

步骤 / 读到 token	这一动作	动作后的完整数值栈
18 / -	先弹右 7，再弹左-120；(-120) - (7) = -127	[-127]
19 / 8	压入数字 8	[-127, 8]
20 / -	先弹右 8，再弹左-127；(-127) - (8) = -135	[-135]
21 / 9	压入数字 9	[-135, 9]
22 / +	先弹右 9，再弹左-135；(-135) + (9) = -126	[-126]

最终值为-126，数值栈峰值 3 项。

30.5 第 4 式

原式：`(0+1+2)/3!/4!*5!*6*(7+8)*9`

括号和为 3。依次除 3!=6 与 4!=24，得到 1/48，再乘 5!=120 得到 5/2，继续乘 6、15、9。若每次除法都截成整数，第一步 3/6 就变 0，整式会完全换掉。

```
0 1 + 2 + 3 ! / 4 ! / 5 ! * 6 * 7 8 + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / 1	压入数字 1	[0, 1]
3 / +	先弹右 1，再弹左 0；(0) + (1) = 1	[1]
4 / 2	压入数字 2	[1, 2]
5 / +	先弹右 2，再弹左 1；(1) + (2) = 3	[3]
6 / 3	压入数字 3	[3, 3]
7 / !	弹 3，算 3!=6，压回	[3, 6]
8 / /	先弹右 6，再弹左 3；(3) / (6) = 1/2	[1/2]
9 / 4	压入数字 4	[1/2, 4]
10 / !	弹 4，算 4!=24，压回	[1/2, 24]
11 / /	先弹右 24，再弹左 1/2；(1/2) / (24) = 1/48	[1/48]
12 / 5	压入数字 5	[1/48, 5]
13 / !	弹 5，算 5!=120，压回	[1/48, 120]
14 / *	先弹右 120，再弹左 1/48；(1/48) * (120) = 5/2	[5/2]
15 / 6	压入数字 6	[5/2, 6]
16 / *	先弹右 6，再弹左 5/2；(5/2) * (6) = 15	[15]
17 / 7	压入数字 7	[15, 7]
18 / 8	压入数字 8	[15, 7, 8]
19 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[15, 15]
20 / *	先弹右 15，再弹左 15；(15) * (15) = 225	[225]
21 / 9	压入数字 9	[225, 9]
22 / *	先弹右 9，再弹左 225；(225) * (9) = 2025	[2025]

最终值为 2025，数值栈峰值 3 项。

30.6 第 5 式

原式：`(0!^1/2*3!!/(4+5)*6-7-8)*9`

0!=1，1^1=1。接下来除 2、乘 3!!=3、除(4+5)=9、乘 6，连续乘除结果是 1。括号内再减 7、减 8 得−14，最后乘 9。这里的负值由明确的减法链产生。

```
0 ! 1 ^ 2 / 3 !! * 4 5 + / 6 * 7 - 8 - 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / `0`	压入数字 0	`[0]`
2 / `!`	弹 0，算 0!=1，压回	`[1]`
3 / `1`	压入数字 1	`[1, 1]`
4 / `^`	先弹右 1，再弹左 1；`(1) ^ (1) = 1`	`[1]`
5 / `2`	压入数字 2	`[1, 2]`
6 / `/`	先弹右 2，再弹左 1；`(1) / (2) = 1/2`	`[1/2]`
7 / `3`	压入数字 3	`[1/2, 3]`
8 / `!!`	弹 3，算 3!!=3，压回	`[1/2, 3]`
9 / `*`	先弹右 3，再弹左 1/2；`(1/2) * (3) = 3/2`	`[3/2]`
10 / `4`	压入数字 4	`[3/2, 4]`
11 / `5`	压入数字 5	`[3/2, 4, 5]`
12 / `+`	先弹右 5，再弹左 4；`(4) + (5) = 9`	`[3/2, 9]`
13 / `/`	先弹右 9，再弹左 3/2；`(3/2) / (9) = 1/6`	`[1/6]`
14 / `6`	压入数字 6	`[1/6, 6]`
15 / `*`	先弹右 6，再弹左 1/6；`(1/6) * (6) = 1`	`[1]`
16 / `7`	压入数字 7	`[1, 7]`
17 / `-`	先弹右 7，再弹左 1；`(1) - (7) = -6`	`[-6]`
18 / `8`	压入数字 8	`[-6, 8]`
19 / `-`	先弹右 8，再弹左-6；`(-6) - (8) = -14`	`[-14]`
20 / `9`	压入数字 9	`[-14, 9]`
21 / `*`	先弹右 9，再弹左-14；`(-14) * (9) = -126`	`[-126]`

最终值为-126，数值栈峰值 3 项。

30.7 第 6 式

原式：`((0*(1-2)+3+4)*5*6+(7+8))*9`

先算 1−2=−1，再乘 0 得到 0；随后加 3、加 4 得 7。7×5×6=210，另一括号 7+8=15，相加 225 再乘 9。乘零会消去一项数值，但解析器仍应正确识别其完整子表达式。

```
0 1 2 - * 3 + 4 + 5 * 6 * 7 8 + + 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / `0`	压入数字 0	`[0]`
2 / `1`	压入数字 1	`[0, 1]`

步骤 / 读到 token	这一动作	动作后的完整数值栈
3 / 2	压入数字 2	[0, 1, 2]
4 / -	先弹右 2，再弹左 1；(1) - (2) = -1	[0, -1]
5 / *	先弹右-1，再弹左 0；(0) * (-1) = 0	[0]
6 / 3	压入数字 3	[0, 3]
7 / +	先弹右 3，再弹左 0；(0) + (3) = 3	[3]
8 / 4	压入数字 4	[3, 4]
9 / +	先弹右 4，再弹左 3；(3) + (4) = 7	[7]
10 / 5	压入数字 5	[7, 5]
11 / *	先弹右 5，再弹左 7；(7) * (5) = 35	[35]
12 / 6	压入数字 6	[35, 6]
13 / *	先弹右 6，再弹左 35；(35) * (6) = 210	[210]
14 / 7	压入数字 7	[210, 7]
15 / 8	压入数字 8	[210, 7, 8]
16 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[210, 15]
17 / +	先弹右 15，再弹左 210；(210) + (15) = 225	[225]
18 / 9	压入数字 9	[225, 9]
19 / *	先弹右 9，再弹左 225；(225) * (9) = 2025	[2025]

最终值为 2025，数值栈峰值 3 项。

30.8 第 7 式

原式：`(0!-1!+2)*3!/4!*5*6*(7+8)*9`

0!和 1!都等于 1，所以第一个括号 1-1+2=2，后续链与第 2 式相同。这样可以把两个表达式做变形核对：不同写法确实产生同一个中间值 2。

```
0 ! 1 ! - 2 + 3 ! * 4 ! / 5 * 6 * 7 8 + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / !	弹 0，算 0!=1，压回	[1]
3 / 1	压入数字 1	[1, 1]
4 / !	弹 1，算 1!=1，压回	[1, 1]
5 / -	先弹右 1，再弹左 1；(1) - (1) = 0	[0]
6 / 2	压入数字 2	[0, 2]
7 / +	先弹右 2，再弹左 0；(0) + (2) = 2	[2]
8 / 3	压入数字 3	[2, 3]
9 / !	弹 3，算 3!=6，压回	[2, 6]
10 / *	先弹右 6，再弹左 2；(2) * (6) = 12	[12]
11 / 4	压入数字 4	[12, 4]

步骤 / 读到 token	这一动作	动作后的完整数值栈
12 / !	弹 4，算 4!=24，压回	[12, 24]
13 / /	先弹右 24，再弹左 12；(12) / (24) = 1/2	[1/2]
14 / 5	压入数字 5	[1/2, 5]
15 / *	先弹右 5，再弹左 1/2；(1/2) * (5) = 5/2	[5/2]
16 / 6	压入数字 6	[5/2, 6]
17 / *	先弹右 6，再弹左 5/2；(5/2) * (6) = 15	[15]
18 / 7	压入数字 7	[15, 7]
19 / 8	压入数字 8	[15, 7, 8]
20 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[15, 15]
21 / *	先弹右 15，再弹左 15；(15) * (15) = 225	[225]
22 / 9	压入数字 9	[225, 9]
23 / *	先弹右 9，再弹左 225；(225) * (9) = 2025	[2025]

最终值为 2025，数值栈峰值 3 项。

30.9 第 8 式

原式：`((0-1)^2*(3!!/4-5+6*7+8))*9`

0−1=−1，其平方为 1。另一个括号中 3!!/4=3/4，6×7=42，合并成 3/4−5+42+8=183/4，最后乘 9，得到 1647/4。不要把 3/4 提前舍去。

```
0 1 - 2 ^ 3 !! 4 / 5 - 6 7 * + 8 + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / 1	压入数字 1	[0, 1]
3 / -	先弹右 1，再弹左 0；(0) - (1) = -1	[-1]
4 / 2	压入数字 2	[-1, 2]
5 / ^	先弹右 2，再弹左-1；(-1) ^ (2) = 1	[1]
6 / 3	压入数字 3	[1, 3]
7 / !!	弹 3，算 3!!=3，压回	[1, 3]
8 / 4	压入数字 4	[1, 3, 4]
9 / /	先弹右 4，再弹左 3；(3) / (4) = 3/4	[1, 3/4]
10 / 5	压入数字 5	[1, 3/4, 5]
11 / -	先弹右 5，再弹左 3/4；(3/4) - (5) = -17/4	[1, -17/4]
12 / 6	压入数字 6	[1, -17/4, 6]
13 / 7	压入数字 7	[1, -17/4, 6, 7]

步骤 / 读到 token	这一动作	动作后的完整数值栈
14 / *	先弹右 7，再弹左 6；(6) * (7) = 42	[1, -17/4, 42]
15 / +	先弹右 42，再弹左-17/4；(-17/4) + (42) = 151/4	[1, 151/4]
16 / 8	压入数字 8	[1, 151/4, 8]
17 / +	先弹右 8，再弹左 151/4；(151/4) + (8) = 183/4	[1, 183/4]
18 / *	先弹右 183/4，再弹左 1；(1) * (183/4) = 183/4	[183/4]
19 / 9	压入数字 9	[183/4, 9]
20 / *	先弹右 9，再弹左 183/4；(183/4) * (9) = 1647/4	[1647/4]

最终值为 1647/4，数值栈峰值 4 项。

30.10 第 9 式

原式：`((0-1)^2*(3!!/4-5-6+7*8))*9`

左侧平方仍为 1。右括号是 3/4−5−6+7×8=3/4−11+56=183/4，与第 8 式巧合相等，但子表达式不同，分别算清才能确认。

```
0 1 - 2 ^ 3 !! 4 / 5 - 6 - 7 8 * + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / 1	压入数字 1	[0, 1]
3 / -	先弹右 1，再弹左 0；(0) - (1) = -1	[-1]
4 / 2	压入数字 2	[-1, 2]
5 / ^	先弹右 2，再弹左-1；(-1) ^ (2) = 1	[1]
6 / 3	压入数字 3	[1, 3]
7 / !!	弹 3，算 3!!=3，压回	[1, 3]
8 / 4	压入数字 4	[1, 3, 4]
9 / /	先弹右 4，再弹左 3；(3) / (4) = 3/4	[1, 3/4]
10 / 5	压入数字 5	[1, 3/4, 5]
11 / -	先弹右 5，再弹左 3/4；(3/4) - (5) = -17/4	[1, -17/4]
12 / 6	压入数字 6	[1, -17/4, 6]
13 / -	先弹右 6，再弹左-17/4；(-17/4) - (6) = -41/4	[1, -41/4]
14 / 7	压入数字 7	[1, -41/4, 7]

步骤 / 读到 token	这一动作	动作后的完整数值栈
15 / 8	压入数字 8	[1, -41/4, 7, 8]
16 / *	先弹右 8，再弹左 7；(7) * (8) = 56	[1, -41/4, 56]
17 / +	先弹右 56，再弹左-41/4；(-41/4) + (56) = 183/4	[1, 183/4]
18 / *	先弹右 183/4，再弹左 1；(1) * (183/4) = 183/4	[183/4]
19 / 9	压入数字 9	[183/4, 9]
20 / *	先弹右 9，再弹左 183/4；(183/4) * (9) = 1647/4	[1647/4]

最终值为 1647/4，数值栈峰值 4 项。

30.11 第 10 式

原式：`(0!+1+2)!+(3!-4)^(5+6)-7*8+9`

0!+1+2=4，外面的阶乘得到 24。3!-4=2，5+6=11，乘方 2^11=2048。最后 24+2048-56+9=2025。阶乘作用于整个括号，而不是只作用于末项 2。

```
0 ! 1 + 2 + ! 3 ! 4 - 5 6 + ^ + 7 8 * - 9 +
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / !	弹 0，算 0!=1，压回	[1]
3 / 1	压入数字 1	[1, 1]
4 / +	先弹右 1，再弹左 1；(1) + (1) = 2	[2]
5 / 2	压入数字 2	[2, 2]
6 / +	先弹右 2，再弹左 2；(2) + (2) = 4	[4]
7 / !	弹 4，算 4!=24，压回	[24]
8 / 3	压入数字 3	[24, 3]
9 / !	弹 3，算 3!=6，压回	[24, 6]
10 / 4	压入数字 4	[24, 6, 4]
11 / -	先弹右 4，再弹左 6；(6) - (4) = 2	[24, 2]
12 / 5	压入数字 5	[24, 2, 5]
13 / 6	压入数字 6	[24, 2, 5, 6]
14 / +	先弹右 6，再弹左 5；(5) + (6) = 11	[24, 2, 11]
15 / ^	先弹右 11，再弹左 2；(2) ^ (11) = 2048	[24, 2048]
16 / +	先弹右 2048，再弹左 24；(24) + (2048) = 2072	[2072]
17 / 7	压入数字 7	[2072, 7]
18 / 8	压入数字 8	[2072, 7, 8]
19 / *	先弹右 8，再弹左 7；(7) * (8) = 56	[2072, 56]

步骤 / 读到 token	这一动作	动作后的完整数值栈
20 / -	先弹右 56，再弹左 2072；(2072) - (56) = 2016	[2016]
21 / 9	压入数字 9	[2016, 9]
22 / +	先弹右 9，再弹左 2016；(2016) + (9) = 2025	[2025]

最终值为 2025，数值栈峰值 4 项。

30.12 第 11 式

原式：(0+(1+2))^3/(4+5)*6!-(7+8)*9

内括号 1+2=3，加 0 仍 3，立方 27。4+5=9，所以 27/9=3，再乘 6!=720 得到 2160；减去(7+8)×9=135，结果 2025。

```
0 1 2 + + 3 ^ 4 5 + / 6 ! * 7 8 + 9 * -
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / 1	压入数字 1	[0, 1]
3 / 2	压入数字 2	[0, 1, 2]
4 / +	先弹右 2，再弹左 1；(1) + (2) = 3	[0, 3]
5 / +	先弹右 3，再弹左 0；(0) + (3) = 3	[3]
6 / 3	压入数字 3	[3, 3]
7 / ^	先弹右 3，再弹左 3；(3) ^ (3) = 27	[27]
8 / 4	压入数字 4	[27, 4]
9 / 5	压入数字 5	[27, 4, 5]
10 / +	先弹右 5，再弹左 4；(4) + (5) = 9	[27, 9]
11 / /	先弹右 9，再弹左 27；(27) / (9) = 3	[3]
12 / 6	压入数字 6	[3, 6]
13 / !	弹 6，算 6!=720，压回	[3, 720]
14 / *	先弹右 720，再弹左 3；(3) * (720) = 2160	[2160]
15 / 7	压入数字 7	[2160, 7]
16 / 8	压入数字 8	[2160, 7, 8]
17 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[2160, 15]
18 / 9	压入数字 9	[2160, 15, 9]
19 / *	先弹右 9，再弹左 15；(15) * (9) = 135	[2160, 135]
20 / -	先弹右 135，再弹左 2160；(2160) - (135) = 2025	[2025]

最终值为 2025，数值栈峰值 3 项。

30.13 第 12 式

原式：((0-1)^2!*(3!!/4-5+6*7+8))*9

与第 8 式相比，指数由 2 变成 2!，二者恰好都为 2，因此数值仍 1647/4。RPN 必须保留“先算 2!再乘方”的过程，不能因为最后相同就抄成同一个记号串。

```
0 1 - 2 ! ^ 3 !! 4 / 5 - 6 7 * + 8 + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / `0`	压入数字 0	`[0]`
2 / `1`	压入数字 1	`[0, 1]`
3 / `-`	先弹右 1，再弹左 0；`(0) - (1) = -1`	`[-1]`
4 / `2`	压入数字 2	`[-1, 2]`
5 / `!`	弹 2，算 2!=2，压回	`[-1, 2]`
6 / `^`	先弹右 2，再弹左-1；`(-1) ^ (2) = 1`	`[1]`
7 / `3`	压入数字 3	`[1, 3]`
8 / `!!`	弹 3，算 3!!=3，压回	`[1, 3]`
9 / `4`	压入数字 4	`[1, 3, 4]`
10 / `/`	先弹右 4，再弹左 3；`(3) / (4) = 3/4`	`[1, 3/4]`
11 / `5`	压入数字 5	`[1, 3/4, 5]`
12 / `-`	先弹右 5，再弹左 3/4；`(3/4) - (5) = -17/4`	`[1, -17/4]`
13 / `6`	压入数字 6	`[1, -17/4, 6]`
14 / `7`	压入数字 7	`[1, -17/4, 6, 7]`
15 / `*`	先弹右 7，再弹左 6；`(6) * (7) = 42`	`[1, -17/4, 42]`
16 / `+`	先弹右 42，再弹左-17/4；`(-17/4) + (42) = 151/4`	`[1, 151/4]`
17 / `8`	压入数字 8	`[1, 151/4, 8]`
18 / `+`	先弹右 8，再弹左 151/4；`(151/4) + (8) = 183/4`	`[1, 183/4]`
19 / `*`	先弹右 183/4，再弹左 1；`(1) * (183/4) = 183/4`	`[183/4]`
20 / `9`	压入数字 9	`[183/4, 9]`
21 / `*`	先弹右 9，再弹左 183/4；`(183/4) * (9) = 1647/4`	`[1647/4]`

最终值为 1647/4，数值栈峰值 4 项。

30.14 第 13 式

原式：`((0-1)^2!*(3!!/4-5-6+7*8))*9`

与第 9 式同理，指数 2!先得到 2；括号 3/4−5−6+56=183/4，结果 1647/4。这里可用独立简化式核对栈结果。

```
0 1 - 2 ! ^ 3 !! 4 / 5 - 6 - 7 8 * + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / `0`	压入数字 0	`[0]`
2 / `1`	压入数字 1	`[0, 1]`

步骤 / 读到 token	这一动作	动作后的完整数值栈
3 / -	先弹右 1，再弹左 0；(0) - (1) = -1	[-1]
4 / 2	压入数字 2	[-1, 2]
5 / !	弹 2，算 2!=2，压回	[-1, 2]
6 / ^	先弹右 2，再弹左-1；(-1) ^ (2) = 1	[1]
7 / 3	压入数字 3	[1, 3]
8 / !!	弹 3，算 3!!=3，压回	[1, 3]
9 / 4	压入数字 4	[1, 3, 4]
10 / /	先弹右 4，再弹左 3；(3) / (4) = 3/4	[1, 3/4]
11 / 5	压入数字 5	[1, 3/4, 5]
12 / -	先弹右 5，再弹左 3/4；(3/4) - (5) = -17/4	[1, -17/4]
13 / 6	压入数字 6	[1, -17/4, 6]
14 / -	先弹右 6，再弹左-17/4；(-17/4) - (6) = -41/4	[1, -41/4]
15 / 7	压入数字 7	[1, -41/4, 7]
16 / 8	压入数字 8	[1, -41/4, 7, 8]
17 / *	先弹右 8，再弹左 7；(7) * (8) = 56	[1, -41/4, 56]
18 / +	先弹右 56，再弹左-41/4；(-41/4) + (56) = 183/4	[1, 183/4]
19 / *	先弹右 183/4，再弹左 1；(1) * (183/4) = 183/4	[183/4]
20 / 9	压入数字 9	[183/4, 9]
21 / *	先弹右 9，再弹左 183/4；(183/4) * (9) = 1647/4	[1647/4]

最终值为 1647/4，数值栈峰值 4 项。

30.15 第 14 式

原式：`(0!+1)*(2+3)*(4+5)/6*(7+8)*9`

三个括号分别为 2、5、9。乘除链先得 2×5×9/6=15，再乘 15 和 9，得到 2025。除法不是一个括号把后面所有乘项包起来，必须尊重左结合。

```
0 ! 1 + 2 3 + * 4 5 + * 6 / 7 8 + * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / !	弹 0，算 0!=1，压回	[1]
3 / 1	压入数字 1	[1, 1]

步骤 / 读到 token	这一动作	动作后的完整数值栈
4 / +	先弹右 1，再弹左 1；(1) + (1) = 2	[2]
5 / 2	压入数字 2	[2, 2]
6 / 3	压入数字 3	[2, 2, 3]
7 / +	先弹右 3，再弹左 2；(2) + (3) = 5	[2, 5]
8 / *	先弹右 5，再弹左 2；(2) * (5) = 10	[10]
9 / 4	压入数字 4	[10, 4]
10 / 5	压入数字 5	[10, 4, 5]
11 / +	先弹右 5，再弹左 4；(4) + (5) = 9	[10, 9]
12 / *	先弹右 9，再弹左 10；(10) * (9) = 90	[90]
13 / 6	压入数字 6	[90, 6]
14 / /	先弹右 6，再弹左 90；(90) / (6) = 15	[15]
15 / 7	压入数字 7	[15, 7]
16 / 8	压入数字 8	[15, 7, 8]
17 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[15, 15]
18 / *	先弹右 15，再弹左 15；(15) * (15) = 225	[225]
19 / 9	压入数字 9	[225, 9]
20 / *	先弹右 9，再弹左 225；(225) * (9) = 2025	[2025]

最终值为 2025，数值栈峰值 3 项。

30.16 第 15 式

原式：`(0!^1/2*3!!/(4+5)*6-(7+8))*9`

前半连续乘除仍得到 1；后面明确减去括号 7+8=15，所以括号值−14，乘 9 得−126。它与第 5 式的“先减 7 再减 8”在这个位置等价。

```
0 ! 1 ^ 2 / 3 !! * 4 5 + / 6 * 7 8 + - 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / !	弹 0，算 0!=1，压回	[1]
3 / 1	压入数字 1	[1, 1]
4 / ^	先弹右 1，再弹左 1；(1) ^ (1) = 1	[1]
5 / 2	压入数字 2	[1, 2]
6 / /	先弹右 2，再弹左 1；(1) / (2) = 1/2	[1/2]
7 / 3	压入数字 3	[1/2, 3]
8 / !!	弹 3，算 3!!=3，压回	[1/2, 3]
9 / *	先弹右 3，再弹左 1/2；(1/2) * (3) = 3/2	[3/2]
10 / 4	压入数字 4	[3/2, 4]
11 / 5	压入数字 5	[3/2, 4, 5]

步骤 / 读到 token	这一动作	动作后的完整数值栈
12 / +	先弹右 5，再弹左 4；(4) + (5) = 9	[3/2, 9]
13 / /	先弹右 9，再弹左 3/2；(3/2) / (9) = 1/6	[1/6]
14 / 6	压入数字 6	[1/6, 6]
15 / *	先弹右 6，再弹左 1/6；(1/6) * (6) = 1	[1]
16 / 7	压入数字 7	[1, 7]
17 / 8	压入数字 8	[1, 7, 8]
18 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[1, 15]
19 / -	先弹右 15，再弹左 1；(1) - (15) = -14	[-14]
20 / 9	压入数字 9	[-14, 9]
21 / *	先弹右 9，再弹左-14；(-14) * (9) = -126	[-126]

最终值为-126，数值栈峰值 3 项。

30.17 第 16 式

原式：`((0-1)^2*(3!!/4-5*(6-7-8)))*9`

0-1 的平方为 1。最内层 6-7-8 按左结合是-9，不是 6-(7-8)=7。于是 3/4-5×(-9)=183/4，最后乘 9，得 1647/4。

```
0 1 - 2 ^ 3 !! 4 / 5 6 7 - 8 - * - * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / 1	压入数字 1	[0, 1]
3 / -	先弹右 1，再弹左 0；(0) - (1) = -1	[-1]
4 / 2	压入数字 2	[-1, 2]
5 / ^	先弹右 2，再弹左-1；(-1) ^ (2) = 1	[1]
6 / 3	压入数字 3	[1, 3]
7 / !!	弹 3，算 3!!=3，压回	[1, 3]
8 / 4	压入数字 4	[1, 3, 4]
9 / /	先弹右 4，再弹左 3；(3) / (4) = 3/4	[1, 3/4]
10 / 5	压入数字 5	[1, 3/4, 5]
11 / 6	压入数字 6	[1, 3/4, 5, 6]
12 / 7	压入数字 7	[1, 3/4, 5, 6, 7]
13 / -	先弹右 7，再弹左 6；(6) - (7) = -1	[1, 3/4, 5, -1]
14 / 8	压入数字 8	[1, 3/4, 5, -1, 8]
15 / -	先弹右 8，再弹左-1；(-1) - (8) = -9	[1, 3/4, 5, -9]

步骤 / 读到 token	这一动作	动作后的完整数值栈
16 / *	先弹右-9，再弹左 5；(5) * (-9) = -45	[1, 3/4, -45]
17 / -	先弹右-45，再弹左 3/4；(3/4) - (-45) = 183/4	[1, 183/4]
18 / *	先弹右 183/4，再弹左 1；(1) * (183/4) = 183/4	[183/4]
19 / 9	压入数字 9	[183/4, 9]
20 / *	先弹右 9，再弹左 183/4；(183/4) * (9) = 1647/4	[1647/4]

最终值为 1647/4，数值栈峰值 5 项。

30.18 第 17 式

原式：`((0-1)^2!*(3!!/4-5*(6-7-8)))*9`

与第 16 式相比，指数为 2!，仍等于 2；其他过程相同。这种配对适合检查解析器是否把后缀阶乘正确附到指数上。

```
0 1 - 2 ! ^ 3 !! 4 / 5 6 7 - 8 - * - * 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / 1	压入数字 1	[0, 1]
3 / -	先弹右 1，再弹左 0；(0) - (1) = -1	[-1]
4 / 2	压入数字 2	[-1, 2]
5 / !	弹 2，算 2!=2，压回	[-1, 2]
6 / ^	先弹右 2，再弹左-1；(-1) ^ (2) = 1	[1]
7 / 3	压入数字 3	[1, 3]
8 / !!	弹 3，算 3!!=3，压回	[1, 3]
9 / 4	压入数字 4	[1, 3, 4]
10 / /	先弹右 4，再弹左 3；(3) / (4) = 3/4	[1, 3/4]
11 / 5	压入数字 5	[1, 3/4, 5]
12 / 6	压入数字 6	[1, 3/4, 5, 6]
13 / 7	压入数字 7	[1, 3/4, 5, 6, 7]
14 / -	先弹右 7，再弹左 6；(6) - (7) = -1	[1, 3/4, 5, -1]
15 / 8	压入数字 8	[1, 3/4, 5, -1, 8]
16 / -	先弹右 8，再弹左-1；(-1) - (8) = -9	[1, 3/4, 5, -9]

步骤 / 读到 token	这一动作	动作后的完整数值栈
17 / *	先弹右-9，再弹左 5；(5) * (-9) = -45	[1, 3/4, -45]
18 / -	先弹右-45，再弹左 3/4；(3/4) - (-45) = 183/4	[1, 183/4]
19 / *	先弹右 183/4，再弹左 1；(1) * (183/4) = 183/4	[183/4]
20 / 9	压入数字 9	[183/4, 9]
21 / *	先弹右 9，再弹左 183/4；(183/4) * (9) = 1647/4	[1647/4]

最终值为 1647/4，数值栈峰值 5 项。

30.19 第 18 式

原式：`((0!+1!)^2+(3!-4)+5*6)*(7*8)+9`

0!+1!=2，平方为 4；3!−4=2；5×6=30，括号合计 36。7×8=56，36×56=2016，再加 9 得 2025。

```
0 ! 1 ! + 2 ^ 3 ! 4 - + 5 6 * + 7 8 * * 9 +
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / !	弹 0，算 0!=1，压回	[1]
3 / 1	压入数字 1	[1, 1]
4 / !	弹 1，算 1!=1，压回	[1, 1]
5 / +	先弹右 1，再弹左 1；(1) + (1) = 2	[2]
6 / 2	压入数字 2	[2, 2]
7 / ^	先弹右 2，再弹左 2；(2) ^ (2) = 4	[4]
8 / 3	压入数字 3	[4, 3]
9 / !	弹 3，算 3!=6，压回	[4, 6]
10 / 4	压入数字 4	[4, 6, 4]
11 / -	先弹右 4，再弹左 6；(6) - (4) = 2	[4, 2]
12 / +	先弹右 2，再弹左 4；(4) + (2) = 6	[6]
13 / 5	压入数字 5	[6, 5]
14 / 6	压入数字 6	[6, 5, 6]
15 / *	先弹右 6，再弹左 5；(5) * (6) = 30	[6, 30]
16 / +	先弹右 30，再弹左 6；(6) + (30) = 36	[36]
17 / 7	压入数字 7	[36, 7]
18 / 8	压入数字 8	[36, 7, 8]
19 / *	先弹右 8，再弹左 7；(7) * (8) = 56	[36, 56]
20 / *	先弹右 56，再弹左 36；(36) * (56) = 2016	[2016]
21 / 9	压入数字 9	[2016, 9]

步骤 / 读到 token	这一动作	动作后的完整数值栈
22 / +	先弹右 9，再弹左 2016；(2016) + (9) = 2025	[2025]

最终值为 2025，数值栈峰值 3 项。

30.20 第 19 式

原式：((0!+1)*2)!+(3!-4)^(5+6)-7*8+9

0!+1=2，再乘 2 得 4，整个括号阶乘为 24。另一路 3!−4=2、5+6=11，得到 2048，最终 24+2048−56+9=2025。

```
0 ! 1 + 2 * ! 3 ! 4 - 5 6 + ^ + 7 8 * - 9 +
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / 0	压入数字 0	[0]
2 / !	弹 0，算 0!=1，压回	[1]
3 / 1	压入数字 1	[1, 1]
4 / +	先弹右 1，再弹左 1；(1) + (1) = 2	[2]
5 / 2	压入数字 2	[2, 2]
6 / *	先弹右 2，再弹左 2；(2) * (2) = 4	[4]
7 / !	弹 4，算 4!=24，压回	[24]
8 / 3	压入数字 3	[24, 3]
9 / !	弹 3，算 3!=6，压回	[24, 6]
10 / 4	压入数字 4	[24, 6, 4]
11 / -	先弹右 4，再弹左 6；(6) - (4) = 2	[24, 2]
12 / 5	压入数字 5	[24, 2, 5]
13 / 6	压入数字 6	[24, 2, 5, 6]
14 / +	先弹右 6，再弹左 5；(5) + (6) = 11	[24, 2, 11]
15 / ^	先弹右 11，再弹左 2；(2) ^ (11) = 2048	[24, 2048]
16 / +	先弹右 2048，再弹左 24；(24) + (2048) = 2072	[2072]
17 / 7	压入数字 7	[2072, 7]
18 / 8	压入数字 8	[2072, 7, 8]
19 / *	先弹右 8，再弹左 7；(7) * (8) = 56	[2072, 56]
20 / -	先弹右 56，再弹左 2072；(2072) - (56) = 2016	[2016]
21 / 9	压入数字 9	[2016, 9]
22 / +	先弹右 9，再弹左 2016；(2016) + (9) = 2025	[2025]

最终值为 2025，数值栈峰值 4 项。

30.21 第 20 式

原式：((0!^1+2)!!/(3!-4-5+6)-(7+8))*9

0!^1=1，加 2 为 3，双阶乘 3!!=3。分母 3!−4−5+6 按左结合为 6−4−5+6=3，所以商 1；减 15 得−14，再乘 9 得−126。

```
0 ! 1 ^ 2 + !! 3 ! 4 - 5 - 6 + / 7 8 + - 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / `0`	压入数字 0	`[0]`
2 / `!`	弹 0，算 0!=1，压回	`[1]`
3 / `1`	压入数字 1	`[1, 1]`
4 / `^`	先弹右 1，再弹左 1；`(1) ^ (1) = 1`	`[1]`
5 / `2`	压入数字 2	`[1, 2]`
6 / `+`	先弹右 2，再弹左 1；`(1) + (2) = 3`	`[3]`
7 / `!!`	弹 3，算 3!!=3，压回	`[3]`
8 / `3`	压入数字 3	`[3, 3]`
9 / `!`	弹 3，算 3!=6，压回	`[3, 6]`
10 / `4`	压入数字 4	`[3, 6, 4]`
11 / `-`	先弹右 4，再弹左 6；`(6) - (4) = 2`	`[3, 2]`
12 / `5`	压入数字 5	`[3, 2, 5]`
13 / `-`	先弹右 5，再弹左 2；`(2) - (5) = -3`	`[3, -3]`
14 / `6`	压入数字 6	`[3, -3, 6]`
15 / `+`	先弹右 6，再弹左-3；`(-3) + (6) = 3`	`[3, 3]`
16 / `/`	先弹右 3，再弹左 3；`(3) / (3) = 1`	`[1]`
17 / `7`	压入数字 7	`[1, 7]`
18 / `8`	压入数字 8	`[1, 7, 8]`
19 / `+`	先弹右 8，再弹左 7；`(7) + (8) = 15`	`[1, 15]`
20 / `-`	先弹右 15，再弹左 1；`(1) - (15) = -14`	`[-14]`
21 / `9`	压入数字 9	`[-14, 9]`
22 / `*`	先弹右 9，再弹左-14；`(-14) * (9) = -126`	`[-126]`

最终值为-126，数值栈峰值 3 项。

30.22 第 21 式

原式：`((0!^1+2)!!/(3!-(4+(5-6)))-(7+8))*9`

分母的嵌套必须逐层算：5−6=−1，4+(−1)=3，3!−3=3。分子仍 3，故商 1，减 15 再乘 9 得−126。它与第 20 式分母数值相同，但 RPN 的减法次序不同。

```
0 ! 1 ^ 2 + !! 3 ! 4 5 6 - + - / 7 8 + - 9 *
```

步骤 / 读到 token	这一动作	动作后的完整数值栈
1 / `0`	压入数字 0	`[0]`
2 / `!`	弹 0，算 0!=1，压回	`[1]`
3 / `1`	压入数字 1	`[1, 1]`
4 / `^`	先弹右 1，再弹左 1；`(1) ^ (1) = 1`	`[1]`
5 / `2`	压入数字 2	`[1, 2]`
6 / `+`	先弹右 2，再弹左 1；`(1) + (2) = 3`	`[3]`

步骤 / 读到 token	这一动作	动作后的完整数值栈
7 / !!	弹 3，算 3!!=3，压回	[3]
8 / 3	压入数字 3	[3, 3]
9 / !	弹 3，算 3!=6，压回	[3, 6]
10 / 4	压入数字 4	[3, 6, 4]
11 / 5	压入数字 5	[3, 6, 4, 5]
12 / 6	压入数字 6	[3, 6, 4, 5, 6]
13 / –	先弹右 6，再弹左 5；(5) – (6) = –1	[3, 6, 4, –1]
14 / +	先弹右-1，再弹左 4；(4) + (–1) = 3	[3, 6, 3]
15 / –	先弹右 3，再弹左 6；(6) – (3) = 3	[3, 3]
16 / /	先弹右 3，再弹左 3；(3) / (3) = 1	[1]
17 / 7	压入数字 7	[1, 7]
18 / 8	压入数字 8	[1, 7, 8]
19 / +	先弹右 8，再弹左 7；(7) + (8) = 15	[1, 15]
20 / –	先弹右 15，再弹左 1；(1) – (15) = –14	[–14]
21 / 9	压入数字 9	[–14, 9]
22 / *	先弹右 9，再弹左-14；(–14) * (9) = –126	[–126]

最终值为-126，数值栈峰值 5 项。

30.23 合上表格，试着自己找错

练习 1。 第 2 式如果使用 C++整数除法，中间 2*6/24 会变成多少？最后还会是 2025 吗？

解答：2×6=12，整数 12/24 截为 0，后面无论乘多少仍为 0，最后结果 0。表达式字符串相同，数值类型与除法语义不同，计算问题就不同。本章有理数表适合精确核对数学值，不能直接冒充整数程序的输出。

练习 2。 第 16 式把 6–7–8 错读成 6–(7–8)，会怎样影响括号？

解答：正确为(6–7)–8=–9，错误为 6–(–1)=7。正确的 3/4–5×(–9)=183/4，错读后成 3/4–35=–137/4。RPN 中应是 6 7 – 8 –；若变成 6 7 8 – –，就是另一棵语法树。比较 RPN 可以在最终乘 9 之前就定位错误。

练习 3。 第 12 式与第 8 式结果相同，能否说明它们是同一个语法结构？

解答：不能。第 12 式需要先算指数 2!，第 8 式直接用 2；只是 2!=2 造成数值相同。换成指数 3 与 3!就会不同。测试解析器时需要既有结果不同的区分例，也有结果相同但结构不同的例子，避免把错误结构碰巧算对当成解析正确。

本章的数值栈峰值是求值 RPN 所需的空间，不是中缀转 RPN 时的运算符栈峰值。后者还取决于优先表、括号与底部哨兵计数；两种栈保存不同对象，不能拿一个峰值替另一个答案。

31 换掉数字以后，仍然能重做的六种推理

本章使用另设的输入讲方法，不公开转载本地受限样卷的原页、完整题面或个人信息。做这些题时，先把中间状态写出来，再用一个公式压缩它；若公式和表不一致，先查端点和计费对象，不急着记答案。

31.1 一、步长也在变，累计进度才决定轮数

考查下面的控制过程，i、j 都从 0 开始。每轮先把旧 j 加到 i，再让 j 加 1：

```
i=0, j=0
当i<n:
    i=i+j
    j=j+1
```

取 n=15，每步完整状态为：

已完成轮数 k	本轮加上的旧 j	轮末 i	轮末 j	下一轮还进吗
0	—	0	0	进
1	0	0	1	进
2	1	1	2	进
3	2	3	3	进
4	3	6	4	进
5	4	10	5	进
6	5	15	6	不进

第一轮 i 不变，但完整状态改变了：j 从 0 变 1。只看 i 会误判死循环。做完 k 轮，j=k，i 是 0 到 k−1 的和，因此

$$i = \frac{k(k-1)}{2}.$$

要停下来，必须且只需$i \ge n$。对 n>0，最小整数轮数是

$$k = \left\lceil \frac{1+\sqrt{1+8n}}{2} \right\rceil.$$

推导来自解二次不等式$k^2 - k - 2n \ge 0$，负根无关，取正根以后向上取整。本例正根 6，恰好 6 轮。n=0 时一开始不进，须单独返回 0，不能套正根式得 1。

只问渐近阶时，无需真的开方：当 k≥2，$k(k-1)/2$介于$k^2/4$与$k^2/2$之间，退出阈值就与$\sqrt{n}$相差固定倍数。所以轮数和常数主体时间均为$\Theta(\sqrt{n})$。

若把增量改成j^2，累计进度为$0^2 + 1^2 + \cdots + (k-1)^2 = \Theta(k^3)$，轮数变成$\Theta(n^{1/3})$；若改成$2^j$，累计为$2^k - 1$，轮数变成$\Theta(\log(n+1))$。每次先写“k 轮总前进多少”，再反解 n，适用于比背循环类型更广的情况。

需要精确机器结果时，浮点根式可能在很大的整数边界附近舍入。可用整数二分找最小 k，使$k(k-1)/2 \ge n$，或先估计再用整数比较修正；乘法也需防溢出。渐近推导和工程实现是两个相连但不同的任务。

31.2 二、容量每次只长平方根，扩容费怎样摊

规则改成：容量 C 满时，新容量为$C + \lceil\sqrt{C}\rceil$。初始容量 1，只做单项尾追加。下面按真规则列出到 16 项时发生过的扩容：

触发追加的序号	旧容量 C	增加格数$\lceil\sqrt{C}\rceil$	新容量	本次搬旧项
2	1	1	2	1
3	2	2	4	2

触发追加的序号	旧容量 C	增加格数$\lceil\sqrt{C}\rceil$	新容量	本次搬旧项
5	4	2	6	4
7	6	3	9	6
10	9	3	12	9
13	12	4	16	12

其余追加只写新项，不搬旧项。前 16 项共搬$1+2+4+6+9+12=34$项，另有 16 次新项写入，总 50。单个表还看不出一般阶，需要把扩容按容量大小分组。

先用一段实际轨迹理解分组。容量不是每个整数都会取到：完整走过`[4,8)`时，旧容量先为 4，扩到 6；再以 6 扩到 9，已经越过 8。这个容量段付了 4+6=10 项复制，共两次扩容。不能要求恰好停在 2M。

一般取足够大的 M，把旧容量落在`[M,2M)`的扩容归到同一组。令$A=\lceil\sqrt{M}\rceil$、$B=\lceil\sqrt{2M}\rceil$，这一组每次增加量都在 A 与 B 之间。进入本组时第一个旧容量C_0满足$M\le C_0<M+B$：上一容量小于 M，最后一步的增量不超过 B。离开本组后的容量C_r满足$2M\le C_r<2M+B$。

若本组共有 r 次扩容，则它们总增加C_r-C_0，一方面不超过 rB，另一方面至少 rA。结合两个端点，得到

$$\frac{M-B}{B}<r<\frac{M+B}{A}.$$

当 M 足够大，B 小于 M/2，并且 A、B 都与$\sqrt{M}$相差固定倍数。因此左界至少是正常数倍$\sqrt{M}$，右界至多是常数倍$\sqrt{M}$，即$r=\Theta(\sqrt{M})$。小的 M 只影响有限初始成本。

每次旧容量介于 M 与 2M，复制项数为$\Theta(M)$。本组总复制于是为

$$\Theta(\sqrt{M})\cdot\Theta(M)=\Theta(M^{3/2}).$$

接着按`[1,2)`、`[2,4)`、`[4,8)`等二倍容量段分组。各段的上界随 M 按固定比$2^{3/2}$增长，有限几何和不超过最后一段上界的固定倍数，给出总复制$O(n^{3/2})$。最后一段即使只走了一部分，也不会超过该段全部走完的费用。

下界也要选真正走完的段。取二的幂 M，使$n/4<M\le n/2$；做 n 次追加后，容量至少 n，所以`[M,2M)`已跨完（若容量刚等于 2M，该段也已走完）。它单独就复制$\Omega(M^{3/2})=\Omega(n^{3/2})$项。因此总复制为$\Theta(n^{3/2})$，加 n 次新项写入仍同阶，除以操作次数 n 得到摊还$\Theta(\sqrt{n})$。

还可以用积分检查同一结果：在容量 C 附近，每增加$\sqrt{C}$格付一次 C 的迁移费，因此每增加 1 格平均约付$C/\sqrt{C}=\sqrt{C}$。从小规模累计到 n，面积为

$$\int_1^n\sqrt{C}\,dC=\frac{2}{3}(n^{3/2}-1).$$

积分解释的是增长趋势，严格上下界来自前面的完整容量段夹逼；不能只把“次数大概除一下”当成离散精确等式。

若每次增加$\Theta(C^{\alpha})$，其中$0\le\alpha<1$固定，在$[M,2M]$中扩容次数为$\Theta(M^{1-\alpha})$，每次搬$\Theta(M)$，总量为$\Theta(M^{2-\alpha})$。全程也为$\Theta(n^{2-\alpha})$。α=0 就是固定增加常数、总平方；α=1 的固定比例增长可另用几何和得到总线性。若 α 随 C 变、存在额外大跳跃或允许删除，原操作序列与夹逼条件变化，要重新分析。

31.3 三、表达式树：先算左边，为什么会占着一格等右边

这里只统计 RPN 执行时的**数值栈**，不计运算符栈，也不计语言递归调用帧。叶子读入时压一个数，二元运算弹两个、压一个。固定先完成左子树、再右子树。

叶子所需峰值$S(\text{叶})=1$。对根 T，左子树为 L、右子树为 R：算左边时最多用$S(L)$格；左边完成后留下一个结果；算右边时这个左值仍在底下，要再加$S(R)$格。因此

$$S(T)=\max\big(\,S(L),1+S(R)\big)\,.$$

这个式子中的 1 是“已经算好的左结果”，不是根运算符自己占一个数值格。

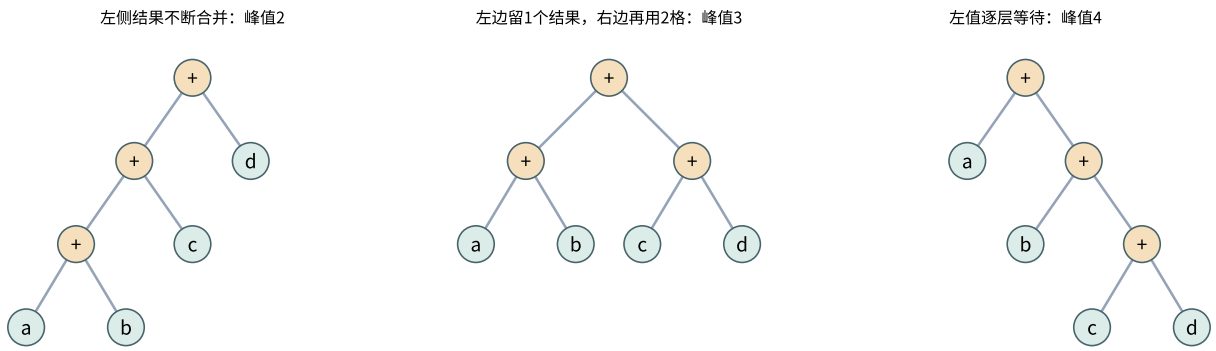


图 35　同样四个叶子的三棵表达式树。固定先算左边时，左倾树不断合并只需 2 格，平衡树需 3 格，右倾树的多个左值同时等待，需 4 格。

对(a+b)+(c+d)，用具体数 1、2、3、4 走一次：

```
读a=1：[1]
读b=2：[1,2]        峰2
读+：  [3]
读c=3：[3,3]
读d=4：[3,3,4]      峰3
读+：  [3,7]
读+：  [10]
```

左树需 2 格、右树需 2 格，根峰值 max(2,1+2)=3，完全吻合。换成左倾(((a+b)+c)+d)，每次刚压第二数就合并，峰值只有 2；换成右倾 a+(b+(c+d))，要先压 a 等，再压 b 等，再压 c、d，峰值 4。三棵树都有 4 个叶、3 个运算符，但求值顺序造成的等待不同。

如果允许先算右树，空间需求改为$\max(S(R), 1+S(L))$。选较小者可以省栈；但仍需保存每个结果的左右身份。计算减法时先算右边，不表示最后可以把“左减右”换成“右减左”；有副作用或规定执行次序的语言表达式也未必允许重排。

31.3.1 额外限制“左子树不少于右子树”时，峰值能有多大

设每个内部节点都满足左子树内部节点数不少于右子树。问达到峰值 s 至少需要多少内部节点，记为N_s。这里只考虑纯二元表达式，叶子不是内部节点。

$N_1 = 0$，一个叶就能达到峰值 1。选一棵达到峰值 s 且内部节点最少的树。如果峰值完全由左子树的$S(L) = s$产生，单独取这棵左子树会用更少节点也达到 s，与最小性冲突。所以在最小构造中，峰值必须由$1 + S(R)$产生，即右子树至少需要峰值 s−1、至少有N_{s-1}个内部节点；左侧按限制至少同样大，再加根，得到

$$N_s \geq 2N_{s-1} + 1.$$

取两棵相同的最小 s−1 构造作为左右孩子，就可以达到等号。因此不是只得到下界，而是

$$N_s = 2^{s-1} - 1.$$

前几个值为 0、1、3、7、15；对应峰值 1、2、3、4、5。若有 k 个内部节点，则$2^{s-1} - 1 \leq k$，取对数得

$$s \leq 1 + \lfloor \log_2(k+1) \rfloor.$$

15 个内部节点的完美树确实需 5 格。15 个内部节点的全左长树却只需 2 格；公式是可能达到的峰值上界，没有说同规模所有形状都同峰值。

31.4 四、全数组有很多逆序，指定区间里至少留下多少

逆序是下标 i<j 但值$a_i > a_j$的一对。长度 n 的数组共有$n(n-1)/2$对下标，指定长度 m 的连续区间内部有$m(m-1)/2$对；其余对都是“至少一端在区间外”。无论怎样摆值，区间外能承担的逆序最多是

$$B = \frac{n(n-1) - m(m-1)}{2}.$$

如果总逆序数是 I，内部至少要接住超过 B 的部分，所以

$$I_{\text{内}} \geq \max(0, I - B).$$

这里 I 是已知总量，B 是外部容量上界，不是已经保证能同时实现的外部实际逆序数。要声称最少“恰好”，还要给达到下界的构造。

取 n=6，指定前 4 项为区间，m=4，总 I=13。全部 15 对，内部最多 6 对，外部容量 B=9，所以内部至少 4。现在真的构造：

```
[5,6,3,4 | 2,1]
```

内部`[5,6,3,4]`的逆序是(5,3)、(5,4)、(6,3)、(6,4)，共 4；前 4 项每项都大于后面的 2、1，跨界有 8；尾部 2>1 又有 1，总 4+8+1=13。所有条件同时满足，所以在这个指定区间例子中最少确实为 4。

这比随意说“把剩下 9 个放外面”多了一步：构造确认三部分的数值大小关系能共存。若区间位置改变或另外要求某些位置固定，未必仍能原样达到。

若 I=5，同样的粗下界只能给 0。反例`[1,3,5,6 | 2,4]`前 4 项有序，内部 0 逆序；跨界逆序为(3,2)、(5,2)、(5,4)、(6,2)、(6,4)，恰 5。于是“总共有 5 个逆序，所以这里至少有一个”的直觉也被实际排列反驳了。

31.5 五、归约：新问题不能制造旧问题没有的答案

设旧问题给互异实数$x_1, \ldots, x_n$，要判断有没有三个不同索引的数和为 0。把每个 x 变成点(x, x^3)，调用一个“找三点共线”的算法。为什么找共线能帮零和，必须证明两个方向。

任取三个互异数 a、b、c。横坐标不同，斜率分母都不为 0。a 与 b 对应点的斜率是

$$\frac{b^3 - a^3}{b - a} = a^2 + ab + b^2,$$

a 与 c 对应点的斜率是$a^2 + ac + c^2$。两斜率相减：

$$ab + b^2 - ac - c^2 = (b - c)(a + b + c).$$

b−c 不为 0，因此斜率相等当且仅当 a+b+c=0。正向说明真解不会丢；反向说明算法找到的共线三点不会是伪解。

a,b,c	和	AB 斜率	AC 斜率	共线吗
−2,−1,3	0	$4 + 2 + 1 = 7$	$4 - 6 + 9 = 7$	是
−2,−1,2	−1	7	$4 - 4 + 4 = 4$	否

生成 n 个点，在实数 RAM 且乘法单位成本的模型中每点只做常数次乘法，总$O(n)$；保留原索引，找到三点后也可常数时间映回原三数。若使用 L 位整数，立方结果约 3L 位，运算成本与输出长度要按位模型另算。若允许重复数值，点可能重合，必须先单独处理重复的零和组合或改变归约条件，不能直接复用互异证明。

把箭头写成“零和问题→共线问题”。这表示共线快算法可以解决零和问题；若已知零和在某个模型中有下界，才可转移成共线下界。仅仅说共线“看起来难”，不能反向证明零和难；本章也没有借这一归约宣称已经证明某个未给出的二次时间下界。

31.6 六、找下一个节点的费用，也属于遍历

先检查输入信息够不够唯一确定目标。普通先序 A,B 和后序 B,A，可以对应 A 左孩子 B，也可以对应 A 右孩子 B；没有空标记或真树限制，不能写一个算法“恢复唯一原树”。把不唯一问题硬编码成任选一种，得到的是一个可能解，承诺已经变了。

如果给 n 条 parent 记录，只能确定父子关系；若还要区分左右，必须另给孩子槽位或顺序。可以先扫描 n 条记录建立孩子表，预处理$O(n)$，再做常规 DFS。这个预处理不是免费，也不能从“父亲可直接读”推断“孩子也可以直接读”。

以树 A 左 B 右 C、B 右 D 为例，显式栈保存待办节点，先序如下：

```
初始[A]
弹A访问，先压C再压B：栈[C,B]
弹B访问，压D：栈[C,D]
弹D访问：栈[C]
弹C访问：栈[]
```

每个孩子只由唯一父亲压一次，每个节点也只弹一次，得到线性总工作。若改为“每次输出后，从根搜索下一个节点”，即使最终输出仍是 A、B、D、C，搜索过程中重复走过的边也可能很多。访问次数与定位费用必须分开。

同理，分治求和有对数层，并不只有对数时间。根处理左右两半，每层会出现许多调用；总时间把所有调用相加，调用栈空间则只看某一时刻同时活着的一条路径。第 32 章 G(G(n))又是另一种情况：先算内层返回值，再以它作为外层入口规模，不能把“两次调用”简单相加成两份同规模费用。

31.7 自己更换输入

1. 容量每次增加$\lceil C^{1/3} \rceil$，到 n 项的总复制是什么阶？说明“每次扩容多少次”的分母来自哪里。
2. `(a+b)*(c+(d+e))`固定左先右后，写 RPN 值栈峰值。随后尝试右先，是否减少？
3. 将 n=6、前 4 项区间、总 I=13 的构造换一个，使内部仍恰 4。不能只报公式。
4. 归约若只证明零和推出共线，会留下哪一种错误可能？

答案：

1. 在容量段$[M, 2M]$里，每次只增加$\Theta(M^{1/3})$格，要填满$\Theta(M)$格需要$\Theta(M^{2/3})$次，每次搬$\Theta(M)$，段总$\Theta(M^{5/3})$；全程同阶为$\Theta(n^{5/3})$。分母是每次实际增加的格数，不是每次复制量。
2. 左树需 2，右树 `c+(d+e)`需 3，先左后右为 max(2,1+3)=4。先右时右树内部仍按原顺序需 3，留 1 值后算左最多 1+2=3，所以根可降到 3；乘法允许该重排且无副作用时可以采用。若连右树内部也重排，还要单独求它的最小需求。
3. 例如`[6,4,3,5,2,1]`：前 4 项内逆序为(6,4)、(6,3)、(6,5)、(4,3)，共 4；跨界 8、尾部 1，总 13。数值不同、分段计数相同。
4. 可能有和不为 0 的三数也被映射成共线，导致新问题返回真而旧问题其实是假。只有双向等价加上合法索引映射，才能安全调用新问题的求解器。

32 习题 1–32：把十六段程序逐层数清楚

PS01 第 16 项让我们独立分析《数据结构习题解析》1-32。本章对应作者官方合体 PDF 第 446 到 455 页，亦即习题解析印刷页 24 到 33；第 456 页为空白。原程序可从作者教材页进入合体 PDF 阅读。下面保留题号顺序，用自己的推导解释每段执行成本，并把容易混淆的返回值、轮数与递归深度分别列出。
题目使用抽象机器，假设字长足够、移位是单位操作、递归不会因真实机器栈限制而失败。这里的 T 表示执行成本，V 表示返回值。把原程序直接交给固定字长 C++运行，可能先遇到溢出或非法移位；若用大整数，又要另算位成本。这些实现边界不改变题目指定模型下的推导，却决定实验能测到什么。

32.1 1：矩形循环区域，行数乘列数

控制代码（按原题端点）：

```
for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; ++j) { /* 计一次 */ }
```

第一段的核心是 n 轮外循环，每轮 n 次内循环。若把一次主体执行画成坐标(i,j)，正好是 n×n 方格，每格访问一次，总 n^2次。
n=4 时，每行 4 次，共 4+4+4+4=16 次。每个循环还要增加变量、检查条件，内层条件每行约 n+1 次，总也为 $\Theta(n^2)$，不会改变阶。基础操作不是只有函数体里面那一条，但主项可由它代表。
把 n=3 的每一个内层状态列出来，就得到坐标格，不依赖“二重循环”这个外观：

```
外i=0：j=0 → 1 → 2，3次；j变3，条件失败。
外i=1：j重新置0，再走0 → 1 → 2，3次。
外i=2：j又重新置0，再走0 → 1 → 2，3次。
i变3，外条件失败；总9次主体。
```

每次重入内循环都会重新初始化 j，所以它不是“j 只从 0 走到 n 一次”。

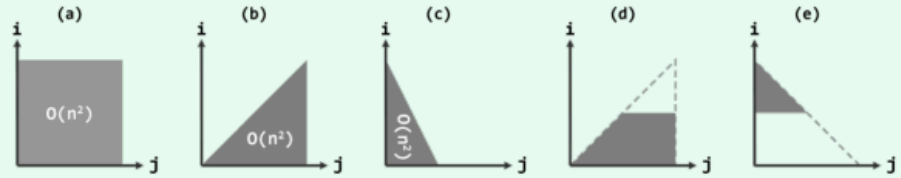
```
void F(int n) {
   for (int i = 0; i < n; i ++)
   for (int j = 0; j < n; j ++);
}
```

【解答】

这是最基本的二重循环模式，其特点是循环控制变量均按算术级数变化。内循环的每次迭代只需$O(1)$时间，故执行时间取决于总体的迭代次数。因外、内循环的范围分别是$i \in [0, n)$和$j \in [0, n)$，故累计迭代次数为：

$$\sum_{i=0}^{n-1}\sum_{j=0}^{n-1}1=\sum_{i=0}^{n-1}n=n\times n=O(n^2)$$

实际上在如图x1.9(a)所示的(i, j)坐标平面上，此二重循环的执行过程对应于匀速地自下而上、由左到右逐行扫过矩形$[0, n) \times [0, n)$。因此，这一过程所需的时间也就对应于该矩形的面积，就渐进意义而言即是$O(n^2)$。



图x1.9 二重循环执行时间的对应图形

相对于以上严格的计算，基于这一图形式理解的估算不仅更加简捷，而且可以更好地体现通过大O记号刻画渐进复杂度总体趋势的要义。而对于接下来更为复杂的情况，这种方法的上述优势则尤为突出。

```
void F(int n) {
   for (int i = 0; i < n; i ++)
   for (int j = i; j < n; j ++);
}
```

【解答】

这也是一种典型的二重循环模式，循环控制变量仍然按算术级数变化。与上例相比，外循环的范围依然是$i \in [0, n)$，但内循环的范围则变成$j \in [i, n)$，故累计迭代次数为：

$$\sum_{i=0}^{n-1}\sum_{j=i}^{n-1}1=\sum_{i=0}^{n-1}(n-i)=\sum_{(k=n-i)=1}^{n}k=n(n+1)/2=O(n^2)$$

也可套用以上基于图形的估算方法，在如图x1.9(b)所示的(i, j)坐标平面上，此二重循环的执行过程对应于匀速地自下而上、由左到右逐行扫过三角形$[0, n) \times [i, n)$。该三角形的面积大致为原矩形的一半——就渐进意义而言，时间复杂度依然是$O(n^2)$。

图 36　官方合体 PDF 第 447 页完整原图，含前两段程序与五种循环区域的几何图。阴影中的一个格代表一次主体动作；本章逐段重新核对边界，尤其第 5 段按实际 n-i 计数。

上界是常数倍 n^2，下界同样至少有 n^2 次主体，因此为 $\Theta(n^2)$。若只写“有两层 for 所以平方”，这份理由在第 6 与第 8 段就会失效；这里正确是因为每行确实有 n 项。

32.2 2：三角形区域，逐行长度不同

控制代码（按原题端点）：

```
for (int i = 0; i < n; ++i)
    for (int j = i; j < n; ++j) { /* 计一次 */ }
```

第二段内层从当前外层位置扫到边界。各行长度依次 n、n−1、…、1，因此

$$N(n)=\sum_{i=0}^{n-1}(n-i)=\frac{n(n+1)}{2}=\Theta(n^2).$$

n=4 时为 4+3+2+1=10。它大约是第一段的一半，但渐近阶相同。判断下界可只看前 n/2 行：每行至少 n/2 次，已足以贡献常数倍 n^2。

n=4 时每行实际 j 值是 `i=0:0,1,2,3`；`i=1:1,2,3`；`i=2:2,3`；`i=3:3`。将第 1 行与第 4 行相加是 4+1=5，第 2 行与第 3 行也是 3+2=5，共两对得到 10。一般把 1 到 n 与倒序 n 到 1 相加，每对为 n+1，共 n 对，再除 2，得到$n(n+1)/2$。求和公式不是额外假设，而是把同一批行长度配对。

如果循环端点改为不含当前位置，各行变 n−1、…、0，精确次数变 n(n−1)/2，仍平方级。端点差一会影响精确值，不能在考试问“执行几次”时直接忽略；问 Θ 时才可在证明后舍去低阶项。

32.3 3：内层步长 2013，常数再大也仍是常数

控制代码（按原题端点）：

```
for (int i = 0; i < n; ++i)
    for (int j = 0; j < i; j += 2013) { /* 计一次 */ }
```

内层每次增加固定 2013，第 i 轮约执行 i/2013 次。取整误差每轮至多常数，总误差 O(n)，主和为

$$N(n)=\sum_{i=0}^{n-1}\left\lceil\frac{i}{2013}\right\rceil=\frac{n(n-1)}{2\cdot 2013}+O(n)=\Theta(n^2).$$

例如将 2013 暂时换成便于手算的 3，外层上限 i 从 1 到 9，相应循环次数可能按端点约定取 ceil(i/3)，依次 1、1、1、2、2、2、3、3、3，总 18；外层规模扩大，仍是二次增长。这个小数值示例解释步长作用，不替换原题中的固定常数。

按原常数取几个确切边界：i=0 时一次也不做；i=1 到 2013，j 只有 0，做 1 次；i=2014 到 4026，j 为 0、2013，做 2 次；i=4027 时 j 为 0、2013、4026，做 3 次。故每行恰为$\lceil i/2013\rceil$。n=4028 时外层最后 i=4027，总主体数为$2013\times 1+2013\times 2+1\times 3=6042$。这个小表同时验证了严格小于边界和取整方向。

原常数很大，1≤n≤2014 时除 i=0 外每行恰做 1 次，主体精确为 n−1 次，所以小规模曲线确实接近线性。渐近结论讨论无界增长的 n，不由小规模测时曲线决定。若步长改成 n，就不再是固定常数，必须重新求和。

32.4 4：只做前半外循环，内层却仍然很长

控制代码（按原题端点）：

```
for (int i = 0; i < n / 2; ++i)
    for (int j = i; j < n; ++j) { /* 计一次 */ }
```

外层只有约 n/2 轮，每轮内循环长度在约 n/2 到 n 之间。上下夹逼为

$$c_1n^2\le N(n)\le c_2n^2$$

其中 c_1、c_2为与 n 无关的正常数，所以 $\Theta(n^2)$。n=8 时，若内层长度为 8、7、6、5，总 26，已经能看出不是只执行 4 次。

n=8 时外层 i 只取 0、1、2、3，因为整数 `n/2` 等于 4；内层分别取 `0..7`、`1..7`、`2..7`、`3..7`，行长 8、7、6、5。若 n=9，外层仍只有 4 行，行长 9、8、7、6；不能把 `i<n/2` 在整数代码里擅自当作$i<4.5$而多走 i=4。精确式可令$h=\lfloor n/2\rfloor$，总数$hn-h(h-1)/2$。

这里不必为了得到 Θ 而求一个很长的精确闭式。行数 Θ(n)、每行长度 Θ(n)，且界对所选所有行都成立，直接相乘即可。若某些行短某些行长，不能只取最小或最大后就宣称紧界，应检查足够多行都提供所需下界。

32.5 5：后半个三角形，仍有平方数量的格子

控制代码（按原题端点）：

```
for (int i = n / 2; i < n; ++i)
    for (int j = 0; j < n - i; ++j) { /* 计一次 */ }
```

外层走后半段，内层长度从约 n/2 逐步减到 1。总和约为 1+2+…+n/2，所以 Θ(n²)。n=8 的简化行长 4、3、2、1，总 10；n 翻倍时行数与最长行都翻倍，面积约变 4 倍。

n=8 时，外 i=4、5、6、7，对应内 j 分别为 0..3、0..2、0..1、0，长度 4、3、2、1。n=9 时从 i=4 开始，行长 5、4、3、2、1，共 15。一般令$s = \lceil n/2 \rceil$，主体数为$s(s+1)/2$。原页说明中内范围有一处写成[0,i)，但实际代码是 j<n-i；这里必须跟代码而不是跟那句说明。

旧解答旁的说明若写成另一个区间，应回到实际循环条件数格子，而不是同时抄两套互相冲突的文字。求和下标的范围是程序语义的一部分。

前 5 段的共同结论都是平方，但理由不全一样：矩形、完整三角形、固定步长三角形、梯形、半三角形。把每种区域画出来，后续遇到换边界的程序就能重新分析，而不必记住第几个答案。

32.6 6：外面 n 轮，里面翻倍到 n

控制代码（按原题端点）：

```
for (int i = 0; i < n; ++i)
    for (int j = 1; j < n; j <<= 1) { /* 计一次 */ }
```

内层状态 1、2、4、8……，在达到 n 以前执行约 $\log_2 n$ 次。外层每次都重新从 1 开始，所以每一轮都要重新支付这份成本，总 Θ(n log n)。

n=16 时，若条件为 j<n，内层取 1、2、4、8，共 4 次，外层 16 轮，总 64 次；若条件为 j≤n，则每轮多一次取 16，总 80 次。两者同阶，精确计数却不同。

精确地，第 t 次主体时$j = 2^t$，t 从 0 开始。合法条件$2^t < n$，因此 n>1 时每轮主体数为$\lceil \log_2 n \rceil$。n=9 时 j 是 1、2、4、8，做 4 次；n=8 时没有最后的 8，只做 3 次。外层 n 次重置 j，总主体数就是$n\lceil \log_2 n \rceil$，端点已经明确。

不要因为内层变量翻倍就说“所有轮合起来是一个几何和”。这里每个 j 对应一次常数操作，而不是做 j 次操作；内层总成本是项数 log n。若主体改成重复 j 次，才需相加 1+2+4+...得到 Θ(n)，再乘外层 n 会变平方级。

32.7 7：翻倍到固定上限，关于 n 就是常数

控制代码（按原题端点）：

```
for (int i = 0; i < n; ++i)
    for (int j = 1; j < 2013; j <<= 1) { /* 计一次 */ }
```

内层翻倍却只到 2013，与输入规模 n 无关。因为 2^{10}=1024、2^{11}=2048，不论 n 多大，内层都只有约 11 次检查范围内的主体动作。外层 n 轮，总 Θ(n)。

完整内层轨迹是 1,2,4,8,16,32,64,128,256,512,1024，共 11 次；下一值 2048 不小于 2013 而退出。因此主体精确执行 11n 次，内层退出判断另做 n 次。n 从 100 涨到百万，只改变重复这条固定轨迹的次数。

第 6 与第 7 的代码可能只差一个上限，渐近结论就从 n log n 变为 n。分析要问“哪个量会随输入增长”，不能只搜源代码里的乘法或移位符号。

若常数 2013 本身变成另一个输入参数 M，则应写 Θ(n log M)，再根据 n、M 之间是否有关联决定是否可化简成单变量形式。

32.8 8：步长随外层变化，出现调和和式

控制代码（按原题端点）：

```
for (int i = 1; i < n; ++i)
    for (int j = 0; j < n; j += i) { /* 计一次 */ }
```

先固定 n=8，看每个 i 到底让 j 访问哪些位置，再决定公式。j 总从 0 起，所以大步长也往往不只执行一次。n=8 各行真正访问的 j 是：

i	j 的值	次数
1	0,1,2,3,4,5,6,7	8
2	0,2,4,6	4
3	0,3,6	3
4	0,4	2
5	0,5	2
6	0,6	2
7	0,7	2

i=7 时并不是 1 次，因为 j=0 也执行一次。由$j = ti < n$得$t < n/i$，从 t=0 计数恰为$\lceil n/i \rceil$，所以这里不能误用向下取整。每项介于n/i与$n/i+1$之间，求和误差才小于 n。于是一般总数为

$$N(n) = \sum_{i=1}^{n-1} \left\lceil \frac{n}{i} \right\rceil = nH_{n-1} + O(n),$$

其中$H_{n-1} = 1 + 1/2 + \cdots + 1/(n-1)$叫调和和。本例精确为 8+4+3+2+2+2+2=23。

H_n 为对数级，可把分母按 2 的幂分组：1 一组，2 到 3 一组，4 到 7 一组，每组总贡献在正常数范围内，共 Θ(log n)组。这个解释也说明为什么“大量后面很小的项”仍会累积出一个对数因子。

32.9 9：一个循环，累计步长是三角数

控制代码（按原题端点）：

```
for (int i = 0, j = 0; i < n; i += j, ++j) { /* 计一次 */ }
```

i、j 从 0 开始，每轮做 i+=j，再让 j 加 1。k 轮后 i=k(k−1)/2、j=k。n=10 时状态依次(0,1)、(1,2)、(3,3)、(6,4)、(10,5)，5 轮结束。

退出需要 k(k−1)/2≥n，故 k=Θ(√n)，每轮常数工作，总 Θ(√n)。第一轮 i 没变，但 j 变了；若只盯 i 就说“状态不动”，会漏掉下一轮增量来源。

这里的输入 n 以一个整数给出；若按 n 的二进制位数 b 计，√n 仍可约为$2^{b/2}$。模型中所谓“平方根时间”是关于数值 n，不自动等于关于输入编码长度的多项式时间。

32.10 10：i 的指数也在翻倍，得到双对数

控制代码（按原题端点）：

```
for (int i = 1, r = 1; i < n; i <<= r, r <<= 1) { /* 计一次 */ }
```

r 从 1 开始每轮加倍，i 从 1 开始每轮左移 r 位。左移 r 位是乘2^r，不是乘 r，也不是只乘 2。

逐轮列出：初始 i=1、r=1；第一轮 i=2、r=2；第二轮 i=8、r=4；第三轮 i=128、r=8；第四轮 i=32768、r=16。k 轮后 i 的二进制指数是 1+2+4+…+2^{k-1}=2^k−1，所以

$$i = 2^{2^k-1}.$$

两条逗号更新从左向右执行，i 使用的是本轮旧 r。若 n=1000：

已完成轮数 k	i	r	下一次 i<n
0	1	1	真
1	2	2	真
2	8	4	真
3	128	8	真
4	32768	16	假，停止

所以恰 4 轮。把“左移 r 位”误当“左移 1 位”会得到完全不同的轨迹；把更新顺序反过来也会变成 2、8 等幂次错位。

让 i 达到 n，需$2^k-1\geq\log_2 n$，故 k=Θ(log log n)。这是对数的对数，来源是指数本身在几何增长。真实 C++在移位位数达到类型位宽前就要特别处理，不能把这张抽象表无限照跑。

32.11 11：指数塔增长，反复取对数数回去

控制代码（按原题端点）：

```
for (int i = 1; i < n; i = 1 << i) { /* 计一次 */ }
```

i 每次变为2^i，状态 1、2、4、16、65536……。到达一个极大阈值 n，倒着问：对 n 反复取 $\log_2$，多少次降到不大于 1？这个次数记为 $\log_2^*$ n，读作迭代对数。

例如 n=65536，依次取 log 得到 16、4、2、1，共 4 次；正向从 1 到 65536 也是 4 次指数化。成本 Θ($\log^*$ n)，虽然现实规模下数值很小，作为无界函数仍会增长，不能严格写成 Θ(1)。

取 n=17，正向为 1→2→4→16→65536，4 次主体后结束；反向取对数是 17→约 4.087→约 2.031→约 1.022→约 0.032，也是 4 次降到不大于 1。这里 log 星数的是操作“反复取对数”的次数，不能把星号读成乘号。n≤1 时循环 0 次，讨论渐近阶时默认 n 足够大。

它也不是 log log n。若 n 是高度不断增长的指数塔，双对数仍随塔高后的庞大数值变化，log*只记录需要剥掉多少层指数。

32.12 12：G 返回平方，第二次调用吃的是返回值

控制代码（按原题端点）：

```
int F(int n) { return n > 0 ? G(G(n - 1)) : 0; }
int G(int n) { return n > 0 ? G(n - 1) + 2*n - 1 : 0; }
```

辅助 G(m)把前 m 个奇数相加：1+3+…+(2m−1)=m^2，所以返回值 V_G(m)=m^2，执行成本 T_G(m)=Θ(m)。主函数先算 G(n−1)，再将返回的$(n-1)^2$作为另一次 G 的参数。

取 n=5，第一次 G(4)加 4 个奇数得到 16；第二次 G(16)加 16 个奇数得到 256。总加法次数约 4+16=20，而返回 256。一般返回$(n-1)^4$，运行成本 Θ(n)+Θ($(n-1)^2$)=Θ(n^2)。

以 G(4)展开，调用依次是 G4→G3→G2→G1→G0；返回依次 0、1、4、9、16，新增的数为 1、3、5、7。每次从$k-1$项的平方加上$2k-1$，恰为k^2，因此返回值与成本都能归纳证明。第二次 G(16)则有 17 个调用、16 次加奇数；加上第一次 5 个调用，共 22 个 G 调用，而不是 256 个。若统计的是非基础递归步，则是 4+16=20，两个计数口径只差基例，均为平方阶。

把“返回多大”和“执行多久”分成两列，就不会把四次方返回值误当四次方时间。相反，一个函数也可以花很久最后只返回 0，输出大小本身并不决定所有计算成本。

32.13 13：昂贵的是循环条件里的开方子过程

控制代码（按原题端点）：

```
void F(int n) { for (int i = 1; i < n / G(i, 0); ++i) {} }
int G(int n, int k) {
    return n < 1 ? k : G(n - 2*k - 1, k + 1);
}
```

G 从输入 m 中依次减去 1、3、5……，第 k 次累计减去 k^2，首次使剩余非正时返回 k，所以 G(m,0)=ceil(√m)，成本 Θ(√m)。

例如 m=10，依次减 1 后余 9，减 3 余 6，减 5 余 1，减 7 余−6，共 4 步，ceil√10=4。m=9 时减 1、3、5 恰到 0，共 3 步，边界也吻合。

F 在第 i 次检查中调用 G(i,0)，条件大致 i<n/G(i,0)。以 G 约√i 代入，外层能走到 I 满足$I^{3/2}$约 n，所以 I=Θ($n^{2/3}$)。这只是条件被检查的轮数，不是总成本，因为每次检查本身花 Θ(√i)。

因此

$$T(n)=\Theta\left(\sum_{i=1}^{I}\sqrt{i}\right)=\Theta(I^{3/2})=\Theta(n).$$

平方根和为何是$I^{3/2}$？上界 I 项每项至多√I；下界后 I/2 项每项至少√(I/2)，两者相乘即可。若只写外循环 $n^{2/3}$，会漏算条件调用；若再无脑乘一个最大√I，则能给上界，但还需后半段提供匹配下界。

原解答一处前面得$n^{2/3}$，后面却写$n^{1/3}$的文字，应以前面阈值与程序重新推导为准。整数截断版本的条件 `i < floor(n/ceil√i)`可准确变成`(i+1)ceil√i <= n`，便于用纯整数检查。

把 n=30 的所有条件检查都列出来。G 返回的是向上取整平方根；整数除法先截断，因此条件$i<\lfloor 30/G(i,0)\rfloor$等价于$(i+1)G(i,0)\le 30$：

i	G(i,0)	整除 30/G	条件通过吗	本次 G 减奇数次数
1	1	30	是	1
2	2	15	是	2
3	2	15	是	2
4	2	15	是	2
5	3	10	是	3
6	3	10	是	3
7	3	10	是	3
8	3	10	是	3
9	3	10	是	3
10	4	7	否，退出	4

主体 9 轮，条件 10 次，条件中减奇数的总次数 26。最后一次虽然不进空主体，G(10,0)仍真的执行了，不能漏算。阈值接近$n^{2/3}$而非$n^{1/3}$；原页后续把阈值估成三次方根的优化文字同样不适用于原条件，不能只修前面的一个指数就继续引用后面的错误建议。

本书示例/复杂度计数.py 把逐个减奇数与独立整数平方根核对，再将逐项条件阈值与单调二分核对。它验证有限范围算例，没有拿排版结论自己核对自己。

32.14 14：F 只有一条递归链，G 却会越来越长

控制代码（按原题端点）：

```
int F(int n) { return n > 0 ? G(2, F(n - 1)) : 1; }
int G(int n, int m) { return m > 0 ? n + G(n, m - 1) : 0; }
```

G(a,b)通过把 a 累加 b 次返回 ab，成本 Θ(b)。F 从 F(0)=1 起，每层通过 G 把前次返回值乘 2，故 F(n)=2^n。

n=4 时，F 各层返回 1、2、4、8、16，对应 G 的第二参数依次 1、2、4、8，工作量总 1+2+4+8=15，加 F 自身 4 层常数工作，仍 Θ(16)。一般

$$T(n)=\Theta\left(n+\sum_{k=1}^{n}2^{k-1}\right)=\Theta(2^n).$$

最高层的调用关系应分成“先返回一个数，再拿这个数当重复次数”：

```
F4 等F3返回8，再执行G(2,8)，8次加法，返回16。
 F3 等F2返回4，再执行G(2,4)，4次加法，返回8。
  F2 等F1返回2，再执行G(2,2)，2次加法，返回4。
   F1 等F0返回1，再执行G(2,1)，1次加法，返回2。
    F0 直接返回1。
```

F 的调用链最深只有 5 帧，但最后的 G(2,8)有 9 个含基例的 G 帧。一般在顶层 G 执行时，有$2^{n-1}+1$个 G 帧，外面还等着 1 个 F(n)帧，总峰值为$2^{n-1}+2$帧（n≥1），所以不止运行时间指数，朴素递归实现的最大额外调用栈也指数。不要只数 F 那一条短链就答空间 O(n)。

这里指数时间来自每层 G 参数越来越大，不是 F 有两个递归孩子。G 自身的递归深度为 Θ(b)；代入最高层 b=2^{n-1}以后，最大深度才对 n 呈指数。不能把 G(a,b)直接说成深度2^b，那又多套了一层指数。

32.15 15：孩子规模不同，总量甚至可以低于线性

控制代码（按原题端点）：

```
int F(int n) { return n > 3 ? F(n >> 1) + F(n >> 2) : n; }
```

先看规模 64 的孩子如何缩小，再列调用数和一般式。每次调用的常数工作相同，因此总时间与递归树节点数同阶。这里不需要先知道 Fibonacci 的增长结论。

例如规模 64 的孩子为 32、16；32 再分 16、8；16 再分 8、4。虽然每个非叶节点有两个孩子，孩子总规模只有父的 3/4，不像两半分治保留全部规模。只数孩子个数会丢掉缩小比例。

若以“一次进入 F 计 1 个调用”作精确成本，可从基础情况逐项算：

n	两孩子	总调用数 C(n)	$(C(n)+1)/2$
1	基例	1	1
2	基例	1	1
4	2、1	3	2
8	4、2	5	3
16	8、4	9	5
32	16、8	15	8
64	32、16	25	13

令$n=2^k$，$S_k=C(2^k)$，基础情况是$S_0=S_1=1$，非基础情况满足$S_k=S_{k-1}+S_{k-2}+1$。定义$U_k=(S_k+1)/2$，把等式两边代入：

$$U_k=\frac{S_{k-1}+S_{k-2}+2}{2}=U_{k-1}+U_{k-2}.$$

初值$U_0=U_1=1$，表的最后一列正是 1、1、2、3、5、8、13。现在证明它为何是黄金比例的指数级，而不把“与 Fibonacci 同阶”当作额外黑箱。

取$\varphi=(1+\sqrt{5})/2$，它满足$\varphi^2=\varphi+1$。对所有 k≥0，有

$$\varphi^{k-1}\le U_k\le\varphi^k.$$

k=0、1 时直接代入都成立。若前两项成立，将上下界分别相加：下界为$\varphi^{k-2}+\varphi^{k-3}=\varphi^{k-3}(\varphi+1)=\varphi^{k-1}$，上界同理为$\varphi^{k-1}+\varphi^{k-2}=\varphi^k$。归纳完成，两界只相差固定倍数 φ。

由$S_k = 2U_k - 1$得到$S_k = \Theta(\varphi^k)$。再代$k = \log_2 n$，使用$\varphi^{\log_2 n} = n^{\log_2 \varphi}$，得到

$$T(n) = \Theta(n^{\log_2 \varphi}) \approx \Theta(n^{0.69424}).$$

调用总数 25 不是 F(64)返回值；返回值由基例的数值决定，是另一个问题。

另一种解法令指数 p 满足$2^{-p}+4^{-p}=1$，置 $x=2^{-p}$，则 $x+x^2=1$，正根 $x=1/\varphi$，得 $p=\log_2\varphi$。整数向下取整与很小规模的基础分支，需用单调性和相邻二次幂夹逼处理，改变常数而非指数。

32.16 16：随机分支，把每轮期望加起来

控制代码（按原题端点）：

```
for (int i = n; i > 0; --i)
    if (!(rand() % i))
        for (int j = 0; j < n; ++j) { /* 计一次 */ }
```

外层 i 由 n 降到 1，每轮若随机数对 i 余 0，就执行 n 次内层。理想模型假设每个模数下余数均匀，触发概率 1/i。记该轮是否触发为 X_i，不管各轮是否独立，都有 E[X_i]=1/i。总成本含外循环 n 次及 nΣX_i，所以

$$E[T] = \left(\Theta n + n\sum_{i=1}^{n}\frac{1}{i}\right) = \Theta(n\log n).$$

n=4 时，触发概率依次 1/4、1/3、1/2、1，期望触发数 $H_4=25/12$，内层期望工作 $4H_4=25/3$。这个数可以不是整数，它是不同运行整数成本的平均。

对 n=4，拿一条确定的随机数输出记录`[8,5,6,7]`来单步看，分别用于 i=4、3、2、1：

i	rand 本次给的数	余数	内层执行次数
4	8	0	4
3	5	2	0
2	6	0	4
1	7	0	4

这一条运行内层 12 次，既不是平均25/3也不是最坏 16。期望是对所有可能随机记录加权平均，不能要求每一条记录都等于它。

最坏每轮触发，$\Theta(n^2)$；最好仍需 n 轮外循环，且 i=1 必触发一次 n 内层，故 $\Theta(n)$。同一程序给出三种不同答案，各自描述一个不同问题，不能把期望当最坏保证。

若真实 rand 在 0 到 R 均匀，余 0 的数为 0,i,2i,...,floor(R/i)i，共 floor(R/i)+1 个，因此准确概率

$$p_i = \frac{\lfloor R/i \rfloor + 1}{R+1}.$$

它一般不恰等于 1/i。如果 R+1≥n，每轮误差至多约 1/(R+1)，合计内层期望误差 $O(n^2/(R+1))=O(n)$，不改变 n log n 主阶。如果固定 R 而让 n 无限增长，i>R 时触发只对应随机数 0，概率固定 1/(R+1)，累积可能带来平方主项，不能继续引用同一理想渐近结论。

32.17 练习与答案

练习 1。 一个函数返回 n^8，能否仅由这个式子推出它至少需要 n^8步？

解答：不能。在固定字或抽象无限字模型中，连续常数次乘法即可计算 n^2、n^4、n^8。位模型下要考虑输出位数与乘法位成本，但输出数值 n^8本身不等于执行次数。第 12 段正是在防止把返回值套进时间结论。

练习 2。 第 13 段若预先用 O(1)表查得到每个 ceil√i，外循环时间还为 $\Theta(n)$吗？

解答：单看已准备好的表且查询常数成本，循环轮数为 $\Theta(n^{2/3})$，每轮条件成本变常数，总循环成本也变成该阶。但预处理表与内存成本要另算，不能拿修改后的程序成本冒充原程序。题目中昂贵 G 调用的存在是结论的一部分。

练习 3。 第 16 段证明期望时，为什么没用各次随机分支独立？

解答：只求期望总和，线性期望永远成立，知道每轮触发概率就够。若求“所有分支都触发”的概率，或集中界，则可能需要独立性或其他相关条件。每份概率证明只使用它真正需要的假设，既不能少写，也不用额外乱加。

33 教材、习题解析、PS 与 PA，怎样对着解决同一个问题

遇到一段看不懂的证明，不必在同一页上反复硬读。先判断缺的是定义、具体执行、证明方法还是代码约定，再去对应材料补。教材给完整概念，课件常保留课堂推导的图与中间步骤，PS 提出容易混淆的条件，PA 则要求你把它们落实成在规模内可运行的程序。

课程名是数据结构。教材书名、旧版章号、课程课件节号与本书章号各有一套，不能拿某套号直接代替另一套。2026 教材本地 PDF 含完整十四章，覆盖算法、向量、列表、栈队列、树、搜索结构、词典、图、优先级队列、串与排序。旧版第三版与习题解析的章序不同，不能把此前只读到的范围写成文件本身只有六章。

33.1 先知道自己翻的是哪一份书

本地 2026 教材为 352 页 PDF；作者官方合体 PDF 为 669 页，含第三版主教材与《习题解析》，习题解析从 PDF 第 413 页开始。官方入口是作者教材页，公开原件本地保存在`资料/教材与参考/数据结构（第 3 版）与习题解析（官方合体）.pdf`。

2026 版十四章在 PDF 中的起始页如下，按实际书签与原页核对：

章	主题	PDF 起始页	本书主要对应
1	算法概论	9	01—03
2	向量	43	04—06
3	列表	75	07
4	栈与队列	93	08—09
5	二叉树	123	10—11
6	二叉搜索树	151	12
7	搜索应用	183	14
8	高级搜索树	197	13
9	词典	239	15
10	图算法基础	259	16
11	图算法进阶	277	17
12	优先级队列	289	18
13	串	307	19
14	排序	327	20

PDF 前置目录目前只列到第六章，尚未更新到后八章；后半请按 PDF 书签或本表定位，不能据前置目录判断文件只含六章。这是文件内容与定位表，不表示本书已逐题解完教材全部习题。引用算法时还须核对具体版本，例如 2026 版算法 14.2 的 LUG 代码与旧习题 12-1 的单扫描划分不是同一个循环。

一份 PDF 中印刷页可能从前言重新编号，习题解析又从自己的页码开始。因此引用“第 24 页”不够；本书 32 章明确说“官方合体 PDF446 页、习题解析印刷 24 页”，两者定位到同一位置。搜索提取文字能帮助定位名词，涉及树形、箭头、求和上下限与分式时，仍应看对应原页。

下面按官方书签列出旧版习题解析的 292 道大题。许多题还有小问；它们与 PS01—05 的 184 个编号不是同一套，题量不能相加后就宣称全部完成。

旧版习题范围	大题数	官方 PDF 页	本书对应
1-1 至 1-32，绪论	32	424 至 456	01—03、22、28、31；32 专讲 1-32

旧版习题范围	大题数	官方 PDF 页	本书对应
2-1 至 2-41，向量	41	458 至 493	04—06、23
3-1 至 3-19，列表	19	496 至 508	06—07、24
4-1 至 4-26，栈与队列	26	510 至 524	08—09、25、30
5-1 至 5-30，二叉树	30	526 至 540	10—11、26
6-1 至 6-32，图	32	542 至 565	16—17
7-1 至 7-20，搜索树	20	568 至 578	12
8-1 至 8-20，高级搜索树	20	580 至 599	13—14
9-1 至 9-26，词典	26	602 至 615	15
10-1 至 10-22，优先级队列	22	618 至 632	11、18
11-1 至 11-10，串	10	634 至 640	19
12-1 至 12-14，排序	14	642 至 650	20

33.2 原书十二章，各完整走一类代表题

以下题号都来自旧版《习题解析》，定位均为官方 669 页合体 PDF 的物理页。它们让你看到题目与本书的具体接点，并不代替原书所有题。题意采用转述，算例另设。

33.2.1 绪论 1–1，PDF424 页：尺规算法也要证明输出关系

任务是在直线 l 外给定点 P，作过 P 的平行线。取 l 上一点 A，以 AP 为半径作圆，与 l 交于 B；再分别以 B、P 为圆心、同半径作圆，两圆除 A 外的另一交点为 C，连接 PC。

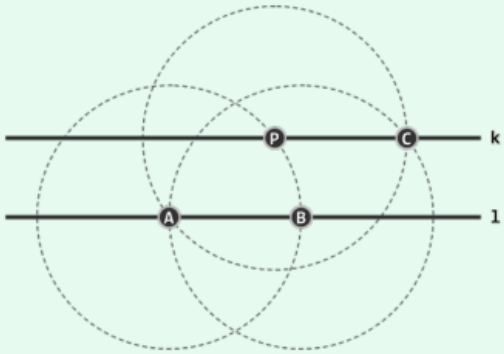
[1-1] 试借助基本的几何作图操作描述一个算法过程，实现“过直线外一点作其平行线”的功能。

【解答】

算法x1.1[①]给出了一个可行的算法，其原理及操作过程如图x1.1所示。

```
parallel(l, P)
输入：直线l及其外一点P
输出：经过P且平行于l的直线
1. 以l上任意一点A为中心、以|AP|为半径作圆
2. 在该圆与l的两个交点中，任取其一记作B
3. 分别以B和P为中心、以|AP|为半径各作一圆
4. 两圆相交于A及另一点，将后者记作C
5. 过P和C绘制一条直线k
```

算法x1.1 过直线外一点作其平行线



图x1.1 过直线外一点作其平行线

[1-2] 《海岛算经》讨论了如下遥测海岛高度的问题：

> **今有望海岛，立两表，齐高三丈，前后相去千步，令后表与前表参相直。从前表却行一百二十三步，人目著地取望岛峰，与表末参合。从后表却行一百二十七步，人目著地取望岛峰，亦与表末参合。问岛高及去表各几何？**

刘徽所给出的解法是：

> **以表高乘表间为实；相多为法，除之。所得加表高，即得岛高。**
> **求前表去岛远近者，以前表却行乘表间为实。相多为法。除之，得岛去表数。**

a） 该算法的原理是什么？

【解答】

刘徽通过设立两根立柱（表），利用光线直线传播的性质，根据海岛峰顶经过立柱顶端后到地面的投影位置，巧妙地计算出远方海岛的高度。

具体地如图x1.2所示，若将立柱高度（表高）记作h，后立柱、前立柱间距（表间）记作d，对应的投影距离分别记作d_1和d_2，则不难导出：

$$\text{海岛高度}H = d \times h / (d_1 - d_2) + h$$
$$= 1000 \times 3 / (127 - 123) + 3$$
$$= 753 \text{（丈）}$$
$$\text{海岛距离}D = d \times d_2 / (d_1 - d_2)$$

2

① 为与主教材相互区别，本习题解析中所有插图、表格、算法、代码等编号前，均增加“x”标识

图 37 官方合体 PDF 第 424 页完整原页。图中三个同半径的圆分别把 AP 的长度传到 AB、BC、PC，最后连 PC；原页下半另有海岛题，本节只解 1-1。

按图一步步核对，而不是看到最终平行线就结束：第一圆以 A 为圆心，P 在圆上；B 取这圆与 l 的交点，所以 AB=AP。第二、三圆分别以 B 和 P 为圆心、以同一个半径 AP 作图；A 在这两圆上，因此取两圆交点时要明确取另一个 C，而不是又取回 A。于是 BC=PC=AP。最后 P 与 C 不同，可以连成直线。

P 不在 l 上使 A、B、P 不共线，两个相交圆不是相切于 A 的一种退化情形，因此另一个交点存在。输入条件保证了操作本身可执行。

为什么 PC 平行 l？AB=AP，且 BC=PC=AP，所以 A、B、C、P 依次构成菱形，PC 与 AB 平行，AB 又在 l 上。P 不在 l 上保证相关三角形不退化。算法的基本操作是定半径作圆、取交点、连直线，并非必须先写成 C++才叫算法。01 章说明输入条件与基本操作模型，尺规题同样要交代这些前提。

33.2.2 向量 2–2，PDF458 页：重叠复制怎样选方向

源区间与目标可能重叠时，按地址判断。数组[A,B,C,D,E]将前三项复制到从下标 1 起的位置，目标在源右侧，必须倒着：先 C 到 3，再 B 到 2，再 A 到 1，结果[A,A,B,C,E]。若正向，A 先覆盖 B，后面会不断复制 A。

右移的每一格如下，尚未读出的源用原大写字母保持：

```
初始        [A,B,C,D,E]
先a[3]=a[2] [A,B,C,C,E]
再a[2]=a[1] [A,B,B,C,E]
后a[1]=a[0] [A,A,B,C,E]
```

若反过来先写 a[1]=a[0]，源 B 被覆盖；下次读取 a[1]得到 A，错误已不能靠最后一行补救。判断方向时画的是地址区间，不是比较元素值大小。

反过来将下标 1 到 3 的 B,C,D 复制到 0 到 2，目标在左，正向得到[B,C,D,D,E]。若倒着，先 D 覆盖下标 2 的 C，再向前读时已丢原 C。一般规则是右移重叠区从后向前，左移重叠区从前向后；不重叠时两向都可。04 章的插入和删除正是在使用这条覆盖依赖关系。

33.2.3 列表 3–3，PDF496 页：唯一数据节点删除以后是什么

双哨兵链表 H↔X↔T，X 既是首数据节点也是末数据节点。先保存 p=X.pred=H、q=X.succ=T；令 p.succ=q、q.pred=p，再释放 X，size 从 1 变 0。得到 H↔T，恰好是空表的不变式。

对两条真正的接线分别画中间状态：先写 H.succ=T，正向已经绕过 X，但 T.pred 暂时还指 X；再写 T.pred=H，双向关系恢复。到这时才销毁 X。循环或回调若可能在两行之间观察表，应按并发或重入协议处理；本题普通单线程删除把这几行作为一个局部修改完成。

```
删前：H <-> X <-> T，size=1
保存：left=H，right=T
接好：H <--------> T，size=0
释放：X不再拥有或连接后续节点
```

如果 X 析构会自动删除 succ，则本来独立的尾哨兵可能被误删；课程节点只按其约定释放自己的 data 与存储，容器统一拥有链，不能把两种所有权策略混用。

没有特殊分支仍能成立，是因为H、T虽然不属于用户数据，却提供合法邻居。不能将X删除后再读X.succ，也不能把 H 或 T 当作可删除数据。07 章的同一套 remove 对中间、首尾与单节点情形统一适用，前提是哨兵确实存在且不被当作数据。

33.2.4 栈与队列 4–1，PDF510 页：用列表实现栈，接口不止换个名字

令列表末端为栈顶：push(e)调用 insertAsLast(e)，pop 调用 remove(last())，top 返回 last.data；size 与 empty 沿用列表管理信息。双向列表有尾哨兵时，取末节点与在末端插删都是 O(1)，因此标准栈接口都可最坏常数。

空栈的 pop、top 必须拒绝或返回失败，不能把尾哨兵的数据当结果。若还继承了 find 接口，应核对失败返回：列表返回空位置，向量可能返回负秩，调用代码必须相应修改。原页代码用 last 与 insertAsLast，旁边个别“首元素”注释应按实际调用理解为末端；不能把互相冲突的注释当作两套同时成立的方向。

用链画四个接口的实际变化，H、T 为哨兵：

```
empty: H↔T, size=0, true
push A: H↔A↔T, size=1
push B: H↔A↔B↔T, size=2, top读T.pred即B
pop: 保存B.data, 令A.succ=T、T.pred=A, 释放B, size=1, 返回B
```

每次末端读 T.pred、插接或摘接都是固定条数，因而最坏常数。单链即使有 tail 指针，也没有 B 的前驱 A，尾删可能先扫描全链；原题基于有前后链接的 List，复杂度依据不能漏掉这一表示条件。

具体执行 push A、push B、pop 得到 B，剩 A，top 仍 A。若换成首端做栈顶也可，但遍历显示顺序会反过来，使用者若依赖“底到顶”的输出，需统一约定。08 章强调的是后进先出关系，而非底层一定用数组。

33.2.5 二叉树 5–1，PDF526 页：深度加高度为何不超过整树高

取任意节点 v，从根到 v 长 depth(v)，再取 v 子树里最深叶 x，v 到 x 长 height(v)。两段接起来是整树中一条根叶路径，所以 depth(v)+height(v)=depth(x)≤height(T)。若 v 子树包含至少一个整树最深叶，便取等；若不包含，子树内所有叶都更浅，不等号严格。

符号可以直接对应到一张树：

```
        R              depth(R)=0, height(R)=3
       / \
      A   Q            A深1高2, Q深1高0
     /
    B                  B深2高1
   /
  C                    C深3高0
```

取 v=A，子树里最深的是 C，根到 A 的 1 条边接上 A 到 C 的 2 条边，得到 3；取 v=Q 只能得到 1。一般式子里，max 只是“在 v 子树里挑最深后代”的操作，换成全树挑，候选更多，所以不会更小。

例如根 R 左边有三条边长链，右孩子 Q 是叶。整树高 3，Q 深 1、高 0，相加 1<3；左链上的节点均位于某条最深路径，相加 3。证明不要求树平衡，也不要求最深叶唯一。10 章统一的按边计高约定可直接用于这道题。

33.2.6 图 6–1，PDF542 页：关联矩阵乘自己转置，究竟保留了什么

无向链 A-B-C，两条边分别为列，关联矩阵为 `I=[[1,0],[1,1],[0,1]]`。两行点积共同计数两顶点关联的边，故 `I I^T=[[1,1,0],[1,2,1],[0,1,1]]`。对角是度 1,2,1，非对角为相邻边数；简单无环图下，非对角就是邻接矩阵。

先解释矩阵相乘怎样读取。I 的行是顶点，列是边 e1=AB、e2=BC：

$$I = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{pmatrix}.$$

乘II^T的第 A 行第 B 列，就是 I 的 A 行与 B 行的点积：$1\cdot 1 + 0\cdot 1 = 1$，共同碰到 1 条边。A 与 C 点积$1\cdot 0 + 0\cdot 1 = 0$，不相邻。B 与自己点积$1^2 + 1^2 = 2$，是度 2。每一格都来自这个规则，得到

$$II^T = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 1 \end{pmatrix}.$$

有平行边时非对角可能大于 1，表示公共关联边的条数；本例没有自环。

有向链 A→B→C，把出发端记 −1、到达端记 +1，则 `I=[[-1,0],[1,-1],[0,1]]`，乘积为 `[[1,-1,0],[-1,2,-1],[0,-1,1]]`。它记录总度与忽略方向的连接数，却无法恢复边方向，因为将某列整体取反不改变乘积。别误把这个对称乘积叫作原有向邻接矩阵。

把 A⟶B 方向反过来，I 的第 1 列由`[-1,1,0]`改成`[1,-1,0]`，整列取负。任意两行在这一列相乘都同时多两个负号，贡献不变，所以乘积无法区分 A⟶B 与 B⟶A。对角度数与连接关系能保留，原方向却丢了；这个信息损失可以由一条边的实际变换直接证明。

应用例子是差分约束 x_v−x_u≤w，转成边 u⟶v 权 w。给各点一个超级源 0 权到达边，最短距离满足所有`d[v]≤d[u]+w`，可作可行赋值；若存在负环，将环上不等式相加会得 0≤负数，矛盾。17 章 Bellman–Ford 检测负环因此能处理这类可行性问题。另做一个完整可行性例子：

$$x_2 - x_1 \le 3, \qquad x_3 - x_2 \le -1, \qquad x_3 - x_1 \le 4.$$

转成 1⟶2 权 3、2⟶3 权 −1、1⟶3 权 4，并从超级源向三个点各连 0 边。初始 d1=d2=d3=0；扫 1⟶2 时 0≤0+3，不改；扫 2⟶3 时 0>0−1，把 d3 改 −1；扫 1⟶3 时 −1≤4，不改。下一轮全部不改，取赋值$(x_1, x_2, x_3) = (0, 0, -1)$，三项左侧依次 0、−1、−1，都满足对应上界。

若再加入$x_1 - x_3 \le -3$，沿 1⟶2⟶3⟶1 的环权和为 3−1−3=−1。把三个约束直接相加，变量全部抵消，得到$0 \le -1$，不可能。因此负环不是算法随意定的失败信号，它对应原约束之间的矛盾。

矩阵方向编码负责表达约束，乘积本身并没有保留全部有向信息。

33.2.7 搜索树 7–1，PDF568 页：局部孩子大小还不够

BST 要求左子树所有后代≤根，右子树所有后代≥根。中序按左、根、右列出，因此非降。反过来若中序非降，对任意根，所有左子树项都出现在根前且不大于根，所有右项都在根后且不小于根，故满足这种允许重复的 BST 定义。

仅检查孩子会漏错：

```
      10
     /
    5
     \
      12
```

根的左孩子 5<10，5 的右孩子 12>5，每条边局部都像 BST，但 12 属于 10 的整个左子树，违反≤10。中序实际输出 5、12、10，在 12⟶10 处下降。

检查时可以把祖先累积的范围传下去：根 10 允许任意实数；左孩子 5 继承“≤10”；5 的右孩子继承“≥5”还必须保留原来的“≤10”，所以允许范围为[5,10]，12 立即被拒绝。若递归时只传当前父亲限制，忘掉更高祖先的上界，正好会漏掉本例。12 章先用互异键得严格递增；原书这题允许非严格关系，重复键的策略需另加约定。

33.2.8 高级搜索树 8–1，PDF580 页：重复记录需要身份，整组查询需要边界

原题要伸展树允许多个相等数据项，并增加 searchAll 与 removeAll。可把内部比较键设成`(用户 key,唯一递增 id)`，按字典序严格比较；旋转只依赖这个总序，不会把相等用户键随机挤到不相干位置。对外查找仍只看用户 key，所有相等 key 在中序中连续。

依次插`(5,A,id1)`、`(3,B,id2)`、`(5,C,id3)`，每次把新项伸展到根。三次状态可以全部画出：

```
插A后:    5A        插B并右旋后:  3B        插C并两次左旋后:  5C
                                  \                          /
                                   5A                       5A
                                                           /
                                                          3B
```

内部次序始终`3B < 5A < 5C`。查询第一个用户 key 不小于 5，根 5C 是候选，向左 5A 也是候选，再向左 3B 太小、其右边空，故最左候选是 5A。把 5A 绕 5C 右旋到根：

```
      5A
     /  \
    3B   5C
```

一次 search(5)可返回 A，单项 remove(5)也只删除找到的一份，C 仍应存在。删除全部不能仅删根，也不能用一个布尔“5 存在”代表两条记录。

批量接口可以用**两次直接分裂**得到$O(\log n + k)$摊还界。分裂不仅要把等值范围找对，还要真正伸展走完搜索的节点；只旋转一个很早记录的候选，可能遗漏长搜索路径的成本。

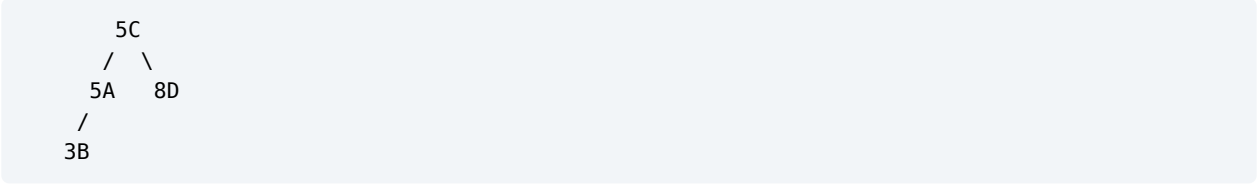
第一次分成 L（用户 key<x）与 R（用户 key≥x），按以下完整规则执行：

1. 从根搜索到空指针，不在命中等值时提前停。当前 key<x 就向右走，否则向左走；每走到真实节点，都把它记为 last。
2. 空树直接得到两个空树。非空树把**实际最后访问的 last** 伸展到根，正常修正每次旋转的 parent、孩子和 size。
3. 若新根 key<x，它就是小于 x 的最后一项。根及其左边归 L，根的右子树归 R：断 `root.right`。
4. 若新根 key≥x，它就是第一个不小于 x 的项。根及其右边归 R，根的左子树归 L：断 `root.left`。

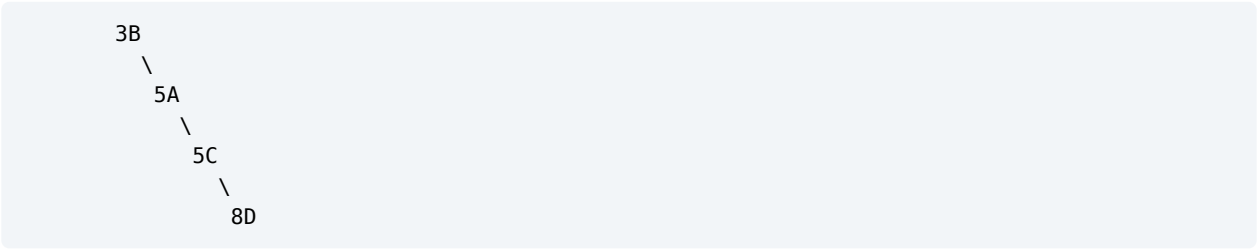
为什么 last 只能是边界两侧紧邻的项？搜索要么在“<x”的节点向右走到空，说明它与 x 之间已没有更大但仍<x 的节点；要么在“≥x”的节点向左走到空，说明没有更小但仍≥x 的节点。祖先给出的另一侧限制也一直保留。伸展保持中序，所以旋转后按根的一条边切开，仍恰好把整个全序切在 x 之前。

第二次把 R 分为 M（用户 key≤x）与 G（用户 key>x）。搜索规则相应改为：key≤x 向右，否则向左，仍一路到空、伸展 last。新根若≤x，断 `root.right`，左份为 M、右份为 G；新根若>x，断 `root.left`，左份为 M、右份为 G。R 原本全≥x，所以 M 中的项恰好全等于 x。每次断边后，把新分离的根 parent 置空，重算仍保留孩子的原根 size；若某份为空，直接保留为空。

把两边都非空的例子走全。查询 x=5，初态为一棵合法伸展树：

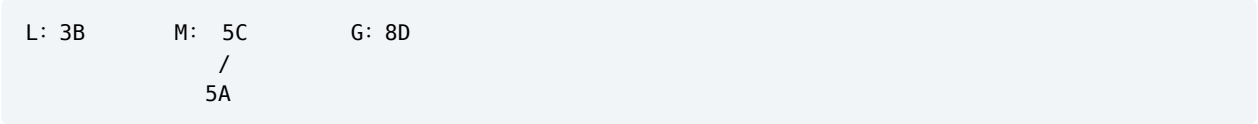


第一搜索依次访问 5C、5A、3B，再从 3B 的右边走到空，last 是 3B。3B 是左−左孙，先绕 5C 右旋得到 5A 为根、左 3B、右 5C（5C 右 8D）；再绕 5A 右旋，得到：



新根 3<5，断它的**右边**，得到 L 仅 3B，R 为 5A 右连 5C、5C 右连 8D。第一步并非找到 5A 就停，3B 的访问也被实际伸展覆盖了。

第二搜索在 R 中依次访问 5A、5C、8D，从 8D 左边到空，last 是 8D。它是右−右孙，连续两次左旋：先绕 5A，得到根 5C、左 5A、右 8D；再绕 5C，得到根 8D、左孩子 5C、5C 左孩子 5A。新根 8>5，断它的**左边**：



searchAll 中序遍历 M，得到 A、C 共 k=2 项，再合回 L、M、G；removeAll 销毁 M 的两个节点，再合 L 与 G，删除结果为 3B 右连 8D。若没有等值项，M 为空；若全表都小于 x，第一次新根是全表最大项、右边本来空，R 自然为空，不需要读取不存在的候选。

一个能检查复杂度口径的反例是：根 10，左边为 1→2→…→9 的右链，查询 x=9.5。第一个≥x 的候选一直是根 10，但真实搜索会走到 9。如果只伸展候选 10，树完全不变，反复查询每次都线性。这里会伸展实际 last=9，再按 9<9.5 断右边，整条访问路径都进入正常伸展分析。

合并两棵所有内部键满足左<右的伸展树，可把左树最大项伸展到左根，此时没有右孩子，再把右树接到其右边并修 parent、size。若其中一棵为空，直接返回另一棵。刚才 searchAll 后的合回也可执行：先将 M 接在 L 的 3B 右边，再把合树最大项 5C 伸展到根，此时左边是 3B、3B 右孩子为 5A，最后把 8D 接到 5C 右边。中序仍是 3B、5A、5C、8D；查询可以改变形状，不能改变记录内容与总序。每次分裂都搜索到 last 并将 last 伸展，搜索长度与这次伸展路径相配；合并也在找到左树最大项后将它伸展。各自按访问引理摊还$O(\log n)$，切掉一条边不会增加按子树大小计算的森林总势能；中间树遍历或销毁需$\Theta(k)$，所以总$O(\log n + k)$摊还时间、输出/删除量另计。k=0 时也要处理空 M。单次伸展可能很长，不能把摊还结论改成单次最坏对数。

仅仅说“找到首项后不断求后继”还不足以自动保证这个批量界：如果每次后继从根重查，可能多出 k 个对数；如果结束边界要走一条很深的非等值链，也需要分析那段代价。两次分裂明确圈出全部输出的子树，扫描成本才可以直接按 k 记账。第 13 章给伸展旋转与访问引理，本节把它们接到重复记录这个具体接口。

33.2.9 词典 9–2，PDF602 页：一座塔不要重复复制整个词条

跳转表某键高 4 层，若每层都复制一份长字符串和值对象，会浪费 4 份大载荷。可让底层数据节点唯一拥有 payload，高层索引只存指向同一 payload 或底层节点的引用及横纵链接。

例如某条记录 P 的 key 为 7、value 是一段长文本，塔高 3。把载荷只留一份：

```
第2层:  ... ↔ index7 ----payload----> P(key=7,value=长文本)
                  |                    ↑
第1层:  ... ↔ index7 ----payload---------+
                  |                    ↑
第0层:  ... ↔ data7  ----payload---------+
```

横向链接负责这一层前后跳，纵向链接负责上下，payload 引用只负责找到那一份记录。更新 value 把 P.value 改一次，不复制三份；key 通常不能直接原地修改，否则横向有序性会被破坏，应删后重插。

删除时先在每层把索引前后邻居接好，拆掉塔，再释放唯一载荷 P 一次；若先删 P，高层比较 key 时就可能读到悬空引用。若接口允许外部长期持有记录，则还要用共享所有权或稳定句柄规定什么时候可释放，塔高本身没有解决外部生命期。

查询在高层停止时，沿引用返回那一份数据。链接数期望 O(n)的前提不变，但每层不再重复存可能很大的键和值；13 章的跳转索引因此与对象所有权接起来。

33.2.10 优先级队列 10–1，PDF618 页：四个基础实现，把三个接口都算一遍

无序向量尾部插入摊还 O(1)，找最大 O(n)，删最大可扫描后用末项填洞，总 O(n)。无序列表可常数插节点，找最大与删前寻找仍 O(n)。若缓存最大位置，getMax 可 O(1)，但删最大后重找下一最大仍需 O(n)，缓存没有免除这笔工作。

用插 4、1、7、6，再删最大，实际动作如下：

结构	插完前 3 项	插 6 时做什么	取得/删除 7 时做什么
无序向量	`[4,1,7]`	尾写 6，成`[4,1,7,6]`	扫到下标 2 的 7，用末项 6 填洞，成`[4,1,6]`
有序向量	`[1,4,7]`	把 7 右移，6 填进去，成`[1,4,6,7]`	直接读/删末项，成`[1,4,6]`
无序双链	`4↔1↔7`	尾接 6	扫出 7 的节点，常数摘链
有序双链	`1↔4↔7`	先走链接找到 4 与 7 之间，再常数接 6	尾指针直接读/摘 7

无序的代价落在寻找最大，有序的代价落在寻找并保持插入位置。链表“已知位置后插删常数”没有免除寻找位置。

有序升序向量在正确秩插入需寻找并搬移，O(n)，最大在尾，getMax 与尾部删除 O(1)或按收缩策略给摊还界。有序双向列表插入需 O(n)寻找位置，末端取最大与删末节点 O(1)。保持全序使删除极值便宜，却使插入较贵。18 章的完全二叉堆只维护父子序，getMax O(1)，插入与删最大 O(log n)，正是放松全序换来的平衡。

33.2.11 串 11–1，PDF634 页：前缀、后缀与子串的坐标

以 S=ABCDE 为例，长度为 5，长 2 前缀 AB，长 2 后缀 DE。按原题 Excel 式从 1 编号，mid(S,2,2)取 BC；换成本书从 0 起的区间，就是 S[1:3]。相等比较要求长度相同且每个对应字符相同，ABC 与 AbC 在区分大小写的 exact 语义下不同。

把同一个 S 从两种坐标写出来：

```
字符       A B C D E
Excel位置  1 2 3 4 5
本书下标   0 1 2 3 4
```

合法参数下，`mid(S,i,k)`的本书半开区间是`[i-1,i-1+k)`。例如 i=2、k=2，读下标 1、2 的 B、C，右端 3 不读；`left(S,2)`是`[0,2)`，`right(S,2)`是`[3,5)`。三个参数分别是起点、长度与排除的右边界，不能把 k 直接当结束下标。这里用 ASCII 与合法范围说明映射；不同工具的非法参数、Unicode 计数细节还需按其各自接口规则核对，不能把这张坐标表当作所有软件错误处理也完全相同。

这道看似简单的接口题在提醒你：课件的秩、语言切片与工具界面起点可能不同。字符长度也要看采用字节、码点还是其他单位；不能拿一个软件的显示长度直接当另一个 API 的字节偏移。19 章先固定字符模型，再定义 π 与匹配起点，正是为了防止这类坐标错位。

33.2.12 排序 12–1，PDF642 页：一个扫描指针也能完成轴点划分

原题版本 C 把轴值先放 lo，mi 指向已知小于轴的区域末尾，k 从 lo+1 向右扫。遇 a[k]<pivot，就先 mi 加 1，再交换 a[mi]与 a[k]；否则只前进 k。最后交换 a[lo]与 a[mi]，返回 mi。

习题[12-1] 第12章 排序

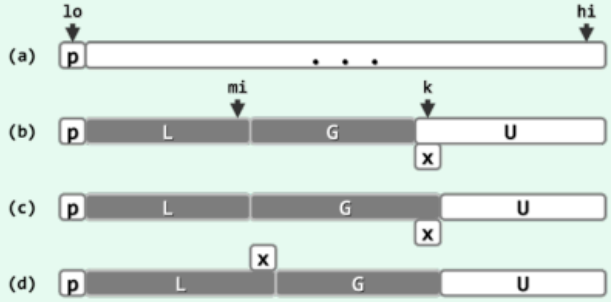
[12-1] 构造轴点的另一更为快捷的策略，思路如图 x12.1 所示：

始终将整个向量 V[lo, hi]划分为四个区间：

V[lo]，L = V(lo, mi]，G = V(mi, k)，U = V[k, hi]

其中 V[lo]为候选轴点，L/G 中的元素均不大/不小于 V[lo]，U 中元素的大小未知

初始时取 k - 1 = mi = lo，L 和 G 均为空；此后随着 k 不断递增，逐一检查元素 V[k]，并根据 V[k]相对于候选轴点的大小，相应地扩展区间 L（图(d)）或区间 G（图(c)），同时压缩区间 U。最终当 k - 1 = hi 时，U 不含任何元素，于是只需将候选轴点放至 V[mi]，即成为真正的轴点。



图x12.1 轴点构造算法（版本C）

a） 试依此思路，实现对应的划分算法 Vector::partition()；

【解答】

一种可行的实现方式，如代码x12.1所示。

```
1  template <typename T> //轴点构造算法：通过调整元素位置构造区间[lo, hi]的轴点，并返回其秩
2  Rank Vector<T>::partition ( Rank lo, Rank hi ) { //版本C
3     swap ( _elem[lo], _elem[lo + rand() % ( hi - lo + 1 ) ] ); //任选一个元素与首元素交换
4     T pivot = _elem[lo]; //以首元素为候选轴点——经以上交换，等效于随机选取
5     int mi = lo;
6     //++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
7     //   [ ---- < [lo] ----- ] [ ----- [lo] <= --- ] [ ----- unknown ----- ]
8     // X x . . . . . . . . . . x . . . . . . . . . . . x . . . . . . . . . . x
9     // |                       |                       |                     |
10    // lo (pivot)             mi                      k                     hi
11    //++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
12    for ( int k = lo + 1; k <= hi; k++ ) //自左向右扫描
13       if ( _elem[k] < pivot ) //若当前元素_elem[k]小于pivot，则
14          swap ( _elem[++mi], _elem[k] ); //将_elem[k]交换至原mi之后，使L子序列向右扩展
15    //++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
16    //   [ --------- < [lo] ---------- ] [ ----------- [lo] <= ----------- ]
```


图 38　官方合体 PDF 第 642 页完整原图。四块依次是暂存轴值、小于轴区、不小于轴区、尚未知区；图下给出版本 C 代码。这里使用原题闭区间，最后下标 hi 参与扫描。

本段遵循原题闭区间[lo,hi]。若数组长度 n，整表调用应传 hi=n−1，不能传 n。为专看分区过程，下面固定首项作轴；原题随机选轴只是先交换一个位置到首项，不改变扫描不变量。

```
def partition_closed(a, lo, hi):
    pivot = a[lo]
    mi = lo
    for k in range(lo + 1, hi + 1):
        if a[k] < pivot:
            mi += 1
            a[mi], a[k] = a[k], a[mi]
    a[lo], a[mi] = a[mi], a[lo]
    return mi
```

前置条件是 0<=lo<=hi<len(a)；空区间由外层快排的基础分支直接跳过，不应调用本函数去读 a[lo]。
以[4,7,2,5,1]、lo=0、轴 4 为例：7 不动；2 小，mi=1，交换得[4,2,7,5,1]；5 不动；1 小，mi=2，得[4,2,1,5,7]；最后 4 与下标 2 交换，得[1,2,4,5,7]，轴点秩 2。扫描中[lo+1,mi]严格小于轴，[mi+1,k)不小于轴，未知段从 k 开始；每步都缩小未知段，扫描线性。
用竖线把每轮的四块画出来，轴一直单独放首格：

```
初始: 4 | 空 | 空       | 7,2,5,1       mi=0, k=1
看7: 4 | 空 | 7        | 2,5,1         7>=4, k前进
看2: 4 | 2  | 7        | 5,1           mi=1, 交换2与7
看5: 4 | 2  | 7,5      | 1             5>=4, 只扩G
看1: 4 | 2,1| 5,7      | 空            mi=2, 1与7交换
末换: 1,2 | 4 | 5,7                    返回mi=2
```

第三块内部不需要有序，7,5 在扫描中完全合法；划分只承诺相对轴的大小。每步要么把当前小项换到 L 末端，同时把原 G 首项转移到 G 末端；要么直接把当前项归 G。未知区每轮少 1，终止后轴与 mi 交换便让左侧全<轴、右侧全≥轴。后续递归是[lo,mi-1]和[mi+1,hi]，轴本人不再递归。
这里返回单个轴点；20 章的 LUG、DUP 也返回轴点，而另列的三路划分返回整个等值区间，递归边界不能混用。若所有值相等，版本 C 每次 mi 仍为 lo，递归侧大小 n−1，仍可能平方；随机选轴无法改变“所有值相同”这个事实，三路划分才会一次跳过全部等值项。

33.3 读完例子，再按薄弱处回到材料

33.4 路线 1：复杂度题，先把“要数什么”固定

先读 01 章的计算模型，明确一条移位、大整数乘法和数组访问是否算常数。再读 02 章的三种和式与递归树，亲自写出每行次数。随后做 PS01：给程序一个小 n，列出循环状态，再写一般阈值。遇到 1-32，32 章按 16 段分别区分返回值、外层轮数和总调用成本。
例如第 13 段 G 算 ceil√i，主循环走到 i 约$n^{2/3}$。若只看外循环轮数，会答错；把每次条件调用的√i 成本求和，才得到线性。看答案之前要问：“这里哪一句是在数轮数，哪一句是在数工作？”
到 PA 时再将复杂度与规模核对。n=40 的二选一组合不能轻易做2^{40}枚举，可考虑两半；字符串长度很大但预算 k 小，可考虑带状 DP。理论阶是选择算法的约束，不是写完代码后用来装饰的结尾。

33.5 路线 2：向量与列表，同一个插入动作付不同的账

读向量课件原图时，追踪容量、size 与复制量，不要只记“扩容摊还常数”。04 章给纯追加序列，28 章再把势能总账写成等式。然后用 PS02 检查不同增长策略、成功失败查找长度和二分接口。
转到列表，先在纸上画前驱、当前节点、后继，写出四根链接。07 章删除例子专门保存旧 next，防止释放后使用。PS03 再检查稳定性与比较次数：列表节点摘接便宜，不等于在有序列表中寻找插入位置也便宜。
若程序输出排序值正确，仍应给相等项加 A、B 标签检查稳定性。选择“最后一个最大值”后远距离交换，可能让另一种相等键反序；06 章给了可手算的反例。用具体记录比“这个算法通常稳定吗”的印象判断可靠。

33.6 路线 3：栈队列，先写状态再谈峰值

08 章先手算括号和 RPN，再读老师中缀优先表。课程原表的乘方左结合与数学常用右结合应分开。PS04 有些题要求原程序，有些要求修改程序，不能把修改后的规则倒灌回原版。30 章对 21 式列出每次数字压入与每次运算后的完整数值栈，便于定位哪次减法或除法先算错。

09 章用窗口最大值和直方图说明删除候选的理由。之后读 PS04 里“单步空间”“整段峰值”“随机期望”的问题，再去 28 章看记录变量与并合界。固定一个窗口的平均候选数小，不代表无限长流的最大候选数同样小。

PA 中不一定直接出现栈，但同样需要问“哪些旧状态仍可能影响未来”。Filename 保留带内可达状态，Gift 保留有重复身份的半和，偏序 DP 保留每个 b 前缀的最佳值。这些压缩都应给出不会丢掉更优解的理由。

33.7 路线 4：树，把所有计数约定写在第一张纸上

先固定高度按边数还是节点数，空树高度取多少，二叉树是否区分左右，是否允许单孩子。10 章统一按边数，给增强先序解析；PS05 再检查普通遍历与带空标记遍历的唯一性。

扩充二叉树是在每个缺失孩子处补一个外部空叶。n 个原节点有 n+1 个空槽，所以补后外叶 n+1，所有原节点都成为双分支节点，共 n 个；不是只给原单孩子节点补一次。原叶的左右两个缺口也必须补。这个约定与增强先序中 n+1 个空孩子标记是同一份结构信息的两种写法。

BST、AVL 先用 12 章小树执行删除与旋转，再回到教材的通用节点重构；不要只看最终树形。Huffman 先用 11 章算合并权重总和，再读贪心交换证明和比较下界。最优编码的成本、构造算法的复杂度、输出所有码字的长度，是三个不同的量。

33.8 范围说明

这本书按现有章节讲解核心概念，按 22—26 章逐项处理本学期五张 PS，32 章专门重做原书 1-32。取得 669 页原件并建立 12 章索引，不等于 292 道原书大题及所有小问都已经逐题完整解答。查到一个题号以后，仍要看对应章节是否实际给出了该题的证明与计算。

课内样卷中带使用限制的原件留在本地，31 章用另设实例提炼方法。个人作业、转系资料和群聊照片也不因写教材而进入公开仓库。公开教材中的例子用于教学与检查，不替代课程发布的正式约束与独立提交。

资料覆盖、原图出处与使用说明

这是一份独立组织、重新讲解的教材式初稿。课程范围首先按教师资料；并非教师审核的官方教材，也不把补充证明和预习部分自动列作考试要求。

章节覆盖表

教学来源	本书对应
教材/整套讲义第 1 章算法概论	01—03、22、28、31
第 2 章向量	04—06、23
第 3 章列表	07、24
第 4 章栈与队列	08—09、25、30
第 5 章二叉树	10—11、26、29
第 6 章 BST 与 AVL	12
第 7 章搜索树应用	14
第 8 章高级搜索树	13
第 9 章词典/散列	15
第 10 章图基础	16
第 11 章图进阶	16—17
第 12 章优先级队列	18
第 13 章串	19
第 14 章排序	20
已发布 PS01—PS05	22—26 逐题导航，题数 32+43+37+35+37=184
PA1 讲解、作业实现与实验解答	仅本地保留；公开版不含第 21、27 章
编程/白盒指引与模板	27：规格、复杂度、边界与验证
两份本地样卷	31：只提炼通用方法，不转载课内受限原题

原生解析与视觉核对

PDF 提取原生文字与页码，PPT/PPTX 提取文字和内嵌图片，同时原生兼容导出完整幻灯片以保留箭头、布局、曲线与公式。机器已为可用课件生成完整页级联系表；写入笔记的关键原图逐幅视觉核对。未声称已对数千页的每一个细节完成逐字人工审校。PPT 导出中缺失中文字体替换为 Noto Sans CJK SC，原件不覆盖。

原生提取和视觉复核需要互相补足：抽出的字不能告诉你箭头指谁，图片也不总能准确给出公式符号。课件中的省略或不一致处，本书在相关段落说明，不把它们无条件当作定理。

原图逐项出处

- 老师原图-向量.png：数据结构/slides/分章课件/02 向量.pdf，PDF 第 29 页。直接渲染原始 PDF 页，不改图中结构。
- 老师原图-单调栈.png：数据结构/slides/分章课件/04 栈与列表.pdf，PDF 第 95 页。直接渲染原始 PDF 页，不改图中结构。
- 老师原图-Huffman.png：数据结构/slides/分章课件/05 二叉树.pdf，PDF 第 105 页。直接渲染原始 PDF 页，不改图中结构。

- 老师原图-增强遍历.png：数据结构/slides/分章课件/05 二叉树.pdf，PDF 第 87 页。直接渲染原始 PDF 页，不改图中结构。
- 老师原图-AVL.png：数据结构/slides/整套讲义/电子讲义(按章节).7z，DSA-2026F-Chapter/06.二叉搜索树.pdf，PDF 第 30 页。直接渲染原始 PDF 页，不改图中结构。
- 老师原图-KMP.png：数据结构/slides/整套讲义/电子讲义（按章节）.7z，DSA-2026F-Chapter/13.串.pdf，PDF 第 28 页。直接渲染原始 PDF 页，不改图中结构。
- 课程完整原页-最大区间接口.png、课程完整原页-最大区间扫描.png：数据结构/slides/分章课件/01 算法概论.pdf，PDF 第 94、99 页（印刷第 93、98 页）。保留完整原页，03 章解释允许空区间、同和短者与靠后者优先，以及从右向左扫描。这一部分属于本地 119 页分章版，不能按 7z 内 90 页同名章节定位。
- ds-v4-19-teacher-next29.png：7z 内 DSA-2026F-Chapter/13.串.pdf，PDF 第 29 页、页脚 4；19 章完整展示教师改进 next 的递推图和程序，正文用同一模式逐项构表。
- ds-v4-20-teacher-LUG06.png：7z 内 DSA-2026F-Chapter/14.排序.pdf，PDF 第 6 页、页脚 2；20 章保留完整 LUG 原码，并解释其中左扫描比较方向和 for 结束括号的勘误，不修改原图冒充修订课件。
- ds-v4-20-textbook-LUG329.png：数据结构/资料/教材与参考/数据结构教材（2026 版）.pdf，PDF 第 329 页、印刷 303 页；算法 14.2 第 8 行及正文给出正确的比较方向，图 14.3D 的小标签仍残留反向符号，20 章明确指出应核对哪一处。

原创 SVG 图由工具脚本/教材/配图.py 生成，文字转为轮廓，可离线显示；它们是本书的补充示意，不伪装成教师原图。

连续步骤图演示-*.svg 由工具脚本/教材/逐步图.py 生成，含当前状态、执行动作和原因。PDF 显示每一步，网页在同一记录上提供前后步、播放、拖动时间轴和预测后揭示；关闭交互后仍可展开连续静态图。只有专门标记的自编 SVG 会启用播放器，教师原页保持完整图片。

本轮还实际对照 Sedgewick、Wayne 的归并讲解、平衡搜索树、最短路，以及 Erickson 动态规划章。参考其如何把过程显式展开，例子与中文讲解按本课程重新组织，不混用不同版本的算法约定。

版本和复核边界

2026-10-25，用新版 OneTHU 默认共享后台只读核验课程文件清单，三门课文件枚举均返回 complete=true。数据结构的综合资源接口对作业全量枚举仍给出未核实状态，因此本书不宣称覆盖了独立 OJ 未来/全部题目。随后已从作者官方入口取得 669 页主教材与《习题解析》合体 PDF；数据结构 32–33 章补充 1-32 的 16 种程序推导和全部 12 章交叉索引。取得全文不等于已经把其中 292 道大题所有小问逐一重做，具体覆盖按各章说明。

完整文字、公式与图在 Markdown 和 Typst 间共享；每次改正文后应运行构建器，Pandoc 公式转换警告将阻止成功构建。重要例题按推导、量纲、边界和独立小规模程序交叉检查，发现错误应同时修正文并重建两版本。

延伸阅读：MIT 算法课讲义。本书的章节编号与课程本地教材对应，不照搬另一门课的组织。

本版补充阅读的具体落点

33 章现在不仅列目录，还转述并独立讲解旧版习题 1-1、2-2、3-3、4-1、5-1、6-1、7-1、8-1、9-2、10-1、11-1、12-1，逐项给官方合体 PDF 物理页；相关页已结合原生文字与原页图形核对。13 章额外对照习题 8-2 的 PDF580 至 582 页，展开伸展访问引理的三种局部势能推导。32 章按 1-32 的 16 段分别给状态、阈值、求和与模型条件；这仍不表示 292 道原书大题及其全部小问已逐题解答。

30 章的 21 式从只有结果扩成每次实际运算与栈状态表，除法用精确有理数。教师原版乘方左结合与数学常见右结合现已分别说明；这 21 式没有无括号连续乘方链，数值不受该区别影响。代码中的乘方 token 使用自定义运算语义，不能直接按 C++异或解释。

原生提取用于准确检索定义和程序，视觉核对用于检查箭头、树形、上下限、缺失孩子和优先表；本版没有把生成联系表等同于已经逐页读完全部源材料。实际讲解范围以正文例题与题号为准。

写法另参考用户提供的《啊哈 C 语言 !逻辑的挑战（修订版）》与《啊哈算法》。已分工读完两书正文、相关图与代码，包括算法书后半的整页扫描。借鉴同一任务怎样从具体操作推进到新方法，正文、算例与配图独立编写；原书电子文件和解包内容仅保存在本地忽略目录。课程接口与技术结论仍按教师资料核对。

读图、读程序和读公式时怎样查符号

本章用于遇到陌生写法时回查。正文第一次使用符号时仍会解释，读者不需要先背完这张表。一个字母在不同问题里可能表示不同的量，因此每道例题会重新说明它的含义。

数组格子和区间

数组 a=[8,3,6]是依次排列的三个元素。下标是位置的编号，本书从 0 开始：a[0]=8，a[1]=3，a[2]=6。长度记为n，本例$n=3$；合法下标为 0、1、2。下标 3 指向末尾之后，不能拿来读取元素。

a[lo,hi)表示从下标 lo 开始，一直取到 hi 之前；左方括号表示包含 lo，右圆括号表示不包含 hi。例如 a[1,3)取出 3、6，共两项。长度为何是$hi-lo$？这些位置依次是$lo, lo+1, \ldots, hi-1$，末项减首项再加 1，得到$(hi-1)-lo+1=hi-lo$。若$lo=hi$，没有位置可取，称为空区间。这里逗号分隔两个边界，不是两个独立数组元素。

代码里的 a[i:j]有时用来说明同一个半开区间；C++原生数组不提供这种切片语法，实际程序一般把两个边界作为参数传进去。a[i]则只指一个元素。

图中格子内是当前值，格子下方是下标。指针或游标写在格子上方。颜色只作辅助：橙色格表示本步正在检查或修改，具体改变仍会在图下写明。搬移中可能同时出现两个相同值，那是暂时复制；还在局部变量里保存的待插入值也必须一起看，不能只看数组就认定元素丢失。

赋值、比较和状态

x ← 5 读作“把 5 保存到 x”；C/C++代码写作 x = 5。它会改变状态。x == 5 是询问当前 x 是否等于 5，不进行保存。数学式$x=5$表达相等关系，需根据上下文与程序赋值区分。

程序写 x=x+1 时，先用右侧旧 x 算出结果，再覆盖左侧 x。例如旧值 4，右侧得到 5，最后 x 变为 5。这里并不是要求同一个数在数学上等于它自己加 1。为了在推导中消除歧义，可以写$x_{t+1}=x_t+1$，其中t是执行轮次，x_t是第t轮开始时的值。

状态是继续执行所必需的信息。例如插入排序进行到一半，状态至少包含数组 a、暂存值 x、正在检查的下标 j。若只画 a 而遗漏 x，就无法判断下一步要把什么填回去。不变量是某个指定时刻始终成立的陈述，例如“每轮外层循环开始时，a 的前 i 项已有序”；它不代表每条机器指令中间都保持同一句形式。

求和、乘积、取整与对数

符号$\sum_{i=1}^{4} i$读作“把 i 依次取 1、2、3、4 时得到的值加起来”，实际就是$1+2+3+4=10$。下面的 1 是起点，上面的 4 是终点，i是轮流替换的求和下标。若写$\sum_{i=1}^{4} 2i$，每一项先乘 2，得到$2+4+6+8=20$。下标不是额外相乘的一项。

$\prod_{i=1}^{4} i$使用大写希腊字母 Π 形状的乘积号，表示$1\times2\times3\times4=24$；它与求和号 Σ 不同。$n!$读作 n 的阶乘，等于$1\times2\times\cdots\times n$，约定$0!=1$。计算排列数时，“第一位 n 种、第二位 n−1 种……”相乘便得到$n!$。

$\lfloor x\rfloor$为不大于 x 的最大整数。例如$\lfloor 2.8\rfloor=2$、$\lfloor -2.8\rfloor=-3$。$\lceil x\rceil$为不小于 x 的最小整数，因此$\lceil 2.8\rceil=3$。负数向下取整与 C++整数除法朝 0 截断不同，程序中需要单独核对。

2^k表示 k 个 2 相乘；$\log_2 n$问“2 的几次方等于 n”。例如$2^3=8$，所以$\log_2 8=3$。当二分每次把候选数量减半，8 项依次变成 4、2、1，共减半 3 次，这就是对数出现的具体原因。$\lg n$在算法教材中常指$\log_2 n$，$\ln n$指以自然常数 e 为底的对数；本书在容易混淆处直接写底数。

复杂度记号比较的是什么

用$T(n)$表示输入大小为 n 时所计基本操作的数量。先确定操作模型，才知道一次比较、移动或大整数乘法各算多贵。O、Ω、Θ都是比较函数随 n 增长的速度；它们本身不指定最好、平均或最坏情况。

说$T(n)=O(n^2)$，意思是存在固定常数$c>0$和门槛n_0，使所有$n\ge n_0$都满足$T(n)\le cn^2$。例如$T(n)=3n^2+2n+7$，当$n\ge1$时，$2n\le2n^2$、$7\le7n^2$，于是$T(n)\le12n^2$。可取$c=12$、$n_0=1$。常数可以依赖具体算法，但不能随着 n 增长而增长。

说$T(n)=\Omega(n^2)$是相反方向的下界，要求充分大的 n 上$T(n)\ge cn^2$。同一个例子中$3n^2+2n+7\ge 3n^2$，所以也满足下界。上下两边都成立才写$T(n)=\Theta(n^2)$。仅给上界并没有说明它紧：这个函数也属于$O(n^3)$，但三次上界描述得较松。

最坏复杂度会先在长度 n 的所有合法输入里取最大操作数，再比较这个新函数的增长；平均复杂度则要先说输入遵循什么概率分布。均摊复杂度计算一串操作的总成本再除以操作数，不能把一个昂贵操作单独保证为常数，也不能无条件当成随机平均。

树、图与字符串中的局部记号

树的根是没有父节点的起点；孩子是从某节点向下一层直接连接的节点；后代还包括继续向下走多层的节点。叶节点没有孩子。空孩子位置表示没有节点，与键值为 0 的节点不同。树高按最长向下路径的边数计算：叶高 0、空树高 −1。一个节点的子树含这个节点自身及全部后代，所以子树规模不是只数后代。

栈按后进先出取元素。本书横画时明确标“底→顶”，最右边才是可弹出的栈顶；竖画时会另标顶部。队列按先入先出，图中会标队首与队尾。哨兵是为了统一边界处理加上的辅助标记或节点，通常不算真实数据；统计栈峰值等题若把哨兵计入，则必须在题目约定里写出。

图的顶点是对象，边表示对象间的关系。有向边$u\to v$只保证从 u 到 v 可走，不能自动反向。V有时指顶点集合，有时在复杂度中简写顶点数；严谨写法$|V|$是集合中元素的个数。边集合 E 也同理。$w(u,v)$是边 u→v 的权重，不必等于两个顶点的编号差。$d[v]$在最短路算法中是当前找到的候选距离，在被证明确定前不等于最终答案。

字符串 T 是文本，P 是要寻找的模式。前缀从字符串开头开始，后缀在末尾结束；“真前缀”排除整个字符串自身。子串要求字符连续，子序列允许跳过中间字符但保持先后顺序。`abc` 的 `ac` 是子序列而非子串。KMP 保留的是已经匹配片段的前后缀关系；LCS 求的是两个串的最长公共子序列，两者回答不同问题。